Lecture Notes on Quantum Algorithms

Andrew M. Childs

Department of Computer Science,
Institute for Advanced Computer Studies, and
Joint Center for Quantum Information and Computer Science
University of Maryland

17 April 2025

Contents

Preface **vii**

1 Preliminaries **1**
1.1 Quantum data . . . 1
1.2 Quantum circuits . . . 1
1.3 Universal gate sets . . . 2
1.4 Reversible computation . . . 2
1.5 Uniformity . . . 3
1.6 Quantum complexity . . . 3
1.7 Fault tolerance . . . 3

I Quantum circuits **5**

2 Efficient universality of quantum circuits **7**
2.1 Subadditivity of errors . . . 7
2.2 The group commutator and a net around the identity . . . 8
2.3 Proof of the Solovay-Kitaev Theorem . . . 8
2.4 Proof of Lemma 2.3 . . . 9

3 Quantum circuit synthesis over Clifford+T **11**
3.1 Converting to Matsumoto-Amano normal form . . . 11
3.2 Uniqueness of Matsumoto-Amano normal form . . . 12
3.3 Algebraic characterization of Clifford+T unitaries . . . 13
3.4 From exact to approximate synthesis . . . 14

II Quantum algorithms for algebraic problems **15**

4 The abelian quantum Fourier transform and phase estimation **17**
4.1 Quantum Fourier transform . . . 17
4.2 QFT over $\mathbb{Z}_{2^n}$ . . . 17
4.3 Phase estimation . . . 19
4.4 QFT over $\mathbb{Z}_N$ and over a general finite abelian group . . . 19

5 Discrete log and the hidden subgroup problem **21**
5.1 Discrete log . . . 21
5.2 Diffie-Hellman key exchange . . . 21
5.3 The hidden subgroup problem . . . 22
5.4 Shor's algorithm . . . 23

6 The abelian HSP and decomposing abelian groups **25**
6.1 The abelian HSP . . . 25
6.2 Decomposing abelian groups . . . 27

7 Quantum attacks on elliptic curve cryptography **29**
7.1 Elliptic curves . . . 29
7.2 Elliptic curve cryptography . . . 32
7.3 Shor's algorithm for discrete log over elliptic curves . . . 32

8 Quantum algorithms for number fields **33**

8.1 Review: Order finding . . . 33
8.2 Pell's equation . . . 33
8.3 Some basic algebraic number theory . . . 34
8.4 A periodic function for the units of $\mathbb{Z}[\sqrt{d}]$ . . . 35

9 Period finding from $\mathbb{Z}$ to $\mathbb{R}$ 37
9.1 Period finding over the integers . . . 37
9.2 Period finding over the reals . . . 39
9.3 Other algorithms for number fields . . . 41

10 Quantum query complexity of the HSP 43
10.1 The nonabelian HSP and its applications . . . 43
10.2 The standard method . . . 44
10.3 Query complexity of the HSP . . . 45

11 Fourier analysis in nonabelian groups 47
11.1 A brief introduction to representation theory . . . 47
11.2 Fourier analysis for nonabelian groups . . . 49

12 Fourier sampling 51
12.1 Weak Fourier sampling . . . 51
12.2 Normal subgroups . . . 52
12.3 Strong Fourier sampling . . . 53

13 Kuperberg's algorithm for the dihedral HSP 55
13.1 The HSP in the dihedral group . . . 55
13.2 Fourier sampling in the dihedral group . . . 56
13.3 Combining states . . . 56
13.4 The Kuperberg sieve . . . 57
13.5 Analysis of the Kuperberg sieve . . . 57
13.6 Entangled measurements . . . 58

14 The HSP in the Heisenberg group 59
14.1 The Heisenberg group . . . 59
14.2 Fourier sampling . . . 60
14.3 Two states are better than one . . . 60

15 Schur-Weyl duality 63
15.1 The Schur decomposition . . . 63
15.2 Weak Schur sampling . . . 64
15.3 The swap test . . . 65

16 Approximating the Jones polynomial 67
16.1 The Hadamard test . . . 67
16.2 The Jones polynomial . . . 67
16.3 Links from braids . . . 68
16.4 Representing braids in the Temperley-Lieb algebra . . . 68
16.5 A quantum algorithm . . . 69
16.6 Quality of approximation . . . 69
16.7 Other algorithms . . . 70

III Quantum walk 71

17 Continuous-time quantum walk 73
17.1 Continuous-time quantum walk . . . 73
17.2 Random and quantum walks on the hypercube . . . 74
17.3 Random and quantum walks in one dimension . . . 75
17.4 Black-box traversal of the glued trees graph . . . 75
17.5 Quantum walk algorithm to traverse the glued trees graph . . . 76
17.6 Classical and quantum mixing . . . 78

17.7 Classical lower bound . . . 80

18 Discrete-time quantum walk **81**
18.1 Discrete-time quantum walk . . . 81
18.2 How to quantize a Markov chain . . . 82
18.3 Spectrum of the quantum walk . . . 83
18.4 Hitting times . . . 84

19 Unstructured search **87**
19.1 Unstructured search . . . 87
19.2 Quantum walk algorithm . . . 87
19.3 Amplitude amplification and quantum counting . . . 88
19.4 Search on graphs . . . 89

20 Quantum walk search **91**
20.1 Element distinctness . . . 91
20.2 Quantum walk algorithm . . . 92
20.3 Quantum walk search algorithms with auxiliary data . . . 93

IV Quantum query complexity **95**

21 Query complexity and the polynomial method **97**
21.1 Quantum query complexity . . . 97
21.2 Quantum queries . . . 98
21.3 Quantum algorithms and polynomials . . . 98
21.4 Symmetrization . . . 99
21.5 Parity . . . 99
21.6 Unstructured search . . . 100

22 The collision problem **103**
22.1 Problem statement . . . 103
22.2 Classical query complexity . . . 103
22.3 Quantum algorithm . . . 104
22.4 Toward a quantum lower bound . . . 104
22.5 Constructing the functions . . . 105
22.6 Finishing the proof . . . 106

23 The quantum adversary method **109**
23.1 Quantum adversaries . . . 109
23.2 The adversary method . . . 110
23.3 Example: Unstructured search . . . 113

24 Span programs and formula evaluation **115**
24.1 The dual of the adversary method . . . 115
24.2 Span programs . . . 116
24.3 Unstructured search . . . 117
24.4 Formulas and games . . . 117
24.5 Function composition . . . 118
24.6 An algorithm from a dual adversary solution . . . 119

25 Learning graphs **121**
25.1 Learning graphs and their complexity . . . 121
25.2 Unstructured search . . . 121
25.3 From learning graphs to span programs . . . 122
25.4 Element distinctness . . . 123
25.5 Other applications . . . 124

26 The compressed oracle method **125**
26.1 The compressed oracle . . . 125
26.2 Relating the standard and compressed oracles . . . 126

26.3 Unstructured search . . . 128
26.4 Collision finding . . . 129

V Quantum simulation 131

27 Simulating Hamiltonian dynamics 133
27.1 Hamiltonian dynamics . . . 133
27.2 Efficient simulation . . . 133
27.3 Product formulas . . . 134
27.4 Sparse Hamiltonians . . . 135
27.5 Measuring an operator . . . 136

28 Fast quantum simulation algorithms 139
28.1 No fast-forwarding . . . 139
28.2 Quantum walk . . . 140
28.3 Linear combinations of unitaries . . . 140

29 Quantum signal processing 143
29.1 Block encoding . . . 143
29.2 Quantum signal processing . . . 144
29.3 Qubitization . . . 146
29.4 Application to Hamiltonian simulation . . . 147

VI Adiabatic quantum computing 149

30 The quantum adiabatic theorem 151
30.1 Adiabatic evolution . . . 151
30.2 Proof of the adiabatic theorem . . . 152

31 Adiabatic optimization 157
31.1 An adiabatic optimization algorithm . . . 157
31.2 The running time and the gap . . . 158
31.3 Adabatic optimization algorithm for unstructured search . . . 159

32 An example of the success of adiabatic optimization 163
32.1 The ring of agrees . . . 163
32.2 The Jordan-Wigner transformation: From spins to fermions . . . 164
32.3 Diagonalizing a system of free fermions . . . 166
32.4 Diagonalizing the ring of agrees . . . 168

33 Universality of adiabatic quantum computation 171
33.1 The Feynman quantum computer . . . 171
33.2 An adiabatic variant . . . 172
33.3 Locality . . . 175

Bibliography 177

Preface

This is a set of lecture notes on quantum algorithms. It is primarily intended for graduate students who have already taken an introductory course on quantum information. Such a course typically covers only the early breakthroughs in quantum algorithms, namely Shor's factoring algorithm and Grover's searching algorithm. Here we show that there is much more to quantum computing by exploring some of the many quantum algorithms that have been developed subsequently.

These notes cover several major topics in quantum algorithms, divided into six parts:

- In Part I, we discuss *quantum circuits*—in particular, the problem of expressing a quantum algorithm using a given universal set of quantum gates.
- In Part II, we discuss *quantum algorithms for algebraic problems*. Many of these algorithms generalize the main idea of Shor's algorithm. These algorithms use the quantum Fourier transform and typically achieve an exponential (or at least superpolynomial) speedup over classical computers. In particular, we explore a group-theoretic problem called the *hidden subgroup problem*. A solution of this problem for abelian groups leads to several applications; we also discuss what is known about the nonabelian case.
- In Part III, we explore the concept of *quantum walk*, a quantum generalization of random walk. This concept leads to a powerful framework for solving search problems, generalizing Grover's search algorithm.
- In Part IV, we discuss the model of *quantum query complexity*. We cover the two main methods for proving lower bounds on quantum query complexity (the polynomial method and the adversary method), demonstrating limitations on the power of quantum algorithms. We also discuss how the concept of *span programs* turns the quantum adversary method into an upper bound, giving optimal quantum algorithms for evaluating Boolean formulas.
- In Part V, we describe quantum algorithms for *simulating the dynamics of quantum systems*. We also discuss an application of quantum simulation to an algorithm for linear systems.
- In Part VI, we discuss *adiabatic quantum computing*, a general approach to solving optimization problems (in a similar spirit to simulated annealing). Related ideas may also provide insights into how one might build a quantum computer.

These notes were originally prepared for a course that was offered three times at the University of Waterloo: in the winter terms of 2008 (as CO 781) and of 2011 and 2013 (as CO 781/CS 867/QIC 823). The course has also been offered three times at the University of Maryland (in 2017, 2021, and 2025). I thank the students in these courses for their feedback on the lecture notes. Each offering of the course has covered a somewhat different set of topics. This document collects the material from all versions of the course and includes a few subsequent improvements.

The material on quantum algorithms for algebraic problems has been collected into a review article that was written with Wim van Dam [38]. I thank Wim for his collaboration on that project, which strongly influenced the presentation in Part II.

Please keep in mind that these are rough lecture notes; they are not meant to be a comprehensive treatment of the subject, and they surely contain mistakes. Corrections (by email to amchilds@umd.edu) are welcome.

I hope you find these notes to be a useful resource for learning about quantum algorithms.

Chapter 1

Preliminaries

This chapter briefly reviews some background material on quantum computation. We cover these topics at a very high level, just to give a sense of what you should know to understand the rest of the lecture notes. If any of these topics are unfamiliar, you can learn more about them from a text on quantum computation such as Nielsen and Chuang [88]; Kitaev, Shen, and Vyalyi [70]; or Kaye, Laflamme, and Mosca [68].

1.1 Quantum data

A quantum computer is a device that uses a quantum mechanical representation of information to perform calculations. Information is stored in quantum bits, the states of which can be represented as ℓ_2-normalized vectors in a complex vector space. For example, we can write the state of n qubits as

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} a_x |x\rangle \tag{1.1}$$

where the $a_x \in \mathbb{C}$ satisfy $\sum_x |a_x|^2 = 1$. We refer to the basis of states $|x\rangle$ as the *computational basis*.

It will often be useful to think of quantum states as storing data in a more abstract form. For example, given a group G, we could write $|g\rangle$ for a basis state corresponding to the group element $g \in G$, and

$$|\phi\rangle = \sum_{g \in G} b_g |g\rangle \tag{1.2}$$

for an arbitrary superposition over the group. We assume that there is some canonical way of efficiently representing group elements using bit strings; it is usually unnecessary to make this representation explicit.

If a quantum computer stores the state $|\psi\rangle$ *and* the state $|\phi\rangle$, its overall state is given by the tensor product of those two states. This may be denoted $|\psi\rangle \otimes |\phi\rangle = |\psi\rangle|\phi\rangle = |\psi, \phi\rangle$.

1.2 Quantum circuits

The allowed operations on (pure) quantum states are those that map normalized states to normalized states, namely *unitary operators* U, satisfying $UU^\dagger = U^\dagger U = I$. (You probably know that there are more general quantum operations, but for the most part we will not need to use them in this course.)

To have a sensible notion of *efficient* computation, we require that the unitary operators appearing in a quantum computation are realized by *quantum circuits*. We are given a set of gates, each of which acts on one or two qubits at a time (meaning that it is a tensor product of a one- or two-qubit operator with the identity operator on the remaining qubits). A quantum computation begins in the $|0\rangle$ state, applies a sequence of one- and two-qubit gates chosen from the set of allowed gates, and finally reports an outcome obtained by measuring in the computational basis.

1.3 Universal gate sets

In principle, any unitary operator on n qubits can be implemented using only 1- and 2-qubit gates. Thus we say that the set of all 1- and 2-qubit gates is *(exactly) universal*. Of course, some unitary operators may take many more 1- and 2-qubit gates to realize than others, and indeed, a counting argument shows that most unitary operators on n qubits can only be realized using an exponentially large circuit of 1- and 2-qubit gates.

In general, we are content to give circuits that give good approximations of our desired unitary transformations. We say that a circuit with gates $U_1, U_2, \ldots, U_t$ approximates U with precision ϵ if

$$\|U - U_t \ldots U_2 U_1\| \leq \epsilon. \tag{1.3}$$

Here $\|\cdot\|$ denotes some appropriate matrix norm, which should have the property that if $\|U - V\|$ is small, then U should be hard to distinguish from V no matter what quantum state they act on. A natural choice (which will be suitable for our purposes) is the spectral norm

$$\|A\| := \max_{|\psi\rangle} \frac{\|A|\psi\rangle\|}{\||\psi\rangle\|}, \tag{1.4}$$

(where $\||\psi\rangle\| = \sqrt{\langle\psi|\psi\rangle}$ denotes the vector 2-norm of $|\psi\rangle$), i.e., the largest singular value of A. Then we call a set of elementary gates *universal* if any unitary operator on a fixed number of qubits can be approximated to any desired precision ϵ using elementary gates.

It turns out that there are finite sets of gates that are universal: for example, the set $\{H, T, C\}$ with

$$H := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \qquad T := \begin{pmatrix} e^{i\pi/8} & 0 \\ 0 & e^{-i\pi/8} \end{pmatrix} \qquad C := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{1.5}$$

There are situations in which we say a set of gates is *effectively* universal, even though it cannot actually approximate any unitary operator on n qubits. For example, the set $\{H, T^2, \text{Tof}\}$, where

$$\text{Tof} := \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \tag{1.6}$$

is universal, but only if we allow the use of ancilla qubits (qubits that start and end in the $|0\rangle$ state). Similarly, the basis $\{H, \text{Tof}\}$ is universal in the sense that, with ancillas, it can approximate any *orthogonal* matrix. It clearly cannot approximate complex unitary matrices, since the entries of H and Tof are real; but the effect of arbitrary unitary transformations can be simulated using orthogonal ones by simulating the real and imaginary parts separately.

1.4 Reversible computation

Unitary matrices are invertible: in particular, $U^{-1} = U^\dagger$. Thus any unitary transformation is a reversible operation. This may seem at odds with how we often define classical circuits, using irreversible gates such as AND and OR. But in fact, any classical computation can be made reversible by replacing any irreversible gate $x \mapsto g(x)$ by the reversible gate $(x, y) \mapsto (x, y \oplus g(x))$, and running it on the input $(x, 0)$, producing $(x, g(x))$. In other words, by storing all intermediate steps of the computation, we make it reversible.

On a quantum computer, storing all intermediate computational steps could present a problem, since two identical results obtained in different ways would not be able to interfere. However, there is an easy way to

remove the accumulated information. After performing the classical computation with reversible gates, we simply XOR the answer into an ancilla register, and then perform the computation in reverse. Thus we can implement the map $(x, y) \mapsto (x, y \oplus f(x))$ even when f is a complicated circuit consisting of many gates.

Using this trick, any computation that can be performed efficiently on a classical computer can be performed efficiently on a quantum computer: if we can efficiently implement the map $x \mapsto f(x)$ on a classical computer, we can efficiently perform the transformation $|x, y\rangle \mapsto |x, y \oplus f(x)\rangle$ on a quantum computer. This transformation can be applied to any superposition of computational basis states, so for example, we can perform the transformation

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x, 0\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x, f(x)\rangle. \tag{1.7}$$

Note that this does not necessarily mean we can efficiently implement the map $|x\rangle \mapsto |f(x)\rangle$, even when f is a bijection (so that this is indeed a unitary transformation). However, if we can efficiently invert f, then we can indeed do this efficiently by computing $f(x)$ in another register and then reversibly uncomputing x using the inverse of the reversible circuit for computing f^{-1}.

1.5 Uniformity

When we give an algorithm for a computational problem, we consider inputs of varying sizes. Typically, the circuits for instances of different sizes will be related to one another in a simple way. But this need not be the case; and indeed, given the ability to choose an arbitrary circuit for each input size, we could have circuits computing uncomputable languages. Thus we require that our circuits be *uniformly generated*: say, that there exists a fixed (classical) Turing machine that, given a tape containing the symbol '1' n times, outputs a description of the nth circuit in time poly(n).

1.6 Quantum complexity

We say that an algorithm for a problem is *efficient* if the circuit describing it contains a number of gates that is polynomial in the number of bits needed to write down the input. For example, if the input is a number modulo N, the input size is $\lceil \log_2 N \rceil$.

With a quantum computer, as with a randomized (or noisy) classical computer, the final result of a computation may not be correct with certainty. Instead, we are typically content with an algorithm that can produce the correct answer with high enough probability (for a decision problem, bounded above $1/2$; for a non-decision problem for which we can check a correct solution, $\Omega(1)$). By repeating the computation many times, we can make the probability of outputting an incorrect answer arbitrarily small.

In addition to considering explicit computational problems, in which the input is a string, we will also consider the concept of *query complexity*. Here the input is a black box transformation, and our goal is to discover some property of the transformation by making as few queries as possible. For example, in Simon's problem, we are given a transformation $f\colon \mathbb{Z}_2^n \to S$ satisfying $f(x) = f(y)$ iff $y = x \oplus t$ for some unknown $t \in \mathbb{Z}_2^n$, and the goal is to learn t. The main advantage of considering query complexity is that it allows us to prove lower bounds on the number of queries required to solve a given problem. Furthermore, if we find an efficient algorithm for a problem in query complexity, then if we are given an explicit circuit realizing the black-box transformation, we will have an efficient algorithm for an explicit computational problem.

Sometimes, we care not just about the size of a circuit for implementing a particular unitary operation, but also about its *depth*, the maximum number of gates on any path from an input to an output. The depth of a circuit tells us how long it takes to implement if we can perform gates in parallel.

1.7 Fault tolerance

In any real computer, operations cannot be performed perfectly. Quantum gates and measurements may be performed imprecisely, and errors may happen even to stored data that is not being manipulated. Fortu-

nately, there are protocols for dealing with faults that may occur during the execution of a quantum computation. Specifically, the *threshold theorem* states that as long as the noise level is below some threshold (depending on the noise model, but typically in the range of 10^{-3} to 10^{-4}), an arbitrarily long computation can be performed with an arbitrarily small amount of error (see for example [55]).

In this course, we will always assume implicitly that fault-tolerant protocols have been applied, such that we can effectively assume a perfectly functioning quantum computer.

Part I

Quantum circuits

Chapter 2

Efficient universality of quantum circuits

Are some universal gate sets better than others? Classically, this is not an issue: the set of possible operations is discrete, so any gate acting on a constant number of bits can be simulated exactly using a constant number of gates from any given universal gate set. But we might imagine that some quantum gates are much more powerful than others. For example, given two rotations about strange axes by strange angles, it may not be obvious how to implement a Hadamard gate, and we might worry that implementing such a gate to high precision could take a very large number of elementary operations, scaling badly with the required precision.

Fortunately, this is not the case: a unitary operator that can be realized efficiently with one set of 1- and 2-qubit gates can also be realized efficiently with another such set. In particular, we have the following (see [88, Appendix 3], [43], and [70, Chapter 8]).

Theorem 2.1 (Solovay-Kitaev)**.** *Fix two universal gate sets that are closed under inverses. Then any t-gate circuit using one gate set can be implemented to precision ϵ using a circuit of $t \cdot \mathrm{poly}(\log \frac{t}{\epsilon})$ gates from other set (indeed, there is a classical algorithm for finding this circuit in time $t \cdot \mathrm{poly}(\log \frac{t}{\epsilon})$).*

Thus, not only are the two gate sets equivalent under polynomial-time reduction, but the running time of an algorithm using one gate set is the same as that using the other gate set up to logarithmic factors. This means that even polynomial quantum speedups are robust with respect to the choice of gate set.

2.1 Subadditivity of errors

We begin with the basic fact that errors in the approximation of one quantum circuit by another accumulate at most linearly.

Lemma 2.2. *Let U_i, V_i be unitary matrices satisfying $\|U_i - V_i\| \leq \epsilon$ for all $i \in \{1, 2, \ldots, t\}$. Then*

$$\|U_t \ldots U_2 U_1 - V_t \ldots V_2 V_1\| \leq t\epsilon. \tag{2.1}$$

Proof. We use induction on t. For $t = 1$ the lemma is trivial. Now suppose the lemma holds for a particular value of t. Then by the triangle inequality and the fact that the norm is unitarily invariant ($\|UAV\| = \|A\|$ for any unitary matrices U, V),

$$\begin{aligned}
&\|U_{t+1}U_t \ldots U_1 - V_{t+1}V_t \ldots V_1\| \\
&\quad = \|U_{t+1}U_t \ldots U_1 - U_{t+1}V_t \ldots V_1 + U_{t+1}V_t \ldots V_1 - V_{t+1}V_t \ldots V_1\| && (2.2)\\
&\quad \leq \|U_{t+1}U_t \ldots U_1 - U_{t+1}V_t \ldots V_1\| + \|U_{t+1}V_t \ldots V_1 - V_{t+1}V_t \ldots V_1\| && (2.3)\\
&\quad = \|U_{t+1}(U_t \ldots U_1 - V_t \ldots V_1)\| + \|(U_{t+1} - V_{t+1})V_t \ldots V_1\| && (2.4)\\
&\quad = \|U_t \ldots U_1 - V_t \ldots V_1\| + \|U_{t+1} - V_{t+1}\| && (2.5)\\
&\quad \leq (t+1)\epsilon, && (2.6)
\end{aligned}$$

so the lemma follows by induction. □

Thus, in order to simulate a t-gate quantum circuit with total error at most ϵ, it suffices to simulate each individual gate with error at most ϵ/t.

2.2 The group commutator and a net around the identity

To simulate an arbitrary individual gate, the strategy is to first construct a very fine net covering a very small ball around the identity using the *group commutator*,

$$[\![U, V]\!] := UVU^{-1}V^{-1}. \tag{2.7}$$

To approximate general unitaries, we will effectively translate them close to the identity.

Note that it suffices to consider unitary gates with determinant 1 (i.e., elements of SU(2)) since a global phase is irrelevant. Let

$$S_\epsilon := \{U \in \mathrm{SU}(2) \colon \|I - U\| \leq \epsilon\} \tag{2.8}$$

denote the ϵ-ball around the identity. Given sets $\Gamma, S \subseteq \mathrm{SU}(2)$, we say that Γ is an *ϵ-net* for S if for any $A \in S$, there is a $U \in \Gamma$ such that $\|A - U\| \leq \epsilon$. The following result (to be proved later on) indicates how the group commutator helps us to make a fine net around the identity.

Lemma 2.3. *If Γ is an ϵ^2-net for S_ϵ, then $[\![\Gamma, \Gamma]\!] := \{[\![U, V]\!] \colon U, V \in \Gamma\}$ is an $O(\epsilon^3)$-net for S_{ϵ^2}.*

To make an arbitrarily fine net, we apply this idea recursively. But first it is helpful to derive a consequence of the lemma that is more suitable for recursion. We would like to maintain the quadratic relationship between the size of the ball and the quality of the net. If we aim for a $k^2\epsilon^3$-net (for some constant k), we would like it to apply to arbitrary points in $S_{k\epsilon^{3/2}}$, whereas the lemma only lets us approximate points in S_{ϵ^2}. To handle an arbitrary $A \in S_{k\epsilon^{3/2}}$, we first let W be the closest gate in Γ to A. For sufficiently small ϵ we have $k\epsilon^{3/2} < \epsilon$, so $S_{k\epsilon^{3/2}} \subset S_\epsilon$, and therefore $A \in S_\epsilon$. Since Γ is an ϵ^2-net for S_ϵ, we have $\|A - W\| \leq \epsilon^2$, i.e., $\|AW^\dagger - I\| \leq \epsilon^2$, so $AW^\dagger \in S_{\epsilon^2}$. Then we can apply the lemma to find $U, V \in \Gamma$ such that $\|AW^\dagger - [\![U, V]\!]\| = \|A - [\![U, V]\!]W\| \leq k^2\epsilon^3$. In other words, if Γ is an ϵ^2-net for S_ϵ, then $[\![\Gamma, \Gamma]\!]\Gamma := \{[\![U, V]\!]W \colon U, V, W \in \Gamma\}$ is a $k^2\epsilon^3$-net for $S_{k\epsilon^{3/2}}$.

Now suppose that Γ_0 is an ϵ_0^2-net for S_{ϵ_0}, and let $\Gamma_i := [\![\Gamma_{i-1}, \Gamma_{i-1}]\!]\Gamma_{i-1}$ for all positive integers i. Then Γ_i is an ϵ_i^2-net for S_{ϵ_i}, where $\epsilon_i = k\epsilon_{i-1}^{3/2}$. Solving this recursion gives $\epsilon_i = (k^2\epsilon_0)^{(3/2)^i}/k^2$.

2.3 Proof of the Solovay-Kitaev Theorem

With these tools in hand, we are prepared to establish the main result.

Proof of Theorem 2.1. It suffices to consider how to approximate an arbitrary $U \in \mathrm{SU}(2)$ to precision ϵ by a sequence of gates from a given universal gate set Γ.

First we take products of elements of Γ to form a new universal gate set Γ_0 that is an ϵ_0^2-net for SU(2), for some sufficiently small constant ϵ_0. We know this can be done since Γ is universal. Since ϵ_0 is a constant, the overhead in constructing Γ_0 is constant.

Now we can find $V_0 \in \Gamma_0$ such that $\|U - V_0\| \leq \epsilon_0^2$. Since $\|U - V_0\| = \|UV_0^\dagger - I\|$, we have $UV_0^\dagger \in S_{\epsilon_0^2}$. If ϵ_0 is sufficiently small, then $\epsilon_0^2 < k\epsilon_0^{3/2} = \epsilon_1$, so $UV_0^\dagger \in S_{\epsilon_1}$.

Since Γ_0 is an ϵ_0^2-net for SU(2), in particular it is an ϵ_0^2-net for S_{ϵ_0}. Thus by the above argument, Γ_1 is an ϵ_1^2-net for S_{ϵ_1}, so we can find $V_1 \in \Gamma_1$ such that $\|UV_0^\dagger - V_1\| \leq \epsilon_1^2 < k\epsilon_1^{3/2} = \epsilon_2$, i.e., $UV_0^\dagger V_1^\dagger \in S_{\epsilon_2}$.

In general, suppose we are given $V_0, V_1, \ldots, V_{i-1}$ such that $UV_0^\dagger V_1^\dagger \ldots V_{i-1}^\dagger \in S_{\epsilon_i}$. Since Γ_i is an ϵ_i^2-net for S_{ϵ_i}, we can find $V_i \in \Gamma_i$ such that $\|UV_0^\dagger V_1^\dagger \ldots V_{i-1}^\dagger - V_i\| \leq \epsilon_i^2$. In turn, this implies that $UV_0^\dagger V_1^\dagger \ldots V_i^\dagger \in S_{\epsilon_{i+1}}$.

Repeating this process t times gives a very good approximation of U by $V_t \ldots V_1 V_0$: in particular, we have $\|U - V_t \ldots V_1 V_0\| \leq \epsilon_t^2$. Suppose we consider a gate from Γ_0 to be elementary. (These gates can be implemented using only a constant number of gates from Γ, so there is a constant factor overhead if we only count gates in Γ as elementary.) The number of elementary gates needed to implement a gate from Γ_i is 5^i,

so the total number of gates in the approximation is $\sum_{i=0}^{t} 5^i = (5^{t+1}-1)/4 = O(5^t)$. To achieve an overall error at most ϵ, we need $\epsilon_t^2 = ((k^2\epsilon_0)^{(3/2)^t}/k^2)^2 \le \epsilon$, i.e.,

$$\left(\frac{3}{2}\right)^t > \frac{\frac{1}{2}\log(k^4\epsilon)}{\log(k^2\epsilon_0)}. \tag{2.9}$$

Thus the number of gates used is $O(\log^\nu \frac{1}{\epsilon})$ where $\nu = \log 5/\log\frac{3}{2}$.

At this point, it may not be clear that the approximation can be found quickly, since Γ_i contains a large number of points, so we need to be careful about how we find a good approximation $V_i \in \Gamma_i$ of $UV_0^\dagger V_1^\dagger \dots V_{i-1}^\dagger$. However, by constructing the approximation recursively, it can be shown that the running time of this procedure is poly$(\log \frac{1}{\epsilon})$. It will be clearer how to do this after we prove the lemma, but we leave the details as an exercise. □

2.4 Proof of Lemma 2.3

It remains to prove the lemma. A key idea is to move between the Lie group SU(2) and its Lie algebra, i.e., the Hamiltonians generating these unitaries. In particular, we can represent any $A \in \mathrm{SU}(2)$ as $A = e^{i\vec{a}\cdot\vec{\sigma}}$, where $\vec{a} \in \mathbb{R}^3$ and $\vec{\sigma} = (\sigma_x, \sigma_y, \sigma_z)$ is a vector of Pauli matrices. Note that we can choose $\|\vec{a}\| \le \pi$ without loss of generality.

In the proof, the following basic facts about SU(2) will be useful.

(i) $\|I - e^{i\vec{a}\cdot\vec{\sigma}}\| = 2\sin\frac{\|\vec{a}\|}{2} = \|\vec{a}\| + O(\|\vec{a}\|^3)$

(ii) if $\|\vec{b}\|, \|\vec{c}\| \le \epsilon$ then $\|e^{i\vec{b}\cdot\vec{\sigma}} - e^{i\vec{c}\cdot\vec{\sigma}}\| = \|\vec{b}-\vec{c}\| + O(\epsilon^3)$

(iii) $[\vec{b}\cdot\vec{\sigma}, \vec{c}\cdot\vec{\sigma}] = 2i(\vec{b}\times\vec{c})\cdot\vec{\sigma}$

(iv) $\|[\![e^{i\vec{b}\cdot\vec{\sigma}}, e^{i\vec{c}\cdot\vec{\sigma}}]\!] - e^{-[\vec{b}\cdot\vec{\sigma},\vec{c}\cdot\vec{\sigma}]}\| = O(\|\vec{b}\|\|\vec{c}\|(\|\vec{b}\| + \|\vec{c}\|))$

Here the big-O notation is with respect to $\|\vec{a}\| \to 0$ in (i) and with respect to $\|\vec{b}\|, \|\vec{c}\| \to 0$ in (iv).

Proof of Lemma 2.3. Let $A \in S_{\epsilon^2}$. Our goal is to find $U, V \in \Gamma$ such that $\|A - [\![U, V]\!]\| = O(\epsilon^3)$.

Choose $\vec{a} \in \mathbb{R}^3$ such that $A = e^{i\vec{a}\cdot\vec{\sigma}}$. Since $A \in S_{\epsilon^2}$, by (i) we can choose $\vec{a}$ so that $\|\vec{a}\| = O(\epsilon^2)$.

Then choose $\vec{b}, \vec{c} \in \mathbb{R}^3$ such that $-2\vec{b}\times\vec{c} = \vec{a}$. We can choose these vectors to be orthogonal and of equal length, so that $\|\vec{b}\| = \|\vec{c}\| = \sqrt{\|\vec{a}\|/2} = O(\epsilon)$. Let $B = e^{i\vec{b}\cdot\vec{\sigma}}$ and $C = e^{i\vec{c}\cdot\vec{\sigma}}$. Then the only difference between A and $[\![B, C]\!]$ is the difference between the commutator and the group commutator, which is $O(\epsilon^3)$ by (iv).

However, we need to choose points from the net Γ. So let $U = e^{i\vec{u}\cdot\vec{\sigma}}$ be the closest element of Γ to B, and let $V = e^{i\vec{v}\cdot\vec{\sigma}}$ be the closest element of Γ to C. Since Γ is an ϵ^2-net for S_ϵ, we have $\|U - B\| \le \epsilon^2$ and $\|V - C\| \le \epsilon^2$, so in particular (by (ii)), $\|\vec{u} - \vec{b}\| = O(\epsilon^2)$ and $\|\vec{v} - \vec{c}\| = O(\epsilon^2)$.

Now by the triangle inequality, we have

$$\|A - [\![U, V]\!]\| \le \|A - e^{2i(\vec{u}\times\vec{v})\cdot\vec{\sigma}}\| + \|e^{2i(\vec{u}\times\vec{v})\cdot\vec{\sigma}} - [\![U, V]\!]\|. \tag{2.10}$$

For the first term, using (ii), we have

$$\begin{aligned}
\|A - e^{2i(\vec{u}\times\vec{v})\cdot\vec{\sigma}}\| &= \|e^{2i(\vec{b}\times\vec{c})\cdot\vec{\sigma}} - e^{2i(\vec{u}\times\vec{v})\cdot\vec{\sigma}}\| && (2.11)\\
&\le 2\|\vec{b}\times\vec{c} - \vec{u}\times\vec{v}\| + O(\epsilon^3) && (2.12)\\
&= 2\|(\vec{b}-\vec{u}+\vec{u})\times(\vec{c}-\vec{v}+\vec{v}) - \vec{u}\times\vec{v}\| + O(\epsilon^3) && (2.13)\\
&= 2\|(\vec{b}-\vec{u})\times(\vec{c}-\vec{v}) + (\vec{b}-\vec{u})\times\vec{v} + \vec{u}\times(\vec{c}-\vec{v})\| + O(\epsilon^3) && (2.14)\\
&= O(\epsilon^3). && (2.15)
\end{aligned}$$

For the second term, using (iii) and (iv) gives

$$\|e^{2i(\vec{u}\times\vec{v})\cdot\vec{\sigma}} - [\![U, V]\!]\| = \|e^{-[\vec{u}\cdot\vec{\sigma},\vec{v}\cdot\vec{\sigma}]} - [\![U, V]\!]\| = O(\epsilon^3). \tag{2.16}$$

The lemma follows. □

Note that it is possible to improve the construction somewhat over the version described above. Furthermore, it can be generalized to SU(N) for arbitrary N. In general, the cost is exponential in N^2, but for any fixed N this is just a constant.

The Solovay-Kitaev theorem has a technical condition that the gate set is closed under inverses. This is essential for the proof strategy described above, since the group commutator includes inverses of the available gates. However, it is also possible to prove a version of the theorem without this condition [26].

Chapter 3

Quantum circuit synthesis over Clifford+T

As we discussed in Chapter 2, the Solovay-Kitaev Theorem tells us that we can convert between gate sets with overhead that is only $\mathrm{poly}(\log(1/\epsilon))$. However, the overhead may not be that small in practice (we upper bounded the power of the log by $\log 5/\log \frac{3}{2} \approx 3.97$), and it is natural to ask if we can do better. A counting argument shows that the best possible exponent is 1 (see [72] and [88, Section 4.5.4]). Can we get close to this lower bound—ideally while retaining a fast algorithm?

In general, no such result is known (even if we do not require a fast algorithm). However, there are strong circuit synthesis results for particular gate sets with nice structure. In particular, one can perform fast, nearly-optimal synthesis for the set of single-qubit Clifford+T circuits. Not only does it admit fast synthesis, but this gate set is also the most common choice for fault-tolerant quantum computation, so it is likely to be relevant in practice.

To understand the synthesis of Clifford+T circuits, we focus here on the problem of *exactly* expressing a given unitary operation over that gate set, assuming such a representation is possible. This result can be extended to give an algorithm for approximately synthesizing arbitrary single-qubit gates, although the details are beyond the scope of this lecture. (Note that some of these ideas can also be applied to the synthesis of multi-qubit circuits, but that is also beyond our scope.)

3.1 Converting to Matsumoto-Amano normal form

An algorithm for exact synthesis of Clifford+T circuits was first presented in [71]. However, our presentation here is based on a simpler analysis [53] that uses a normal form for such circuits introduced by Matsumoto and Amano [84].

The single-qubit Clifford group $\mathcal{C} = \langle H, S, \omega \rangle$ is generated by the Hadamard gate H, the phase gate S, and the phase $\omega := e^{i\pi/4}$, where

$$H := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \qquad T := \begin{pmatrix} 1 & 0 \\ 0 & \omega \end{pmatrix} \qquad S := T^2 = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}. \tag{3.1}$$

By adding the T gate, we get a universal gate set—in other words, the set $\langle H, T, \omega \rangle$ is dense in U(2). We call any unitary operation that can be represented exactly over this gate set a Clifford+T operation.

Clearly, any single-qubit Clifford+T operation M can be written in the form

$$M = C_n T C_{n-1} T \cdots C_1 T C_0 \tag{3.2}$$

where $C_0, \ldots, C_n \in \mathcal{C}$. Our goal is to rewrite such an expression into a simpler form.

Let $\mathcal{S} := \langle S, X, \omega \rangle \leq \mathcal{C}$. Any element of $\mathcal{S}$ can be pushed through T (say, to the right), since we have

$$ST = TS \tag{3.3}$$
$$XT = \omega^{-1} TXS \tag{3.4}$$
$$\omega T = T\omega. \tag{3.5}$$

Thus we can assume $C_1, \ldots, C_{n-1} \notin \mathcal{S}$. (In some cases, pushing elements of $\mathcal{S}$ to the right might cause two T gates to merge into $S \in \mathcal{C}$; we take n to be the number of Clifford gates after any such cancellations.)

An explicit calculation shows that $|\mathcal{S}| = 64$, whereas $|\mathcal{C}| = 192$. Since I, H, and SH are in different left cosets of $\mathcal{S}$ in $\mathcal{C}$, they can be chosen as the three coset representatives, and we can write every element of the Clifford group in the form $\tilde{H}\tilde{S}$, where $\tilde{H} \in \{I, H, SH\}$ and $\tilde{S} \in \mathcal{S}$. Similarly, every element of $\mathcal{C} \setminus \mathcal{S}$ can be written in the same form, where $\tilde{H} \in \{H, SH\}$. Thus we can write M in the form

$$M = H_n S_n T H_{n-1} S_{n-1} T \cdots H_1 S_1 T C_0 \tag{3.6}$$

where $C_0 \in \mathcal{C}$, $H_1, \ldots, H_{n-1} \in \{H, SH\}$, $H_n \in \{I, H, SH\}$ and $S_1, \ldots, S_n \in \mathcal{S}$.

Now we can further simplify this expression by again pushing elements of $\mathcal{S}$ to the right. We have already seen that such operators can be pushed through T gates, giving new elements of $\mathcal{S}$. But furthermore, they can also be pushed through elements of $\{H, SH\}$, since

$$SH \in \{H, SH\} \qquad SSH = HX \tag{3.7}$$
$$XH = HZ = HS^2 \qquad XSH = SHY = SH(\omega^2 XS^2) \tag{3.8}$$
$$\omega H = H\omega \qquad \omega SH = SH\omega. \tag{3.9}$$

After applying these rules, we are left with an expression of the form

$$M = H_k T H_{k-1} T \cdots H_1 T C_0 \tag{3.10}$$

where $H_1, \ldots, H_{k-1} \in \{H, SH\}$ and $H_k \in \{I, H, SH\}$. (Note that we can have $k < n$, since again we could find cancellations as we push gates to the right.) This expression is now in *Matsumoto-Amano (MA) normal form*. In terms of regular expressions, we can write this form as $(\varepsilon \mid T)\,(HT \mid SHT)^* \mathcal{C}$, where ε denotes the empty string.

Since the above argument is constructive, it also gives an algorithm for converting circuits to MA normal form. A naive implementation would take time $O(n^2)$, since we might make a pass through $O(n)$ gates to find a simplification, and we might have to repeat this $O(n)$ times before reaching MA normal form. However, we can reduce to MA normal form in linear time by simplifying the given circuit gate-by-gate, maintaining MA normal form along the way. If N is in MA normal form and $C \in \mathcal{C}$, then NC can be reduced to MA normal form in constant time (we simply combine the rightmost two Clifford operators). On the other hand, case analysis shows that reducing NT to MA normal form only requires updating the rightmost 5 gates, so it can also be reduced in constant time. Overall, this approach takes $O(n)$ steps, each taking time $O(1)$, for a total running time of $O(n)$.

An important parameter of a Clifford+T circuit is its *T-count*, which is simply the number of T gates it contains. Clearly there is a way of writing any Clifford+T circuit in MA normal form such that the T-count is minimal, simply because the reduction procedure described above never increases the T-count.

3.2 Uniqueness of Matsumoto-Amano normal form

In fact, the MA normal form is unique, so the procedure described above always produces a circuit with minimal T-count. Furthermore, the proof of this helps to develop an algebraic characterization of Clifford+T unitaries that facilitates approximate synthesis.

Given a single-qubit unitary U, let $\hat{U}$ denote its Bloch sphere representation. If $U(xX + yY + zZ)U^\dagger = x'X + y'Y + z'Z$, then $\hat{U}\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$, and this relationship serves to define $\hat{U}$ by linearity. We have

$$\hat{H} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \qquad \hat{S} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \qquad \hat{T} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & \sqrt{2} \end{pmatrix}. \tag{3.11}$$

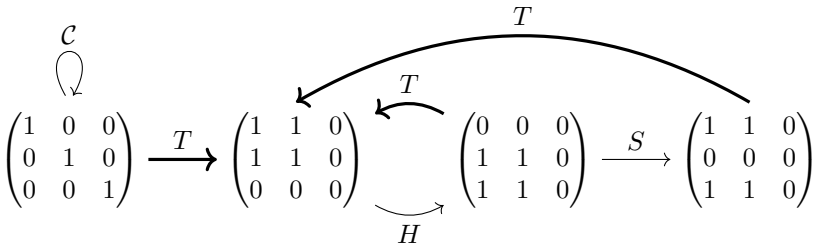
These generators belong to the ring $\mathbb{Z}[\frac{1}{\sqrt{2}}] = \{\frac{a+b\sqrt{2}}{\sqrt{2}^k} : a, b \in \mathbb{Z}, k \in \mathbb{N}\}$, so clearly the Bloch sphere representation of any Clifford+T operator has entries in this ring.

We say that $k \in \mathbb{N}$ is a *denominator exponent* of $x \in \mathbb{Z}[\frac{1}{\sqrt{2}}]$ if $\sqrt{2}^k x \in \mathbb{Z}[\sqrt{2}] = \{a + b\sqrt{2} : a, b \in \mathbb{Z}\}$. We call the smallest such k the *least denominator exponent* of x.

Define the *parity* of $x \in \mathbb{Z}[\sqrt{2}]$, denoted $p(x)$, such that $p(a+b\sqrt{2})$ is the parity of a (i.e., 0 if a is even and 1 if a is odd). If k is a denominator exponent for x, define the *k-parity* of $x \in \mathbb{Z}[\frac{1}{\sqrt{2}}]$ as $p_k(x) := p(\sqrt{2}^k x)$.

Observe that the Bloch sphere representation of a Clifford operator is a signed permutation matrix, so it has denominator exponent 0, and its parity (applied to the matrix elementwise) is a permutation.

We can define an equivalence relation on (k-)parity matrices of Bloch sphere representations of Clifford+T operators such that they are equivalent if they differ by right-multiplication by the parity matrix of a Clifford operator (in other words, by permutation of the columns). Now consider what happens to the k-parity matrix of the operator as we proceed through the MA normal form, where we increase k by one every time we multiply by a T gate. A simple calculation shows that transitions between the resulting equivalence classes are as follows:



(3.12)

Here the matrices are representatives of equivalence classes of k-parity matrices and the labels on the arrows show what gates induce the transitions. We have $k = 0$ at the leftmost (starting) matrix, and the value of k is increased by 1 along each thick arrow. For example, for the transitions under a T gate, we have

$$\begin{pmatrix} a_{11}+b_{11}\sqrt{2} & a_{12}+b_{12}\sqrt{2} & a_{13}+b_{13}\sqrt{2} \\ a_{21}+b_{21}\sqrt{2} & a_{22}+b_{22}\sqrt{2} & a_{23}+b_{23}\sqrt{2} \\ a_{31}+b_{31}\sqrt{2} & a_{32}+b_{32}\sqrt{2} & a_{33}+b_{33}\sqrt{2} \end{pmatrix} \mapsto \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & \sqrt{2} \end{pmatrix} \begin{pmatrix} a_{11}+b_{11}\sqrt{2} & a_{12}+b_{12}\sqrt{2} & a_{13}+b_{13}\sqrt{2} \\ a_{21}+b_{21}\sqrt{2} & a_{22}+b_{22}\sqrt{2} & a_{23}+b_{23}\sqrt{2} \\ a_{31}+b_{31}\sqrt{2} & a_{32}+b_{32}\sqrt{2} & a_{33}+b_{33}\sqrt{2} \end{pmatrix}$$
$$= \frac{1}{\sqrt{2}} \begin{pmatrix} (a_{11}-a_{21})+(b_{11}-b_{21})\sqrt{2} & (a_{12}-a_{22})+(b_{12}-b_{22})\sqrt{2} & (a_{13}-a_{23})+(b_{13}-b_{23})\sqrt{2} \\ (a_{11}+a_{21})+(b_{11}+b_{21})\sqrt{2} & (a_{12}+a_{22})+(b_{12}+b_{22})\sqrt{2} & (a_{13}+a_{23})+(b_{13}+b_{23})\sqrt{2} \\ 2b_{31}+a_{31}\sqrt{2} & 2b_{32}+a_{32}\sqrt{2} & 2b_{33}+a_{33}\sqrt{2} \end{pmatrix}. \tag{3.13}$$

At the leftmost matrix, we have a_{11}, a_{22}, a_{33} odd and a_{ij} even for $i \neq j$. Clearly the resulting 1-parity matrix is of the indicated form. Similar calculations verify the other transitions.

From this transition diagram, we can easily see that the MA normal form is unique. If we remain at the leftmost matrix, the operation is Clifford. On the other hand, if we end at one of the next three matrices to the right, the leftmost syllable of M is T, HT, or SHT, respectively. Given a matrix M, let k be its least denominator exponent. By computing $p_k(M)$ and determining which equivalence class it belongs to, we can determine the final syllable of its MA normal form. By induction, the entire MA normal form is determined. Note that this also shows that the least denominator exponent of M is its minimal T-count.

This argument also implies an algorithm for exact synthesis given the matrix of a Clifford+T operation (instead of some initial Clifford+T circuit). We simply convert to the Bloch matrix representation, compute the least denominator exponent k, use $p_k(M)$ to determine the leftmost syllable of the MA normal form, and recurse until we are left with a Clifford operation. In this algorithm, the number of arithmetic operations performed is $O(k)$.

3.3 Algebraic characterization of Clifford+T unitaries

We can use the ideas of the previous section to give an algebraic characterization of Clifford+T operations. Specifically, a matrix $\hat{U} \in \mathrm{SO}(3)$ is the Bloch sphere representation of a Clifford+T unitary if and only if its entries are in $\mathbb{Z}[\frac{1}{\sqrt{2}}]$. As noted above, the "only if" part is trivial; it remains to show that any such matrix corresponds to a Clifford+T operation.

The proof of this statement uses the orthogonality condition on $\hat{U}$ to characterize the possible values of $p_k(\hat{U})$ (specifically, to show it is one of the forms in the above transition diagram, up to permutation of the columns), and then shows that the least denominator can always be reduced by multiplying from the left by the inverse of a matrix from $\{T, HT, SHT\}$. The proofs of these statements are straightforward, but involve some explicit calculation and case analysis; see [53] for details.

As a simple corollary, we can establish that U is a Clifford+T unitary if and only if its entries are in $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ Again the "only if" direction is trivial. For the other direction, simply observe that if the entries of U are in $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$, then the entries of $\hat{U}$ are in $\mathbb{Z}[\frac{1}{\sqrt{2}}]$, and we can apply the characterization of Bloch matrices of Clifford+T unitaries. Note that this only determines the actual matrix up to a phase, but this phase must be a power of ω, so indeed the original U must be a Clifford+T unitary.

In summary, we have seen that any Clifford+T unitary can be synthesized into a Clifford+T circuit, with the minimal number of T gates (equal to the least denominator exponent of its Bloch sphere representation), in time linear in the T-count.

3.4 From exact to approximate synthesis

The methods described above can only be directly applied if the unitary operation in question can be performed exactly using Clifford+T gates. However, the methods can be adapted to perform approximate synthesis of arbitrary gates. The basic idea is to "round" a given unitary to a nearby element of $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ and then to synthesize that operation over Clifford+T. This is far from straightforward since a naive rounding procedure (say, rounding the matrix elementwise) will generally yield an operation that is not unitary, and thus not amenable to synthesis. However, by a careful rounding procedure, we can produce a nearby unitary matrix over $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$, and thus produce an ϵ-approximation of length $O(\log(1/\epsilon))$.

Part II

Quantum algorithms for algebraic problems

Chapter 4

The abelian quantum Fourier transform and phase estimation

4.1 Quantum Fourier transform

Perhaps the most important unitary transformation in quantum computing is the *quantum Fourier transform* (QFT). Later, we will discuss the QFT over arbitrary finite groups, but for now we will focus on the case of an abelian group G. Here the transformation is

$$F_G := \frac{1}{\sqrt{|G|}} \sum_{x \in G} \sum_{y \in \hat{G}} \chi_y(x) |y\rangle\langle x| \tag{4.1}$$

where $\hat{G}$ is a complete set of characters of G, and $\chi_y(x)$ denotes the yth character of G evaluated at x. (You can verify that this is a unitary operator using the orthogonality of characters.) Since G and $\hat{G}$ are isomorphic, we can label the elements of $\hat{G}$ using elements of G, and it is often useful to do so.

The simplest QFT over a family of groups is the QFT over $G = \mathbb{Z}_2^n$. The characters of this group are $\chi_y(x) = (-1)^{x \cdot y}$, so the QFT is simply

$$F_{\mathbb{Z}_2^n} = \frac{1}{\sqrt{2^n}} \sum_{x,y \in \mathbb{Z}_2^n} (-1)^{x \cdot y} |y\rangle\langle x| = H^{\otimes n}. \tag{4.2}$$

You have presumably seen how this transformation is used in the solution of Simon's problem [107].

4.2 QFT over $\mathbb{Z}_{2^n}$

A more complex quantum Fourier transform is the QFT over $G = \mathbb{Z}_{2^n}$:

$$F_{\mathbb{Z}_{2^n}} = \frac{1}{\sqrt{2^n}} \sum_{x,y \in \mathbb{Z}_{2^n}} \omega_{2^n}^{xy} |y\rangle\langle x| \tag{4.3}$$

where $\omega_m := \exp(2\pi \mathrm{i}/m)$ is a primitive mth root of unity. To see how to realize this transformation by a quantum circuit, it is helpful to represent the input x as a string of bits, $x = x_{n-1} \ldots x_1 x_0$, and to consider

how an input basis vector is transformed:

$$|x\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{y \in \mathbb{Z}_{2^n}} \omega_{2^n}^{xy} |y\rangle \tag{4.4}$$

$$= \frac{1}{\sqrt{2^n}} \sum_{y \in \mathbb{Z}_{2^n}} \omega_{2^n}^{x(\sum_{k=0}^{n-1} y_k 2^k)} |y_{n-1} \dots y_1 y_0\rangle \tag{4.5}$$

$$= \frac{1}{\sqrt{2^n}} \sum_{y \in \mathbb{Z}_{2^n}} \prod_{k=0}^{n-1} \omega_{2^n}^{x y_k 2^k} |y_{n-1} \dots y_1 y_0\rangle \tag{4.6}$$

$$= \frac{1}{\sqrt{2^n}} \bigotimes_{k=0}^{n-1} \sum_{y_k \in \mathbb{Z}_2} \omega_{2^n}^{x y_k 2^k} |y_k\rangle \tag{4.7}$$

$$= \bigotimes_{k=0}^{n-1} |z_k\rangle \tag{4.8}$$

where

$$|z_k\rangle := \frac{1}{\sqrt{2}} \sum_{y_k \in \mathbb{Z}_2} \omega_{2^n}^{x y_k 2^k} |y_k\rangle \tag{4.9}$$

$$= \frac{1}{\sqrt{2}} (|0\rangle + \omega_{2^n}^{x 2^k} |1\rangle) \tag{4.10}$$

$$= \frac{1}{\sqrt{2}} (|0\rangle + \omega_{2^n}^{\sum_{j=0}^{n-1} x_j 2^{j+k}} |1\rangle) \tag{4.11}$$

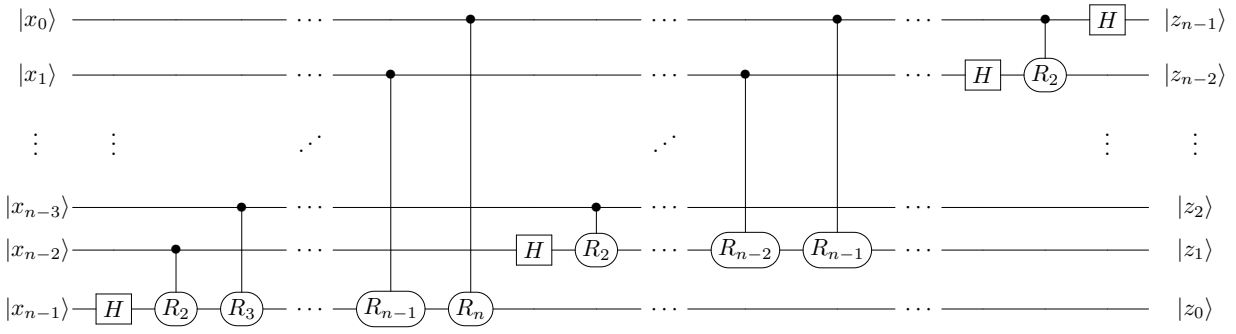
$$= \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi \mathrm{i} (x_0 2^{k-n} + x_1 2^{k+1-n} + \dots + x_{n-1-k} 2^{-1})} |1\rangle). \tag{4.12}$$

(A more succinct way to write this is $|z_k\rangle = \frac{1}{\sqrt{2}}(|0\rangle + \omega_{2^{n-k}}^{x} |1\rangle)$, but the above expression is more helpful for understanding the circuit.) In other words, $F|x\rangle$ is a tensor product of single-qubit states, where the kth qubit only depends on the $n-k$ least significant bits of x.

This decomposition immediately gives a circuit for the QFT over $\mathbb{Z}_{2^n}$. Let R_k denote the single-qubit unitary operator

$$R_k := \begin{pmatrix} 1 & 0 \\ 0 & \omega_{2^k} \end{pmatrix}. \tag{4.13}$$

Then the circuit can be written as follows:



This circuit uses $O(n^2)$ gates. However, there are many rotations by small angles that do not affect the final result very much. If we simply omit the gates R_k with $k = \Omega(\log n)$, then we obtain a circuit with $O(n \log n)$ gates that implements the QFT with precision $1/\operatorname{poly}(n)$.

4.3 Phase estimation

Aside from being directly useful in quantum algorithms, such as Shor's algorithm, The QFT over $\mathbb{Z}_{2^n}$ provides a useful quantum computing primitive called *phase estimation* [40, 70]. In the phase estimation problem, we are given a unitary operator U (either as an explicit circuit, or as a black box that lets us apply a controlled-U^j operation for integer values of j). We are also given a state $|\phi\rangle$ that is promised to be an eigenvector of U, namely $U|\phi\rangle = e^{\mathrm{i}\phi}|\phi\rangle$ for some $\phi \in \mathbb{R}$. The goal is to output an estimate of ϕ to some desired precision.

The procedure for phase estimation is straightforward. To get an n-bit estimate of ϕ, prepare the quantum computer in the state

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_{2^n}} |x, \phi\rangle, \tag{4.14}$$

apply the operator

$$\sum_{x \in \mathbb{Z}_{2^n}} |x\rangle\langle x| \otimes U^x \tag{4.15}$$

to give the state

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_{2^n}} e^{\mathrm{i}\phi x}|x, \phi\rangle, \tag{4.16}$$

apply an inverse Fourier transform on the first register, and measure. If the binary expansion of $\phi/2\pi$ terminates after at most n bits (i.e., if $\phi = 2\pi y/2^n$ for some $y \in \mathbb{Z}_{2^n}$), then the state (4.16) is $F_{2^n}|y\rangle \otimes |\phi\rangle$, so the result is guaranteed to be the binary expansion of $\phi/2\pi$. In general, we obtain a good approximation with high probability. In particular, the probability of obtaining the result y (corresponding to the estimate $2\pi y/2^n$ for the phase) is

$$\Pr(y) = \frac{1}{2^{2n}} \cdot \frac{\sin^2(2^{n-1}\phi)}{\sin^2(\frac{\phi}{2} - \frac{\pi y}{2^n})}, \tag{4.17}$$

which is strongly peaked around the best n-bit approximation (in particular, it gives the best n-bit approximation with probability at least $4/\pi^2$). We will see the details of a similar calculation when we discuss period finding.

4.4 QFT over $\mathbb{Z}_N$ and over a general finite abelian group

One useful application of phase estimation is to implement the QFT over an arbitrary cyclic group $\mathbb{Z}_N$:

$$F_{\mathbb{Z}_N} = \frac{1}{\sqrt{N}} \sum_{x,y \in \mathbb{Z}_N} \omega_N^{xy} |y\rangle\langle x|. \tag{4.18}$$

The circuit we derived using the binary representation of the input and output only works when N is a power of two (or, with a slight generalization, some other small integer). But there is a simple way to realize $F_{\mathbb{Z}_N}$ (approximately) using phase estimation.

We would like to perform the transformation that maps $|x\rangle \mapsto |\tilde{x}\rangle$, where $|\tilde{x}\rangle := F_{\mathbb{Z}_N}|x\rangle$ denotes a Fourier basis state. (By linearity, if the transformation acts correctly on a basis, it acts correctly on all states.) It is straightforward to perform the transformation $|x, 0\rangle \mapsto |x, \tilde{x}\rangle$; then it remains to erase the register $|x\rangle$ from such a state.

Consider the unitary operator that adds 1 modulo N:

$$U := \sum_{x \in \mathbb{Z}_N} |x+1\rangle\langle x|. \tag{4.19}$$

The eigenstates of this operator are precisely the Fourier basis states $|\tilde{x}\rangle := F_{\mathbb{Z}_N}|x\rangle$, since (as a simple calculation shows)

$$F_{\mathbb{Z}_N}^\dagger U F_{\mathbb{Z}_N} = \sum_{x \in \mathbb{Z}_N} \omega_N^{-x} |x\rangle\langle x|. \tag{4.20}$$

Thus, using phase estimation on U (with n bits of precision where $n = O(\log N)$), we can perform the transformation

$$|\tilde{x}, 0\rangle \mapsto |\tilde{x}, x\rangle \tag{4.21}$$

(actually, phase estimation only gives an approximation of x, so we implement this transformation only approximately). By running this operation in reverse, we can erase $|x\rangle$, and thereby produce the desired QFT.

Given the Fourier transform over $\mathbb{Z}_N$, it is straightforward to implement the QFT over an arbitrary finite abelian group: any finite abelian group can be written as a direct product of cyclic factors, and the QFT over a direct product of groups is simply the tensor product of QFTs over the individual groups.

Chapter 5

Discrete log and the hidden subgroup problem

In this lecture we will discuss the discrete logarithm problem and its relevance to cryptography. We will introduce the general hidden subgroup problem, and show how Shor's algorithm solves a particular instance of it, giving an efficient quantum algorithm for discrete log.

5.1 Discrete log

Let $G = \langle g \rangle$ be a cyclic group generated by g, written multiplicatively. Given an element $x \in G$, the *discrete logarithm of x in G with respect to g*, denoted $\log_g x$, is the smallest non-negative integer α such that $g^\alpha = x$. The *discrete logarithm problem* is the problem of calculating $\log_g x$.

Here are some simple examples of discrete logarithms:

- For any $G = \langle g \rangle$, $\log_g 1 = 0$
- For $G = \mathbb{Z}_7^\times$, $\log_3 2 = 2$
- For $G = \mathbb{Z}_{541}^\times$, $\log_{126} 282 = 101$

The discrete logarithm seems like a good candidate for a one-way function. We can efficiently compute g^α, even if α is exponentially large (in $\log |G|$), using repeated squaring. But given x, it is not immediately clear how to compute $\log_g x$, other than by checking exponentially many possibilities. (There are better algorithms than brute force search, but none is known that works in polynomial time.)

5.2 Diffie-Hellman key exchange

The apparent hardness of the discrete logarithm problem is the basis of the *Diffie-Hellman key exchange protocol*, the first published public-key cryptographic protocol.

The goal of key exchange is for two distant parties, Alice and Bob, to agree on a secret key using only an insecure public channel. The Diffie-Hellman protocol works as follows:

1. Alice and Bob publicly agree on a large prime p and an integer g of high order. For simplicity, suppose they choose a g for which $\langle g \rangle = \mathbb{Z}_p^\times$ (i.e., a primitive root modulo p). (In general, finding such a g might be hard, but it can be done efficiently given certain restrictions on p.)

2a. Alice chooses some $a \in \mathbb{Z}_{p-1}$ uniformly at random. She computes $A := g^a \bmod p$ and sends the result to Bob (keeping a secret).

2b. Bob chooses some $b \in \mathbb{Z}_{p-1}$ uniformly at random. He computes $B := g^b \bmod p$ and sends the result to Alice (keeping b secret).

3a. Alice computes $K := B^a \bmod p = g^{ab} \bmod p$.

3b. Bob computes $A^b \bmod p = g^{ab} \bmod p = K$.

At the end of the protocol, Alice and Bob share a key K, and Eve has only seen p, g, A, and B.

The security of the Diffie-Hellman protocol relies on the assumption that discrete log is hard. Clearly, if Eve can compute discrete logarithms, she can recover a and b, and hence the key. (Note that it is an open question whether, given the ability to break the protocol, Eve can calculate discrete logarithms, though some partial results in this direction are known.)

This protocol only provides a means of exchanging a secret key, not of sending private messages. However, very similar ideas can be used to create a public-key cryptosystem (similar in spirit to RSA).

5.3 The hidden subgroup problem

It turns out that discrete logarithms can be calculated efficiently on a quantum computer, rendering cryptographic protocols such as Diffie-Hellman key exchange insecure. The quantum algorithm for discrete log solves a particular instance of the *hidden subgroup problem* (HSP).

In the general HSP, we are given a black box function $f : G \to S$, where G is a known group and S is a finite set. The function is promised to satisfy

$$\begin{aligned} f(x) = f(y) \text{ if and only if } & x^{-1}y \in H \\ & \text{i.e., } y = xh \text{ for some } h \in H \end{aligned} \tag{5.1}$$

for some unknown subgroup $H \leq G$. We say that such a function *hides* H. The goal of the HSP is to learn H (say, specified in terms of a generating set) using queries to f.

It's clear that H can in principle be reconstructed if we are given the entire truth table of f. Notice in particular that $f(1) = f(x)$ if and only if $x \in H$: the hiding function is constant on the hidden subgroup, and does not take that value anywhere else.

But the hiding function has a lot more structure as well. If we fix some element $g \in G$ with $g \notin H$, we see that $f(g) = f(x)$ if and only if $x \in gH$, a left coset of H in G with coset representative g. So f is constant on the left cosets of H in G, and distinct on different left cosets.

In the above definition of the HSP, we have made an arbitrary choice to multiply by elements of H on the right, which is why the hiding function is constant on left cosets. We could just as well have chosen to multiply by elements of H on the left, in which case the hiding function would be constant on right cosets; the resulting problem would be equivalent. Of course, in the case where G is abelian, we don't need to make such a choice. For reasons that we will see later, this case turns out to be considerably simpler than the general case; and indeed, there is an efficient quantum algorithm for the HSP in any abelian group, whereas there are only a few nonabelian groups for which efficient algorithms are known.

You should be familiar with Simon's problem, the HSP with $G = \mathbb{Z}_2^n$ and $H = \{0, s\}$ for some $s \in \mathbb{Z}_2^n$. There is a very simple quantum algorithm for this problem, yet one can prove that any classical algorithm for finding s must query the hiding function exponentially many times (in n) [106]. The gist of the argument is that, since the set S is unstructured, we can do no better than querying random group elements so long as we do not know two elements x, y for which $f(x) = f(y)$. But by the birthday problem, we are unlikely to see such a collision until we make $\Omega(\sqrt{|G|/|H|})$ random queries.

A similar argument applies to any HSP with a large number of trivially intersecting subgroups. More precisely, we have

Theorem 5.1. *Suppose that G has a set $\mathcal{H}$ of N subgroups whose only common element is the identity. Then to solve the HSP, a deterministic classical computer must make $\Omega(\sqrt{N})$ queries.*

Proof. Suppose the oracle does not a priori hide a particular subgroup, but instead behaves adversarially, as follows. On the ℓth query, the algorithm queries g_ℓ, which we assume to be different from $g_1, \ldots, g_{\ell-1}$ without loss of generality. If there is any subgroup $H \in \mathcal{H}$ for which $g_k \notin g_j H$ for all $1 \leq j < k \leq \ell$ (i.e., there is some consistent way the oracle could assign g_ℓ to an as-yet-unqueried coset of a hidden subgroup from $\mathcal{H}$), then the oracle simply outputs ℓ; otherwise the oracle concedes defeat and outputs a generating set for some $H \in \mathcal{H}$ consistent with its answers so far (which must exist, by construction).

The goal of the algorithm is to force the oracle to concede, and we want to lower bound the number of queries required. (Given an algorithm for the HSP in G, there is clearly an algorithm that forces this oracle to concede using only one more query.) Now consider an algorithm that queries the oracle t times before

forcing the oracle to concede. This algorithm simply sees a fixed sequence of responses $1, 2, \ldots, t$, so for the first t queries, the algorithm cannot be adaptive. But observe that, regardless of which t group elements are queried, there are at most $t(t-1)$ non-identity values of $g_k g_j^{-1}$, whereas there are N possible subgroups in $\mathcal{H}$. Thus, to satisfy the N conditions that for all $H \in \mathcal{H}$, there is some pair j, k such that $g_k g_j^{-1} \in H$, we must have $t(t-1) \geq N$, i.e., $t = \Omega(\sqrt{N})$.

Note that if the adversarial oracle cannot be beaten after t queries, there are two or more standard oracles that cannot be distinguished by the algorithm after t queries, so the lower bound applies in the setting where the algorithm encounters a fixed oracle. □

This argument can be adapted to show hardness for randomized algorithms by considering an adversary that chooses its responses to queries uniformly at random from a sufficiently large set.

Note that there are cases where a classical algorithm *can* find the hidden subgroup with a polynomial number of queries. In particular, since a classical computer can easily test whether a certain subgroup is indeed the hidden one, the HSP is easy for a group with only a polynomial number of subgroups. Thus, for example, a classical computer can easily solve the HSP in $\mathbb{Z}_p$ for p prime (since it has only 2 subgroups) and in $\mathbb{Z}_{2^n}$ (since it has only $n+1$ subgroups).

5.4 Shor's algorithm

Now we will see how Shor's algorithm can be used to calculate discrete logarithms [105]. This is a nice example because it's simpler than the factoring algorithm, but the problem it solves is actually at least as hard: factoring N can be reduced to calculating discrete log in $\mathbb{Z}_N^\times$. (Unfortunately, this does not by itself give a quantum algorithm for factoring, because Shor's algorithm for discrete log in G requires us to know the order of G—but computing $|\mathbb{Z}_N^\times| = \phi(N)$ is as hard as factoring N.)

Recall that we are given some element x of a cyclic group $G = \langle g \rangle$, we would like to calculate $\log_g x$, the smallest integer α such that $g^\alpha = x$. For simplicity, let us assume that the order of G, $N := |G|$, is known. (For example, if $G = \mathbb{Z}_p^\times$, then we know $N = p - 1$. In general, if we do not know N, we can learn it using Shor's period-finding algorithm, but we will not discuss that here.) We can also assume that $x \neq g$ (i.e., $\log_g x \neq 1$), since it is easy to check whether this is the case.

The discrete log problem can be cast as an abelian HSP in the (additive) group $\mathbb{Z}_N \times \mathbb{Z}_N$. Define a function $f : \mathbb{Z}_N \times \mathbb{Z}_N \to G$ as follows:

$$f(\alpha, \beta) = x^\alpha g^\beta. \tag{5.2}$$

Since $f(\alpha, \beta) = g^{\alpha \log_g x + \beta}$, f is constant on the lines

$$L_\lambda := \{(\alpha, \beta) \in \mathbb{Z}_N^2 : \alpha \log_g x + \beta = \lambda\}. \tag{5.3}$$

In other words, it hides the subgroup

$$H = L_0 = \{(0,0), (1, -\log_g x), (2, -2\log_g x), \ldots, (N-1, -(N-1)\log_g x)\}. \tag{5.4}$$

The cosets of H in $\mathbb{Z}_N \times \mathbb{Z}_N$ are of the form $(\gamma, \delta) + H$ with $\gamma, \delta \in \mathbb{Z}_N$. In particular, the set of cosets of the form

$$(0, \delta) + H = \{(\alpha, \delta - \alpha \log_g x) : \alpha \in \mathbb{Z}_N\} = L_\delta \tag{5.5}$$

varying over all $\delta \in \mathbb{Z}_N$ gives a complete set of cosets (so the set $\{0\} \times \mathbb{Z}_N$ is a complete set of coset representatives, i.e., a *transversal of H in $\mathbb{Z}_N \times \mathbb{Z}_N$*).

Shor's algorithm for finding H proceeds as follows. We start from the uniform superposition over $\mathbb{Z}_N \times \mathbb{Z}_N$ and compute the hiding function:

$$|\mathbb{Z}_N \times \mathbb{Z}_N\rangle := \frac{1}{N} \sum_{\alpha,\beta \in \mathbb{Z}_N} |\alpha, \beta\rangle \mapsto \frac{1}{N} \sum_{\alpha,\beta \in \mathbb{Z}_N} |\alpha, \beta, f(\alpha, \beta)\rangle. \tag{5.6}$$

Next we discard the third register. To see what this does, it may be conceptually helpful to imagine that we actually measure the third register. Then the post-measurement state is a superposition over group

elements consistent with the observed function value, which by definition is some coset of H. In particular, if the measurement outcome is g^δ, we are left with the *coset state* corresponding to $(0, \delta) + H$, namely

$$|(0,\delta)+H\rangle = |L_\delta\rangle = \frac{1}{\sqrt{N}} \sum_{\alpha \in \mathbb{Z}_N} |\alpha, \delta - \alpha \log_g x\rangle \tag{5.7}$$

However, note that the measurement outcome is unhelpful: each possible value occurs with equal probability, and we cannot obtain δ from g^δ unless we know how to take discrete logarithms. This is why we may as well simply discard the third register, leaving the system in the mixed state described by the ensemble of pure states (5.7) where δ is uniformly random and unknown.

Now we can exploit the symmetry of the quantum state by performing a QFT over $\mathbb{Z}_N \times \mathbb{Z}_N$; then the state becomes

$$\frac{1}{N^{3/2}} \sum_{\alpha,\mu,\nu \in \mathbb{Z}_N} \omega_N^{\mu\alpha + \nu(\delta - \alpha \log_g x)} |\mu, \nu\rangle = \frac{1}{N^{3/2}} \sum_{\mu,\nu \in \mathbb{Z}_N} \omega_N^{\nu\delta} \sum_{\alpha \in \mathbb{Z}_N} \omega_N^{\alpha(\mu - \nu \log_g x)} |\mu, \nu\rangle, \tag{5.8}$$

and using the identity $\sum_{\alpha \in \mathbb{Z}_N} \omega_N^{\alpha\beta} = N\delta_{\beta,0}$, we have

$$\frac{1}{\sqrt{N}} \sum_{\nu \in \mathbb{Z}_N} \omega_N^{\nu\delta} |\nu \log_g x, \nu\rangle. \tag{5.9}$$

Now suppose we measure this state in the computational basis. Then we obtain some pair $(\nu \log_g x, \nu)$ for uniformly random $\nu \in \mathbb{Z}_N$. If ν has a multiplicative inverse modulo N, we can divide the first register by ν to get the desired answer. If ν does not have a multiplicative inverse, we simply repeat the entire procedure again. The probability of success for each independent attempt is $\phi(N)/N = \Omega(1/\log\log N)$, so we don't have to repeat the procedure many times before we find an invertible ν.

This algorithm can be carried out for any cyclic group G so long as we have a unique representation of the group elements, and we are able to efficiently compute products in G. (We need to be able to compute high powers of a group element, but recall that this can be done quickly by repeated squaring.)

Chapter 6

The abelian HSP and decomposing abelian groups

Here we describe an algorithm to solve the HSP in any finite abelian group of known structure. We also explain how related ideas can be used to determine the structure of a black-box abelian group.

6.1 The abelian HSP

We now consider the HSP for a general abelian group. When the group elements commute, it often makes more sense to use additive notation for the group operation. We use this convention here, writing the condition that f hides H as $f(x) = f(y)$ iff $x - y \in H$.

The strategy for the general abelian HSP closely follows the algorithm for the discrete log problem. We begin by creating a uniform superposition over the group,

$$|G\rangle := \frac{1}{\sqrt{|G|}} \sum_{x \in G} |x\rangle. \tag{6.1}$$

Then we compute the function value in another register, giving

$$\frac{1}{\sqrt{|G|}} \sum_{x \in G} |x, f(x)\rangle. \tag{6.2}$$

Discarding the second register then gives a uniform superposition over the elements of some randomly chosen coset $x + H := \{x + h \colon h \in H\}$ of H in G,

$$|x + H\rangle = \frac{1}{\sqrt{|H|}} \sum_{h \in H} |x + h\rangle. \tag{6.3}$$

Such a state is commonly called a *coset state*. Equivalently, since the coset is unknown and uniformly random, the state can be described by the density matrix

$$\rho_H := \frac{1}{|G|} \sum_{x \in G} |x + H\rangle\langle x + H|. \tag{6.4}$$

Next we apply the QFT over G. Then we obtain the state

$$|\widehat{x + H}\rangle := F_G |x + H\rangle \tag{6.5}$$

$$= \frac{1}{\sqrt{|H| \cdot |G|}} \sum_{y \in \hat{G}} \sum_{h \in H} \chi_y(x + h)|y\rangle \tag{6.6}$$

$$= \sqrt{\frac{|H|}{|G|}} \sum_{y \in \hat{G}} \chi_y(x)\chi_y(H)|y\rangle \tag{6.7}$$

where

$$\chi_y(H) := \frac{1}{|H|} \sum_{h \in H} \chi_y(h). \tag{6.8}$$

Note that applying the QFT was the right thing to do because the state ρ_H is G-invariant. In other words, it commutes with the regular representation of G, the unitary matrices $U(x)$ satisfying $U(x)|y\rangle = |x+y\rangle$ for all $x, y \in G$: we have

$$U(x)\rho_H = \frac{1}{|G|} \sum_{y \in G} |x+y+H\rangle\langle y+H| \tag{6.9}$$

$$= \frac{1}{|G|} \sum_{z \in G} |z+H\rangle\langle z-x+H| \tag{6.10}$$

$$= \rho_H U(-x)^\dagger \tag{6.11}$$

$$= \rho_H U(x). \tag{6.12}$$

It follows that $\hat{\rho}_H := F_G \rho_H F_G^\dagger$ is diagonal (indeed, we verify this explicitly below), so we can measure without losing any information. We will talk about this phenomenon more when we discuss nonabelian Fourier sampling.

Note that χ_y is a character of H if we restrict our attention to that subgroup. If $\chi_y(h) = 1$ for all $h \in H$, then clearly $\chi_y(H) = 1$. On the other hand, if there is any $h' \in H$ with $\chi_y(h') \neq 1$ (i.e., if the restriction of χ_y to H is not the trivial character of H), then since $h' + H = H$, we have

$$\chi_y(H) = \frac{1}{|H|} \sum_{h \in h'+H} \chi_y(h) \tag{6.13}$$

$$= \frac{1}{|H|} \sum_{h \in H} \chi_y(h'+h) \tag{6.14}$$

$$= \chi_y(h')\chi_y(H), \tag{6.15}$$

which implies that $\chi_y(H) = 0$. (This also follows from the orthogonality of characters of H,

$$\frac{1}{|H|} \sum_{x \in H} \chi_y(x)\chi_{y'}(x)^* = \delta_{y,y'}, \tag{6.16}$$

taking y' to be the trivial character.) Thus we have

$$\widehat{|x+H\rangle} = \sqrt{\frac{|H|}{|G|}} \sum_{y\colon \chi_y(H)=1} \chi_y(x)|y\rangle \tag{6.17}$$

or, equivalently, the mixed quantum state

$$\hat{\rho}_H = \frac{|H|}{|G|^2} \sum_{x \in G} \sum_{y,y'\colon \chi_y(H)=\chi_{y'}(H)=1} \chi_y(x)\chi_{y'}(x)^* |y\rangle\langle y'| = \frac{|H|}{|G|} \sum_{y\colon \chi_y(H)=1} |y\rangle\langle y|. \tag{6.18}$$

Next we measure in the computational basis. Then we obtain some character χ_y that is trivial on the hidden subgroup H. This information narrows down the possible elements of the hidden subgroup: we can restrict our attention to those elements $g \in G$ satisfying $\chi_y(g) = 1$. The set of such elements is called the *kernel* of χ_y,

$$\ker \chi_y := \{g \in G\colon \chi_y(g) = 1\}; \tag{6.19}$$

it is a subgroup of G. Now our strategy is to repeat the entire sampling procedure many times and compute the intersection of the kernels of the resulting characters. After only polynomially many steps, we claim that the resulting subgroup is H with high probability. It clearly cannot be smaller than H (since the kernel of every sampled character contains H), so it suffices to show that each sample is likely to reduce the size of H by a substantial fraction until H is reached.

Suppose that at some point in this process, the intersection of the kernels is $K \leq G$ with $K \neq H$. Since K is a subgroup of G with $H < K$, we have $|K| \geq 2|H|$ (by Lagrange's theorem). Because each character χ_y of G satisfying $\chi_y(H) = 1$ has probability $|H|/|G|$ of appearing, the probability that we see some χ_y for which $K \leq \ker \chi_y$ is

$$\frac{|H|}{|G|}|\{y \in \hat{G} \colon K \leq \ker \chi_y\}|. \tag{6.20}$$

But the number of such ys is precisely $|G|/|K|$, since we know that if the subgroup K were hidden, we would sample such ys uniformly, with probability $|K|/|G|$. Therefore the probability that we see a y for which $K \leq \ker \chi_y$ is precisely $|H|/|K| \leq 1/2$. Now if we observe a y such that $K \not\leq \ker \chi_y$, then $|K \cap \ker \chi_y| \leq |K|/2$; furthermore, this happens with probability at least $1/2$. Thus, if we repeat the process $O(\log |G|)$ times, it is extremely likely that the resulting subgroup is in fact H.

6.2 Decomposing abelian groups

To apply the above algorithm, we must understand the structure of the group G; in particular, we must be able to apply the Fourier transform F_G. For some applications, we might not know the structure of G a priori. But if we assume only that we have a unique encoding of each element of G, the ability to perform group operations on these elements, and a generating set for G, then there is an efficient quantum algorithm [31] that decomposes the group as

$$G = \langle \gamma_1 \rangle \oplus \langle \gamma_2 \rangle \oplus \cdots \oplus \langle \gamma_t \rangle \tag{6.21}$$

in terms of generators $\gamma_1, \gamma_2, \ldots, \gamma_t$. Here $\oplus$ denotes an internal direct sum, meaning that the groups $\langle \gamma_i \rangle$ intersect only in the identity element; in other words, we have

$$G \cong \mathbb{Z}_{|\langle \gamma_1 \rangle|} \times \mathbb{Z}_{|\langle \gamma_2 \rangle|} \times \cdots \times \mathbb{Z}_{|\langle \gamma_t \rangle|}. \tag{6.22}$$

Given such a decomposition, it is straightforward to implement F_G and thereby solve HSPs in G. We might also use this tool to decompose the structure of the hidden subgroup H output by the HSP algorithm, e.g., to compute $|H|$.

First, it is helpful to simplify the problem by reducing to the case of a p-group for some prime p. For each given generator g of G, we compute its order, the smallest non-negative integer r such that $rg = 0$ (where we are using additive notation; in multiplicative notation we would write $g^r = 1$). Recall that there is an efficient quantum algorithm for order finding. Furthermore, there is an efficient quantum algorithm for factoring, so suppose we can write $r = st$ for some relatively prime integers s, t. By Euclid's algorithm, we can find a, b such that $as + bt = 1$, so $asg + btg = g$. Therefore, we can replace the generator g by the two generators sg and tg and still have a generating set. By repeating this procedure, we eventually obtain a generating set in which all the generators have prime power order.

For a given prime p, let G_p be the group generated by all the generators of G whose order is a power of p. Then $G = \bigoplus_p G_p$: every element of G can be written as a sum of elements from the G_ps (since together they include a generating set), and since G_p is a p-group (i.e., the orders of all its elements are powers of p), $G_p \cap G_{p'} = \{0\}$. Thus, it suffices to focus on the generators of G_p and determine the structure of this p-group. So from now on we assume that the order of G is a power of p.

Now, given a generating set $\{g_1, \ldots, g_d\}$ for G, let q (which is some power of p) be the largest order of any of the generators. We consider a hidden subgroup problem in the group $\mathbb{Z}_q^d$ whose solution allows us to determine the structure of G. Define $f \colon \mathbb{Z}_q^d \to G$ by

$$f(x_1, \ldots, x_d) = x_1 g_1 + \cdots + x_d g_d.$$

Now $f(x_1, \ldots, x_d) = f(y_1, \ldots, y_d)$ if and only if $(x_1 - y_1)g_1 + \cdots + (x_d - y_d)g_d = 0$, i.e., if and only if $f(x - y) = 0$. The elements of G for which f takes the value 0,

$$K := \{x \in \mathbb{Z}_q^d \colon f(x) = 0\},$$

form a subgroup of G called the *kernel* of f. Using the algorithm for the hidden subgroup problem in $\mathbb{Z}_q^d$, we can find generators for K. Suppose this generating set is $W = \{w_1, \ldots, w_m\}$, where $w_i \in \mathbb{Z}_q^d$.

The function f is clearly a homomorphism from $\mathbb{Z}_q^d$ to G, and it is also surjective (i.e., onto, meaning that the image of f is all of G), which implies that $\mathbb{Z}_q^d/K \cong G$ (this is called the *first isomorphism theorem*). Thus, to determine the structure of G, it suffices to determine the structure of the quotient $\mathbb{Z}_q^d/K$. In particular, if $\mathbb{Z}_q^d/K = \langle u_1 + K\rangle \oplus \cdots \oplus \langle u_t + K\rangle$, then $G = \langle f(u_1)\rangle \oplus \cdots \oplus \langle f(u_t)\rangle$. The final ingredient is a polynomial-time classical algorithm that produces such a direct sum decomposition of a quotient group.

To find such a decomposition, it is helpful to view the problem in terms of linear algebra. With $x \in \mathbb{Z}_q^d$, we have $x + K = K$ (so that $f(x) = 0$, and there is no need to include x as a generator) if and only if $x \in \mathrm{span}_{\mathbb{Z}_q} W$ (recall that W is a generating set for K). We can easily modify this to allow arbitrary integer vectors $x \in \mathbb{Z}^d$: then $x + K = K$ if and only if $x \in \mathrm{span}_{\mathbb{Z}}(W \cup \{qe_1, \ldots, qe_d\})$, where e_i is the ith standard basis vector. In other words, as x varies over the integer span of the vectors $w_1, \ldots, w_m, qe_1, \ldots, qe_d$, we obtain redundant vectors.

Now we use a tool from integer linear algebra called the *Smith normal form*. A square integer matrix is called *unimodular* if it has determinant ± 1. Given an integer matrix M, its Smith normal form is a decomposition $M = UDV^{-1}$, where U, V are unimodular and $D = \mathrm{diag}(1, \ldots, 1, d_1, \ldots, d_t, 0, \ldots 0)$ is an integer diagonal matrix with its positive diagonal entries satisfying $d_1 \mid d_2 \mid \ldots \mid d_t$. The Smith normal form can be computed classically in polynomial time.

In the present context, let M be the matrix with columns $w_1, \ldots, w_m, qe_1, \ldots, qe_d$. Let $M = UDV^{-1}$ be its Smith normal form, and let $u_1, \ldots, u_t$ be the columns of U corresponding to diagonal entries of D that are not 0 or 1 (i.e., if the ith diagonal entry of D is not 0 or 1, the ith column of U is included). We claim that $\mathbb{Z}_q^d/K = \langle u_1 + K\rangle \oplus \cdots \oplus \langle u_t + K\rangle$.

Since U is nonsingular, it is clear that we still have a generating set if we take all the columns of U. We're claiming that the columns corresponding to 0 or 1 diagonal entries of D are redundant. Let u be the jth column of U; we know that $u + K = K$ (i.e., u is redundant) if $u \in \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(M)$ (where $\mathrm{cols}(M)$ denotes the set of columns of M). Since V is unimodular, $\mathrm{span}_{\mathbb{Z}} \mathrm{cols}(M) = \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(MV)$. So $u + K = K$ if $u \in \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(MV)$, i.e., if $e_j \in \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(U^{-1}MV) = \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(D)$. If the jth diagonal entry of D is 0 or 1, then clearly this is true, so $u + K = K$. This shows that the cosets $u_1 + K, \ldots, u_t + K$ alone indeed generate $\mathbb{Z}_q^d/K$.

It remains to show that they generate $\mathbb{Z}_q^d/K$ as a direct sum. The above argument shows that $d_i u_i + K = K$, and this is not true for any smaller value than d_i, so the order of $u_i + K$ is d_i. Now suppose $\sum_i x_i u_i + K = K$. Then $\sum_i x_i u_i \in \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(M) = \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(MV)$, or in other words, $x \in \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(U^{-1}MV) = \mathrm{span}_{\mathbb{Z}} \mathrm{cols}(D)$. But this implies that x_i is an integer multiple of d_i, which shows that $\langle u_1 + K\rangle \oplus \cdots \oplus \langle u_t + K\rangle$ is indeed a direct sum decomposition.

Chapter 7

Quantum attacks on elliptic curve cryptography

In Chapter 5 we discussed Shor's algorithm, which can calculate discrete logarithms over any cyclic group. In particular, this algorithm can be used to break the Diffie-Hellman key exchange protocol, which assumes that the discrete log problem in $\mathbb{Z}_p^\times$ (p prime) is hard. However, Shor's algorithm also breaks *elliptic curve cryptography*, the main competitor to RSA. In this lecture we will introduce elliptic curves and show how they give rise to abelian groups that can be used to define cryptosystems.

This lecture is only intended to be a survey of the main ideas behind elliptic curve cryptography. While breaking such cryptosystems is a major potential application of quantum computers, only a few implementation details differ between the algorithms for discrete log over the integers and over elliptic curves; no new quantum ideas are required.

7.1 Elliptic curves

Fix a field $\mathbb{F}$ whose characteristic is not equal to 2 or 3. (Cryptographic applications often use the field $\mathbb{F}_{2^n}$ of characteristic 2, but the definition of an elliptic curve is slightly more complicated in this case, so we will not consider it here.) Consider the equation

$$y^2 = x^3 + ax + b \tag{7.1}$$

where $a, b \in \mathbb{F}$ are parameters. The set of points $(x, y) \in \mathbb{F}^2$ satisfying this equation, together with a special point $\mathcal{O}$ called the *point at infinity*, is called the *elliptic curve* $E_{a,b}$. A curve is called *nonsingular* if its *discriminant*, $\Delta := -16(4a^3 + 27b^2)$, is nonzero, and we will assume that this is the case for all curves we consider.

Figure 7.1 shows a few examples of elliptic curves over $\mathbb{R}^2$. Such pictures are helpful for developing intuition. However, for cryptographic applications it is useful to have a curve whose points can be represented exactly with a finite number of bits, so we use curves over finite fields. For simplicity, we will only consider the case $\mathbb{F}_p$ where p is a prime different from 2 or 3.

As an example, consider the curve

$$E_{-2,1} = \{(x, y) \in \mathbb{F}_7^2 : y^2 = x^3 - 2x + 1\} \tag{7.2}$$

over $\mathbb{F}_7$. This curve has $4a^3 + 27b^2 = -32 + 27 = -5 = 2 \bmod 7$, so it is nonsingular. It is tedious but straightforward to check that the points on this curve are

$$E_{-2,1} = \{\mathcal{O}, (0,1), (0,6), (1,0), (3,1), (3,6), (4,1), (4,6), (5,2), (5,5), (6,3), (6,4)\}. \tag{7.3}$$

In general, the number of points on the curve depends on the parameters a and b. However, for large p it is quite close to p for all curves. Specifically, a theorem of Hasse says it is $p + 1 - t$, where $|t| \leq 2\sqrt{p}$. (Note that for elliptic curves, there is a classical algorithm, *Schoof's algorithm*, that computes the number

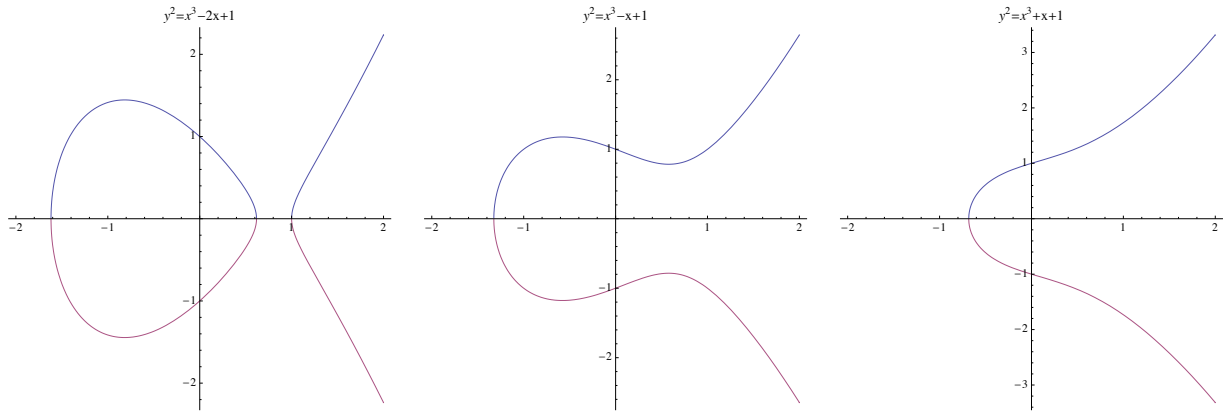


Figure 7.1: Examples of elliptic curves over $\mathbb{R}^2$.

of points on the curve in time poly($\log p$). For more general curves defined by polynomial equations over finite fields, there are similar estimates to the one provided by Hasse's theorem, yet computing the precise number of points may be a classically hard problem. But for some such curves, there is an efficient *quantum* algorithm, *Kedlaya's algorithm*, for counting the number of points on the curve [69].)

It turns out that an elliptic curve defines an abelian group. Specifically, there is a binary operation '+' that maps a pair of points on the curve to a new point on the curve, in a way that satisfies all the group axioms. To motivate this definition, we go back to the case where $\mathbb{F} = \mathbb{R}$ (see Figure 7.2 for an illustration). Given two points $P, Q \in E_{a,b}$, their sum $P + Q$ is defined geometrically, as follows. For now, assume that neither point is $\mathcal{O}$. Draw a line through the points P and Q (or, if $P = Q$, draw the tangent to the curve at P), and let R denote the third point of intersection (defined to be $\mathcal{O}$ if the line is vertical). Then $P + Q$ is defined as the reflection of R about the x axis (where the reflection of $\mathcal{O}$ is $\mathcal{O}$). If one of P or Q is $\mathcal{O}$, we draw a vertical line through the other point, giving the result that $P + \mathcal{O} = P$: $\mathcal{O}$ acts as the additive identity. Thus we define $\mathcal{O} + \mathcal{O} = \mathcal{O}$. Note that reflection about the x axis corresponds to negation, so we can think of the rule as saying that the three points of intersection of any line with the curve sum to 0.

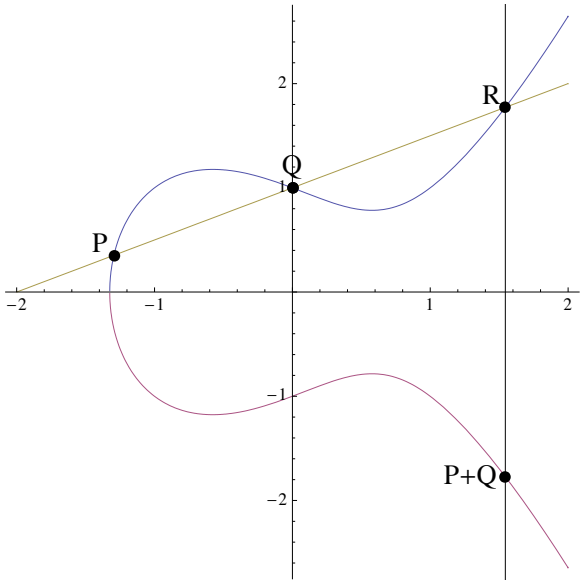


Figure 7.2: The group operation for elliptic curves.

It turns out that this law makes $E_{a,b}$ into an abelian group for which the identity is $\mathcal{O}$ and the inverse of $P = (x, y)$ is $-P = (x, -y)$. By definition, it is clear that $(E_{a,b}, +)$ is abelian (the line through P and Q does not depend on which point is chosen first) and closed (we always choose $P+Q$ to be some point on the curve). The only remaining group axiom to check is associativity: we must show that $(P + Q) + T = P + (Q + T)$. Using a diagram of a typical curve, and picking three arbitrary points, you should be able to convince yourself that associativity appears to hold. Actually proving it in these geometric terms requires a little algebraic geometry.

For calculations, it is helpful to produce an algebraic description of the definition of elliptic curve point addition. Let $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$. The slope of the line through P and Q (with $P \neq Q$) is

$$\lambda = \frac{y_Q - y_P}{x_Q - x_P}. \tag{7.4}$$

Thus the set of points (x, y) on this line is $y = \lambda x + y_0$, where $y_0 = y_P - \lambda x_P$. Substituting this into (7.1) gives the equation

$$x^3 - \lambda^2 x^2 + (a - 2\lambda y_0)x + b - y_0^2 = 0, \tag{7.5}$$

and solving this equation with the cubic formula shows that $x_P + x_Q + x_R = \lambda^2$. Thus we have

$$\begin{aligned} x_{P+Q} &= x_R && (7.6)\\ &= \lambda^2 - x_P - x_Q && (7.7)\\ &= \left(\frac{y_Q - y_P}{x_Q - x_P}\right)^2 - x_P - x_Q && (7.8) \end{aligned}$$

and

$$\begin{aligned} y_{P+Q} &= -y_R && (7.9)\\ &= -\lambda x_{P+Q} - y_0 && (7.10)\\ &= \lambda(x_P - x_{P+Q}) - y_P && (7.11)\\ &= \frac{y_Q - y_P}{x_Q - x_P}(x_P - x_{P+Q}) - y_P. && (7.12) \end{aligned}$$

A similar formula can be derived for the case where $P = Q$ (i.e., we are computing $2P$). It is straightforward to compute the slope of the tangent to the curve at P; if $y_P = 0$ then the slope is infinite, so $2P = \mathcal{O}$, but otherwise

$$\lambda = \frac{3x_P^2 + a}{2y_P}. \tag{7.13}$$

The rest of the calculation proceeds as before, and we have

$$\begin{aligned} x_{2P} &= \lambda^2 - 2x_P && (7.14)\\ &= \left(\frac{3x_P^2 + a}{2y_P}\right)^2 - 2x_P && (7.15) \end{aligned}$$

and

$$\begin{aligned} y_{2P} &= \lambda(x_P - x_{2P}) - y_P && (7.16)\\ &= \frac{3x_P^2 + a}{2y_P}(x_P - x_{2P}) - y_P. && (7.17) \end{aligned}$$

While the geometric picture does not necessarily make sense for the case where $\mathbb{F}$ is a finite field, we can take its algebraic description as a definition of the + operation. Even over a finite field, it turns out that this operation defines an abelian group. It is a nice exercise to check explicitly (say, using Mathematica) that when addition of points is defined by these algebraic expressions, it is commutative, closed, and associative, thereby proving that $(E_{a,b}, +)$ is an abelian group over any field.

7.2 Elliptic curve cryptography

Suppose we fix an elliptic curve $E_{a,b}$ and choose a point $g \in E_{a,b}$. Then we can consider the subgroup $\langle g \rangle$ (which is possibly the entire group if it happens to be cyclic). Using exponentiation in this group (which is simply multiplication in our additive notation), we can define analogs of Diffie-Hellman key exchange and related cryptosystems such as ElGamal. The security of such a cryptosystem then relies on the assumption that the discrete log problem on $\langle g \rangle$ is hard.

In practice, there are many details to consider when choosing an elliptic curve for cryptographic purposes. Algorithms are known for calculating discrete log on "supersingular" and "anomalous" curves that run faster than algorithms for the general case, so such curves should be avoided. Also, g should be chosen to be a point of high order—ideally, the elliptic curve group should be cyclic, and g should be a generator. Such curves can be found efficiently, and in the general case, it is not known how to solve the discrete log problem over an elliptic curve classically any faster than by general methods (e.g., Pollard's rho algorithm), which run in time $O(\sqrt{p})$.

7.3 Shor's algorithm for discrete log over elliptic curves

It is straightforward to use Shor's algorithm to solve the discrete log problem for an elliptic curve over $\mathbb{F}_p$ in time $\text{poly}(\log p)$. Points on the curve can be represented uniquely by their coordinates, with a special symbol used to denote $\mathcal{O}$, the point at infinity. Addition of points on the curve can be computed using the formulas described above, which involve only elementary arithmetic operations in the field. The most complex of these operations is the calculation of modular inverses, which can easily be done using the extended Euclidean algorithm.

Elliptic curve cryptosystems are commonly viewed as being more secure than RSA for a given key size, since the best classical algorithms for factoring run faster than the best classical algorithms for calculating discrete log in an elliptic curve. Thus, in practice, much smaller key sizes are used in elliptic curve cryptography than in factoring-based cryptography. Ironically, Shor's algorithm takes a comparable number of steps for both factoring and discrete log (regardless of the group involved), with the caveat that group operations on an elliptic curve take more time to calculate than ordinary multiplication of integers.

Chapter 8

Quantum algorithms for number fields

In this and the next lecture, we will explore a natural extension of the abelian hidden subgroup problem, namely an algorithm discovered by Hallgren for solving a quadratic diophantine equation known as *Pell's equation* [61, 67]. This algorithm is interesting for at least two reasons. First, it gives an application of quantum algorithms to a new area of mathematics, algebraic number theory (and indeed, subsequent work has shown that quantum computers can also efficiently solve other problems in this area). Second, it extends the solution of the abelian HSP to the case of an infinite group, namely the real numbers.

There are two main parts to the quantum algorithm for solving Pell's equation. First, we define a periodic function whose period encodes the solution to the problem. To define this function, we must introduce some notions from algebraic number theory. Second, we show how to find the period of a black-box function defined over the real numbers even when the period is irrational.

8.1 Review: Order finding

Recall that in order finding, we are given an element g of a finite group G and our goal is to find the order of g, the smallest $r \in \mathbb{N}$ such that $g^r = 1$. We consider the function $f : \mathbb{Z} \to G$ defined by $f(x) = g^x$. This function is periodic with period r, and there is a quantum algorithm to efficiently find this period.

8.2 Pell's equation

Given a squarefree integer d (i.e., an integer not divisible by any perfect square), the Diophantine equation

$$x^2 - dy^2 = 1 \tag{8.1}$$

is known as *Pell's equation*. Amusingly, Pell had nothing whatsoever to do with the equation. The misattribution is apparently due to Euler, who confused Pell with a contemporary, Brouncker, who had actually worked on the equation. In fact, Pell's equation was studied in ancient India, where (inefficient) methods for solving it were developed about a century before Pell.

The left hand side of Pell's equation can be factored as

$$x^2 - dy^2 = (x + y\sqrt{d})(x - y\sqrt{d}). \tag{8.2}$$

Note that a solution of the equation $(x, y) \in \mathbb{Z}^2$ can be encoded uniquely as the real number $x + y\sqrt{d}$: since $\sqrt{d}$ is irrational, $x + y\sqrt{d} = w + z\sqrt{d}$ if and only if $(x, y) = (w, z)$. (Proof: $\frac{x-w}{z-y} = \sqrt{d}$.) Thus we can also refer to the number $x + y\sqrt{d}$ as a solution of Pell's equation.

There is clearly no loss of generality in restricting our attention to *positive* solutions of the equation, namely those for which $x > 0$ and $y > 0$. It is straightforward to show that if $x_1 + y_1\sqrt{d}$ is a positive solution, then $(x_1 + y_1\sqrt{d})^n$ is also a positive solution for any $n \in \mathbb{N}$. In fact, one can show that *all* positive solutions are obtained in this way, where $x_1 + y_1\sqrt{d}$ is the *fundamental solution*, the smallest positive solution

of the equation. Thus, even though Pell's equation has an infinite number of solutions, we can in a sense find them all by finding the fundamental solution.

Some examples of fundamental solutions for various values of d are shown in the following table. Notice that while the size of the fundamental solution generally increases with increasing d, the behavior is far from monotonic: for example, x_1 has 44 decimal digits when $d = 6009$, but only 11 decimal digits when $d = 6013$. But it is possible for the solutions to be very large—the size of $x_1 + y_1\sqrt{d}$ is only upper bounded by $2^{O(\sqrt{d}\log d)}$. Thus it is not even possible to *write down* the fundamental solution with $\mathrm{poly}(\log d)$ bits.

d	x_1	y_1
2	3	2
3	2	1
5	9	4
⋮		
13	649	180
14	15	4
⋮		
6009	1316340106327253158925944695105994738840139 75 $\approx 1.3 \times 10^{44}$	1698114661157803451688949237883146576681644 $\approx 1.6 \times 10^{42}$
6013	40929908599	527831340
⋮		

To get around this difficulty, we define the *regulator* of the fundamental solution,

$$R := \ln(x_1 + y_1\sqrt{d}). \tag{8.3}$$

Since $R = O(\sqrt{d}\log d)$, we can write down $\lceil R \rceil$ using $O(\log d)$ bits. Now R is an irrational number, so determining only its integer part may seem unsatisfactory. But in fact, given the integer part of R, there is a classical algorithm to compute n digits of R in time $\mathrm{poly}(\log d, n)$. Thus it suffices to give an algorithm that finds the integer part of R in time $\mathrm{poly}(\log d)$. The best known classical algorithm for this problem takes time $2^{O(\sqrt{\log d \log\log d})}$ assuming the generalized Riemann hypothesis, or time $O(d^{1/4}\,\mathrm{poly}(\log d))$ with no such assumptions.

8.3 Some basic algebraic number theory

As mentioned above, there are two main parts to the quantum algorithm for Pell's equation: first, the definition of a periodic function over the reals whose period encodes the regulator, and second, a solution of the period-finding problem in the case where the period might be irrational. We will start by showing how to define the periodic function. To do this, we need to introduce some concepts from algebraic number theory.

Given a squarefree positive integer d, the *quadratic number field* $\mathbb{Q}(\sqrt{d})$ is defined as

$$\mathbb{Q}(\sqrt{d}) := \{x + y\sqrt{d} : x, y \in \mathbb{Q}\}. \tag{8.4}$$

You can easily check that this is a field with the usual addition and multiplication operations. We can also define an operation called *conjugation*, defined by

$$\overline{x + y\sqrt{d}} := x - y\sqrt{d}. \tag{8.5}$$

Conjugation of elements of $\mathbb{Q}(\sqrt{d})$ has many of the same properties as complex conjugation, and indeed $\mathbb{Q}(\sqrt{d})$ behaves in many respects like $\mathbb{C}$, with $\sqrt{d}$ taking the place of the imaginary unit $\mathrm{i} = \sqrt{-1}$. Defining the ring $\mathbb{Z}[\sqrt{d}] \subset \mathbb{Q}(\sqrt{d})$ as

$$\mathbb{Z}[\sqrt{d}] := \{x + y\sqrt{d} : x, y \in \mathbb{Z}\}, \tag{8.6}$$

we see that solutions of Pell's equation correspond to $\xi \in \mathbb{Z}[\sqrt{d}]$ satisfying $\xi\bar{\xi} = 1$.

Notice that any solution of Pell's equation, $\xi \in \mathbb{Z}[\sqrt{d}]$, has the property that its multiplicative inverse over $\mathbb{Q}(\sqrt{d})$, $\xi^{-1} = \bar{\xi}/\xi\bar{\xi} = \bar{\xi}$, is also an element of $\mathbb{Z}[\sqrt{d}]$. In general, an element of a ring with an inverse that is also an element of the ring is called a *unit*. In $\mathbb{Z}$, the only units are ± 1, but in other rings it is possible to have more units. It should not be a surprise that the set of units of $\mathbb{Z}[\sqrt{d}]$ is closely related to the set of solutions of Pell's equation. Specifically, we have

Proposition 8.1. *$\xi = x + y\sqrt{d}$ is a unit in $\mathbb{Z}[\sqrt{d}]$ if and only if $\xi\bar{\xi} = x^2 - dy^2 = \pm 1$.*

Proof. We have

$$\xi^{-1} = \frac{\bar{\xi}}{\xi\bar{\xi}} = \frac{x - y\sqrt{d}}{x^2 - dy^2}. \tag{8.7}$$

If $x^2 - dy^2 = \pm 1$, then clearly $\xi^{-1} \in \mathbb{Z}[\sqrt{d}]$. Conversely, if $\xi^{-1} \in \mathbb{Z}[\sqrt{d}]$, then so is

$$\xi^{-1}\overline{\xi^{-1}} = \frac{(x - y\sqrt{d})(x + y\sqrt{d})}{(x^2 - dy^2)^2} = \frac{1}{x^2 - dy^2}, \tag{8.8}$$

which shows that $x^2 - dy^2 = \pm 1$. □

It is not hard to show that the set of all units in $\mathbb{Z}[\sqrt{d}]$ is given by $\{\pm\epsilon_1^n : n \in \mathbb{Z}\}$, where ϵ_1 is the *fundamental unit*, the smallest unit greater than 1. The proof is essentially the same as the proof that all solutions of Pell's equation are powers of the fundamental solution.

If we can find ϵ_1, then it is straightforward to find all the solutions of Pell's equation. If $\epsilon_1 = x + y\sqrt{d}$ has $x^2 - dy^2 = +1$, then the units are precisely the solutions of Pell's equation. On the other hand, if $x^2 - dy^2 = -1$, then $\epsilon_2 := \epsilon_1^2$ satisfies $\epsilon_2\bar{\epsilon}_2 = \epsilon_1^2\bar{\epsilon}_1^2 = (-1)^2 = 1$; in this case the solutions of Pell's equation are $\{\pm\epsilon_1^{2n} : n \in \mathbb{Z}\}$. Thus our goal is to find ϵ_1. Just as in our discussion of the solutions to Pell's equation, ϵ_1 is too large to write down, so instead we will compute the *regulator of the fundamental unit*, $\mathcal{R} := \ln \epsilon_1$.

To define a periodic function that encodes $\mathcal{R}$, we need to introduce the concept of an *ideal* of a ring (and more specifically, a *principal ideal*). For any ring R, we say that $I \subseteq R$ is an ideal if it is closed under integer linear combinations and under multiplication by arbitrary elements of of R. For example, $2\mathbb{Z}$ is an ideal of $\mathbb{Z}$.

We say that an ideal is *principal* if it is generated by a single element of the ring, i.e., if it is of the form αR for some $\alpha \in R$. In the example above, $2\mathbb{Z}$ is a principal ideal. (Not all ideals are principal; for example, consider $x\mathbb{Z}[x, y] + y\mathbb{Z}[x, y] \subseteq \mathbb{Z}[x, y]$, an ideal in the ring of polynomials in x, y with integer coefficients.)

8.4 A periodic function for the units of $\mathbb{Z}[\sqrt{d}]$

Principal ideals are useful because the function mapping the ring element $\xi \in \mathbb{Z}[\sqrt{d}]$ to the principal ideal ξR is periodic, and its periodicity corresponds to the units of $\mathbb{Z}[\sqrt{d}]$. Specifically, we have

Proposition 8.2. *$\xi\mathbb{Z}[\sqrt{d}] = \zeta\mathbb{Z}[\sqrt{d}]$ if and only if $\xi = \zeta\epsilon$ where ϵ is a unit in $\mathbb{Z}[\sqrt{d}]$.*

Proof. If ϵ is a unit, then $\xi\mathbb{Z}[\sqrt{d}] = \zeta\epsilon\mathbb{Z}[\sqrt{d}] = \zeta\mathbb{Z}[\sqrt{d}]$ since $\epsilon\mathbb{Z}[\sqrt{d}] = \mathbb{Z}[\sqrt{d}]$ by the definition of a unit. Conversely, suppose that $\xi\mathbb{Z}[\sqrt{d}] = \zeta\mathbb{Z}[\sqrt{d}]$. Since $1 \in \mathbb{Z}[\sqrt{d}]$, $\xi \in \xi\mathbb{Z}[\sqrt{d}] = \zeta\mathbb{Z}[\sqrt{d}]$, so there is some $\mu \in \mathbb{Z}[\sqrt{d}]$ satisfying $\xi = \zeta\mu$. Similarly, $\zeta \in \zeta\mathbb{Z}[\sqrt{d}] = \xi\mathbb{Z}[\sqrt{d}]$, so there is some $\nu \in \mathbb{Z}[\sqrt{d}]$ satisfying $\zeta = \xi\nu$. Thus we have $\xi = \zeta\mu = \xi\nu\mu$. This shows that $\nu\mu = 1$, so μ and ν are units (indeed, $\nu = \mu^{-1}$). □

Thus the function $g(\xi) = \xi\mathbb{Z}[\sqrt{d}]$ is (multiplicatively) periodic with period ϵ_1. In other words, letting $\xi = e^z$, the function

$$h(z) = e^z\mathbb{Z}[\sqrt{d}] \tag{8.9}$$

is (additively) periodic with period $\mathcal{R}$. However, we cannot simply use this function since it is not possible to succinctly represent the values it takes.

To define a more suitable periodic function, Hallgren uses the concept of a *reduced* ideal, and a way of measuring the distance between principal ideals. The definition of a reduced ideal is rather technical, and we will not go into the details. For our purposes, it is sufficient to note that there are only finitely many

reduced principal ideals, and in fact only $O(d)$ of them, so we can represent a reduced principal ideal using $\mathrm{poly}(\log d)$ bits.

Hallgren also uses a function that measures the distance of any principal ideal from the *unit ideal*, $\mathbb{Z}[\sqrt{d}]$. This function is defined as

$$\delta(\xi\mathbb{Z}[\sqrt{d}]) := \ln \left| \frac{\xi}{\bar{\xi}} \right| \bmod \mathcal{R}. \tag{8.10}$$

Notice that the unit ideal has distance $\delta(1\mathbb{Z}[\sqrt{d}]) = \ln|1/1| \bmod \mathcal{R} = 0$, as required. Furthermore, the distance function does not depend on which generator we choose to represent an ideal, since (by the above proposition) two equivalent ideals have generators that differ by some unit ϵ, and

$$\delta(\epsilon\mathbb{Z}[\sqrt{d}]) = \ln \left| \frac{\epsilon}{\bar{\epsilon}} \right| \bmod \mathcal{R} = \ln \left| \frac{\epsilon}{\epsilon^{-1}} \right| \bmod \mathcal{R} = \ln|\epsilon^2| \bmod \mathcal{R} = 2\ln|\epsilon| \bmod \mathcal{R} = 0. \tag{8.11}$$

With this definition of distance, one can show that the reduced ideals are not too far apart, so that there is a reduced ideal close to any non-reduced ideal.

The periodic function used in Hallgren's algorithm, $f(z)$, is defined as the reduced principal ideal whose distance from the unit ideal is maximal among all reduced principal ideals of distance at most z (together with the distance from $z \bmod \mathcal{R}$, to ensure that the function is one-to-one within each period). In other words, we select the reduced principal ideal "to the left of or at z".

This function is periodic with period $\mathcal{R}$, and can be computed in time $\mathrm{poly}(\log d)$. Thus it remains to show how to perform period finding when the period of the function might be irrational.

Chapter 9

Period finding from $\mathbb{Z}$ to $\mathbb{R}$

In the previous chapter, we defined a periodic function over $\mathbb{R}$ whose period is an irrational number (the regulator) encoding the solutions of Pell's equation. Here we review Shor's approach to period finding, and show how it can be adapted to find an irrational period.

9.1 Period finding over the integers

Shor's factoring algorithm is based on a reduction of factoring to order finding (observed by Miller in the 1970s). This reduction is typically covered in a first course on quantum computing, so we do not discuss the details in these notes.

In the order finding problem for a group G, we are given an element $g \in G$ and our goal is to find the order of g, the smallest $r \in \mathbb{N}$ such that $g^r = 1$. In other words, we are trying to find the smallest positive integer r such that $g^x \bmod L = g^{x+r} \bmod L$ for all $x \in \mathbb{Z}$. (Factoring L reduces to order finding in $G = \mathbb{Z}_L$ with a random element $g \in \mathbb{Z}_L^\times$.) One way to approach this problem is to consider the function $f\colon \mathbb{Z} \to G$ defined by $f(x) = g^x$. This function is periodic with period r, and there is an efficient quantum algorithm to find this period, which we now review.

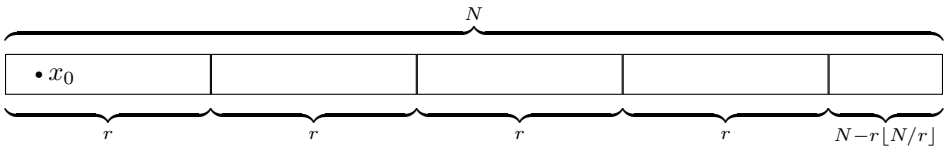
Note that since the period does not, in general, divide a known number N, we cannot simply reduce this task to period finding over $\mathbb{Z}_N$; rather, we should really think of it as period finding over $\mathbb{Z}$ (or, equivalently, the hidden subgroup problem over $\mathbb{Z}$).

Of course, we cannot hope to represent arbitrary integers on a computer with finitely many bits of memory. Instead, we will consider the function only on the inputs $\{0, 1, \ldots, N-1\}$ for some chosen N, and we will perform Fourier sampling over $\mathbb{Z}_N$. We will see that this procedure can work even when the function is not precisely periodic over $\mathbb{Z}_N$. Of course, this can only have a chance of working if the period is sufficiently small, since otherwise we could miss the period entirely. Later, we will see how to choose N if we are given an a priori upper bound of M on the period. If we don't initially have such a bound, we can simply start with $M = 2$ and repeatedly double M until it's large enough for period finding to work. The overhead incurred by this procedure is only $\mathrm{poly}(\log r)$.

Given a value of N, we prepare a uniform superposition over $\{0, 1, \ldots, N-1\}$ and compute the function in another register, giving

$$\frac{1}{\sqrt{N}} \sum_{x \in \{0,\ldots,N-1\}} |x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{x \in \{0,\ldots,N-1\}} |x, f(x)\rangle. \tag{9.1}$$

Next we measure the second register, leaving the first register in a uniform superposition over those values consistent with the measurement outcome. When f is periodic with minimum period r, we obtain a superposition over points separated by the period r. The number of such points, n, depends on where the first point, $x_0 \in \{0, 1, \ldots, r-1\}$, appears. When restricted to $\{0, 1, \ldots, N-1\}$, the function has $\lfloor N/r \rfloor$ full periods and $N - r\lfloor N/r \rfloor$ remaining points, as depicted below. Thus $n = \lfloor N/r \rfloor + 1$ if $x_0 < N - r\lfloor N/r \rfloor$ and $n = \lfloor N/r \rfloor$ otherwise.



Discarding the measurement outcome, we are left with the quantum state

$$\frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} |x_0 + jr\rangle \tag{9.2}$$

where x_0 occurs nearly uniformly random (it appears with probability n/N) and is unknown. To obtain information about the period, we apply the Fourier transform over $\mathbb{Z}_N$, giving

$$\frac{1}{\sqrt{nN}} \sum_{j=0}^{n-1} \sum_{k \in \mathbb{Z}_N} \omega_N^{k(x_0+jr)} |k\rangle = \frac{1}{\sqrt{nN}} \sum_{k \in \mathbb{Z}_N} \omega_N^{kx_0} \sum_{j=0}^{n-1} \omega_N^{jkr} |k\rangle. \tag{9.3}$$

Now if we were lucky enough to choose a value of N for which $r \mid N$, then in fact $n = N/r$ regardless of the value of x_0, and the sum over j above is

$$\sum_{j=0}^{n-1} \omega_N^{jkr} = \sum_{j=0}^{n-1} \omega_n^{jk} \tag{9.4}$$

$$= n\delta_{k \bmod n, 0}. \tag{9.5}$$

In this especially simple case, the quantum state is

$$\frac{n}{\sqrt{nN}} \sum_{k \in \mathbb{Z}_N} \omega_N^{kx_0} \delta_{k \bmod n, 0} |k\rangle = \frac{1}{\sqrt{r}} \sum_{k \in n\mathbb{Z}_r} \omega_N^{kx_0} |k\rangle, \tag{9.6}$$

and measurement of k is guaranteed to give an integer multiple of $n = N/r$, with each of the r multiples occurring with probability $1/r$. But more generally, the sum over j in (9.3) is the geometric series

$$\sum_{j=0}^{n-1} \omega_N^{jkr} = \frac{\omega_N^{krn} - 1}{\omega_N^{kr} - 1} \tag{9.7}$$

$$= \omega_N^{(n-1)kr/2} \frac{\sin \frac{\pi krn}{N}}{\sin \frac{\pi kr}{N}}. \tag{9.8}$$

The probability of seeing a particular value k is given by the normalization factor $1/nN$ times the magnitude squared of this sum, namely

$$\Pr(k) = \frac{\sin^2 \frac{\pi krn}{N}}{nN \sin^2 \frac{\pi kr}{N}}. \tag{9.9}$$

From the case where $n = N/r$, we expect this distribution to be strongly peaked around values of k that are close to integer multiples of N/r. The probability of seeing $k = \lfloor jN/r \rceil = jN/r + \epsilon$ for some $j \in \mathbb{Z}$, where $\lfloor x \rceil$ denotes the nearest integer to x, is

$$\Pr(k = \lfloor jN/r \rceil) = \frac{\sin^2(\pi jn + \frac{\pi \epsilon rn}{N})}{nN \sin^2(\pi j + \frac{\pi \epsilon r}{N})} \tag{9.10}$$

$$= \frac{\sin^2 \frac{\pi \epsilon rn}{N}}{nN \sin^2 \frac{\pi \epsilon r}{N}}. \tag{9.11}$$

Now, to upper bound the denominator, we use $\sin^2 x \le x^2$. To lower bound the numerator, observe that since $|\epsilon| \le 1/2$ and $rn/N \le 1 + O(1/n)$, we have $|\frac{\pi \epsilon rn}{N}| \le \frac{\pi}{2} + O(1/n)$; thus $\sin^2 \frac{\pi \epsilon rn}{N} \ge c(\frac{\pi \epsilon rn}{N})^2$ for some

constant c (in particular, we can take $c \approx \frac{4}{\pi^2}$ for large n). Thus we have

$$\Pr(k = \lfloor jN/r \rceil) \geq \frac{c(\frac{\pi \epsilon r n}{N})^2}{nN(\frac{\pi \epsilon r}{N})^2} \tag{9.12}$$

$$= \frac{cn}{N} \tag{9.13}$$

$$\approx \frac{c}{r}. \tag{9.14}$$

This bound shows that Fourier sampling produces a value of k that is the closest integer to one of the r integer multiples of N/r with probability lower bounded by a constant.

To discover r given one of the values $\lfloor jN/r \rceil$, we can divide by N to obtain a rational approximation to j/r that deviates by at most $1/2N$. Then consider the continued fraction expansion

$$\frac{\lfloor jN/r \rceil}{N} = \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \cdots}}}. \tag{9.15}$$

Truncating this expansion after a finite number of terms gives a *convergent* of the expansion. The convergents provide a sequence of successively better approximations to $\lfloor jN/r \rceil / N$ by fractions that can be computed in polynomial time (see for example Knuth's *The Art of Computer Programming*, volume 2). Furthermore, it can be shown that any fraction p/q with $|p/q - \lfloor jN/r \rceil / N| < 1/2q^2$ will appear as one of the convergents (see for example Hardy and Wright, Theorem 184). Since j/r differs by at most $1/2N$ from $\lfloor jN/r \rceil / N$, the fraction j/r will appear as a convergent provided $r^2 < N$. By taking N is sufficiently large, this gives an efficient means of recovering the period.

9.2 Period finding over the reals

Now suppose we are given a function $f\colon \mathbb{R} \to S$ satisfying $f(x + r) = f(x)$ for some $r \in \mathbb{R}$, and as usual, assume that f is injective within each (minimal) period. Now we'll see how to adapt Shor's procedure to find an approximation to r, even if it happens to be irrational.

To perform period finding on a digital computer, we must of course discretize the function. We have to be careful about how we perform this discretization. For example, suppose that $S = \mathbb{R}$. If we simply evaluate f at equally spaced points and round the resulting values (perhaps rescaled) to get integers, there is no reason for the function values corresponding to inputs separated by an amount close to the period to be related in any way whatsoever. It could be that the discretized function is injective, carrying absolutely no information about the period.

Instead we will discretize in such a way that the resulting function is *pseudoperiodic*. We say that $f : \mathbb{Z} \to S$ is *pseudoperiodic at $k \in \mathbb{Z}$ with period $r \in \mathbb{R}$* if for each $\ell \in \mathbb{Z}$, either $f(k) = f(k + \lfloor \ell r \rfloor)$ or $f(k) = f(k + \lceil \ell r \rceil)$. We say that f is *ϵ-pseudoperiodic* if it is pseudoperiodic for at least an ϵ fraction of the values $k = 0, 1, \ldots, \lfloor r \rfloor$. We assume that the discretized function is ϵ-pseudoperiodic for some constant ϵ, and that it is injective on the subset of inputs where it is pseudoperiodic. Note that the periodic function encoding the regulator of Pell's equation can be constructed so that it satisfies these conditions.

Now let's consider what happens when we apply Fourier sampling to a pseudoperiodic function. As before, we will Fourier sample over $\mathbb{Z}_N$, with N to be determined later (again, depending on some a priori upper bound M on the period r). We start by computing the pseudoperiodic function on a uniform superposition:

$$\sum_{x \in \{0,\ldots,N-1\}} |x\rangle \mapsto \sum_{x \in \{0,\ldots,N-1\}} |x, f(x)\rangle. \tag{9.16}$$

Now measuring the second register gives, with constant probability, a value for which f is pseudo-periodic. Say that this value is $f(x_0)$ where $0 \leq x_0 \leq r$. As before, we see $n = \lfloor N/r \rfloor + 1$ points if $x_0 < N - r\lfloor N/r \rfloor$ or $n = \lfloor N/r \rfloor$ points otherwise (possibly offset by 1 depending on how the rounding occurs for the largest

value of x, but let's not be concerned with this detail). We will write $[\ell]$ to denote an integer that could be either $\lfloor \ell \rfloor$ or $\lceil \ell \rceil$. With this notation, we obtain

$$\frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} |x_0 + [jr]\rangle. \tag{9.17}$$

Next, performing the Fourier transform over $\mathbb{Z}_N$ gives

$$\frac{1}{\sqrt{nN}} \sum_{j=0}^{n-1} \sum_{k \in \mathbb{Z}_N} \omega_N^{k(x_0+[jr])} |k\rangle = \frac{1}{\sqrt{nN}} \sum_{k \in \mathbb{Z}_N} \omega_N^{kx_0} \sum_{j=0}^{n-1} \omega_N^{k[jr]} |k\rangle. \tag{9.18}$$

Now we have $[jr] = jr + \delta_j$, where $-1 < \delta_j < 1$, so the sum over j is

$$\sum_{j=0}^{n-1} \omega_N^{k[jr]} = \sum_{j=0}^{n-1} \omega_N^{kjr} \omega_N^{k\delta_j}. \tag{9.19}$$

We would like this to be close to the corresponding sum in the case where the offsets δ_j are zero (which, when normalized, is $\Omega(1/\sqrt{r})$ by the same calculation as in the case of period finding over $\mathbb{Z}$). Consider the deviation in amplitude,

$$\left| \sum_{j=0}^{n-1} \omega_N^{kjr} \omega_N^{k\delta_j} - \sum_{j=0}^{n-1} \omega_N^{kjr} \right| \leq \sum_{j=0}^{n-1} |\omega_N^{k\delta_j} - 1| \tag{9.20}$$

$$= 2 \sum_{j=0}^{n-1} \left| \sin \frac{\pi k \delta_j}{N} \right| \tag{9.21}$$

$$\leq 2 \sum_{j=0}^{n-1} \left| \frac{\pi k \delta_j}{N} \right| \tag{9.22}$$

$$\leq \frac{2\pi k n}{N}. \tag{9.23}$$

At least insofar as this bound is concerned, the amplitudes may not be close for all values of k. However, suppose we only consider values of k less than $N/\log r$. (We will obtain such a k with probability about $1/\log r$, so we can condition on this event with only polynomial overhead.) For such a k, we have

$$\left| \frac{1}{\sqrt{nN}} \sum_{j=0}^{n-1} \omega_N^{k[jr]} \right| = \Omega(1/\sqrt{r}) - O(\tfrac{1}{\sqrt{nN}} \cdot \tfrac{n}{\log r}) \tag{9.24}$$

$$= \Omega(1/\sqrt{r}) - O(\tfrac{1}{\sqrt{r} \log r}) \tag{9.25}$$

$$= \Omega(1/\sqrt{r}). \tag{9.26}$$

Thus, as in the case of period finding over $\mathbb{Z}$, Fourier sampling allows us to sample from a distribution for which some value $k = \lfloor jN/r \rceil$ (with $j \in \mathbb{Z}$) appears with reasonably large probability (now $\Omega(1/\operatorname{poly}(\log r))$ instead of $\Omega(1)$).

Finally, we must obtain an approximation to r using these samples. Since r is not an integer, the procedure used in Shor's period-finding algorithm does not suffice. However, we can perform Fourier sampling sufficiently many times that we obtain two values $\lfloor jN/r \rceil, \lfloor j'N/r \rceil$ such that j and j' are relatively prime, again with only polynomial overhead. We prove below that if $N \geq 3r^2$, then j/j' is guaranteed to be one of the convergents in the continued fraction expansion for $\lfloor jN/r \rceil / \lfloor j'N/r \rceil$. Thus we can learn j, and hence compute $jN/\lfloor jN/r \rceil$, which gives a good approximation to r: in particular, $|r - \lfloor jN/\lfloor jN/r \rceil \rceil| \leq 1$.

Lemma 9.1. *If $N \geq 3r^2$, then j/j' appears a convergent in the continued fraction expansion of $\lfloor jN/r \rceil / \lfloor j'N/r \rceil$. Furthermore, $|r - \lfloor jN/\lfloor jN/r \rceil \rceil| \leq 1$.*

Proof. A standard result on the theory of approximation by continued fractions says that if $a, b \in \mathbb{Z}$ with $|x - \frac{a}{b}| \leq \frac{1}{2b^2}$, then a/b appears as a convergent in the continued fraction expansion of x (see for example Hardy and Wright, *An Introduction to the Theory of Numbers*, Theorem 184.) Thus it is sufficient to show that

$$\left| \frac{\lfloor jN/r \rceil}{\lfloor j'N/r \rceil} - \frac{j}{j'} \right| < \frac{1}{2j'^2}. \tag{9.27}$$

Letting $\lfloor jN/r \rceil = jN/r + \mu$ and $\lfloor j'N/r \rceil = j'N/r + \nu$ with $|\mu|, |\nu| \leq 1/2$, we have

$$\left| \frac{\lfloor jN/r \rceil}{\lfloor j'N/r \rceil} - \frac{j}{j'} \right| = \left| \frac{jN/r + \mu}{j'N/r + \nu} - \frac{j}{j'} \right| \tag{9.28}$$

$$= \left| \frac{jN + \mu r}{j'N + \nu r} - \frac{j}{j'} \right| \tag{9.29}$$

$$= \left| \frac{r(\mu j' - \nu j)}{j'(j'N + \nu r)} \right| \tag{9.30}$$

$$\leq \left| \frac{r(j + j')}{2j'^2 N - j'r} \right| \tag{9.31}$$

$$\leq \frac{r}{j'N - r/2} \tag{9.32}$$

where in the last step we have assumed $j < j'$ wlog. This is upper bounded by $1/2j'^2$ provided $j'N \geq r/2 + 2j'^2 r$, which certainly holds if $N \geq 3r^2$ (using the fact that $j' < r$).

Finally

$$r - \frac{jN}{\left\lfloor \frac{jN}{r} \right\rceil} = r - \frac{jN}{\frac{jN}{r} + \mu} \tag{9.33}$$

$$= r - \frac{jNr}{jN + \mu r} \tag{9.34}$$

$$= \frac{\mu r^2}{jN + \mu r} \tag{9.35}$$

which is at most 1 in absolute value since $N \geq 3r^2$, $|\mu| \leq 1/2$, and $j \geq 1$. □

9.3 Other algorithms for number fields

To conclude, we mention some further applications of quantum computing to computational algebraic number theory.

Hallgren's original paper on Pell's equation [61] also solves another problem, the *principal ideal problem*, which is the problem of deciding whether an ideal is principal, and if so, finding a generator of the ideal. Factoring reduces to the problem of solving Pell's equation, and Pell's equation reduces to the principal ideal problem; but no reductions in the other direction are known. Motivated by the possibility that the principal ideal problem is indeed harder than factoring, Buchmann and Williams designed a key exchange protocol based on it. Hallgren's algorithm shows that quantum computers can break this cryptosystem.

Subsequently, further related algorithms for problems in algebraic number theory have been found by Hallgren [60] and, independently, by Schmidt and Vollmer [99]. Specifically, they found polynomial-time algorithms for computing the unit group and the class group of a number field of constant degree. These algorithms require generalizing period finding over $\mathbb{R}$ to a similar problem over $\mathbb{R}^d$.

More recently, some of these algorithms have been extended to the case of arbitrary-degree number fields [44].

Chapter 10

Quantum query complexity of the HSP

So far, we have considered the hidden subgroup problem in abelian groups. We now turn to the case where the group might be nonabelian. We will look at some of the potential applications of the HSP, and then show that the general problem has polynomial quantum query complexity.

10.1 The nonabelian HSP and its applications

Recall that in the hidden subgroup problem for a group G, we are given a black box function $f\colon G \to S$, where S is a finite set. We say that f *hides* a subgroup $H \leq G$ provided

$$f(x) = f(y) \text{ if and only if } x^{-1}y \in H. \tag{10.1}$$

In other words, f is constant on left cosets $H, g_1H, g_2H, \ldots$ of H in G, and distinct on different left cosets. When G is a nonabelian group, we refer to this problem as the nonabelian HSP.

The nonabelian HSP is of interest not only because it generalizes the abelian case in a natural way, but because a solution of certain nonabelian hidden subgroup problems would have particularly useful applications. The most well-known (and also the most straightforward) applications are to the *graph automorphism problem* and the *graph isomorphism problem*, problems for which no efficient classical algorithm is currently known.

In the graph automorphism problem, we are given a graph Γ on n vertices, and the goal is to determine whether it has some nontrivial automorphism. In other words, we would like to know whether there is any nontrival permutation $\pi \in S_n$ such that $\pi(\Gamma) = \Gamma$. The automorphisms of Γ form a subgroup $\operatorname{Aut}\Gamma \leq S_n$; if $\operatorname{Aut}\Gamma$ is trivial then we say Γ is *rigid*. We may cast the graph automorphism problem as an HSP over S_n by considering the function $f(\pi) := \pi(\Gamma)$, which hides $\operatorname{Aut}\Gamma$. If we could solve the HSP in S_n, then by checking whether or not the automorphism group is trivial, we could decide graph automorphism.

In the graph isomorphism problem, we are given two graphs Γ, Γ', each on n vertices, and our goal is to determine whether there is any permutation $\pi \in S_n$ such that $\pi(\Gamma) = \Gamma'$, in which case we say that Γ and Γ' are isomorphic. We can cast graph isomorphism as an HSP in the wreath product $S_n \wr S_2 \leq S_{2n}$, the subgroup of S_{2n} generated by permutations of the first n points, permutations of the second n points, and swapping the two sets of points. Writing elements of $S_n \wr S_2$ in the form (σ, τ, b) where $\sigma, \tau \in S_n$ represent permutations of Γ, Γ', respectively, and $b \in \{0, 1\}$ denotes whether to swap the two graphs, we can define a function

$$f(\sigma, \tau, b) := \begin{cases} (\sigma(\Gamma), \tau(\Gamma')) & b = 0 \\ (\sigma(\Gamma'), \tau(\Gamma)) & b = 1. \end{cases} \tag{10.2}$$

This function hides the automorphism group of the disjoint union of Γ and Γ', which contains an element that swaps the two graphs if and only if they are isomorphic. In particular, if Γ and Γ' are rigid (which seems to be the hardest case for the HSP approach to graph isomorphism), the hidden subgroup is trivial

when Γ, Γ' are non-isomorphic; and has order two, with its nontrival element the involution $(\pi, \pi^{-1}, 1)$, when $\Gamma = \pi(\Gamma')$.

The second major potential application of the hidden subgroup problem is to lattice problems. An *n-dimensional lattice* is the set of all integer linear combinations of n linearly independent vectors in $\mathbb{R}^n$ (a *basis* for the lattice). In the *shortest vector problem*, we are asked to find a shortest nonzero vector in the lattice. In particular, in the *g(n)-unique shortest vector problem*, we are promised that the shortest nonzero vector is unique (up to its sign), and is shorter than any other non-parallel vector by a factor $g(n)$. This problem can be solved in polynomial time on a classical computer if $g(n)$ is sufficiently large (say, if it is exponentially large), and is NP-hard if $g(n) = O(1)$. Less is known about intermediate cases, but the problem is suspected to be classically hard even for $g(n) = \text{poly}(n)$, to the extent that cryptosystems have been designed based on this assumption.

Regev showed that an efficient quantum algorithm for the dihedral hidden subgroup problem based on the so-called *standard method* (described below) could be used to solve the poly(n)-unique shortest vector problem. Such an algorithm would be significant since it would break lattice cryptosystems, which are some of the few proposed cryptosystems that are not compromised by Shor's algorithm.

So far, only the symmetric and dihedral hidden subgroup problems are known to have significant applications. Nevertheless, there has been considerable interest in understanding the complexity of the HSP for general groups. There are at least three reasons for this. First, the problem is simply of fundamental interest: it appears to be a natural setting for exploring the extent of the advantage of quantum computers over classical ones. Second, techniques developed for other HSPs may eventually find application to the symmetric or dihedral groups. Finally, exploring the limitations of quantum computers for HSPs may suggest cryptosystems that could be robust even to quantum attacks.

10.2 The standard method

Nearly all known algorithms for the nonabelian hidden subgroup problem use the black box for f in essentially the same way as in the abelian HSP. This approach has therefore come to be known as the *standard method*.

In the standard method, we begin by preparing a uniform superposition over group elements:

$$|G\rangle := \frac{1}{\sqrt{|G|}} \sum_{g \in G} |g\rangle. \tag{10.3}$$

We then compute the value $f(g)$ in an ancilla register, giving the state

$$\frac{1}{\sqrt{|G|}} \sum_{g \in G} |g, f(g)\rangle. \tag{10.4}$$

Finally, we measure the second register and discard the result (or equivalently, simply discard the second register). If we obtain the outcome $s \in S$, then the state is projected onto the uniform superposition of those $g \in G$ such that $f(g) = s$, which by the definition of f is simply some left coset of H. Since every coset contains the same number of elements, each left coset occurs with equal probability. Thus this procedure produces the *coset state*

$$|gH\rangle := \frac{1}{\sqrt{|H|}} \sum_{h \in H} |gh\rangle \text{ with } g \in G \text{ uniformly random} \tag{10.5}$$

(or, equivalently, we can view g as being chosen uniformly at random from some left transversal of H in G).

Depending on context, it may be more convenient to view the outcome either as a random pure state, or equivalently, as the mixed quantum state

$$\rho_H := \frac{1}{|G|} \sum_{g \in G} |gH\rangle\langle gH| \tag{10.6}$$

which we refer to as a *hidden subgroup state*. In the standard approach to the hidden subgroup problem, we attempt to determine H using samples of this hidden subgroup state. In other words, given $\rho_H^{\otimes k}$ for some $k = \text{poly}(\log |G|)$, we try to find a generating set for H.

10.3 Query complexity of the HSP

As a first step toward understanding the quantum complexity of the HSP, we can ask how many queries of the hiding function are required to solve the problem. If we could show that an exponential number of quantum queries were required, then we would know that there was no efficient quantum algorithm. But it turns out that this is not the case: as shown by Ettiner, Høyer, and Knill, $\mathrm{poly}(\log |G|)$ queries to f suffice to determine H [46]. In particular, they showed this within the framework of the standard method: $\rho_H^{\otimes \mathrm{poly}(\log |G|)}$ contains enough information to recover H. Of course, this does not necessarily mean that the quantum *computational complexity* of the HSP is polynomial, since it is not clear in general how to perform the quantum post-processing of the hidden subgroup states efficiently. Nevertheless, this is an important observation since it already shows a difference between quantum and classical computation, and offers some clues as to how we might design efficient quantum algorithms.

To show that the query complexity of the HSP is polynomial, it is sufficient to show that the (single-copy) hidden subgroup states are pairwise statistically distinguishable, as measured by the quantum fidelity

$$F(\rho, \rho') := \mathrm{tr}\, |\sqrt{\rho}\sqrt{\rho'}|. \tag{10.7}$$

This follows from a result of Barnum and Knill [16], who showed the following.

Theorem 10.1. *Suppose ρ is drawn from an ensemble $\{\rho_1, \ldots, \rho_N\}$, where each ρ_i occurs with some fixed prior probability p_i. Then there exists a quantum measurement (namely, the so-called* pretty good measurement*) that identifies ρ with probability at least*

$$1 - N\sqrt{\max_{i \neq j} F(\rho_i, \rho_j)}. \tag{10.8}$$

In fact, by the minimax theorem, this holds even without assuming a prior distribution for the ensemble.

Given only one copy of the hidden subgroup state, (10.8) will typically give only a trivial bound. However, by taking multiple copies of the hidden subgroup states, we can ensure that the overall states are nearly orthogonal, and hence distinguishable. In particular, using k copies of ρ, we see that there is a measurement for identifying ρ with probability at least

$$1 - N\sqrt{\max_{i \neq j} F(\rho_i^{\otimes k}, \rho_j^{\otimes k})} = 1 - N\sqrt{\max_{i \neq j} F(\rho_i, \rho_j)^k} \tag{10.9}$$

(since the fidelity is multiplicative under tensor products). Setting this expression equal to $1 - \epsilon$ and solving for k, we see that arbitrarily small error probability ϵ can be achieved provided we use

$$k \geq \left\lceil \frac{2(\log N - \log \epsilon)}{\log\left(1/\max_{i \neq j} F(\rho_i, \rho_j)\right)} \right\rceil \tag{10.10}$$

copies of ρ.

Provided that G does not have too many subgroups, and that the fidelity between two distinct hidden subgroup states is not too close to 1, this shows that polynomially many copies of ρ_H suffice to solve the HSP. The total number of subgroups of G is $2^{O(\log^2 |G|)}$, which can be seen as follows. Any group K can be specified in terms of at most $\log_2 |K|$ generators, since every additional (non-redundant) generator increases the size of the group by at least a factor of 2 (by Lagrange's Theorem). This implies that every subgroup of G can be specified by a subset of at most $\log_2 |G|$ elements of G, so the number of subgroups of G is upper bounded by $|G|^{\log_2 |G|} = 2^{(\log_2 |G|)^2}$. This shows that we can take $\log N = \mathrm{poly}(\log |G|)$ in (10.10). Thus $k = \mathrm{poly}(\log |G|)$ copies of ρ_H suffice to identify H with constant probability provided the maximum fidelity is bounded away from 1 by at least $1/\mathrm{poly}(\log |G|)$.

To upper bound the fidelity between two states ρ, ρ', consider the two-outcome measurement that projects onto the support of ρ or its orthogonal complement. The classical fidelity of the resulting distribution is an upper bound on the quantum fidelity, so

$$F(\rho, \rho') \leq \sqrt{\mathrm{tr}\, \Pi_\rho \rho\, \mathrm{tr}\, \Pi_\rho \rho'} + \sqrt{\mathrm{tr}((1 - \Pi_\rho)\rho)\, \mathrm{tr}((1 - \Pi_\rho)\rho')} \tag{10.11}$$

$$= \sqrt{\mathrm{tr}\, \Pi_\rho \rho'}. \tag{10.12}$$

where Π_ρ denotes the projector onto the support of ρ.

Now consider the fidelity between ρ_H and $\rho_{H'}$ for two distinct subgroups $H, H' \leq G$. Let $|H| \geq |H'|$ without loss of generality. We can write (10.6) as

$$\rho_H = \frac{1}{|G|} \sum_{g \in G} |gH\rangle\langle gH| = \frac{|H|}{|G|} \sum_{g \in T_H} |gH\rangle\langle gH|. \tag{10.13}$$

where T_H denotes some left transversal of H in G. Since the right-hand expression is a spectral decomposition of ρ_H, we have

$$\Pi_{\rho_H} = \sum_{g \in T_H} |gH\rangle\langle gH| = \frac{1}{|H|} \sum_{g \in G} |gH\rangle\langle gH|. \tag{10.14}$$

Then we have

$$F(\rho_H, \rho_{H'})^2 \leq \operatorname{tr} \Pi_{\rho_H} \rho_{H'} \tag{10.15}$$

$$= \frac{1}{|H| \cdot |G|} \sum_{g,g' \in G} |\langle gH | g'H' \rangle|^2 \tag{10.16}$$

$$= \frac{1}{|H| \cdot |G|} \sum_{g,g' \in G} \frac{|gH \cap g'H'|^2}{|H| \cdot |H'|} \tag{10.17}$$

$$= \frac{1}{|G| \cdot |H|^2 \cdot |H'|} \sum_{g,g' \in G} |gH \cap g'H'|^2. \tag{10.18}$$

Now

$$|gH \cap g'H'| = |\{(h, h') \in H \times H' : gh = g'h'\}| \tag{10.19}$$

$$= |\{(h, h') \in H \times H' : hh' = g^{-1}g'\}| \tag{10.20}$$

$$= \begin{cases} |H \cap H'| & \text{if } g^{-1}g' \in HH' \\ 0 & \text{if } g^{-1}g' \notin HH', \end{cases} \tag{10.21}$$

so

$$\sum_{g,g' \in G} |gH \cap g'H'|^2 = |G| \cdot |HH'| \cdot |H \cap H'|^2 \tag{10.22}$$

$$= |G| \cdot |H| \cdot |H'| \cdot |H \cap H'|. \tag{10.23}$$

Thus we have

$$F(\rho_H, \rho_{H'})^2 = \frac{|G| \cdot |H| \cdot |H'| \cdot |H \cap H'|}{|G| \cdot |H|^2 \cdot |H'|} \tag{10.24}$$

$$= \frac{|H \cap H'|}{|H|} \tag{10.25}$$

$$\leq \frac{1}{2}. \tag{10.26}$$

This shows that $F(\rho_H, \rho_{H'}) \leq 1/\sqrt{2}$, thereby establishing that the query complexity of the HSP is poly$(\log |G|)$.

Chapter 11

Fourier analysis in nonabelian groups

We have seen that hidden subgroup states contain sufficient information to determine the hidden subgroup. Now we would like to know whether this information can be extracted efficiently. In this lecture, we will introduce the theory of Fourier analysis over general groups, an important tool for getting a handle on this problem.

11.1 A brief introduction to representation theory

To understand nonabelian Fourier analysis, we first need to introduce some notions from group representation theory. For further information on this subject, a good basic reference is the book *Linear Representations of Finite Groups* by Serre [101].

A *linear representation* (or simply *representation*) of a group G over the vector space $\mathbb{C}^n$ is a *homomorphism* $\sigma\colon G \to \mathrm{GL}(\mathbb{C}^n)$, i.e., a map from group elements to nonsingular $n \times n$ complex matrices satisfying $\sigma(x)\sigma(y) = \sigma(xy)$ for all $x, y \in G$. Clearly, $\sigma(1) = 1$ and $\sigma(x^{-1}) = \sigma(x)^{-1}$. We call $\mathbb{C}^n$ the *representation space* of σ, where n is called its *dimension* (or *degree*), denoted d_σ.

Two representations σ and σ' with representation spaces $\mathbb{C}^n$ are called *isomorphic* (denoted $\sigma \sim \sigma'$) if there is an invertible linear transformation $M \in \mathbb{C}^{n\times n}$ such that $M\sigma(x) = \sigma'(x)M$ for all $x \in G$. Otherwise they are called *non-isomorphic* (denoted $\sigma \not\sim \sigma'$). In particular, representations of different dimensions are non-isomorphic. Every representation of a finite group is isomorphic to a *unitary representation*, i.e., one for which $\sigma(x)^{-1} = \sigma(x)^\dagger$ for all $x \in G$. Thus we can restrict our attention to unitary representations without loss of generality.

The simplest representations are those of dimension one, such that $\sigma(x) \in \mathbb{C}$ with $|\sigma(x)| = 1$ for all $x \in G$. Every group has a one-dimensional representation called the *trivial representation*, defined by $\sigma(x) = 1$ for all $x \in G$.

Two particularly useful representations of a group G are the *left regular representation* and the *right regular representation*. Both of these representations have dimension $|G|$, and their representation space is the *group algebra* $\mathbb{C}G$, the $|G|$-dimensional complex vector space spanned by basis vectors $|x\rangle$ for $x \in G$. The left regular representation L satisfies $L(x)|y\rangle = |xy\rangle$, and the right regular representation R satisfies $R(x)|y\rangle = |yx^{-1}\rangle$. In particular, both regular representations are *permutation representations*: each of their representation matrices is a permutation matrix.

Given two representations $\sigma\colon G \to V$ and $\sigma'\colon G \to V'$, we can define their *direct sum*, a representation $\sigma \oplus \sigma'\colon G \to V \oplus V'$ of dimension $d_{\sigma\oplus\sigma'} = d_\sigma + d_{\sigma'}$. The representation matrices of $\sigma \oplus \sigma'$ are block diagonal, of the form

$$(\sigma \oplus \sigma')(x) = \begin{pmatrix} \sigma(x) & 0 \\ 0 & \sigma'(x) \end{pmatrix} \tag{11.1}$$

for all $x \in G$.

A representation is called *irreducible* if it cannot be decomposed as the direct sum of two other representations. Any representation of a finite group G can be written as a direct sum of irreducible representations (or *irreps*) of G.

Another way to combine two representations is with the *tensor product*. The tensor product of $\sigma\colon G \to V$ and $\sigma'\colon G \to V'$ is $\sigma \otimes \sigma'\colon G \to V \otimes V'$, a representation of G of dimension $d_{\sigma\otimes\sigma'} = d_\sigma d_{\sigma'}$.

The *character* of a representation σ is the function $\chi_\sigma\colon G \to \mathbb{C}$ defined by $\chi_\sigma(x) := \operatorname{tr}\sigma(x)$. We have

- $\chi_\sigma(1) = d_\sigma$ (since $\sigma(1)$ is I_d, the d-dimensional identity matrix)
- $\chi_\sigma(x^{-1}) = \chi_\sigma(x)^*$ (since we can assume that σ is unitary), and
- $\chi_\sigma(yx) = \chi_\sigma(xy)$ for all $x, y \in G$ (since the trace is cyclic).

In particular, $\chi_\sigma(yxy^{-1}) = \chi_\sigma(x)$, so characters are constant on conjugacy classes. For two representations σ, σ', we have $\chi_{\sigma\oplus\sigma'} = \chi_\sigma + \chi_{\sigma'}$ and $\chi_{\sigma\otimes\sigma'} = \chi_\sigma \cdot \chi_{\sigma'}$.

The most useful result in representation theory is probably *Schur's Lemma*, which can be stated as follows:

Theorem 11.1 (Schur's Lemma)**.** *Let σ and σ' be two irreducible representations of G, and let $M \in \mathbb{C}^{d_\sigma \times d_{\sigma'}}$ be a matrix satisfying $\sigma(x)M = M\sigma'(x)$ for all $x \in G$. Then if $\sigma \not\sim \sigma'$, $M = 0$; and if $\sigma = \sigma'$, M is a scalar multiple of the identity matrix.*

Schur's Lemma can be used to prove the following orthogonality relation for irreducible representations:

Theorem 11.2 (Orthogonality of irreps)**.** *For two irreps σ and σ' of G, we have*

$$\frac{d_\sigma}{|G|} \sum_{x\in G} \sigma(x)^*_{i,j}\, \sigma'(x)_{i',j'} = \delta_{\sigma,\sigma'}\delta_{i,i'}\delta_{j,j'}, \tag{11.2}$$

where we interpret $\delta_{\sigma,\sigma'}$ to mean 1 if $\sigma \sim \sigma'$, and 0 otherwise.

This implies a corresponding orthogonality relation for the *irreducible characters* (i.e., the characters of the irreducible representations):

Theorem 11.3 (Orthogonality of characters)**.** *For two irreps σ and σ' of G, we have*

$$(\chi_\sigma, \chi_{\sigma'}) := \frac{1}{|G|} \sum_{x\in G} \chi_\sigma(x)^*\, \chi_{\sigma'}(x) = \delta_{\sigma,\sigma'}. \tag{11.3}$$

The characters of G supply an orthonormal basis for the space of *class functions*, functions that are constant on conjugacy classes of G. (Recall that the characters themselves are class functions.) This is expressed by the orthonormality of the *character table* of G, the square matrix whose rows are labeled by irreps, whose columns are labeled by conjugacy classes, and whose entries are the corresponding characters. The character orthogonality theorem says that the rows of this matrix are orthonormal, provided each entry is weighted by the square root of the size of the corresponding conjugagcy class divided by $|G|$. In fact the columns are orthonormal in the same sense.

Any representation of G can be broken up into its irreducible components. The regular representations of G are useful for understanding such decompositions, since they contain every possible irreducible representation of G, with each irrep occuring a number of times equal to its dimension. Let $\hat{G}$ denote a complete set of irreps of G (which are unique up to isomorphism). Then we have

$$L \cong \bigoplus_{\sigma\in\hat{G}} \big(\sigma \otimes I_{d_\sigma}\big), \quad R \cong \bigoplus_{\sigma\in\hat{G}} \big(I_{d_\sigma} \otimes \sigma^*\big). \tag{11.4}$$

In fact, this holds with the same isomorphism for both L and R, since the left and right regular representations commute. This isomorphism is simply the Fourier transform over G, which we discuss further below.

Considering $\chi_L(1) = \chi_R(1) = |G|$ and using this decomposition, we find the well-known identity

$$\sum_{\sigma\in\hat{G}} d_\sigma^2 = |G|. \tag{11.5}$$

Also, noting that $\chi_L(x) = \chi_R(x) = 0$ for any $x \in G \setminus \{1\}$, we see that

$$\sum_{\sigma\in\hat{G}} d_\sigma\, \chi_\sigma(x) = 0. \tag{11.6}$$

In general, the multiplicity of the irrep $\sigma \in \hat{G}$ in an arbitrary representation τ of G is given by $\mu_\sigma^\tau := (\chi_\sigma, \chi_\tau)$. This gives the decomposition

$$\tau \cong \bigoplus_{\sigma \in \hat{G}} \sigma \otimes I_{\mu_\sigma^\tau}. \tag{11.7}$$

Characters also provide a simple test for irreducibility: for any representation σ, $(\chi_\sigma, \chi_\sigma)$ is a positive integer, and is equal to 1 if and only if σ is irreducible.

Any representation σ of G can also be viewed as a representation of any subgroup $H \leq G$, simply by restricting its domain to elements of H. We denote the resulting *restricted representation* by $\mathrm{Res}_H^G \sigma$. Even if σ is irreducible over G, it may not be irreducible over H.

11.2 Fourier analysis for nonabelian groups

The *Fourier transform* is a unitary transformation from the group algebra, $\mathbb{C}G$, to a complex vector space whose basis vectors correspond to matrix elements of the irreps of G, $\bigoplus_{\sigma \in \hat{G}} (\mathbb{C}^{d_\sigma} \otimes \mathbb{C}^{d_\sigma})$. These two spaces have the same dimension by (11.5).

The Fourier transform of the basis vector $|x\rangle \in \mathbb{C}G$ corresponding to the group element $x \in G$ is a weighted superposition over all irreducible representations $\sigma \in \hat{G}$, namely

$$|\hat{x}\rangle := \sum_{\sigma \in \hat{G}} \frac{d_\sigma}{\sqrt{|G|}} |\sigma, \sigma(x)\rangle, \tag{11.8}$$

where $|\sigma\rangle$ is a state that labels the irreducible representation, and $|\sigma(x)\rangle$ is a normalized, d_σ^2-dimensional state whose amplitudes correspond to the entries of the matrix $\sigma(x)/\sqrt{d_\sigma}$:

$$|\sigma(x)\rangle := \sum_{j,k=1}^{d_\sigma} \frac{\sigma(x)_{j,k}}{\sqrt{d_\sigma}} |j, k\rangle. \tag{11.9}$$

(If σ is one-dimensional, then $|\sigma(x)\rangle$ is simply a phase factor $\sigma(x) = \chi_\sigma(x) \in \mathbb{C}$ with $|\sigma(x)| = 1$.) The Fourier transform over G is the unitary matrix

$$F_G := \sum_{x \in G} |\hat{x}\rangle\langle x| \tag{11.10}$$

$$= \sum_{x \in G} \sum_{\sigma \in \hat{G}} \sqrt{\frac{d_\sigma}{|G|}} \sum_{j,k=1}^{d_\sigma} \sigma(x)_{j,k} \, |\sigma, j, k\rangle\langle x|. \tag{11.11}$$

Note that the Fourier transform over G is not uniquely defined, but rather, depends on a choice of basis for each irreducible representation.

It is straightforward to check that F_G is indeed a unitary transformation. Using the identity

$$\langle \sigma(y) | \sigma(x) \rangle = \mathrm{tr}\, \sigma^\dagger(y) \sigma(x) / d_\sigma \tag{11.12}$$

$$= \mathrm{tr}\, \sigma(y^{-1}x) / d_\sigma \tag{11.13}$$

$$= \chi_\sigma(y^{-1}x) / d_\sigma, \tag{11.14}$$

we have

$$\langle \hat{y} | \hat{x} \rangle = \sum_{\sigma \in \hat{G}} \frac{d_\sigma^2}{|G|} \langle \sigma(y) | \sigma(x) \rangle \tag{11.15}$$

$$= \sum_{\sigma \in \hat{G}} \frac{d_\sigma}{|G|} \chi_\sigma(y^{-1}x). \tag{11.16}$$

Hence by (11.5–11.6) above, we see that $\langle \hat{y} | \hat{x} \rangle = \delta_{x,y}$.

F_G is precisely the transformation that decomposes both the left and right regular representations of G into their irreducible components. Let us check this explicitly for the left regular representation L. Recall that this representation satisfies $L(x)|y\rangle = |xy\rangle$, so we have

$$\begin{aligned}
\hat{L}(x) &:= F_G \, L(x) \, F_G^\dagger && (11.17)\\
&= \sum_{y \in G} |\widehat{xy}\rangle\langle\hat{y}| && (11.18)\\
&= \sum_{y \in G} \sum_{\sigma,\sigma' \in \hat{G}} \sum_{j,k=1}^{d_\sigma} \sum_{j',k'=1}^{d_{\sigma'}} \frac{\sqrt{d_\sigma d_{\sigma'}}}{|G|} \sigma(xy)_{j,k} \, \sigma'(y)^*_{j',k'} \, |\sigma, j, k\rangle\langle\sigma', j', k'| && (11.19)\\
&= \sum_{y \in G} \sum_{\sigma,\sigma' \in \hat{G}} \sum_{j,k,\ell=1}^{d_\sigma} \sum_{j',k'=1}^{d_{\sigma'}} \frac{\sqrt{d_\sigma d_{\sigma'}}}{|G|} \sigma(x)_{j,\ell} \, \sigma(y)_{\ell,k} \, \sigma'(y)^*_{j',k'} \, |\sigma, j, k\rangle\langle\sigma', j', k'| && (11.20)\\
&= \sum_{\sigma \in \hat{G}} \sum_{j,k,\ell=1}^{d_\sigma} \sigma(x)_{j,\ell} \, |\sigma, j, k\rangle\langle\sigma, \ell, k| && (11.21)\\
&= \bigoplus_{\sigma \in \hat{G}} \big(\sigma(x) \otimes I_{d_\sigma}\big), && (11.22)
\end{aligned}$$

where in the fourth line we have used the orthogonality relation for irreducible representations.

A similar calculation can be done for the right regular representation defined by $R(x)|y\rangle = |yx^{-1}\rangle$, giving

$$\begin{aligned}
\hat{R}(x) &:= F_G \, R(x) \, F_G^\dagger && (11.23)\\
&= \bigoplus_{\sigma \in \hat{G}} \big(I_{d_\sigma} \otimes \sigma(x)^*\big). && (11.24)
\end{aligned}$$

This identity will be useful when analyzing the application of the quantum Fourier transform to the hidden subgroup problem.

To use the Fourier transform as part of a quantum computation, we must be able to implement it efficiently by some quantum circuit. Efficient quantum circuits for the quantum Fourier transform are known for many, but not all, nonabelian groups. Groups for which an efficient QFT is known include metacyclic groups (i.e., semidirect products of cyclic groups), such as the dihedral group; the symmetric group; and many families of groups that have suitably well-behaved towers of subgroups. There are a few notable groups for which efficient QFTs are *not* known, such as the general linear group $\mathrm{GL}_n(q)$ of $n \times n$ invertible matrices over $\mathbb{F}_q$, the finite field with q elements.

Chapter 12

Fourier sampling

In this lecture, we will see how the Fourier transform can be used to simplify the structure of the states obtained in the standard approach to the hidden subgroup problem. In particular, we will see how *weak Fourier sampling* is sufficient to identify any normal hidden subgroup (generalizing the solution of the abelian HSP). We will also briefly discuss the potential of *strong Fourier sampling* to go beyond the limitations of weak Fourier sampling.

12.1 Weak Fourier sampling

Recall that the standard approach to the HSP allows us to produce a *coset state*

$$|gH\rangle := \frac{1}{\sqrt{|H|}} \sum_{h \in H} |gh\rangle \tag{12.1}$$

where each $g \in G$ occurs uniformly at random; or equivalently, the *hidden subgroup state*

$$\rho_H := \frac{1}{|G|} \sum_{g \in G} |gH\rangle\langle gH|. \tag{12.2}$$

The symmetry of such a state can be exploited using the quantum Fourier transform. In particular, we have

$$|gH\rangle = \frac{1}{\sqrt{|H|}} \sum_{h \in H} R(h)|g\rangle \tag{12.3}$$

where R is the right regular representation of G. Thus the hidden subgroup state can be written

$$\rho_H = \frac{1}{|G| \cdot |H|} \sum_{g \in G} \sum_{h,h' \in H} R(h)|g\rangle\langle g|R(h')^\dagger \tag{12.4}$$

$$= \frac{1}{|G| \cdot |H|} \sum_{h,h' \in H} R(hh'^{-1}) \tag{12.5}$$

$$= \frac{1}{|G|} \sum_{h \in H} R(h). \tag{12.6}$$

Since the right regular representation is block-diagonal in the Fourier basis, the same is true of ρ_H. In particular, we have

$$\hat{\rho}_H := F_G \, \rho_H \, F_G^\dagger \tag{12.7}$$

$$= \frac{1}{|G|} \bigoplus_{\sigma \in \hat{G}} \left(I_{d_\sigma} \otimes \sigma(H)^* \right) \tag{12.8}$$

where

$$\sigma(H) := \sum_{h \in H} \sigma(h). \tag{12.9}$$

Since $\hat{\rho}_H$ is block diagonal, with blocks labeled by irreducible representations, we may now measure the irrep label without loss of information. This procedure is referred to as *weak Fourier sampling*. The probability of observing representation $\sigma \in \hat{G}$ under weak Fourier sampling is

$$\Pr(\sigma) = \frac{1}{|G|} \operatorname{tr}\left(I_{d_\sigma} \otimes \sigma(H)^*\right) \tag{12.10}$$

$$= \frac{d_\sigma}{|G|} \sum_{h \in H} \chi_\sigma(h)^* \tag{12.11}$$

$$= \frac{d_\sigma |H|}{|G|} (\chi_\sigma, \chi_1)_H, \tag{12.12}$$

or in other words, $d_\sigma |H|/|G|$ times the number of times the trivial representation appears in $\operatorname{Res}^G_H \sigma$, the restriction of σ to H. We may now ask whether polynomially many samples from this distribution are sufficient to determine H, and if so, whether H can be reconstructed from this information efficiently.

12.2 Normal subgroups

If G is abelian, then all of its representations are one-dimensional, so weak Fourier sampling reveals all of the available information about ρ_H. (In this case there is no difference between weak Fourier sampling and strong Fourier sampling, which we will discuss later.) Indeed, for an abelian group, we saw that the information provided by Fourier sampling can be used to efficiently determine H.

Weak Fourier sampling succeeds for a similar reason whenever H is a *normal subgroup* of G (denoted $H \trianglelefteq G$), i.e., whenever $gHg^{-1} = H$ for all $g \in G$ [59]. In this case, the hidden subgroup state within the irrep $\sigma \in \hat{G}$ is proportional to

$$\sigma(H)^* = \frac{1}{|G|} \sum_{g \in G, h \in H} \sigma(ghg^{-1})^*. \tag{12.13}$$

This commutes with $\sigma(g)^*$ for all $g \in G$, so by Schur's Lemma, it is a multiple of the identity. Thus $\hat{\rho}_H$ is proportional to the identity within each block, and again weak Fourier sampling reveals all available information about H.

Furthermore, when $H \trianglelefteq G$, the distribution under weak Fourier sampling is a particularly simple generalization of the abelian case: we have

$$\Pr(\sigma) = \begin{cases} d_\sigma^2 |H|/|G| & H \leq \ker \sigma \\ 0 & \text{otherwise,} \end{cases} \tag{12.14}$$

where $\ker \sigma := \{g \in G : \sigma(g) = I_{d_\sigma}\}$ is the kernel of the representation σ (a normal subgroup of G). To see this, note that if $H \not\leq \ker \sigma$, then there is some $h' \in H$ with $\sigma(h') \neq 1$; but then $\sigma(h')\sigma(H) = \sum_{h \in H} \sigma(h'h) = \sigma(H)$, and since $\sigma(h')$ is unitary and $\sigma(H)$ is a scalar multiple of the identity, this can only be satisfied if in fact $\sigma(H) = 0$. On the other hand, if $H \leq \ker \sigma$, then $\chi_\sigma(h) = d_\sigma$ for all $h \in H$, and the result is immediate.

To find H, we can simply proceed as in the abelian case: perform weak Fourier sampling $O(\log |G|)$ times and compute the intersection of the kernels of the resulting irreps (assuming this can be done efficiently). Again, it is clear that the resulting subgroup contains H, and we claim that it is equal to H with high probability. For suppose that at some stage during this process, the intersection of the kernels is $K \trianglelefteq G$ with $K \neq H$; then the probability of obtaining an irrep σ for which $K \leq \ker \sigma$ is

$$\frac{|H|}{|G|} \sum_{\sigma:\, K \leq \ker \sigma} d_\sigma^2 = \frac{|H|}{|K|} \leq \frac{1}{2} \tag{12.15}$$

where we have used the fact that the distribution (12.14) remains normalized if H is replaced by any normal subgroup of G. Since each repetition of weak Fourier sampling has a probability of at least $1/2$ of cutting

the intersection of the kernels at least in half, $O(\log |G|)$ repetitions suffice to converge to H with substantial probability. In fact, applying the same approach when H is not necessarily normal in G gives an algorithm to find the *normal core* of H, the largest subgroup of H that is normal in G.

This algorithm can be applied to find hidden subgroups in groups that are "close to Abelian" in a certain sense. In particular, Grigni et al. showed that if $\kappa(G)$, the intersection of the normalizers of all subgroups of G, is sufficiently large—specifically, if $|G|/|\kappa(G)| = 2^{O(\log^{1/2} n)}$, such as when $G = \mathbb{Z}_3 \rtimes \mathbb{Z}_{2^n}$—then the HSP in G can be solved in polynomial time [56]. The idea is simply to apply the algorithm for normal subgroups to the restriction of G to all subgroups containing $\kappa(G)$; the union of all subgroups obtained in this way gives the hidden subgroup with high probability. This result was subsequently improved (by Gavinsky) to give a polynomial-time quantum algorithm whenever $|G|/|\kappa(G)| = \mathrm{poly}(\log |G|)$.

12.3 Strong Fourier sampling

Despite the examples we have just discussed, weak Fourier sampling does *not* provide sufficient information to recover the hidden subgroup for the majority of hidden subgroup problems. For example, weak Fourier sampling fails to solve the HSP in the symmetric group and the dihedral group.

To obtain more information about the hidden subgroup, we can perform a measurement on the d_σ^2-dimensional state that results when weak Fourier sampling returns the outcome σ. Such an approach is referred to as *strong Fourier sampling*.

Recall that the state $\hat{\rho}_H$ from (12.8) is maximally mixed over the row register, as a consequence of the fact that the left and right regular representations commute. Thus we may discard this register without loss of information, so that strong Fourier sampling is effectively faced with the d_σ-dimensional state

$$\hat{\rho}_{H,\sigma} := \frac{\sigma(H)^*}{\sum_{h \in H} \chi_\sigma(h)^*}. \tag{12.16}$$

In fact, this state is proportional to a projector whose rank is simply the number of times the trivial representation appears in $\mathrm{Res}^G_H \sigma^*$. This follows because

$$\sigma(H)^2 = \sum_{h,h' \in H} \sigma(hh') = |H|\, \sigma(H), \tag{12.17}$$

which gives

$$\hat{\rho}^2_{H,\sigma} = \frac{|H|}{\sum_{h \in H} \chi_\sigma(h)^*} \hat{\rho}_{H,\sigma}, \tag{12.18}$$

so that $\hat{\rho}_{H,\sigma}$ is proportional to a projector with $\mathrm{rank}(\hat{\rho}_{H,\sigma}) = \sum_{h \in H} \chi_\sigma(h)^*/|H|$.

It is not immediately clear how to choose a good basis for strong Fourier sampling, so a natural first approach is to consider the effect of measuring in a random basis (i.e., a basis chosen uniformly with respect to the Haar measure over $\mathbb{C}^{d_\sigma}$). There are a few cases in which such *random strong Fourier sampling* produces sufficient information to identify the hidden subgroup—in particular, Sen showed that it succeeds whenever $\mathrm{rank}(\hat{\rho}_{H,\sigma}) = \mathrm{poly}(\log |G|)$ for all $\sigma \in \hat{G}$ [100].

However, in many cases random strong Fourier sampling is unhelpful. For example, Grigni et al. showed that if H is sufficiently small and G is sufficiently non-Abelian (in a certain precise sense), then random strong Fourier sampling is not very informative [56]. In particular, they showed this for the problem of finding hidden involutions in the symmetric group. Another example was provided by Moore et al., who showed that random strong Fourier sampling fails in the metacyclic groups $\mathbb{Z}_p \rtimes \mathbb{Z}_q$ (subgroups of the affine group $\mathbb{Z}_p \rtimes \mathbb{Z}_p^\times$) when $q < p^{1-\epsilon}$ for some $\epsilon > 0$ [86].

Even when measuring in a random basis is information-theoretically sufficient, it does not give an efficient quantum algorithm, since it is not possible to efficiently measure in a random basis. It would be interesting to find informative pseudo-random bases that can be implemented efficiently. However, in the absence of such techniques, we can instead hope to find explicit bases in which strong Fourier sampling can be performed efficiently, and for which the results give a solution of the HSP. The first such algorithm was provided by Moore et al., for the aforementioned metacyclic groups, but with $q = p/\mathrm{poly}(\log p)$ [86]. Note that for these

values of p, q, unlike the case $q < p^{1-\epsilon}$ mentioned above, measurement in a random basis is information-theoretically sufficient. Indeed, we do not know of *any* example of an HSP for which strong Fourier sampling succeeds, yet random strong Fourier sampling fails; it would be interesting to find any such example (or to prove that none exists).

Note that simply finding an informative basis is not sufficient; it is also important that the measurement results can be efficiently post-processed. This issue arises not only in the context of measurement in a pseudo-random basis, but also in the context of certain explicit bases. For example, Ettinger and Høyer gave a basis for the dihedral HSP in which a measurement gives sufficient classical information to infer the hidden subgroup, but no efficient means of post-processing this information is known [45].

For some groups, it turns out that strong Fourier sampling simply fails. Moore, Russell, and Schulman showed that, regardless of what basis is chosen, strong Fourier sampling provides insufficient information to solve the HSP in the symmetric group [85]. Specifically, they showed that for any measurement basis (indeed, for any POVM applied to a hidden subgroup state), the distribution of outcomes in the cases where the hidden subgroup is trivial and where the hidden subgroup is an involution are exponentially close. Thus, in general one has to consider *entangled measurements* on multiple copies of the hidden subgroup states. (Indeed, entangled measurements on $\Omega(\log |G|)$ copies may be necessary, as Hallgren et al. showed for the symmetric group [58].) In the next two lectures, we will see some examples of quantum algorithms for the HSP that make use of entangled measurements.

Chapter 13

Kuperberg's algorithm for the dihedral HSP

We now discuss a quantum algorithm for the dihedral hidden subgroup problem. No polynomial-time algorithm for this problem is known. However, Kuperberg gave a quantum algorithm that runs in subexponential (though superpolynomial) time—specifically, it runs in time $2^{O(\sqrt{\log |G|})}$ [74].

13.1 The HSP in the dihedral group

The dihedral group of order $2N$, denoted D_N, is the group of symmetries of a regular N-gon. It has the presentation

$$D_N = \langle r, s \colon r^2 = s^N = 1,\ rsr = s^{-1} \rangle. \tag{13.1}$$

Here r can be thought of as a reflection about some fixed axis, and s can be thought of as a rotation of the N-gon by an angle $2\pi/N$.

Using the defining relations, we can write any group element in the form $s^x r^a$ where $x \in \mathbb{Z}_N$ and $a \in \mathbb{Z}_2$. Thus we can equivalently think of the group as consisting of elements $(x, a) \in \mathbb{Z}_N \times \mathbb{Z}_2$. Since

$$\begin{aligned} (s^x r^a)(s^y r^b) &= s^x r^a s^y r^a r^{a+b} && (13.2)\\ &= s^x s^{(-1)^a y} r^{a+b} && (13.3)\\ &= s^{x+(-1)^a y} r^{a+b}, && (13.4) \end{aligned}$$

the group operation '$\cdot$' on such elements can be expressed as

$$(x, a) \cdot (y, b) = (x + (-1)^a y, a + b). \tag{13.5}$$

(In particular, this shows that the dihedral group is the semidirect product $\mathbb{Z}_N \rtimes_\varphi \mathbb{Z}_2$, where $\varphi \colon \mathbb{Z}_2 \to \mathrm{Aut}(\mathbb{Z}_N)$ is defined by $\varphi(a)(y) = (-1)^a y$.) It is also easy to see that the group inverse is

$$(x, a)^{-1} = (-(-1)^a x, a). \tag{13.6}$$

The subgroups of D_N are either cyclic or dihedral. The possible cyclic subgroups are of the form $\langle (x, 0) \rangle$ where $x \in \mathbb{Z}_N$ is either 0 or some divisor of N. The possible dihedral subgroups are of the form $\langle (y, 1) \rangle$ where $y \in \mathbb{Z}_N$, and of the form $\langle (x, 0), (y, 1) \rangle$ where $x \in \mathbb{Z}_N$ is some divisor of N and $y \in \mathbb{Z}_x$. A result of Ettinger and Høyer reduces the general dihedral HSP, in which the hidden subgroup could be any of these possibilities, to the dihedral HSP with the promise that the hidden subgroup is of the form $\langle (y, 1) \rangle = \{(0, 0), (y, 1)\}$, i.e., a subgroup of order 2 generated by the reflection $(y, 1)$.

The basic idea of the Ettinger-Høyer reduction is as follows. Suppose that $f \colon D_N \to S$ hides a subgroup $H = \langle (x, 0), (y, 1) \rangle$. Then we can consider the function f restricted to elements from the abelian group $\mathbb{Z}_N \times \{0\} \leq D_N$. This restricted function hides the subgroup $\langle (x, 0) \rangle$, and since the restricted group is

abelian, we can find x efficiently using Shor's algorithm. Now $\langle (x,0) \rangle \trianglelefteq D_N$ (since $(z,a)(x,0)(z,a)^{-1} = (z + (-1)^a x, a)(-(-1)^a z, a) = ((-1)^a x, 0) \in \mathbb{Z}_N \times \{0\}$), so we can define the quotient group $D_N / \langle (x,0) \rangle$. But this is simply a dihedral group (of order $2N/(N/x) = 2x$), and if we now define a function f' as f evaluated on some coset representative, it hides the subgroup $\langle (y,1) \rangle$. Thus, in the rest of this lecture, we will assume that the hidden subgroup is of the form $\langle (y,1) \rangle$ for some $y \in \mathbb{Z}_N$ without loss of generality.

13.2 Fourier sampling in the dihedral group

When the hidden subgroup is $H = \langle (y,1) \rangle$, one particular left transversal of H in G consists of the left coset representatives $(z,0)$ for all $z \in \mathbb{Z}_N$. The coset state corresponding to the coset $(z,0)H$ is

$$|(z,0)\{(0,0),(y,1)\}\rangle = \frac{1}{\sqrt{2}}(|z,0\rangle + |y+z,1\rangle). \tag{13.7}$$

We would like to determine y using samples of this state.

We have seen that to distinguish coset states in general, one should start by performing weak Fourier sampling: apply a Fourier transform over G and then measure the irrep label. However, in this case we will instead simply Fourier transform the first register over $\mathbb{Z}_N$, leaving the second register alone. It is possible to show that measuring the first register of the resulting state is essentially equivalent to performing weak Fourier sampling over D_N (and discarding the row register), but for simplicity we will just consider the abelian procedure.

Fourier transforming the first register over $\mathbb{Z}_N$, we obtain

$$(F_{\mathbb{Z}_N} \otimes I_2)|(z,0)H\rangle = \frac{1}{\sqrt{2N}} \sum_{k \in \mathbb{Z}_N} (\omega_N^{kz}|k,0\rangle + \omega_N^{k(y+z)}|k,1\rangle \tag{13.8}$$

$$= \frac{1}{\sqrt{N}} \sum_{k \in \mathbb{Z}_N} \omega_N^{kz}|k\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + \omega_N^{ky}|1\rangle). \tag{13.9}$$

If we then measure the first register, we obtain one of the N values of k uniformly at random, and we are left with the post-measurement state

$$|\psi_k\rangle := \frac{1}{\sqrt{2}}(|0\rangle + \omega_N^{yk}|1\rangle). \tag{13.10}$$

Thus we are left with the problem of determining y given the ability to produce single-qubit states $|\psi_k\rangle$ of this form (where k is known).

13.3 Combining states

It would be very useful if we could prepare states $|\psi_k\rangle$ with particular values of k. For example, if we could prepare the state $|\psi_{N/2}\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^y|1\rangle)$, then we could learn the parity of y (i.e., its least significant bit) by measuring in the basis of states $|\pm\rangle := (|0\rangle \pm |1\rangle)/\sqrt{2}$. The main idea of Kuperberg's algorithm is to combine states of the form (13.10) to produce new states of the same form, but with more desirable values of k.

To combine states, we can use the following procedure. Given two states $|\psi_p\rangle$ and $|\psi_q\rangle$, perform a controlled-not gate from the former to the latter, giving

$$|\psi_p, \psi_q\rangle = \frac{1}{2}(|0,0\rangle + \omega_N^{yp}|1,0\rangle + \omega_N^{yq}|0,1\rangle + \omega_N^{y(p+q)}|1,1\rangle) \tag{13.11}$$

$$\mapsto \frac{1}{2}(|0,0\rangle + \omega_N^{yp}|1,1\rangle + \omega_N^{yq}|0,1\rangle + \omega_N^{y(p+q)}|1,0\rangle) \tag{13.12}$$

$$= \frac{1}{\sqrt{2}}(|\psi_{p+q},0\rangle + \omega_N^{yq}|\psi_{p-q},1\rangle). \tag{13.13}$$

Then a measurement on the second qubit leaves the first qubit in the state $|\psi_{p\pm q}\rangle$ (up to an irrelevant global phase), with the + sign occurring when the outcome is 0 and the − sign occurring when the outcome is 1, each outcome occurring with probability 1/2.

This combination operation has a nice representation-theoretic interpretation: the state indices p and q can be viewed as labels of irreducible representations of D_N, and the extraction of $|\psi_{p\pm q}\rangle$ can be viewed as decomposing their tensor product (a reducible representation of D_N) into one of two irreducible components.

13.4 The Kuperberg sieve

Now we are ready to describe how the algorithm works. For simplicity, we will assume from now on that $N = 2^n$ is a power of 2. For such a dihedral group, it is actually sufficient to be able to determine the least significant bit of y, since such an algorithm could be used recursively to determine all the bits of y. This can be seen as follows. The group D_N contains two subgroups isomorphic to $D_{N/2}$, namely $\{(2x, 0), (2x, 1) \colon x \in \mathbb{Z}_{N/2}\}$ and $\{(2x, 0), (2x + 1, 1) \colon x \in \mathbb{Z}_{N/2}\}$. The hidden subgroup is a subgroup of the former if y has even parity, and of the latter if y has odd parity. Thus, once we learn the parity of y, we can restrict our attention to the appropriate $D_{N/2}$ subgroup. The elements of either $D_{N/2}$ subgroup can be represented using only $n - 1$ bits, and finding the least significant bit of the hidden reflection within this subgroup corresponds to finding the second least significant bit of y in D_N. Continuing in this way, we can learn all the bits of y with only n iterations of an algorithm for finding the least significant bit of the hidden reflection.

The idea of Kuperberg's algorithm is to start with a large number of states, and collect them into pairs $|\psi_p\rangle, |\psi_q\rangle$ that share many of their least significant bits, such that $|\psi_{p-q}\rangle$ is likely to have many of its least significant bits equal to zero. Trying to zero out all but the most significant bit in one shot would require an exponential running time, so instead we will proceed in stages, only trying to zero some of the least significant bits in each stage; this will turn out to give an improvement.

Specifically, the algorithm proceeds as follows:

1. Prepare $\Theta(16^{\sqrt{n}})$ coset states of the form (13.10), where each copy has $k \in \mathbb{Z}_{2^n}$ chosen independently and uniformly at random.
2. For each $j = 0, 1, \ldots, m - 1$ where $m := \lceil \sqrt{n} \rceil$, assume the current coset states are all of the form $|\psi_k\rangle$ with at least mj of the least significant bits of k equal to 0. Collect them into pairs $|\psi_p\rangle, |\psi_q\rangle$ that share at least m of the next least significant bits, discarding any qubits that cannot be paired. Create a state $|\psi_{p\pm q}\rangle$ from each pair, and discard it if the + sign occurs. Notice that the resulting states have at least $m(j + 1)$ significant bits equal to 0.
3. The remaining states are of the form $|\psi_0\rangle$ and $|\psi_{2^{n-1}}\rangle$. Measure one of the latter states in the $|\pm\rangle$ basis to determine the least significant bit of y.

Since this algorithm requires $2^{O(\sqrt{n})}$ initial queries and proceeds through $O(\sqrt{n})$ stages, each of which takes at most $2^{O(\sqrt{n})}$ steps, the overall running time is $2^{O(\sqrt{n})}$.

13.5 Analysis of the Kuperberg sieve

To show that this algorithm works, we need to prove that some qubits survive to the final stage of the process with non-negligible probability. Let's analyze a more general version of the algorithm to see why we should try to zero out $\sqrt{n}$ bits at a time, starting with $2^{O(\sqrt{n})}$ states.

Suppose we try to cancel m bits in each stage, so that there are n/m stages (not yet assuming any relationship between m and n), starting with 2^ℓ states. Each combination operation succeeds with probability 1/2, and turns 2 states into 1, so at each step we retain only about 1/4 of the states that can be paired. Now when we pair states that allow us to cancel m bits, there can be at most 2^m unpaired states, since that is the number of values of the m bits to be canceled. Thus if we ensure that there are at least $2 \cdot 2^m$ states at each stage, we expect to retain at least a 1/8 fraction of the states for the next stage. Since we begin with 2^ℓ states,we expect to have at least $2^{\ell-3j}$ states left after the jth stage. Thus, to have $2 \cdot 2^m$ states remaining at the last stage of the algorithm, we require $2^{\ell-3n/m} > 2^{m+1}$, or $\ell > m + 3n/m + 1$. This is minimized by choosing $m \approx \sqrt{n}$, so we see that $\ell \approx 4\sqrt{n}$ suffices.

This analysis is not quite correct because we do not obtain precisely a 1/8 fraction of the paired states for use in the next stage. For most of the stages, we have many more than $2 \cdot 2^m$ states, so nearly all of them can be paired, and the expected fraction remaining for the next stage is close to 1/4. Of course, the precise fraction will experience statistical fluctuations. However, since we are working with a large number of states, the deviations from the expected values are very small, and a more careful analysis (using the Chernoff bound) shows that the procedure succeeds with high probability. For a detailed argument, see section 3.1 of Kuperberg's paper (SICOMP version). That paper also gives an improved algorithm that runs faster and that works for general N.

Note that this algorithm uses not only superpolynomial time, but also superpolynomial space, since all $\Theta(16^{\sqrt{n}})$ coset states are present at the start of the algorithm. However, by creating a smaller number of coset states at a time and combining them according to the solution of a subset sum problem, Regev showed how to make the space requirement polynomial with only a slight increase in the running time [91, 37].

13.6 Entangled measurements

Although this algorithm acts on pairs of coset states at a time, the overall algorithm effectively implements a highly entangled measurement on all $\Theta(16^{\sqrt{n}})$ registers, since the combination operation that produces $|\psi_{p\pm q}\rangle$ entangles the coset states $|\psi_p\rangle$ and $|\psi_q\rangle$. The same is true of Regev's polynomial-space variant.

It is natural to ask whether a similar sieve could be applied to other hidden subgroup problems, such as in the symmetric group, for which highly entangled measurements are necessary. Alagic, Moore, and Russell used a similar approach to give a subexponential-time algorithm for the hidden subgroup problem in the group G^n, where G is a fixed non-Abelian group [9]. (Note that the HSP in G^n can be much harder than solving n instances of the HSP in G, since G^n has many subgroups that are not direct products of subgroups of G.) But unfortunately, this kind of sieve does not seem well-suited to the symmetric group. In particular, Moore, Russell, and Sniady gave the following negative result for the HSP in $S_n \wr \mathbb{Z}_2$, where the hidden subgroup is promised to be either trivial or an involution [87]. Consider any algorithm that works by combining pairs of hidden subgroup states to produce a new state in the decomposition of their tensor product into irreps (i.e., in their *Clebsch-Gordan decomposition*), and uses the sequence of measurement results to guess whether the hidden subgroup is trivial or nontrivial. Any such algorithm must use $2^{\Omega(\sqrt{n})}$ queries. Thus it is not possible to give a significantly better-than-classical algorithm for graph isomorphism in this way, since there are classical algorithms for graph isomorphism that run in time $2^{O(\sqrt{n/\log n})}$.

Note that entangled measurements are *not* information-theoretically necessary for the dihedral HSP: Ettinger and Høyer gave an explicit measurement (i.e., an explicit basis for strong Fourier sampling) from which the measurement results give sufficient information to determine the hidden subgroup [45]. Suppose that, given the state (13.10), we simply measure in the $|\pm\rangle$ basis. Then we obtain the result $|+\rangle$ with probability

$$\left| \left(\frac{\langle 0| + \langle 1|}{\sqrt{2}} \right) \left(\frac{|0\rangle + \omega_N^{yk}|1\rangle}{\sqrt{2}} \right) \right|^2 = \left| \frac{1 + \omega_N^{yk}}{2} \right|^2 = \cos^2 \frac{\pi y k}{N}. \tag{13.14}$$

If we postselect on obtaining this outcome (which happens with probability 1/2 over the uniformly random value of k, assuming $y \neq 0$), then we effectively obtain each value $k \in \mathbb{Z}_N$ with probability $\Pr(k|+) = \frac{2}{N} \cos^2 \frac{\pi y k}{N}$. It is not hard to show that these distributions are statistically far apart for different values of k, so that they can in principle be distinguished with only polynomially many samples. However, no efficient (or even subexponential time) classical (or even quantum) algorithm for doing so is known.

Chapter 14

The HSP in the Heisenberg group

We showed that the quantum query complexity of the general hidden subgroup problem is polynomial by measuring $\rho_H^{\otimes \operatorname{poly}(\log |G|)}$ using a particular measurement strategy (the *pretty good measurement*) that identifies H with high probability. One strategy for finding an efficient quantum algorithm for the HSP is to find an efficient way of implementing that particular measurement [15]. In this lecture, we will describe an efficient quantum algorithm for the HSP in the Heisenberg group that effectively implements the pretty good measurement.

14.1 The Heisenberg group

There are several different ways to define the Heisenberg group. For those familiar with quantum error correcting codes on higher-dimensional systems, perhaps the most familiar definition is as follows. Given a prime number p, define operators X and Z acting on an orthonormal basis of states $\{|x\rangle : x \in \mathbb{Z}_p\}$ by

$$X|x\rangle = |x + 1 \bmod p\rangle \tag{14.1}$$

$$Z|x\rangle = \omega_p^x |x\rangle. \tag{14.2}$$

These operators satisfy the relation $ZX = \omega_p XZ$. Using this relation, any product of X's and Z's can be written in the form $\omega_p^a X^b Z^c$, where $a, b, c \in \mathbb{Z}_p$. Thus the operators X and Z generate a group of order p^3, which is precisely the Heisenberg group. Writing the group elements in the form (a, b, c) with $a, b, c \in \mathbb{Z}_p$, it is straightforward to work out the group law

$$(a, b, c) \cdot (a', b', c') = (a + a' + b'c, b + b', c + c'). \tag{14.3}$$

Equivalently, the Heisenberg group is the group of lower triangular 3×3 matrices

$$\left\{ \begin{pmatrix} 1 & 0 & 0 \\ b & 1 & 0 \\ a & c & 1 \end{pmatrix} : a, b, c \in \mathbb{F}_p \right\} \tag{14.4}$$

over $\mathbb{F}_p$, and the semidirect product $\mathbb{Z}_p^2 \rtimes_\varphi \mathbb{Z}_p$, where $\varphi : \mathbb{Z}_p \to \operatorname{Aut}(\mathbb{Z}_p^2)$ is defined by $\varphi(c)(a, b) = (a + bc, b)$.

To solve the HSP in the Heisenberg group, it is sufficient to be able to distinguish the following cyclic subgroups of order p:

$$H_{a,b} := \langle (a, b, 1) \rangle = \{(a, b, 1)^x : x \in \mathbb{Z}_p\}. \tag{14.5}$$

The reduction to this case is essentially the same as the reduction of the dihedral hidden subgroup problem to the case of a hidden reflection, so we omit the details. The elements of such a subgroup are

$$(a, b, 1)^2 = (2a + b, 2b, 2) \tag{14.6}$$

$$(a, b, 1)^3 = (a, b, 1)(2a + b, 2b, 2) = (3a + 3b, 3b, 3) \tag{14.7}$$

$$(a, b, 1)^4 = (a, b, 1)(3a + 3b, 3b, 3) = (4a + 6b, 4b, 4) \tag{14.8}$$

$$(a, b, 1)^5 = (a, b, 1)(4a + 6b, 4b, 4) = (5a + 10b, 5b, 5) \tag{14.9}$$

etc., and a straightforward inductive argument shows that a general element has the form

$$(a, b, 1)^x = (xa + \tbinom{x}{2}b, xb, x). \tag{14.10}$$

Furthermore, it is easy to see that the p^2 elements $(\ell, m, 0)$ for $\ell, m \in \mathbb{Z}_p$ form a left transversal of $H_{a,b}$ in the Heisenberg group for any $a, b \in \mathbb{Z}_p$.

14.2 Fourier sampling

Suppose we are given a function that hides $H_{a,b}$ in the Heisenberg group. Then the standard method can be used to produce the coset state

$$|(\ell, m, 0)H_{a,b}\rangle = \frac{1}{\sqrt{p}} \sum_{x \in \mathbb{Z}_p} |\ell + xa + \tbinom{x}{2}b, m + xb, x\rangle \tag{14.11}$$

for some uniformly random, unknown $\ell, m \in \mathbb{Z}_p$. Our goal is to determine the parameters $a, b \in \mathbb{Z}_p$ using the ability to produce such states.

At this point, we could perform weak Fourier sampling over the Heisenberg group without discarding any information. However, as in the case of the dihedral group, it will be simpler to consider an abelian Fourier transform instead of the full nonabelian Fourier transform. Using the representation theory of the Heisenberg group, one can show that this procedure is essentially equivalent to nonabelian Fourier sampling.

Fourier transforming the first two registers over $\mathbb{Z}_p^2$, we obtain the state

$$(F_{\mathbb{Z}_p} \otimes F_{\mathbb{Z}_p} \otimes I_p)|(\ell, m, 0)H_{a,b}\rangle = \frac{1}{p^{3/2}} \sum_{x,s,t \in \mathbb{Z}_p} \omega_p^{s(\ell + xa + \binom{x}{2}b) + t(m + xb)} |s, t, x\rangle. \tag{14.12}$$

Now suppose we measure the values s, t appearing in the first two registers. In fact this can be done without loss of information, since the density matrix of the state (mixed over the uniformly random values of ℓ, m) is block diagonal, with blocks labeled by s, t. Collecting the coefficients of the unknown parameters a, b, the resulting p-dimensional quantum state is

$$|\widehat{H_{a,b;s,t}}\rangle := \frac{1}{\sqrt{p}} \sum_{x \in \mathbb{Z}_p} \omega_p^{s(xa + \binom{x}{2}b) + t(xb)} |x\rangle \tag{14.13}$$

$$= \frac{1}{\sqrt{p}} \sum_{x \in \mathbb{Z}_p} \omega_p^{a(sx) + b(s\binom{x}{2} + tx)} |x\rangle. \tag{14.14}$$

where the values $s, t \in \mathbb{Z}_p$ are known, and are obtained uniformly at random. We would like to use samples of this state to determine $a, b \in \mathbb{Z}_p$.

14.3 Two states are better than one

With only one copy of this state, there is insufficient information to recover the hidden subgroup: Holevo's theorem guarantees that a measurement on a p-dimensional quantum state can reliably communicate at most p different outcomes, yet there are p^2 possible values of $(a, b) \in \mathbb{Z}_p^2$. Thus we have to use at least two copies of the state. One can show that there exist single-register measurements on this state that yield enough information to recover a, b with $\text{poly}(\log p)$ samples—in fact, a random measurement has this property with high probability. But no single-register measurement is known from which a and b can be extracted *efficiently* (i.e., in time $\text{poly}(\log p)$).

However, by making a joint measurement on two copies of the state, we can recover the information about a, b that is encoded in a quadratic function in the phase. To see this, consider the two-copy state

$$|\widehat{H_{a,b;s,t}}\rangle \otimes |\widehat{H_{a,b;u,v}}\rangle = \frac{1}{p} \sum_{x,y \in \mathbb{Z}_p} \omega_p^{a(sx+uy) + b(s\binom{x}{2} + tx + u\binom{y}{2} + vy)} |x, y\rangle \tag{14.15}$$

$$= \frac{1}{p} \sum_{x,y \in \mathbb{Z}_p} \omega_p^{\alpha a + \beta b} |x, y\rangle, \tag{14.16}$$

where

$$\alpha := sx + uy \tag{14.17}$$

$$\beta := s\tbinom{x}{2} + tx + u\tbinom{y}{2} + vy \tag{14.18}$$

and where we suppress the dependence of α, β on s, t, u, v, x, y for clarity. If we could replace $|x, y\rangle$ by $|\alpha, \beta\rangle$, then the resulting state would be simply the Fourier transform of $|a, b\rangle$, and an inverse Fourier transform would reveal the solution. So let's compute the values of α, β in ancilla registers, giving the state

$$\frac{1}{p} \sum_{x,y \in \mathbb{Z}_p} \omega_p^{\alpha a + \beta b} |x, y, \alpha, \beta\rangle, \tag{14.19}$$

and attempt to uncompute the first two registers.

For fixed values of $\alpha, \beta, s, t, u, v \in \mathbb{Z}_p$, the quadratic equations (14.17)–(14.18) could have zero, one, or two solutions $x, y \in \mathbb{Z}_p$. Thus we cannot hope to erase the first and second registers by a classical procedure conditioned on the values in the third and fourth registers (and the known values of s, t, u, v). However, it is possible to implement a quantum procedure to erase the first two registers by considering the full set of solutions

$$S_{\alpha,\beta}^{s,t,u,v} := \{(x, y) \in \mathbb{Z}_p^2 : sx + uy = \alpha \text{ and } s\tbinom{x}{2} + tx + u\tbinom{y}{2} + vy = \beta\}. \tag{14.20}$$

The state (14.19) can be rewritten

$$\frac{1}{p} \sum_{\alpha,\beta \in \mathbb{Z}_p} \omega_p^{\alpha a + \beta b} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|}\, |S_{\alpha,\beta}^{s,t,u,v}, \alpha, \beta\rangle, \tag{14.21}$$

where we use the convention that $|S\rangle := \sum_{s \in S} |s\rangle / \sqrt{|S|}$ denotes the normalized uniform superposition over the elements of the set S. Thus, if we could perform a unitary transformation satisfying

$$|S_{\alpha,\beta}^{s,t,u,v}\rangle \mapsto |\alpha, \beta\rangle \text{ for } |S_{\alpha,\beta}^{s,t,u,v}| \neq 0 \tag{14.22}$$

(and defined in any way consistent with unitarity for other values of α, β), we could erase the first two registers of (14.19), producing the state

$$\frac{1}{p} \sum_{\alpha,\beta \in \mathbb{Z}_p} \omega_p^{\alpha a + \beta b} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|}\, |\alpha, \beta\rangle. \tag{14.23}$$

(Note that in fact we could just apply the transformation (14.22) directly to the state (14.16); there is no need to explicitly compute the values α, β in an ancilla register.)

We refer to the inverse of the transformation (14.22) as *quantum sampling*, since the goal is to produce a uniform superposition over the set of solutions, a natural quantum analog of *random sampling* from those solutions.

Since the system of equations (14.17)–(14.18) consists of a pair of quadratic equations in two variables over $\mathbb{F}_p$, it has either zero, one, or two solutions $x, y \in \mathbb{F}_p$. In particular, a straightforward calculation shows that the solutions can be expressed in closed form as

$$x = \frac{\alpha s + sv - tu \pm \sqrt{\Delta}}{s(s+u)} \qquad\qquad y = \frac{\alpha u + tu - sv \mp \sqrt{\Delta}}{u(s+u)} \tag{14.24}$$

where

$$\Delta := (2\beta s + \alpha s - \alpha^2 - 2\alpha t)(s+u)u + (\alpha u + tu - sv)^2. \tag{14.25}$$

Provided $su(s+u) \neq 0$, the number of solutions is completely determined by the value of Δ. If Δ is a nonzero square in $\mathbb{F}_p$, then there are two distinct solutions; if $\Delta = 0$ then there is only one solution; and if Δ is a non-square then there are no solutions. In any event, since we can efficiently compute an explicit list of solutions in each of these cases, we can efficiently perform the transformation (14.22).

It remains to show that the state (14.23) can be used to recover a, b. This state is close to the Fourier transform of $|a, b\rangle$ provided the solutions are nearly uniformly distributed. Since the values of s, t, u, v are uniformly distributed over $\mathbb{F}_p$, it is easy to see that Δ is uniformly distributed over $\mathbb{F}_p$. This means that Δ is a square about half the time, and is a non-square about half the time (with $\Delta = 0$ occurring only with probability $1/p$). Thus there are two solutions about half the time and no solutions about half the time. This distribution of solutions is uniform enough for the procedure to work.

Applying the inverse quantum Fourier transform over $\mathbb{Z}_p \times \mathbb{Z}_p$, we obtain the state

$$\frac{1}{p^2} \sum_{\alpha,\beta,k,\ell \in \mathbb{Z}_p} \omega_p^{\alpha(a-k)+\beta(b-\ell)} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|}\, |k, \ell\rangle. \tag{14.26}$$

Measuring this state, the probability of obtaining the outcome $k = a$ and $\ell = b$ for any particular values of s, t, u, v is

$$\frac{1}{p^4} \left(\sum_{\alpha,\beta \in \mathbb{Z}_p} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|} \right)^2. \tag{14.27}$$

Since those values occur uniformly at random, the overall success probability of the algorithm is

$$\frac{1}{p^8} \sum_{s,t,u,v \in \mathbb{Z}_p} \left(\sum_{\alpha,\beta \in \mathbb{Z}_p} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|} \right)^2 \geq \frac{1}{p^{12}} \left(\sum_{s,t,u,v \in \mathbb{Z}_p} \sum_{\alpha,\beta \in \mathbb{Z}_p} \sqrt{|S_{\alpha,\beta}^{s,t,u,v}|} \right)^2 \tag{14.28}$$

$$\geq \frac{1}{p^{12}} \left(\sum_{\alpha,\beta \in \mathbb{Z}_p} \frac{p^4}{2 + o(1)} \sqrt{2} \right)^2 \tag{14.29}$$

$$= \frac{1}{2}(1 - o(1)), \tag{14.30}$$

which shows that the algorithm succeeds with probability close to $1/2$.

Chapter 15

Schur-Weyl duality

In Chapter 11, we saw that the quantum Fourier transform lets us exploit the symmetry of the left and right actions of a finite group on the group algebra. We applied this in Chapter 12 to reveal the structure of hidden subgroup states.

Another common symmetry in quantum information problems is permutation invariance. In particular, this symmetry arises whenever we are given many copies of the same quantum state. We will see that permutation symmetry is closely related to another symmetry, corresponding to collective unitary transformations (in which we apply the same unitary on all copies). The concept of *Schur-Weyl duality* shows how these symmetries relate to each other, and how to take advantage of them.

Schur-Weyl duality has applications to a wide variety of quantum information processing tasks, including spectrum testing, tomography, state purification, entanglement concentration, and quantum data compression. For the most part, these are tasks with quantum input (and sometimes also with quantum output). Schur-Weyl duality could conceivably be applied in quantum algorithms for classical problems, although it has been shown not to be useful for the hidden subgroup problem (at least in the context of so-called "weak Fourier-Schur sampling") [36].

15.1 The Schur decomposition

As discussed in Chapter 11, the Fourier transform decomposes the group algebra according to the left and right multiplication actions as

$$\mathbb{C}G \overset{G\times G}{\cong} \bigoplus_{\sigma\in\hat{G}} \mathcal{V}_\sigma \otimes \mathcal{V}_\sigma^*, \tag{15.1}$$

where $\mathcal{V}_\sigma$ is the representation space of the irrep σ.

We now consider another way to use symmetry to decompose quantum state space. Specifically, consider k copies of the d-dimensional space $\mathbb{C}^d$, and consider two groups of transformations on $(\mathbb{C}^d)^{\otimes k}$: the symmetric group S_k under the representation P of S_k on $(\mathbb{C}^d)^{\otimes k}$ that permutes the copies as

$$P(\pi)|i_1 \dots i_k\rangle = |i_{\pi^{-1}(1)}, \dots, i_{\pi^{-1}(k)}\rangle \tag{15.2}$$

for $\pi \in S_k$, and the unitary group $\mathrm{U}(d)$ under the representation Q of $\mathrm{U}(d)$ on $(\mathbb{C}^d)^{\otimes k}$ with $Q(U) = U^{\otimes k}$, i.e.,

$$Q(U)|i_1 \dots i_k\rangle = U|i_1\rangle \otimes \cdots \otimes U|i_k\rangle. \tag{15.3}$$

These actions commute, so by Schur's Lemma, the irreps of P can only act on the multiplicity space of the irreps of Q, and vice versa. But it turns out that each irrep of S_k occurs exactly once, and each irrep of $\mathrm{U}(d)$ also occurs exactly once, so that we can label both kinds of irrep in the same way. Specifically, we have the decomposition

$$(\mathbb{C}^d)^{\otimes k} \overset{S_k\times \mathrm{U}(d)}{\cong} \bigoplus_{\lambda\vdash k} \mathcal{P}_\lambda \otimes \mathcal{Q}_\lambda^d. \tag{15.4}$$

Here $\mathcal{P}_\lambda$ is the representation space for an irrep of S_k labeled by λ. The notation $\lambda \vdash k$ means that λ is a *partition* of k: $\lambda = (\lambda_1, \lambda_2, \ldots)$ for positive integers $\lambda_1, \lambda_2, \ldots$ satisfying $\sum_j \lambda_j = k$ and $\lambda_1 \geq \lambda_2 \geq \cdots$. It is convenient to represent a partition by a *Young diagram*, which has boxes at coordinates $\{(i, j) : 1 \leq j \leq \lambda_i\}$. For each $\lambda \vdash k$, there is also a representation space $\mathcal{Q}^d_\lambda$ for an irrep of $\mathrm{U}(d)$. This relationship between the irreps of the symmetric and unitary groups (or more generally, the general linear group) is known as *Schur-Weyl duality*.

The irreps of the symmetric and unitary groups are well understand, but the details are somewhat complicated, so we will not construct them explicitly here. However, there are fairly straightforward formulas for the dimensions of these irreps. The dimension of the symmetric group irrep for a given $\lambda \vdash k$ is the number of *standard Young tableaux*, i.e., the number of ways of filling the corresponding Young diagram with numbers from $\{1, \ldots, k\}$ such that the rows and columns are strictly increasing. This quantity is given by the *hook length formula*. For a cell (i, j) in the Young diagram λ, let $h_\lambda(i, j)$ be its *hook length*, which is 1 plus the number of boxes to the right plus the number of boxes below cell (i, j). Then

$$\dim \mathcal{P}_\lambda = \frac{k!}{\prod_{(i,j)\in\lambda} h_\lambda(i, j)}. \tag{15.5}$$

The dimension of the corresponding unitary group irrep is given by a similar formula:

$$\dim \mathcal{Q}^d_\lambda = \dim \mathcal{P}_\lambda \frac{d^k}{k!} \prod_{(i,j)\in\lambda} \left(1 + \frac{j - i}{d}\right). \tag{15.6}$$

The transformation that implements a change of basis between the standard basis for $(\mathbb{C}^d)^{\otimes k}$ and bases for the irreps of the symmetric and unitary groups is known as the *Schur transform*. Explicit algorithms have been given for performing this transformation [14, 73], though their details are beyond the scope of this lecture.

15.2 Weak Schur sampling

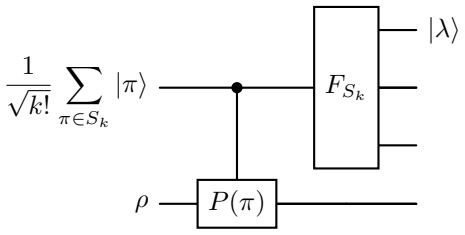
Recall that in weak Fourier sampling, we measure only the irrep label in the Fourier basis. Analogously, in the context of Schur-Weyl duality, we can consider a process where we only measure the label λ of the irrep of the symmetric and unitary groups. This is sometimes called *weak Schur sampling*.

To compute the weak Schur sampling distribution, we can project the given state onto the λ-isotypic subspace $\mathcal{P}_\lambda \otimes \mathcal{Q}^d_\lambda$ and take the trace of the resulting state. The projector onto this space can be written

$$\Pi_\lambda = \frac{\dim \mathcal{P}_\lambda}{k!} \sum_{\pi \in S_k} \chi_\lambda(\pi) P(\pi), \tag{15.7}$$

where $P(\pi)$ is given in (15.2), and χ_λ is the character of the irrep of S_k corresponding to $\lambda \vdash k$. For some applications (discussed further below), this measurement may be sufficient to reveal all available information about the given state.

Weak Schur sampling is much easier to perform than the general Schur transformation. In particular, we can use an approach known as *generalized phase estimation* [62], in which we perform the following circuit.



Here the input state of the top register is a uniform superposition over permutations, which can also be viewed as the inverse Fourier transform (over the symmetric group) of a state corresponding to the trivial

representation. After applying a controlled permutation of the bottom register, we perform the Fourier transform (again over the symmetric group) of the top register, outputting a state labeled by an irrep λ of S_k, together with row and column labels within this irrep. The output of weak Schur sampling is simply the value of λ. Note that the Fourier transform over the symmetric group can be performed efficiently [17].

For some tasks, weak Schur sampling provides all available information about the given state. In particular, suppose our goal is to infer some property of the spectrum of (k copies of) and unknown state ρ. This property is invariant under changes of basis of the state, and also under permutations of the k copies, so there is no information within the representation spaces of either the symmetric or unitary group. In other words, the only information that can be learned from the state is in the distribution over λ that is provided by weak Schur sampling.

A natural case is the task of determining the entire spectrum of ρ from k copies of this state. To answer this question, we should measure $\lambda \vdash k$ and from it, infer the most likely spectrum. Since the spectrum of a d-dimensional density matrix is a probability distribution over d outcomes, the estimate should have the form $(p_1, \ldots, p_d)$ with $\sum_{i=1}^{d} p_i = 1$. Suppose the estimated Young diagram is $(\lambda_1, \ldots, \lambda_d)$ (note that the Young diagram cannot have more than d rows; if so, then $\dim \mathcal{Q}_\lambda^d = 0$, according to (15.6)). Then a natural way to associate a probability distribution is to choose $p_i = \lambda_i/k$, i.e., to simply normalize the Young diagram. In fact, this simple "empirical Young diagram" procedure turns out to be a good estimate: the number of copies it uses to estimate the spectrum to within a given error is nearly optimal.

More generally, suppose we want to learn a specific property of the spectrum of ρ, such as its entropy or whether it is maximally mixed. For any such property, symmetry considerations show that an algorithm should, without loss of generality, perform weak Schur sampling and infer the answer from the resulting Young diagram. This considerably simplifies the task, and the optimal costs of numerous spectrum estimation tasks have been characterized in this way [89].

For any task that takes many identical copies of a given state as input, the Schur transform still block diagonalizes the state into block labeled by Young diagrams. However, if the task does not simply depend on the spectrum of the state—for example, if we want to perform state tomography, i.e., learn a full description of the state—then information remains within the λ-isotypic subspace after performing weak Schur sampling. For such problems, Schur-Weyl duality still provides a useful tool to understand the form of the optimal procedure, but additional considerations may be necessary to understand it fully.

15.3 The swap test

We conclude by briefly discussing the well-known *swap test*, a simple and broadly applicable protocol that arises as a special case of the Schur transform.

Consider the case where we have $k = 2$ copies of $\mathbb{C}^d$. In this case, the representation theory of the relevant symmetric group S_2 is very straightforward. There are only two partitions of 2, namely (2) and (1, 1). These both correspond to 1-dimensional representations of S_2, which is easily verified using (15.5). The $\lambda = (2)$ irrep is trivial, and the $\lambda = (1, 1)$ irrep assigns -1 to the swap of the two items.

Using the corresponding characters of S_2, (15.7) shows that the projectors onto the corresponding λ-isotypic spaces are $\Pi_{(2)} = \frac{1}{2}(I + S)$ and $\Pi_{(1,1)} = \frac{1}{2}(I - S)$, where S is the operator that swaps the two registers. The corresponding subspaces are simply the symmetric subspace

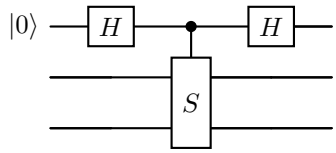
$$\operatorname{span}(\{|a, a\rangle : 1 \leq a \leq d\} \cup \{|a, b\rangle + |b, a\rangle : 1 \leq a < b \leq d\}) \tag{15.8}$$

and the antisymmetric subspace

$$\operatorname{span}\{|a, b\rangle - |b, a\rangle : 1 \leq a < b \leq d\}, \tag{15.9}$$

respectively. It is easy to check consistency with (15.6), which gives $\dim \mathcal{Q}_{(2)}^d = d(d+1)/2$ and $\dim \mathcal{Q}_{(1,1)}^d = d(d-1)/2$.

In this case, weak Schur sampling can be realized by the following circuit.



After performing this circuit, the top qubit is left in the state $|0\rangle$ to indicate $\lambda = (2)$ and the state $|1\rangle$ to indicate $\lambda = (1, 1)$. If the top qubit is measured, then the state of the bottom two registers is projected onto the symmetric or antisymmetric subspace, depending upon the outcome.

Chapter 16

Approximating the Jones polynomial

In this final chapter of the part on algebraic problems, we discuss a very different class of quantum algorithms, ones that approximately solve various #P-complete problems. The best-known example of such a quantum algorithm is for approximating the value of a link invariant called the Jones polynomial. This algorithm is not based on the Fourier transform, but it does use properties of group representations.

16.1 The Hadamard test

The quantum algorithm for approximating the Jones polynomial uses a simple primitive called the *Hadamard test*. This is equivalent to phase estimation with a single bit of precision. Given a unitary operation U and a state $|\psi\rangle$, the Hadamard test provides a means of estimating $\langle\psi|U|\psi\rangle$. The test applies a controlled-U operation to the state $|+\rangle \otimes |\psi\rangle$ and measures the first qubit in the basis $|\pm\rangle := \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle)$. The state before the measurement is

$$\frac{1}{\sqrt{2}}(|0\rangle|\psi\rangle + |1\rangle U|\psi\rangle) = \frac{1}{2}\big(|+\rangle(|\psi\rangle + U|\psi\rangle) + |-\rangle(|\psi\rangle - U|\psi\rangle)\big), \tag{16.1}$$

so

$$\Pr(\pm) = \frac{1}{4}\||\psi\rangle \pm U|\psi\rangle\|^2 \tag{16.2}$$

$$= \frac{1}{2}(1 \pm \mathrm{Re}\langle\psi|U|\psi\rangle). \tag{16.3}$$

In other words, the expected value of the outcome is precisely $\mathrm{Re}\langle\psi|U|\psi\rangle$. Replacing the states $|\pm\rangle$ by the states $|\pm i\rangle := \frac{1}{\sqrt{2}}(|0\rangle \pm i|1\rangle)$, a simple calculation shows that we can approximate $\mathrm{Im}\langle\psi|U|\psi\rangle$.

16.2 The Jones polynomial

The Jones polynomial is a central object in low-dimensional topology with surprising connections to physics. Witten showed that the Jones polynomial is closely related to topological quantum field theory (TQFT). Friedman, Kitaev, Larsen, and Wang investigated the relationship between TQFT and topological quantum computing, showing that quantum computers can efficiently simulate TQFTs (thereby approximating the Jones polynomial), and that in fact TQFTs essentially capture the power of quantum computation [52]. Here we describe the quantum algorithm for approximating the Jones polynomial in a way that does not explicitly refer to TQFT, following the treatment of Aharonov, Jones, and Landau [6].

To define the Jones polynomial, we must first introduce the concepts of knots and links. A *knot* is an embedding of the circle in $\mathbb{R}^3$, i.e., a closed loop of string that may wrap around itself in any way. More generally, a *link* is a collection of any number of knots that may be intertwined. In an *oriented link*, each loop of string is directed. It is natural to identify links that are *isotopic*, i.e., that can be transformed into one another by continuous deformation of the strings.

The Jones polynomial of an oriented link L is a Laurent polynomial $V_L(t)$ in the variable $\sqrt{t}$, i.e., a polynomial in $\sqrt{t}$ and $1/\sqrt{t}$. It is a *link invariant*, meaning that $V_L(t) = V_{L'}(t)$ if the oriented links L and L' are isotopic. While it is possible for the Jones polynomial to take the same value on two non-isotopic links, it can often distinguish links; for example, the Jones polynomials of the two orientations of the trefoil knot are different.

An oriented link L can be specified by a *link diagram*, a drawing of the link in the plane with over- and under-crossings indicated. One way to define the Jones polynomial of a link diagram is as follows. First, let us define the *Kauffman bracket* $\langle L \rangle$, which does not depend on the orientation of L. Each crossing in the link diagram can be opened in one of two ways, and for any given crossing we have

$$\left\langle \times \right\rangle = t^{1/4} \left\langle \;) \; (\; \right\rangle + t^{-1/4} \left\langle \asymp \right\rangle, \tag{16.4}$$

where the rest of the link remains unchanged. Repeatedly applying this rule, we eventually arrive at a link consisting of disjoint unknots. The Kauffman bracket of a single unknot is $\langle \bigcirc \rangle := 1$, and more generally, the Kauffman bracket of n unknots is $(-t^{1/2} - t^{-1/2})^{n-1}$. By itself, the Kauffman bracket is not a link invariant, but it can be turned into one by taking into account the orientation of the link, giving the Jones polynomial. For any oriented link diagram L, we define its *writhe* $w(L)$ as the number of crossings of the form $\nearrow\!\!\!\!\!\!\searrow$ minus the number of crossings of the form $\nwarrow\!\!\!\!\!\!\searrow$. Then the Jones polynomial is defined as

$$V_L(t) := (-t^{-1/4})^{3w(L)} \langle L \rangle. \tag{16.5}$$

Computing the Jones polynomial of a link diagram is quite difficult. A brute-force calculation using the definition in terms of the Kauffman bracket takes time exponential in the number of crossings. Indeed, exactly computing the Jones polynomial is #P-hard (except for a few special values of t), as shown by Jaeger, Vertigan, and Welsh. Here #P is the class of counting problems associated to problems in NP (e.g., computing the number of satisfying assignments of a Boolean formula). Of course, approximate counting can be easier than exact counting, and sometimes #P-hard problems have surprisingly good approximation algorithms.

16.3 Links from braids

It is useful to view links as arising from *braids*. A braid is a collection of n parallel strands, with adjacent strands allowed to cross over or under one another. Two braids on the same number of strands can be composed by placing them end to end. The *braid group* on n strands is an infinite group with generators $\{\sigma_1, \ldots, \sigma_{n-1}\}$, where σ_i denotes a twist in which strand i passes over strand $i+1$, interchanging the two strands. More formally, the braid group is defined by the relations $\sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1}$ and $\sigma_i \sigma_j = \sigma_j \sigma_i$ for $|i - j| > 1$.

Braids and links differ in that the ends of a braid are open, whereas a link consists of closed strands. We can obtain a link from a braid by connecting the ends of the strands in some way. One simple way to close a braid is via the *trace closure*, in which the ith strand of one end is connected to the ith strand of the other end for each $i = 1, \ldots, n$, without crossing the strands. A theorem of Alexander states that any link can be obtained as the trace closure of some braid. Another natural closure (for braids with an even number of strands) is the *plat* closure, which connects the first and second strands, the third and fourth strands, etc., at each end of the braid.

16.4 Representing braids in the Temperley-Lieb algebra

The Jones polynomial of the plat or trace closure of a braid can be expressed in terms of a representation of the braid group defined over an algebra called the Temperley-Lieb algebra. While the definition of this algebra is fairly straightforward, the description of its representations is somewhat technical, and we will not give the details here; instead we only mention some general features.

We consider the case where $t = e^{2\pi i/k}$ is a kth root of unity. For such values, the relevant representation of the braid group is unitary. The dimension of this representation is exponential in n (specifically, it is the

number of paths of length n that start from one end of a path with $k-1$ vertices), so it corresponds to a unitary operation on $\mathrm{poly}(n)$ qubits. The Jones polynomial of the plat closure of a braid is proportional to the expectation $\langle\psi|U|\psi\rangle$ of the associated representation matrix U in a fixed quantum state $|\psi\rangle$.

16.5 A quantum algorithm

The description of the Jones polynomial in terms of a representation of the Temperley-Lieb algebra naturally suggests a quantum algorithm for approximating the Jones polynomial. Suppose that we can efficiently implement unitary operations corresponding to twists of adjacent strands on a quantum computer. By composing such operations, we can implement a unitary operation corresponding to the entire braid. Then we can approximate the desired expectation value using the Hadamard test.

With a suitable choice for an encoding of the basis states of the representation of the braid group using qubits, one can show that the braid group representation operators corresponding to elementary twists can indeed be performed efficiently on a quantum computer. Given an explicit description of the braid group representation, the details of this implementation are fairly straightforward.

Applying this approach to the relevant unitary representation of the braid group, one obtains a quantum algorithm for approximating the Jones polynomial of the plat closure of a braid at a root of unity. In particular, for a braid on n strands, with m crossings, and with $t = e^{2\pi i/k}$, there is an algorithm running in time $\mathrm{poly}(n, m, k)$ that outputs an approximation differing from the actual value $V_L(t)$ of the Jones polynomial by at most $(2\cos\frac{\pi}{k})^{3n/2}/(N \cdot \mathrm{poly}(n, k, m))$, with only exponentially small probability of failure. Here N is an exponentially larger factor derived from the representation of the braid group.

The Jones polynomial of the trace closure of a braid can be similarly approximated by noting that this quantity is given by the *Markov trace* of the representation of the braid. The Markov trace is simply a weighted version of the usual trace, so it can be approximated by sampling $\langle\psi_p|U|\psi_p\rangle$ from an appropriate distribution over states $|\psi_p\rangle$. Performing such a procedure, one obtains an approximation of the Jones polynomial with additive error at most $(2\cos\frac{\pi}{k})^{n-1}/\mathrm{poly}(n, k, m)$, again in polynomial time and with exponentially small failure probability.

16.6 Quality of approximation

Without knowing more about the possible values of the Jones polynomial, it is hard to say whether the approximations described above are good. Notice that the algorithms only provide additive approximations, meaning that the error incurred by the algorithm is independent of the value being approximated, which is undesirable when that value is small. Indeed, the additive error increases exponentially with n, the number of strands in the braid. For some braids, the error might be larger than the value being approximated. It would be preferable to obtain a multiplicative approximation, but no such algorithm is known.

However, it can be shown that obtaining the additive approximation described above for the Jones polynomial of the plat closure of a braid is as hard as any quantum computation. In other words, this quality of Jones polynomial approximation is BQP-complete. This can be shown by demonstrating that, with an appropriate encoding of qubits, the representations of the braid group can be used to implement a universal set of quantum gates. Thus, in principle, any quantum algorithm can be described in terms of some braid whose plat closure has a Jones polynomial encoding the result of the computation, with exponentially differing values corresponding to yes and no outcomes. Therefore, it is unlikely that a classical computer can obtain the same approximation, since this would give a classical algorithm for simulating a general quantum computation.

Approximating the Jones polynomial of the trace closure of a braid to the level described above turns out to be substantially easier: such a computation can be performed using a quantum computer whose initial state has only one pure qubit and many maximally mixed qubits. Such a device can approximate $\mathrm{tr}\, U$ by supplying the maximally mixed state in place of the pure state $|\psi\rangle$ in the Hadamard test. This does not immediately show how to approximate the Jones polynomial of the trace closure, since the Markov trace is a weighted trace. However, by using a different representation of the braid group to describe the Jones polynomial, Jordan and Shor showed that a single pure qubit indeed suffices. Furthermore, they showed

that this problem is complete for the one clean qubit model, and hence apparently unlikely to be solvable by classical computers.

16.7 Other algorithms

The results described above can be generalized to many other related problems. Wocjan and Yard showed how to evaluate the Jones polynomial of a generalized closure of a braid, and how to evaluate a generalization of the Jones polynomial called the HOMFLYPT polynomial [113]. Work of Aharonov, Arad, Eban, and Landau shows how to approximate the Tutte polynomial of a planar graph, which in particular gives an approximation of the partition function of the Potts model on a planar graph; this problem also characterizes the power of quantum computation, albeit only for unphysical choices of parameters [4]. More generally, there are efficient quantum algorithms to compute additive approximations of tensor networks, as shown by Arad and Landau [13]. There are also related quantum algorithms for approximating invariants of manifolds.

Part III

Quantum walk

Chapter 17

Continuous-time quantum walk

We now turn to our second major topic in quantum algorithms, the concept of quantum walk. In this lecture we will introduce continuous-time quantum walk as a natural analog of continuous-time classical random walk, and we'll see some examples of how the two kinds of processes differ.

17.1 Continuous-time quantum walk

Random walks come in two flavors: discrete- and continuous-time. It is easiest to define a quantum analog of a continuous-time random walk [49], so we consider this case first. Given a graph $G = (V, E)$, we define the continuous-time random walk on G as follows. Let A be the *adjacency matrix* of G, the $|V| \times |V|$ matrix with

$$A_{j,k} = \begin{cases} 1 & (j,k) \in E \\ 0 & (j,k) \notin E \end{cases} \tag{17.1}$$

for every pair $j, k \in V$. In particular, if we disallow self loops, then the diagonal of A is zero. There is another matrix associated with G that is nearly as important: the *Laplacian* of G, which has

$$L_{j,k} = \begin{cases} -\deg(j) & j = k \\ 1 & (j,k) \in E \\ 0 & \text{otherwise} \end{cases} \tag{17.2}$$

where $\deg(j)$ denotes the degree of vertex j. (The Laplacian is sometimes defined differently than this—e.g., sometimes with the opposite sign. We use this definition because it makes L a discrete approximation of the Laplacian operator ∇^2 in the continuum.)

The continuous-time random walk on G is defined as the solution of the differential equation

$$\frac{\mathrm{d}}{\mathrm{d}t} p_j(t) = \sum_{k \in V} L_{jk}\, p_k(t). \tag{17.3}$$

Here $p_j(t)$ denotes the probability associated with vertex j at time t. This can be viewed as a discrete analog of the diffusion equation. Note that

$$\frac{\mathrm{d}}{\mathrm{d}t} \sum_{j \in V} p_j(t) = \sum_{j,k \in V} L_{jk}\, p_k(t) = 0 \tag{17.4}$$

(since the columns of L sum to 0), which shows that an initially normalized distribution remains normalized: the evolution of the continuous-time random walk for any time t is a *stochastic process*. The solution of the differential equation can be given in closed form as

$$p(t) = e^{Lt} p(0). \tag{17.5}$$

Now notice that the equation (17.3) is very similar to the Schrödinger equation

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}t}|\psi\rangle = H|\psi\rangle \tag{17.6}$$

except that it lacks the factor of i. If we simply insert this factor, and rename the probabilities $p_j(t)$ as quantum amplitudes $q_j(t) = \langle j|\psi(t)\rangle$ (where $\{|j\rangle \colon j \in V\}$ is an orthonormal basis for the Hilbert space), then we obtain the equation

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}t}q_j(t) = \sum_{k\in V} L_{jk}\, q_k(t), \tag{17.7}$$

which is simply the Schrödinger equation with the Hamiltonian given by the Laplacian of the graph. Since the Laplacian is a Hermitian operator, these dynamics preserve normalization in the sense that $\frac{\mathrm{d}}{\mathrm{d}t}\sum_{j\in V}|q_j(t)|^2 = 0$. Again the solution of the differential equation can be given in closed form, but here it is $|\psi(t)\rangle = e^{-iLt}|\psi(0)\rangle$.

We could also define a continuous-time quantum walk using any Hermitian Hamiltonian that respects the structure of G. For example, we could use the adjacency matrix A of G, even though this matrix cannot be used as the generator of a continuous-time classical random walk.

17.2 Random and quantum walks on the hypercube

Let's begin by investigating a simple, dramatic example of a difference between the behavior of random and quantum walks. Consider the Boolean hypercube, the graph with vertex set $V = \{0,1\}^n$ and edge set $E = \{(x,y) \in V^2 \colon \Delta(x,y) = 1\}$, where $\Delta(x,y)$ denotes the Hamming distance between the strings x and y. When $n = 1$, the hypercube is simply an edge, with adjacency matrix

$$\sigma_x := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}. \tag{17.8}$$

For general n, the graph is the Cartesian product of this graph with itself n times, and the adjacency matrix is

$$A = \sum_{j=1}^{n} \sigma_x^{(j)} \tag{17.9}$$

where $\sigma_x^{(j)}$ denotes the operator acting as σ_x on the jth bit, and as the identity on every other bit.

For simplicity, let's consider the quantum walk with the Hamiltonian given by the adjacency matrix. (In fact, since the graph is regular, the walk generated by the Laplacian would only differ by an overall phase.) Since the terms in the above expression for the adjacency matrix commute, the unitary operator describing the evolution of this walk is simply

$$e^{-\mathrm{i}At} = \prod_{j=1}^{n} e^{-\mathrm{i}\sigma_x^{(j)}t} \tag{17.10}$$

$$= \bigotimes_{j=1}^{n} \begin{pmatrix} \cos t & -\mathrm{i}\sin t \\ -\mathrm{i}\sin t & \cos t \end{pmatrix}. \tag{17.11}$$

After time $t = \pi/2$, this operator flips every bit of the state (up to an overall phase), mapping any input state $|x\rangle$ to the state $|\bar{x}\rangle$ corresponding to the opposite vertex of the hypercube.

In contrast, consider the continuous- or discrete-time random walk starting from the vertex x. It is not hard to show that the probability of reaching the opposite vertex $\bar{x}$ is exponentially small at any time, since the walk rapidly reaches the uniform distribution over all 2^n vertices of the hypercube. So this simple example shows that random and quantum walks can exhibit radically different behavior.

17.3 Random and quantum walks in one dimension

Perhaps the best-known example of a random walk is the case of an infinite path, with $V = \mathbb{Z}$ and $(j,k) \in E$ iff $|j-k| = 1$. It is well known that the random walk on this graph starting from the origin (in either continuous or discrete time) typically moves a distance proportional to $\sqrt{t}$ in time t. Now let's consider the corresponding quantum walk.

To calculate the behavior of the walk, it is helpful to diagonalize the Hamiltonian. The eigenstates of the Laplacian of the graph are the momentum states $|\hat{p}\rangle$ with components

$$\langle j|\hat{p}\rangle = e^{\mathrm{i}pj} \tag{17.12}$$

where $-\pi \leq p \leq \pi$. We have

$$\begin{aligned}
\langle j|L|\hat{p}\rangle &= \langle j+1|\hat{p}\rangle + \langle j-1|\hat{p}\rangle - 2\langle j|\hat{p}\rangle && (17.13)\\
&= (e^{\mathrm{i}p(j+1)} + e^{\mathrm{i}p(j-1)} - 2e^{\mathrm{i}pj}) && (17.14)\\
&= e^{\mathrm{i}pj}(e^{\mathrm{i}p} + e^{-\mathrm{i}p} - 2) && (17.15)\\
&= 2(\cos p - 1)\langle j|\hat{p}\rangle, && (17.16)
\end{aligned}$$

so the corresponding eigenvalue is $2(\cos p - 1)$. Thus the amplitude for the walk to move from j to k in time t is

$$\begin{aligned}
\langle k|e^{-\mathrm{i}Lt}|j\rangle &= \frac{1}{2\pi}\int_{-\pi}^{\pi} e^{-2\mathrm{i}t(\cos p - 1)}\langle k|\hat{p}\rangle\langle\hat{p}|j\rangle \,\mathrm{d}p && (17.17)\\
&= \frac{1}{2\pi}\int_{-\pi}^{\pi} e^{ip(k-j)-2it(\cos p-1)} \,\mathrm{d}p && (17.18)\\
&= e^{2\mathrm{i}t}(-\mathrm{i})^{k-j} J_{k-j}(2t) && (17.19)
\end{aligned}$$

where J_ν is the Bessel function of order ν. This expression can be understood using basic asymptotic properties of the Bessel function. For large values of ν, the function $J_\nu(t)$ is exponentially small in ν for $\nu \gg t$, of order $\nu^{-1/3}$ for $\nu \approx t$, and of order $\nu^{-1/2}$ for $\nu \ll t$. Thus (17.19) describes a wave propagating with speed 2.

We can use a similar calculation to exactly describe the corresponding continuous-time classical random walk, which is simply the analytic continuation of the quantum case with $t \to \mathrm{i}t$. Here the probability of moving from j to k in time t is

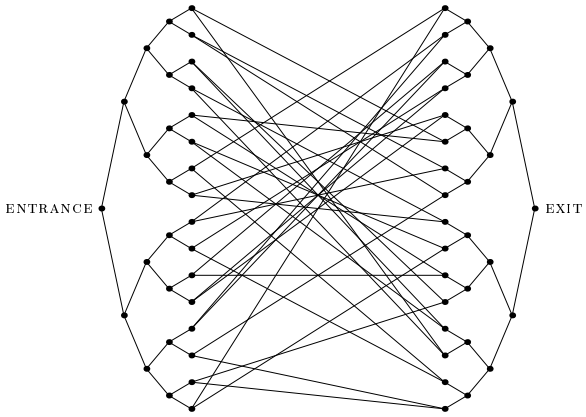
$$[e^{Lt}]_{kj} = e^{-2t} I_{k-j}(2t), \tag{17.20}$$

where I_ν is the modified Bessel function of order ν. For large t, this expression is approximately $\frac{1}{\sqrt{4\pi t}} \exp(-(k-j)^2/4t)$, a Gaussian of width $\sqrt{2t}$, in agreement with our expectations for a classical random walk in one dimension.

17.4 Black-box traversal of the glued trees graph

We have seen that the behavior of a quantum walk can be dramatically different from that of its classical counterpart. Next we will see an even stronger example of the power of quantum walk: a black-box problem that can be solved exponentially faster by a quantum walk than by *any* classical algorithm [34].

Consider a graph obtained by starting from two balanced binary trees of height n, and joining them by a random cycle of length $2 \cdot 2^n$ that alternates between the leaves of the two trees. For example, such a graph for $n = 4$ could look like the following:



Suppose we take a random walk on the graph starting from the root of the left tree. It is not hard to see that such a walk rapidly gets lost in the middle of the graph and never has a substantial probability of reaching the opposite root. In fact, by specifying the graph in such a way that it can only be explored locally, we can ensure that no classical procedure starting from the left root can efficiently reach the right root. However, a quantum walk starting from the left root produces a state with a large (lower bounded by $1/\operatorname{poly}(n)$) overlap on the right root in a short (upper bounded by $\operatorname{poly}(n)$) amount of time.

To establish a provable separation between classical and quantum strategies, we will formulate the graph traversal problem in terms of query complexity.

Let $G = (V, E)$ be a graph with N vertices. To represent G by a black box, let m be such that $2^m \geq N$, and let k be at least as large as the maximum degree of G. For each vertex $a \in V$, assign a distinct m-bit string (called the *name* of a), not assigning $11\ldots1$ as the name of any vertex. For each $b \in V$ with $(a, b) \in E$, assign a unique label from $\{1, 2, \ldots, k\}$ to the ordered pair (a, b). For $a \in \{0,1\}^m$ (identifying the vertex with its name) and $c \in \{1, 2, \ldots, k\}$, define $v_c(a)$ as the name of the vertex reached by following the outgoing edge of a labeled by c, if such an edge exists. If there is no vertex of G named a or no outgoing edge from a labeled c, then let $v_c(a) = 11\ldots1$. The black box for G takes $a \in \{0,1\}^m$ and $c \in \{1, 2, \ldots, k\}$ as input and returns $v_c(a)$.

The black box graph traversal problem is as follows. Let G be a graph and let ENTRANCE and EXIT be two vertices of G. Given a black box for G as described above, with the additional promise that the name of the ENTRANCE is $00\ldots0$, the goal is to output the name of the EXIT. We say an algorithm for this problem is efficient if its running time is polynomial in m.

Of course, a random walk is not necessarily the best classical strategy for this problem. For example, there is an efficient classical algorithm for traversing the n-dimensional hypercube (exercise: what is it?) even though a random walk does not work. However, no classical algorithm can efficiently traverse the glued trees, whereas a quantum walk can.

17.5 Quantum walk algorithm to traverse the glued trees graph

Given a black box for a graph G as specified above, we can efficiently compute a list of neighbors of any desired vertex, provided $k = \operatorname{poly}(m)$ (i.e., provided the maximum degree of the graph is not too large). Thus it is straightforward to simulate the dynamics of the continuous-time quantum walk on any such G, and in particular, on the glued trees graph (which has maximum degree 3). Our strategy for solving the traversal problem is simply to run the quantum walk and show that the resulting state has a substantial overlap on the EXIT for some $t = \operatorname{poly}(n)$.

Let G be the glued trees graph. The dynamics of the quantum walk on this graph are dramatically simplified because of symmetry. Consider the basis of states $|\operatorname{col} j\rangle$ that are uniform superpositions over the vertices at distance j from the ENTRANCE, i.e.,

$$|\operatorname{col} j\rangle := \frac{1}{\sqrt{N_j}} \sum_{\delta(a,\text{ENTRANCE})=j} |a\rangle \tag{17.21}$$

where

$$N_j := \begin{cases} 2^j & 0 \le j \le n \\ 2^{2n+1-j} & n+1 \le j \le 2n+1 \end{cases} \tag{17.22}$$

is the number of vertices at distance j from the ENTRANCE, and where $\delta(a,b)$ denotes the length of the shortest path in G from a to b. It is straightforward to see that the subspace $\text{span}\{|\text{col}\, j\rangle : 0 \le j \le 2n+1\}$ is invariant under the action of the adjacency matrix A of G. At the ENTRANCE and EXIT, we have

$$A|\text{col}\, 0\rangle = \sqrt{2}|\text{col}\, 1\rangle \tag{17.23}$$

$$A|\text{col}\, 2n+1\rangle = \sqrt{2}|\text{col}\, 2n\rangle. \tag{17.24}$$

For any $0 < j < n$, we have

$$A|\text{col}\, j\rangle = \frac{1}{\sqrt{N_j}} \sum_{\delta(a,\text{ENTRANCE})=j} A|a\rangle \tag{17.25}$$

$$= \frac{1}{\sqrt{N_j}} \left(2 \sum_{\delta(a,\text{ENTRANCE})=j-1} |a\rangle + \sum_{\delta(a,\text{ENTRANCE})=j+1} |a\rangle \right) \tag{17.26}$$

$$= \frac{1}{\sqrt{N_j}} (2\sqrt{N_{j-1}}|\text{col}\, j-1\rangle + \sqrt{N_{j+1}}|\text{col}\, j+1\rangle) \tag{17.27}$$

$$= \sqrt{2}(|\text{col}\, j-1\rangle + |\text{col}\, j+1\rangle). \tag{17.28}$$

Similarly, for any $n+1 < j < 2n+1$, we have

$$A|\text{col}\, j\rangle = \frac{1}{\sqrt{N_j}} (\sqrt{N_{j-1}}|\text{col}\, j-1\rangle + 2\sqrt{N_{j+1}}|\text{col}\, j+1\rangle) \tag{17.29}$$

$$= \sqrt{2}(|\text{col}\, j-1\rangle + |\text{col}\, j+1\rangle). \tag{17.30}$$

The only difference occurs at the middle of the graph, where we have

$$A|\text{col}\, n\rangle = \frac{1}{\sqrt{N_n}} (2\sqrt{N_{n-1}}|\text{col}\, n-1\rangle + 2\sqrt{N_{n+1}}|\text{col}\, n+1\rangle) \tag{17.31}$$

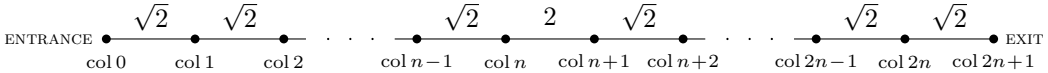
$$= \sqrt{2}|\text{col}\, n-1\rangle + 2|\text{col}\, n+1\rangle \tag{17.32}$$

and similarly

$$A|\text{col}\, n+1\rangle = \frac{1}{\sqrt{N_{n+1}}} (2\sqrt{N_n}|\text{col}\, n\rangle + 2\sqrt{N_{n+2}}|\text{col}\, n+2\rangle) \tag{17.33}$$

$$= 2|\text{col}\, n\rangle + \sqrt{2}|\text{col}\, n+2\rangle. \tag{17.34}$$

In summary, the matrix elements of A between basis states for this invariant subspace can be depicted as follows:



By identifying the subspace of states $|\text{col}\, j\rangle$, we have found that the quantum walk on the glued trees graph starting from the ENTRANCE is effectively the same as a quantum walk on a weighted path of $2n+2$ vertices, with all edge weights the same except for the middle one. Given our example of the quantum walk on the infinite path, we can expect this walk to reach the EXIT with amplitude $1/\text{poly}(n)$ in time linear in n. To prove that the walk indeed reaches the EXIT in polynomial time, we will use the notion of the *mixing time* of a quantum walk.

17.6 Classical and quantum mixing

Informally, the mixing time of a random walk is the amount of time it takes to come close to a stationary distribution. Recall that the continuous-time random walk on a graph $G = (V, E)$ with Laplacian L is defined as the solution of the differential equation $\frac{\mathrm{d}p(t)}{\mathrm{d}t} = Lp(t)$, where $p(t) \in \mathbb{R}^{|V|}$ denotes a vector of probabilities for the walk to be at each vertex at time t. The uniform distribution over the vertices, $u := (1, 1, \ldots, 1)/|V|$, is an eigenvector of L with eigenvalue 0. Indeed, if G is connected, then this is the unique eigenvector with this eigenvalue. Letting v_λ denote a normalized eigenvector of L with eigenvalue λ (so that $L = \sum_{\lambda \neq 0} \lambda v_\lambda v_\lambda^T$), we have

$$p(t) = e^{Lt} p(0) \tag{17.35}$$

$$= \left(|V| u u^T + \sum_{\lambda \neq 0} e^{\lambda t} v_\lambda v_\lambda^T \right) p(0) \tag{17.36}$$

$$= \langle |V| u, p(0) \rangle u + \sum_{\lambda \neq 0} e^{\lambda t} \langle v_\lambda, p(0) \rangle v_\lambda \tag{17.37}$$

$$= u + \sum_{\lambda \neq 0} e^{\lambda t} \langle v_\lambda, p(0) \rangle v_\lambda \tag{17.38}$$

(where in exponentiating L we have used the fact that $\sqrt{|V|}u$ is a normalized eigenvector of L, so that $|V|uu^T$ is the projector onto the corresponding subspace). The Laplacian is a negative semidefinite operator, so the contributions $e^{\lambda t}$ for $\lambda \neq 0$ decrease exponentially in time; thus the walk asymptotically approaches the uniform distribution. The deviation from uniform is small when t is large compared to the inverse of the largest (i.e., least negative) nonzero eigenvalue of L.

Since a quantum walk is a unitary process, we should not expect it to approach a limiting quantum state, no matter how long we wait. Nevertheless, it is possible to define a notion of the limiting distribution of a quantum walk as follows. Suppose we pick a time t uniformly at random between 0 and T, run the quantum walk starting at $a \in V$ for a total time t, and then measure in the vertex basis. The resulting distribution is

$$p_{a \to b}(T) = \frac{1}{T} \int_0^T |\langle b | e^{-iHt} | a \rangle|^2 \mathrm{d}t \tag{17.39}$$

$$= \sum_{\lambda, \lambda'} \langle b|\lambda\rangle\langle\lambda|a\rangle\langle a|\lambda'\rangle\langle\lambda'|b\rangle \frac{1}{T} \int_0^T e^{-i(\lambda - \lambda')t} \mathrm{d}t \tag{17.40}$$

$$= \sum_{\lambda} |\langle a|\lambda\rangle\langle b|\lambda\rangle|^2 + \sum_{\lambda \neq \lambda'} \langle b|\lambda\rangle\langle\lambda|a\rangle\langle a|\lambda'\rangle\langle\lambda'|b\rangle \frac{1 - e^{-i(\lambda - \lambda')T}}{i(\lambda - \lambda')T} \tag{17.41}$$

where we have considered a quantum walk generated by an unspecified Hamiltonian H (it could be the Laplacian or the adjacency matrix, or some other operator as desired), and where we have assumed for simplicity that the spectrum of $H = \sum_\lambda \lambda |\lambda\rangle\langle\lambda|$ is nondegenerate. We see that the distribution $p_{a \to b}(T)$ tends toward a limiting distribution

$$p_{a \to b}(\infty) := \sum_{\lambda} |\langle a|\lambda\rangle\langle b|\lambda\rangle|^2. \tag{17.42}$$

The timescale for approaching this distribution is again governed by the spectrum of H, but now we see that T must be large compared to the inverse of the smallest gap between any pair of distinct eigenvalues, not just the smallest gap between a particular pair of eigenvalues as in the classical case.

Let's apply this notion of quantum mixing to the quantum walk on the glued trees. It will be simplest to consider the walk generated by the adjacency matrix A. Since the subspace of states $|\mathrm{col}\, j\rangle$ has dimension only $2n + 1$, it should not be surprising that the limiting probability of traversing from ENTRANCE to EXIT is bigger than $1/\operatorname{poly}(n)$. To see this, notice that A commutes with the reflection operator R defined as $R|\mathrm{col}\, j\rangle = |\mathrm{col}\, 2n + 1 - j\rangle$, so these two operators can be simultaneously diagonalized. Now $R^2 = 1$, so it

has eigenvalues ± 1, which shows that we can choose the eigenstates $|\lambda\rangle$ of A to satisfy $\langle \textsc{entrance}|\lambda\rangle = \pm\langle \textsc{exit}|\lambda\rangle$. Therefore,

$$p_{\textsc{entrance}\to\textsc{exit}}(\infty) = \sum_\lambda |\langle \textsc{entrance}|\lambda\rangle\langle \textsc{exit}|\lambda\rangle|^2 \tag{17.43}$$

$$= \sum_\lambda |\langle \textsc{entrance}|\lambda\rangle|^4 \tag{17.44}$$

$$\geq \frac{1}{2n+2}\left(\sum_\lambda |\langle \textsc{entrance}|\lambda\rangle|^2\right)^2 \tag{17.45}$$

$$= \frac{1}{2n+2} \tag{17.46}$$

where the lower bound follows by the Cauchy-Schwarz inequality. Thus it suffices to show that the mixing time of the quantum walk is $\mathrm{poly}(n)$.

To see how long we must wait before the probability of reaching the EXIT is close to its limiting value, we can calculate

$$|p_{\textsc{entrance}\to\textsc{exit}}(\infty) - p_{\textsc{entrance}\to\textsc{exit}}(T)|$$

$$= \left|\sum_{\lambda\neq\lambda'} \langle \textsc{exit}|\lambda\rangle\langle\lambda|\textsc{entrance}\rangle\langle\textsc{entrance}|\lambda'\rangle\langle\lambda'|\textsc{exit}\rangle \frac{1-e^{-i(\lambda-\lambda')T}}{i(\lambda-\lambda')T}\right| \tag{17.47}$$

$$\leq \frac{2}{\Delta T}\sum_{\lambda,\lambda'} |\langle \textsc{exit}|\lambda\rangle\langle\lambda|\textsc{entrance}\rangle\langle\textsc{entrance}|\lambda'\rangle\langle\lambda'|\textsc{exit}\rangle| \tag{17.48}$$

$$= \frac{2}{\Delta T}\sum_{\lambda,\lambda'} |\langle \textsc{entrance}|\lambda\rangle|^2 |\langle \textsc{entrance}|\lambda'\rangle|^2 \tag{17.49}$$

$$= \frac{2}{\Delta T}, \tag{17.50}$$

where Δ denotes the smallest gap between any pair of distinct eigenvalues of A. All that remains is to lower bound Δ.

To understand the spectrum of A, recall that an infinite path has eigenstates of the form e^{ipj}. For any value of p, the state $|\lambda\rangle$ with amplitudes $\langle \mathrm{col}\, j|\lambda\rangle = e^{ipj}$ satisfies $\langle \mathrm{col}\, j|A|\lambda\rangle = \lambda\langle \mathrm{col}\, j|\lambda\rangle$, where the eigenvalue is $\lambda = 2\sqrt{2}\cos p$, for all values of j except $0, n, n+1, 2n+1$. We can satisfy the eigenvalue condition for $j = 0, 2n+1$ by taking linear combinations of $e^{\pm ipj}$ that vanish for $j = -1$ and $j = 2n+2$, namely

$$\langle \mathrm{col}\, j|\lambda\rangle = \begin{cases} \sin(p(j+1)) & 0 \leq j \leq n \\ \pm\sin(p(2n+2-j)) & n+1 \leq j \leq 2n+1. \end{cases} \tag{17.51}$$

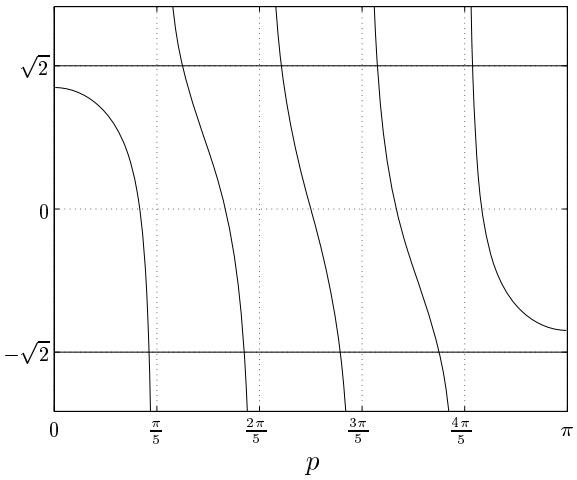
Finally, we can enforce the eigenvalue condition at $j = n$ (which automatically enforces it at $j = n+1$ by symmetry), which restricts the values of p to a finite set. We have

$$\sqrt{2}\sin(pn) \pm 2\sin(p(n+1)) = 2\sqrt{2}\cos(p)\sin(p(n+1)), \tag{17.52}$$

which can be simplified to

$$\frac{\sin(p(n+2))}{\sin(p(n+1))} = \pm\sqrt{2}. \tag{17.53}$$

The left hand side of this equation decreases monotonically, with poles at integer multiples of $\pi/(n+1)$. For example, with $n = 4$, we have the following:



With a bit of analysis (see quant-ph/0209131 for details), one can show that the solutions of this equation give $2n$ values of p, each of which is separated from the integer multiples of $\pi/(n+1)$ by $\Omega(1/n^2)$. The spacings between the corresponding eigenvalues of A, $\lambda = 2\sqrt{2}\cos p$, are $\Omega(1/n^3)$. The remaining two eigenvalues of A can be obtained by considering solutions with p imaginary, and it is easy to show that they are separated from the rest of the spectrum by a constant amount. By taking (say) $T = 5n/\Delta = O(n^4)$, we can ensure that the probability to reach the EXIT is $\Omega(1/n)$. Thus there is an efficient quantum algorithm to traverse the glued trees graph.

17.7 Classical lower bound

It remains to show that this problem is difficult for a classical computer. A formal proof of this fact can be given using a sequence of reductions to problems that are essentially no easier than the original one, but that restrict the nature of the allowed algorithms. Here we will simply sketch the main ideas.

First, note that if we name the vertices at random using strings of about, say, $2\log|V|$ bits, then there are exponentially many more possible names than there are actual vertices. Since the probability that a randomly guessed name corresponds to a vertex of the graph is exponentially small, we can essentially restrict our attention to algorithms that query a connected set of vertices, starting from the ENTRANCE (the only vertex whose name is known initially).

Next, suppose we consider the algorithm to succeed not only if it reaches the EXIT, but also if it manages to find a cycle in the graph. This only makes it easier for the algorithm to succeed, but not significantly so, since it turns out to be hard even to find a cycle.

Now we can restrict our attention to the steps the algorithm takes before it finds a cycle. Notice that for such steps, the names supplied by the black box provide no information whatsoever about the structure of the graph: they could just as well be simulated by a sequence of random responses. Therefore, we can think of an algorithm as simply producing a rooted binary tree and embedding it into the glued trees graph at random. To show that the algorithm fails, it suffices to show that under such a random embedding, the probability of any rooted binary tree giving rise to a cycle or reaching the exit is small. By a fairly straightforward probabilistic argument, one can show that even for exponentially large trees (say, having at most $2^{n/6}$ vertiecs), the probability of the embedded tree giving rise to a cycle or reaching the exit is exponentially small. Thus any classical algorithm for solving the black box glued trees traversal problem must make exponentially many queries to succeed with more than exponentially small probability.

Chapter 18

Discrete-time quantum walk

In the last lecture we introduced the notion of continuous-time quantum walk. We now turn our attention to discrete-time quantum walk, which provides a convenient framework for quantum search algorithms.

18.1 Discrete-time quantum walk

It is trickier to define a quantum analog of a discrete-time random walk than of a continuous-time random walk. In the simplest discrete-time random walk on G, at each time step we simply move from any given vertex to each of its neighbors with equal probability. Thus the walk is governed by the $|V| \times |V|$ matrix M with entries

$$M_{jk} = \begin{cases} 1/\deg(k) & (j,k) \in E \\ 0 & \text{otherwise.} \end{cases} \tag{18.1}$$

for $j, k \in V$: an initial probability distribution p over the vertices evolves to $p' = Mp$ after one step of the walk.

To define a quantum analog of this process, we would like to specify a unitary operator U with the property that an input state $|j\rangle$ corresponding to the vertex $j \in V$ evolves to a superposition of the neighbors of j. We would like this to happen in essentially the same way at every vertex, so we are tempted to propose the definition

$$|j\rangle \stackrel{?}{\mapsto} |\partial_j\rangle := \frac{1}{\sqrt{\deg(j)}} \sum_{k:(j,k)\in E} |k\rangle. \tag{18.2}$$

However, a moment's reflection shows that this typically does not define a unitary transformation, since the orthogonal states $|j\rangle$ and $|k\rangle$ corresponding to adjacent vertices j, k with a common neighbor ℓ evolve to non-orthogonal states. We could potentially avoid this problem using a rule that sometimes introduces phases, but that would violate the spirit of defining a process that behaves in the same way at every vertex. In fact, even if we give that up, there are some graphs that simply do not allow local unitary dynamics [102].

We can get around this difficulty if we allow ourselves to enlarge the Hilbert space, an idea proposed by Watrous as part of a logarithmic-space quantum algorithm for deciding whether two vertices are connected in a graph [111]. Let the Hilbert space consist of states of the form $|j,k\rangle$ where $(j,k) \in E$. We can think of the walk as taking place on the (directed) edges of the graph; the state $|j,k\rangle$ represents a walker at vertex j that will move toward vertex k. Each step of the walk consists of two operations. First, we apply a unitary transformation that operates on the second register conditional on the first register. This transformation is sometimes referred to as a "coin flip," as it modifies the next destination of the walker. A common choice is the Grover diffusion operator over the neighbors of j, namely

$$C := \sum_{j\in V} |j\rangle\langle j| \otimes \big(2|\partial_j\rangle\langle\partial_j| - I\big). \tag{18.3}$$

Next, the walker is moved to the vertex indicated in the second register. Of course, since the process must

be unitary, the only way to do this is to swap the two registers using the operator

$$S := \sum_{(j,k)\in E} |j,k\rangle\langle k,j|. \tag{18.4}$$

Overall, one step of the discrete-time quantum walk is described by the unitary operator SC.

In principle, this construction can be used to define a discrete-time quantum walk on any graph (although care must be taken if the graph is not regular). However, in practice it is often more convenient to use an alternative framework introduced by Szegedy [109], as described in the next section.

18.2 How to quantize a Markov chain

A discrete-time classical random walk on an N-vertex graph can be represented by an $N \times N$ matrix P. The entry P_{kj} represents the probability of making a transition to k from j, so that an initial probability distribution $p \in \mathbb{R}^N$ becomes Pp after one step of the walk. To preserve normalization, we must have $\sum_{k=1}^N P_{kj} = 1$ for any $j \in \{1,\ldots,N\}$; we say that such a matrix is *stochastic*.

For any $N \times N$ stochastic matrix P (not necessarily symmetric), we can define a corresponding discrete-time quantum walk, a unitary operation on the Hilbert space $\mathbb{C}^N \otimes \mathbb{C}^N$. To define this walk, we introduce the states

$$|\psi_j\rangle := |j\rangle \otimes \sum_{k=1}^N \sqrt{P_{kj}}|k\rangle \tag{18.5}$$

$$= \sum_{k=1}^N \sqrt{P_{kj}}|j,k\rangle \tag{18.6}$$

for $j = 1,\ldots,N$. Each such state is normalized since P is stochastic. Now let

$$\Pi := \sum_{j=1}^N |\psi_j\rangle\langle\psi_j| \tag{18.7}$$

denote the projection onto $\mathrm{span}\{|\psi_j\rangle : j = 1,\ldots,N\}$, and let

$$S := \sum_{j,k=1}^N |j,k\rangle\langle k,j| \tag{18.8}$$

be the operator that swaps the two registers. Then a single step of the quantum walk is defined as the unitary operator $U := S(2\Pi - 1)$.

Notice that if $P_{jk} = A_{jk}/\deg(k)$ (i.e., if the walk simply chooses an outgoing edge of an underlying digraph uniformly at random), then this is exactly the coined quantum walk with the Grover diffusion operator as the coin flip.

If we take two steps of the walk, then the corresponding unitary operator is

$$[S(2\Pi - 1)][S(2\Pi - 1)] = [S(2\Pi - 1)S][2\Pi - 1] \tag{18.9}$$

$$= (2S\Pi S - 1)(2\Pi - 1), \tag{18.10}$$

which can be interpreted as the reflection about $\mathrm{span}\{|\psi_j\rangle\}$ followed by the reflection about $\mathrm{span}\{S|\psi_j\rangle\}$ (the states where we condition on the second register to do a coin operation on the first). To understand the behavior of the walk, we will now compute the spectrum of U; but note that it is also possible to compute the spectrum of a product of reflections more generally.

18.3 Spectrum of the quantum walk

To understand the behavior of a discrete-time quantum walk, it will be helpful to compute its spectral decomposition. Let us show the following:

Theorem 18.1. *Fix an $N \times N$ stochastic matrix P, and let $\{|\lambda\rangle\}$ denote a complete set of orthonormal eigenvectors of the $N \times N$ matrix D with entries $D_{jk} = \sqrt{P_{jk}P_{kj}}$ with eigenvalues $\{\lambda\}$. Then the eigenvalues of the discrete-time quantum walk $U = S(2\Pi - 1)$ corresponding to P are ± 1 and $\lambda \pm i\sqrt{1-\lambda^2} = e^{\pm i \arccos \lambda}$.*

Proof. Define an isometry

$$T := \sum_{j=1}^{N} |\psi_j\rangle\langle j| \tag{18.11}$$

$$= \sum_{j,k=1}^{N} \sqrt{P_{kj}}|j,k\rangle\langle j| \tag{18.12}$$

mapping states in $\mathbb{C}^N$ to states in $\mathbb{C}^N \otimes \mathbb{C}^N$, and let $|\tilde{\lambda}\rangle := T|\lambda\rangle$. Notice that

$$TT^\dagger = \sum_{j,k=1}^{N} |\psi_j\rangle\langle j|k\rangle\langle\psi_k| \tag{18.13}$$

$$= \sum_{j=1}^{N} |\psi_j\rangle\langle\psi_j| \tag{18.14}$$

$$= \Pi, \tag{18.15}$$

whereas

$$T^\dagger T = \sum_{j,k=1}^{N} |j\rangle\langle\psi_j|\psi_k\rangle\langle k| \tag{18.16}$$

$$= \sum_{j,k,\ell,m=1}^{N} \sqrt{P_{\ell j}P_{mk}}|j\rangle\langle j,\ell|k,m\rangle\langle k| \tag{18.17}$$

$$= \sum_{j,\ell=1}^{N} P_{\ell j}|j\rangle\langle j| \tag{18.18}$$

$$= I \tag{18.19}$$

and

$$T^\dagger ST = \sum_{j,k=1}^{N} |j\rangle\langle\psi_j|S|\psi_k\rangle\langle k| \tag{18.20}$$

$$= \sum_{j,k,\ell,m=1}^{N} \sqrt{P_{\ell j}P_{mk}}|j\rangle\langle j,\ell|S|k,m\rangle\langle k| \tag{18.21}$$

$$= \sum_{j,k=1}^{N} \sqrt{P_{jk}P_{kj}}|j\rangle\langle k| \tag{18.22}$$

$$= D. \tag{18.23}$$

Applying the walk operator U to $|\tilde{\lambda}\rangle$ gives

$$\begin{aligned} U|\tilde{\lambda}\rangle &= S(2\Pi - 1)|\tilde{\lambda}\rangle && (18.24)\\ &= S(2TT^\dagger - 1)T|\lambda\rangle && (18.25)\\ &= 2ST|\lambda\rangle - ST|\lambda\rangle && (18.26)\\ &= S|\tilde{\lambda}\rangle, && (18.27) \end{aligned}$$

and applying U to $S|\tilde{\lambda}\rangle$ gives

$$\begin{aligned} US|\tilde{\lambda}\rangle &= S(2\Pi - 1)S|\tilde{\lambda}\rangle && (18.28)\\ &= S(2TT^\dagger - 1)ST|\lambda\rangle && (18.29)\\ &= (2STD - T)|\lambda\rangle && (18.30)\\ &= 2\lambda S|\tilde{\lambda}\rangle - |\tilde{\lambda}\rangle. && (18.31) \end{aligned}$$

We see that the subspace $\mathrm{span}\{|\tilde{\lambda}\rangle, S|\tilde{\lambda}\rangle\}$ is invariant under U, so we can find eigenvectors of U within this subspace.

Now let $|\mu\rangle := |\tilde{\lambda}\rangle - \mu S|\tilde{\lambda}\rangle$, and let us choose $\mu \in \mathbb{C}$ so that $|\mu\rangle$ is an eigenvector of U. We have

$$\begin{aligned} U|\mu\rangle &= S|\tilde{\lambda}\rangle - \mu(2\lambda S|\tilde{\lambda}\rangle - |\tilde{\lambda}\rangle) && (18.32)\\ &= \mu|\tilde{\lambda}\rangle + (1 - 2\lambda\mu)S|\tilde{\lambda}\rangle. && (18.33) \end{aligned}$$

Thus μ will be an eigenvalue of U corresponding to the eigenvector $|\mu\rangle$ provided $(1 - 2\lambda\mu) = \mu(-\mu)$, i.e. $\mu^2 - 2\lambda\mu + 1 = 0$, so

$$\mu = \lambda \pm i\sqrt{1 - \lambda^2}. \tag{18.34}$$

Finally, note that for any vector in the orthogonal complement of $\mathrm{span}\{|\tilde{\lambda}\rangle, S|\tilde{\lambda}\rangle\}$, U simply acts as $-S$ (since $\Pi = TT^\dagger = \sum_\lambda T|\lambda\rangle\langle\lambda|T^\dagger = \sum_\lambda |\tilde{\lambda}\rangle\langle\tilde{\lambda}|$ projects onto $\mathrm{span}\{|\tilde{\lambda}\rangle\}$). In this subspace, the eigenvalues are ± 1. □

18.4 Hitting times

We can use random walks to formulate a generic search algorithm, and quantizing this algorithm gives a generic square root speedup [109]. Consider a graph $G = (V, E)$, with some subset $M \subset V$ of the vertices designated as *marked*. We will compare classical and quantum walk algorithms for deciding whether any vertex in G is marked.

Classically, a straightforward approach to this problem is to take a random walk defined by some stochastic matrix P, stopping if we encounter a marked vertex. In other words, we modify the original walk P to give a walk P' defined as

$$P'_{jk} = \begin{cases} 1 & k \in M \text{ and } j = k \\ 0 & k \in M \text{ and } j \neq k \\ P_{jk} & k \notin M. \end{cases} \tag{18.35}$$

Let us assume from now on that the original walk P is symmetric, though the modified walk P' clearly is not provided M is non-empty. If we order the vertices so that the marked ones come last, the matrix P' has the block form

$$P' = \begin{pmatrix} P_M & 0 \\ Q & I \end{pmatrix} \tag{18.36}$$

where P_M is obtained by deleting the rows and columns of P corresponding to vertices in M.

Suppose we take t steps of the walk. A simple calculation shows

$$(P')^t = \begin{pmatrix} P_M^t & 0 \\ Q(I + P_M + \cdots + P_M^{t-1}) & I \end{pmatrix} \tag{18.37}$$

$$= \begin{pmatrix} P_M^t & 0 \\ Q\frac{P_M^t - I}{P_M - I} & I \end{pmatrix}. \tag{18.38}$$

Now if we start from the uniform distribution over unmarked items (if we start from a marked item we are done, so we might as well condition on this not happening), then the probability of not reaching a marked item after t steps is $\frac{1}{N-|M|}\sum_{j,k\notin M}[P_M^t]_{jk} \leq \|P_M^t\| = \|P_M\|^t$, where the inequality follows because the left hand side is the expectation of P_M^t in the normalized state $|V \setminus M\rangle = \frac{1}{\sqrt{N-|M|}}\sum_{j\notin M}|j\rangle$. Now if $\|P_M\| = 1-\Delta$, then the probability of reaching a marked item after t steps is at least $1-\|P_M\|^t = 1-(1-\Delta)^t$, which is $\Omega(1)$ provided $t = O(1/\Delta) = O(\frac{1}{1-\|P_M\|})$.

It turns out that we can bound $\|P_M\|$ away from 1 knowing only the fraction of marked vertices and the spectrum of the original walk. Thus we can upper bound the *hitting time*, the time required to reach some marked vertex with constant probability.

Lemma 18.2. *If the second largest eigenvalue of P (in absolute value) is at most $1-\delta$ and $|M| \geq \epsilon N$, then $\|P_M\| \leq 1 - \delta\epsilon$.*

Proof. Let $|v\rangle \in \mathbb{R}^{N-|M|}$ be the principal eigenvector of P_M, and let $|w\rangle \in \mathbb{R}^N$ be the vector obtained by padding $|v\rangle$ with 0s for all the marked vertices.

We will decompose $|w\rangle$ in the eigenbasis of P. Since P is symmetric, it is actually doubly stochastic, and the uniform vector $|V\rangle = \frac{1}{\sqrt{N}}\sum_j |j\rangle$ corresponds to the eigenvalue 1. All other eigenvectors $|\lambda\rangle$ have eigenvalues at most $1-\delta$ by assumption. Now

$$\|P_M\| = \langle v|P_M|v\rangle \tag{18.39}$$

$$= \langle w|P|w\rangle \tag{18.40}$$

$$= |\langle V|w\rangle|^2 + \sum_{\lambda\neq 1}\lambda|\langle\lambda|w\rangle|^2 \tag{18.41}$$

$$\leq |\langle V|w\rangle|^2 + (1-\delta)\sum_{\lambda\neq 1}|\langle\lambda|w\rangle|^2 \tag{18.42}$$

$$= 1 - \delta\sum_{\lambda\neq 1}|\langle\lambda|w\rangle|^2 \tag{18.43}$$

$$= 1 - \delta(1 - |\langle V|w\rangle|^2). \tag{18.44}$$

But by the Cauchy-Schwarz inequality,

$$|\langle V|w\rangle|^2 = |\langle V|\Pi_{V\setminus M}|w\rangle|^2 \tag{18.45}$$

$$\leq \|\Pi_{V\setminus M}|V\rangle\|^2 \cdot \||w\rangle\|^2 \tag{18.46}$$

$$= \frac{N-|M|}{N} \tag{18.47}$$

$$\leq 1 - \epsilon \tag{18.48}$$

where $\Pi_{V\setminus M} = \sum_{j\notin M}|j\rangle\langle j|$. Thus $\|P_M\| \leq 1-\delta\epsilon$ as claimed. □

Thus we see that the classical hitting time is $O(1/\delta\epsilon)$.

Now we turn to the quantum case. Our strategy will be to perform phase estimation with sufficiently high precision on the operator U, the quantum walk corresponding to P', with the state

$$|\psi\rangle := \frac{1}{\sqrt{N-|M|}}\sum_{j\notin M}|\psi_j\rangle. \tag{18.49}$$

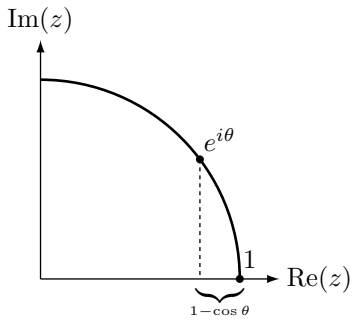


Figure 18.1: The classical gap, $1 - \lambda = 1 - \cos\theta$, appears on the real axis. The quantum phase gap, $\theta = \arccos\lambda$, is quadratically larger, since $\cos\theta \geq 1 - \theta^2/2$, i.e., $\arccos\lambda \geq \sqrt{2(1-\lambda)}$.

This state can easily be prepared by starting from the state

$$T|V\rangle = \frac{1}{\sqrt{N}} \sum_j |\psi_j\rangle \tag{18.50}$$

and measuring whether the first register corresponds to a marked vertex; if it does then we are done, and if not then we have prepared $|\psi\rangle$.

The matrix D for the walk P' is

$$\begin{pmatrix} P_M & 0 \\ 0 & I \end{pmatrix}, \tag{18.51}$$

so according to the spectral theorem, the eigenvalues of the resulting walk operator U are ± 1 and $e^{\pm i \arccos\lambda}$, where λ runs over the eigenvalues of P_M. If the marked set M is empty, then $P' = P$, and $|\psi\rangle$ is an eigenvector of U with eigenvalue 1, so phase estimation on U is guaranteed to return a phase of 0. But if M is non-empty, then the state $|\psi\rangle$ lives entirely within the subspace with eigenvalues $e^{\pm i \arccos\lambda}$. Thus if we perform phase estimation on U with precision $O(\min_\lambda \arccos\lambda)$, we will see a phase different from 0. Since $\arccos\lambda \geq \sqrt{2(1-\lambda)}$ (see Figure 18.1 for an illustration), we see that precision $O(\sqrt{1 - \|P_M\|})$ suffices. So the quantum algorithm can decide whether there is a marked vertex in time $O(1/\sqrt{1 - \|P_M\|}) = O(1/\sqrt{\delta\epsilon})$.

Chapter 19

Unstructured search

Now we begin to discuss applications of quantum walks to search algorithms. We start with the most basic of all search problems, the unstructured search problem (which is solved optimally by Grover's algorithm). We discuss how this problem fits into the framework of quantum walk search, and also describe amplitude amplification and quantum counting in this setting. We also discuss quantum walk algorithms for the search problem under locality constraints.

19.1 Unstructured search

In the unstructured search problem, we are given a black box function $f\colon S \to \{0,1\}$, where S is a finite set of size $|S| = N$. The inputs $x \in M$, where $M := \{x \in S\colon f(x) = 1\}$, are called *marked items*. In the decision version of the problem, our goal is to determine whether M is empty or not. We might also want to find a marked item when one exists.

It is quite easy to see that even the decision problem requires $\Omega(N)$ classical queries, and that N queries suffice, so the classical query complexity of unstructured search is $\Theta(N)$.

You should already be familiar with Grover's algorithm, which solves this problem using $O(\sqrt{N})$ quantum queries [57]. Grover's algorithm works by starting from the state $|S\rangle := \sum_{x\in S} |x\rangle/\sqrt{N}$ and alternately applying the reflection about the set of marked items, $\sum_{x\in M} 2|x\rangle\langle x| - 1$, and the reflection about the state $|S\rangle$, $2|S\rangle\langle S| - 1$. The former can be implemented with two quantum queries to f, and the latter requires no queries to implement. It is straightforward to show that there is some $t = O(\sqrt{N/|M|})$ for which t steps of this procedure give a state with constant overlap on $|M\rangle$ (assuming M is non-empty), so that a measurement will reveal a marked item with constant probability.

It can be shown that unstructured search requires $\Omega(\sqrt{N/|M|})$ queries. We will prove this when we discuss adversary lower bounds.

19.2 Quantum walk algorithm

Consider the discrete-time random walk on the complete graph represented by the stochastic matrix

$$P = \frac{1}{N-1} \begin{pmatrix} 0 & 1 & 1 & \cdots & 1 \\ 1 & 0 & 1 & \cdots & 1 \\ 1 & 1 & 0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 \\ 1 & 1 & \cdots & 1 & 0 \end{pmatrix} \tag{19.1}$$

$$= \frac{N}{N-1}|S\rangle\langle S| - \frac{1}{N-1} I. \tag{19.2}$$

It has eigenvalues 1 (which is non-degenerate) and $-1/(N-1)$ (with degeneracy $N-1$). Since the graph is highly connected, its spectral gap is very large: we have $\delta = 1 + \frac{1}{N-1} = \frac{N}{N-1}$.

This random walk gives rise to a very simple classical algorithm for unstructured search. In this algorithm, we start from a uniformly random item and repeatedly choose a new item uniformly at random from the other $N-1$ possibilities, stopping when we reach a marked item. The fraction of marked items is $\epsilon = |M|/N$, so the hitting time of this walk is

$$O\left(\frac{1}{\delta\epsilon}\right) = \frac{(N-1)N}{N|M|} = O(N/|M|) \tag{19.3}$$

(this is only an upper bound on the hitting time, but in this case we know it is optimal). Of course, if we have no a priori lower bound on $|M|$ in the event that M is non-empty, the best we can say is that $\epsilon \geq 1/N$, giving a running time $O(N)$.

The corresponding quantum walk search algorithm has a hitting time of

$$O\left(\frac{1}{\sqrt{\delta\epsilon}}\right) = O(\sqrt{N/|M|}), \tag{19.4}$$

corresponding to the running time of Grover's algorithm. To see that this actually gives an algorithm using $O(\sqrt{N/|M|})$ queries, we need to see that a step of the quantum walk can be performed using only $O(1)$ quantum queries. In the case where the first item is marked, the modified classical walk matrix is

$$P' = \frac{1}{N-1}\begin{pmatrix} N-1 & 1 & 1 & \cdots & 1 \\ 0 & 0 & 1 & \cdots & 1 \\ 0 & 1 & 0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 \\ 0 & 1 & \cdots & 1 & 0 \end{pmatrix}, \tag{19.5}$$

so that the vectors $|\psi_j\rangle$ are $|\psi_1\rangle = |1,1\rangle$ and $|\psi_j\rangle = |j, S\setminus\{j\}\rangle = \sqrt{\frac{N}{N-1}}|j,S\rangle - \frac{1}{\sqrt{N-1}}|j,j\rangle$ for $j = 2,\ldots,N$. With a general marked set M, the projector onto the span of these states is

$$\Pi = \sum_{j\in M} |j,j\rangle\langle j,j| + \sum_{j\notin M} |j, S\setminus\{j\}\rangle\langle j, S\setminus\{j\}|, \tag{19.6}$$

so the operator $2\Pi - 1$ acts as Grover diffusion over the neighbors when the vertex is unmarked, and as a phase flip when the vertex is marked. (Note that since we start from the state $|\psi\rangle = \sum_{j\notin M}|\psi_j\rangle$, we stay in the subspace of states $\operatorname{span}\{|j,k\rangle : (j,k)\in E\}$, and in particular have zero support on any state $|j,j\rangle$ for $j \in V$, so $2\Pi - 1$ acts as -1 when the first register holds a marked vertex.) Each such step can be implemented using two queries of the black box, one to compute whether we are at a marked vertex and one to uncompute that information; the subsequent swap operation requires no queries. Thus the query complexity is indeed $O(\sqrt{N/|M|})$.

This algorithm is not exactly the same as Grover's; for example, it works in the Hilbert space $\mathbb{C}^N \otimes \mathbb{C}^N$ instead of $\mathbb{C}^N$. Nevertheless, it is clearly closely related. In particular, notice that in Grover's algorithm, the unitary operation $2|S\rangle\langle S| - 1$ can be viewed as a kind of discrete-time quantum walk on the complete graph, where in this particular case no coin is necessary to define the walk.

The algorithm we have described so far only solves the decision version of unstructured search. To find marked item, we could use bisection, but this would introduce a logarithmic overhead. In fact, it can be shown that the final state of the quantum walk algorithm actually encodes a marked item when one exists.

19.3 Amplitude amplification and quantum counting

We briefly mention some other concepts related to unstructured search that provide useful tools for quantum algorithms in general. These ideas are typically presented in the context of Grover's algorithm; here we describe them in the framework of quantum walk search. This is slightly less space efficient, but the essential ideas are the same.

Amplitude amplification is a general method for boosting the success probability of a (classical or quantum) subroutine [27]. It can be implemented by quantum walk search as follows. Suppose we have a procedure that produces a correct answer with probability p (i.e., with an amplitude of magnitude $\sqrt{p}$ if we view it as a quantum process). From this procedure we can define a two-state Markov chain that, at each step, moves from the state where the answer is not known to the state where the answer is known with probability p, and then remains there. This walk has the transition matrix

$$P' = \begin{pmatrix} 1-p & 0 \\ p & 1 \end{pmatrix},$$

so $P_M = 1 - p$, giving a quantum hitting time of $O(1/\sqrt{1 - \|P_M\|}) = O(1/\sqrt{p})$.

For some applications, it may be desirable to estimate the value of p. Quantizing the above two-state Markov chain gives eigenvalues in the non-marked subspace of $e^{\pm i \arccos(1-p)} = e^{\pm i\sqrt{2p}+O(p^{3/2})}$. By applying phase estimation, we can determine $\sqrt{p}$ approximately. Recall that phase estimation gives an estimate with precision μ using $O(1/\mu)$ applications of the given unitary [40] (assuming we cannot apply high powers of the unitary any more efficiently than simply applying it repeatedly). An estimate of $\sqrt{p}$ with precision μ gives an estimate of p with precision $\mu\sqrt{p}$ (since $(\sqrt{p} + O(\mu))^2 = p + O(\mu\sqrt{p})$), so we can produce an estimate of p with precision ν in $O(\sqrt{p}/\nu)$ steps.

In particular, if the Markov chain is a search of the complete graph as described in the previous section, with $|M|$ marked sites out of N, then $p = |M|/N$, and this allows us to count the number of marked items. We obtain an estimate of $|M|/N$ with precision ν in $O(\sqrt{|M|/N}/\nu)$ steps. If we want a multiplicative approximation of $|M|$ with precision ρ, this means we need $O(\sqrt{N/|M|}/\rho)$ steps.

Note that for exact counting, no speedup is possible in general. If $|M| = \Theta(N)$ then we need to estimate p within precision $O(1/N)$ to uniquely determine $|M|$, but then the running time of the above procedure is $O(N)$. In fact, it can be shown that exact counting requires $\Omega(N)$ queries [18].

19.4 Search on graphs

We can also consider a variant of unstructured search with additional locality constraints. Suppose we view the items in S as the vertices of a graph $G = (S, E)$, and we require the algorithm to be local with respect to the graph. More concretely, we require the algorithm to alternate between queries and unitary operations U constrained to satisfy $U|j, \psi\rangle = \sum_{k \in j \cup \partial(j)} \alpha_k |k, \phi_k\rangle$ for any $j \in S$ (where the second register represents possible ancillary space, and recall that $\partial(j)$ denotes the set of neighbors of j in G) [2].

Since we have only added new restrictions that an algorithm must obey, the $\Omega(\sqrt{N})$ lower bound from the non-local version of the problem still applies. However, it is immediately clear that this bound cannot always be achieved. For example, if the graph is a cycle of N vertices, then simply propagating from one vertex of the cycle to an opposing vertex takes time $\Omega(N)$. So we would like to know, for example, how far from complete the graph can be such that we can still perform the search in $O(\sqrt{N})$ steps.

First, note that any expander graph (a graph with degree upper bounded by a constant and second largest eigenvalue bounded away from 1 by a constant) can be searched in time $O(\sqrt{N})$. Such graphs have $\delta = \Omega(1)$, and since $\epsilon \geq 1/N$ when there are marked items, the quantum hitting time is $O(1/\sqrt{\delta\epsilon}) = O(\sqrt{N})$ (whereas the classical hitting time is $O(1/\delta\epsilon) = O(N)$).

There are also many cases in which a quantum search can be performed in time $O(\sqrt{N})$ even though the eigenvalue gap of P is non-constant [103, 35, 12]. For example, consider the n-dimensional hypercube (with $N = 2^n$ vertices). Recall that since the adjacency matrix acts independently as σ_x on each coordinate, the eigenvalues are equally spaced, and the gap of P is $2/n$. Thus the general bound in terms of the eigenvalues of P shows that the classical hitting time is $O(nN) = O(N \log N)$. In fact, this bound is loose; the hitting time is actually $O(N)$, which can be seen by directly computing $\|P_M\|$ with one marked vertex. So there is a local quantum algorithm that runs in the square root of this time, namely $O(\sqrt{N})$.

Perhaps the most interesting example is the d-dimensional square lattice with N sites (i.e., with linear size $N^{1/d}$). This case can be viewed as having N items distributed on a grid in d-dimensional space. For simplicity, suppose we have periodic boundary conditions; then the eigenstates of the adjacency matrix are

given by

$$|\tilde{k}\rangle := \frac{1}{\sqrt{N}} \sum_x e^{2\pi i k \cdot x / N^{1/d}} |x\rangle \tag{19.7}$$

where k is a d-component vector of integers from 0 to $N^{1/d} - 1$. The corresponding eigenvalues are

$$2 \sum_{j=1}^{d} \cos \frac{2\pi k_j}{N^{1/d}}. \tag{19.8}$$

Normalizing to obtain a stochastic matrix, we simply divide these eigenvalues by $2d$. The 1 eigenvector has $k = (0, 0, \ldots, 0)$, and the second largest eigenvalue comes from (e.g.) $k = (1, 0, \ldots, 0)$, with an eigenvalue

$$\frac{1}{d} \left(d - 1 + \cos \frac{2\pi}{N^{1/d}} \right) \approx 1 - \frac{1}{2d} \left(\frac{2\pi}{N^{1/d}} \right)^2. \tag{19.9}$$

Thus the gap of the walk matrix P is about $\frac{2\pi^2}{dN^{2/d}} = O(N^{-2/d})$. This is another case in which the bound on the classical hitting time in terms of eigenvalues of P is too loose (it gives only $O(N^{1+2/d})$), and instead we must directly estimate the gap of P_M. One can show that the classical hitting time is $O(N^2)$ in $d = 1$, $O(N \log N)$ in $d = 2$, and $O(N)$ for any $d \geq 3$. Thus there is a local quantum walk search algorithm that saturates the lower bound for any $d \geq 3$, and one that runs in time time $O(\sqrt{N \log N})$ for $d = 2$. We already argued that there could be no speedup for $d = 1$, and indeed we see that the quantum hitting time in this case is $O(N)$.

Chapter 20

Quantum walk search

In this lecture we will discuss the algorithm that cemented the importance of quantum walk as a tool for quantum query algorithms: Ambainis's algorithm for the element distinctness problem [11]. The key new conceptual idea of this algorithm is to consider walks that store information obtained from many queries at each vertex, but that do not require many queries to update this information for an adjacent vertex. This idea leads to a general, powerful framework for quantum walk search [83, 98].

20.1 Element distinctness

In the *element distinctness problem*, we are given a black-box function $f\colon \{1,\dots,n\} \to S$, where S is some finite set. The goal is to determine whether there are two distinct inputs $x, y \in \{1,\dots,n\}$ such that $f(x) = f(y)$.

It is clear that a classical algorithm must make $\Omega(n)$ queries to solve the problem, since deciding whether there is such a pair is at least as hard as unstructured search (suppose we give the additional promise that if there is a pair, it will be with $x = 1$ for which $f(1) = 1$; then we must search for a $y \in \{2,\dots,n\}$ for which $f(y) = 1$). By the same argument, there is a quantum lower bound for element distinctness of $\Omega(\sqrt{n})$.

There is a simple quantum algorithm that uses Grover's algorithm recursively to improve upon the trivial running time of $O(n)$ [30]. To see how this algorithm works, first consider the following subroutine. Query f in ℓ randomly chosen places, and check whether one of these ℓ places belongs to a pair of inputs that map to the same value by performing a Grover search on the remaining $n - \ell$ inputs. The initial setup takes ℓ queries, and the Grover search takes $O(\sqrt{n-\ell}) = O(\sqrt{n})$ queries, for a total of $\ell + O(\sqrt{n})$. This subroutine fails most of the time, since it is likely that the random choice of ℓ inputs will be unlucky, but it succeeds with probability at least ℓ/n. To boost the success probability, we can use amplitude amplification, which takes $O(\sqrt{n/\ell})$ steps to boost the success probability to a constant. Overall, we can obtain success probability $\Omega(1)$ using

$$(\ell + \sqrt{n})\sqrt{n/\ell} = \sqrt{n\ell} + n/\sqrt{\ell} \tag{20.1}$$

queries. To optimize the query complexity, we set the two terms to be equal, giving $\ell = \sqrt{n}$ and hence a query complexity of $O(n^{3/4})$. (Note that an analysis of the *running time* of this algorithm would include extra logarithmic factors, since the inner use of Grover's algorithm must check whether an element against ℓ queried function values, which can be done in time $O(\log \ell)$ provided S is ordered and we initially sort the queried values.)

So far, we have a quantum upper bound of $O(n^{3/4})$, and a quantum lower bound of $\Omega(n^{1/2})$. It turns out that both of these can be improved. On the lower bound side, Aaronson and Shi proved an $\Omega(n^{1/3})$ lower bound for the closely-related *collision problem*, in which the goal is to distinguish one-to-one from two-to-one functions [3]. This implies an $\Omega(n^{2/3})$ lower bound for element distinctness by the following reduction. Suppose we randomly choose $\sqrt{n}$ inputs of the collision problem function and run the element distinctness algorithm on them. If the function is two-to-one, then there is some pair of elements in this set mapping to the same value with high probability (by the birthday problem), which the element distinctness algorithm

will detect. Hence a k-query element distinctness algorithm implies an $O(\sqrt{k})$-query collision algorithm; or equivalently, a k-query collision lower bound implies an $\Omega(k^2)$ element distinctness lower bound.

Now the question remains, can we close the gap between the $O(n^{3/4})$ upper bound and this $\Omega(n^{2/3})$ lower bound? Ambainis's quantum walk algorithm does exactly this.

20.2 Quantum walk algorithm

The idea of Ambainis's algorithm is to quantize a walk on the Johnson graph $J(n, m)$, where m is chosen appropriately. This graph has $\binom{n}{m}$ vertices corresponding to subsets of $\{1, 2, \ldots, n\}$ of size m, and two vertices are connected by an edge if the subsets differ in exactly one element.

To simplify the analysis slightly, we will use a different graph, the Hamming graph $H(n, m)$. The vertices of this graph are the m-tuples of values from $\{1, 2, \ldots, n\}$ (so there are n^m vertices). Two vertices are connected by an edge if they differ in exactly one coordinate. There are two main differences between the Johnson and Hamming graphs: the Hamming graph allows for repeated elements, and the order of elements is significant. Neither of these differences significantly affects the performance of the algorithm.

At each vertex, we store the values of the function at the corresponding inputs. In other words, the vertex $(x_1, x_2, \ldots, x_m) \in \{1, 2, \ldots, n\}^m$ is represented by the state

$$|x_1, x_2, \ldots, x_m, f(x_1), f(x_2), \ldots, f(x_m)\rangle. \tag{20.2}$$

To prepare such states, we must query the black-box function. In particular, to prepare an initial superposition over vertices of this graph takes m queries. However, we can move from one vertex to an adjacent vertex using only two queries: to replace x by y in any particular coordinate, we use one query to erase $f(x)$ and another to compute $f(y)$.

In this search problem, the marked vertices are those containing some $x \neq y$ with $f(x) = f(y)$. Notice that, given the stored function values, we can check whether we are at a marked vertex with no additional queries. The total number of marked vertices (in the case where the elements are not all distinct) is at least $\binom{m}{2}(n-2)^{m-2}$, so the fraction of marked vertices is

$$\epsilon \geq \frac{m(m-1)(n-2)^{m-2}}{2n^m}. \tag{20.3}$$

To analyze the walk, we also need the eigenvalues of the relevant Markov chain. The adjacency matrix of the Hamming graph $H(n, m)$ is $A = \sum_{i=1}^{m} (J - I)^{(i)}$, where J denotes the $n \times n$ all 1s matrix, and the superscript indicates that this matrix acts on the ith coordinate. The eigenvalues of J are n and 0, so the eigenvalues of $J - I$ are $n - 1$ and -1. Hence the largest eigenvalue of A is $m(n-1)$ (the degree of any vertex of $H(n, m)$) and the second largest eigenvalue is $(m-1)(n-1) - 1 = m(n-1) - n$. Normalizing by the degree, we see that the second largest eigenvalue of the stochastic matrix $A/m(n-1)$ is $(m(n-1) - n)/m(n-1) = 1 - n/m(n-1)$. In other words, the spectral gap is

$$\delta = \frac{n}{m(n-1)}. \tag{20.4}$$

Finally, how many queries does this algorithm use? Taking into account the initial m queries used to prepare the starting state and the 2 queries per step of the walk, we have a total number of queries

$$m + 2 \cdot O\left(\frac{1}{\sqrt{\delta\epsilon}}\right) = m + O\left(\sqrt{\frac{m(n-1)}{n}}\sqrt{\frac{2n^m}{m(m-1)(n-2)^{m-2}}}\right) \tag{20.5}$$

$$= m + O\left(\frac{n}{\sqrt{m}}\right). \tag{20.6}$$

Again we can set the two terms equal to optimize the performance. We have $m^{3/2} = O(n)$, so we should take $m = \Theta(n^{2/3})$. Then the total number of queries is $O(n^{2/3})$, which matches the lower bound, and hence is optimal.

Note that for the classical random walk search algorithm that we have quantized, the corresponding query complexity is $m + O(n^2/m)$, which is optimized by $m = n$. This gives no improvement over querying every input, as we knew must be the case.

20.3 Quantum walk search algorithms with auxiliary data

Algorithms based on similar ideas turn out to be useful for a wide variety of problems, including deciding whether a graph contains a triangle (or various other related graph properties), checking matrix multiplication, and testing whether a group is abelian. In general, as in the element distinctness case, we may need to store some data at each vertex, and we need to take into account the operations on this data when analyzing the walk.

Suppose we have a setup cost S, a cost U to update the state after one step of the walk, and a cost C to check whether a vertex is marked. For example, in Ambainis's algorithm for element distinctness, we had

$$S = m \qquad \text{to query } m \text{ positions} \tag{20.7}$$

$$U = 2 \qquad \text{to remove one of the items and add another} \tag{20.8}$$

$$C = 0 \qquad \text{since the function values for the subset are stored.} \tag{20.9}$$

In general, there is an algorithm to solve such a problem with total cost

$$S + \frac{1}{\sqrt{\delta\epsilon}}(U + C). \tag{20.10}$$

It turns out that for some problems, when the checking cost C is much larger than the update cost U, it is advantageous to take many steps of the walk on the unmarked graph before performing a phase flip on the marked sites. This is how Ambainis's algorithm originally worked, though for element distinctness it is not actually necessary. Using this idea, one can give a general quantum walk search algorithm with total cost [83]

$$S + \frac{1}{\sqrt{\epsilon}}\left(\frac{1}{\sqrt{\delta}}U + C\right). \tag{20.11}$$

In fact, it is also possible to modify the general algorithm so that it finds a marked item when one exists.

Part IV

Quantum query complexity

Chapter 21

Query complexity and the polynomial method

So far, we have discussed several different kinds of quantum algorithms. In the next few chapters, we discuss ways of establishing limitations on the power of quantum algorithms [65]. After reviewing the model of quantum query complexity, this lecture presents the *polynomial method*, an approach that relates quantum query algorithms to properties of polynomials.

21.1 Quantum query complexity

Many of the algorithms we have covered work in the setting of query complexity, where the input for a problem is provided by a black box. This setting is convenient since the black box provides a handle for proving lower bounds: we can often show that many queries are required to compute some given function of the black-box input. In contrast, it is notoriously difficult to prove lower bounds on the complexity of computing some function of explicit input data.

We briefly formalize the model of query complexity. Consider the task of computing a function $f : S \to T$, where $S \subseteq \Sigma^n$ is a set of strings over some input alphabet Σ. If $S = \Sigma^n$ then we say f is *total*; otherwise we say it is *partial*. The input string $x \in S$ is provided to us by a black box that computes x_i for any desired $i \in \{1, \ldots, n\}$. A query algorithm begins from a state that does not depend on the oracle string x. It then alternates between queries to the black box and other, non-query operations. Our goal is to compute $f(x)$ using as few queries to the black box as possible.

Of course, the minimum number of queries (which we call the *query complexity* of f) depends on the kind of computation we allow. There are at least three natural models:

- $D(f)$ denotes the deterministic query complexity, where the algorithm is classical and must always work correctly.

- R_ϵ denotes the randomized query complexity with error probability at most ϵ. Note that this it does not depend strongly on ϵ since we can boost the success probability by repeating the computation several times and taking a majority vote. Therefore $R_\epsilon(f) = \Theta(R_{1/3}(f))$ for any constant ϵ, so sometimes we simply write $R(f)$.

- Q_ϵ denotes the quantum query complexity, again with error probability at most ϵ. Similarly to the randomized case, $Q_\epsilon(f) = \Theta(Q_{1/3}(f))$ for any constant ϵ, so sometimes we simply write $Q(f)$.

We know that $D(\textsc{or}) = n$ and $R(\textsc{or}) = \Theta(n)$. Grover's algorithm shows that $Q(\textsc{or}) = O(\sqrt{n})$. In this lecture we will use the polynomial method to show (among other things) that $Q(\textsc{or}) = \Omega(\sqrt{n})$, a tight lower bound.

21.2 Quantum queries

A quantum query algorithm begins from x-independent state $|\psi\rangle$ and applies a sequence of unitary operations $U_1, \ldots, U_t$ interspersed with queries O_x, resulting in the state

$$|\psi_x^t\rangle := U_t O_x \ldots U_2 O_x U_1 O_x |\psi\rangle. \tag{21.1}$$

To make this precise, we need to specify the action of the oracle O_x.

For simplicity, we will mostly consider the case where the input is a bit string, i.e., $\Sigma = \{0,1\}$. Perhaps the most natural oracle model is the bit flip oracle $\hat{O}_x$, which acts as

$$\hat{O}_x|i,b\rangle = |i, b \oplus x_i\rangle \qquad \text{for } i \in \{1,\ldots,n\},\ b \in \{0,1\}. \tag{21.2}$$

This is simply the linear extension of the natural reversible oracle mapping $(i,b) \mapsto (i, b \oplus x_i)$, which can be performed efficiently given the ability to efficiently compute $i \mapsto x_i$. Note that the algorithm may involve states in a larger Hilbert space; implicitly, the oracle acts as the identity on any ancillary registers.

It is often convenient to instead consider the phase oracle, which is obtained by conjugating the bit-flip oracle by Hadamard gates: by the well-known phase kickback trick, $O_x = (I \otimes H)\hat{O}_x(I \otimes H)$ satisfies

$$O_x|i,b\rangle = (-1)^{bx_i}|i,b\rangle \qquad \text{for } i \in \{1,\ldots,n\},\ b \in \{0,1\}. \tag{21.3}$$

Note that this is slightly wasteful since $O_x|i,0\rangle = |i,0\rangle$ for all i; we could equivalently consider a phase oracle O'_x defined by $O'_x|0\rangle = |0\rangle$ and $O'_x|i\rangle = (-1)^{x_i}|i\rangle$ for all $i \in \{1,\ldots,n\}$. However, it is essential to include the ability to not query the oracle by giving the oracle some eigenstate of known eigenvalue, independent of x. If we could only perform the phase flip $|i\rangle \mapsto (-1)^{x_i}|i\rangle$ for $i \in \{1,\ldots,n\}$, then we could not tell a string x from its bitwise complement $\bar{x}$.

These constructions can easily be generalized to the case of a d-ary input alphabet, say $\Sigma = \mathbb{Z}_d$ (identifying input symbols with integers modulo d). Then for $b \in \Sigma$, we can define an oracle $\hat{O}_x$ by

$$\hat{O}_x|i,b\rangle = |i, b + x_i\rangle \qquad \text{for } i \in \{1,\ldots,n\},\ b \in \mathbb{Z}_d. \tag{21.4}$$

Taking the Fourier transform of the second register gives a phase oracle $O_x = (I \otimes F_{\mathbb{Z}_d})\hat{O}_x(I \otimes F_{\mathbb{Z}_d}^\dagger)$ satisfying

$$O_x|i,b\rangle = \omega_d^{bx_i}|i,b\rangle \qquad \text{for } i \in \{1,\ldots,n\},\ b \in \mathbb{Z}_d \tag{21.5}$$

where $\omega_d := e^{2\pi i/d}$.

21.3 Quantum algorithms and polynomials

The following shows a basic connection between quantum algorithms and polynomials [18].

Lemma 21.1. *The acceptance probability of a t-query quantum algorithm for a problem with black-box input $x \in \{0,1\}^n$ is a polynomial in $x_1, \ldots, x_n$ of degree at most $2t$.*

Proof. We claim that the amplitude of any basis state is a polynomial of degree at most t, so that the probability of any basis state (and hence the probability of success) is a polynomial of degree at most $2t$.

The proof is by induction on t. If an algorithm makes no queries to the input, then its success probability is independent of the input, so it is a constant, a polynomial of degree 0.

For the induction step, a query maps

$$|i,b\rangle \overset{O_x}{\mapsto} (-1)^{bx_i}|i,b\rangle \tag{21.6}$$

$$= (1 - 2bx_i)|i,b\rangle, \tag{21.7}$$

so it increases the degree of each amplitude by at most 1. □

Consider a Boolean function $f\colon \{0,1\}^n \to \{0,1\}$. We say a polynomial $p \in \mathbb{R}[x_1, \ldots, x_n]$ represents f if $p(x) = f(x)$ for all $x \in \{0,1\}^n$. Letting $\deg(f)$ denote the smallest degree of any polynomial representing f, we have $Q_0(f) \geq \deg(f)/2$.

To handle bounded-error algorithms, we introduce the concept of *approximate degree*. We say a polynomial p ϵ-represents f if $|p(x) - f(x)| \leq \epsilon$ for all $x \in \{0,1\}^n$. Then the ϵ-approximate degree of f, denoted $\widetilde{\deg}_\epsilon(f)$, is the smallest degree of any polynomial that ϵ-represents f. Clearly, $Q_\epsilon(f) \geq \widetilde{\deg}_\epsilon(f)/2$. Since bounded-error query complexity does not depend strongly on the particular error probability ϵ, we can define, say, $\widetilde{\deg}(f) := \widetilde{\deg}_{1/3}(f)$.

Now to lower bound the quantum query complexity of a Boolean function, it suffices to lower bound its approximate degree.

21.4 Symmetrization

While polynomials are well-understood objects, the acceptance probability is a multivariate polynomial, so it can be rather complicated. Since $x^2 = x$ for $x \in \{0,1\}$, we can restrict our attention to multilinear polynomials, but it is still somewhat difficult to deal with such polynomials directly. Fortunately, for many functions it suffices to consider a related univariate polynomial obtained by symmetrization.

For a string $x \in \{0,1\}^n$, let $|x|$ denote the Hamming weight of x, the number of 1s in x.

Lemma 21.2. *Given any n-variate multilinear polynomial p, let $P(k) := \mathbb{E}_{|x|=k}[p(x)]$. Then P is a polynomial with $\deg(P) \leq \deg(p)$.*

Proof. Since p is multilinear, it can be written as a sum of monomials, i.e., as

$$p(x) = \sum_{S \subseteq \{1,\ldots,n\}} c_S \prod_{i \in S} x_i \tag{21.8}$$

for some coefficients c_S. Then we have

$$P(k) = \sum_{S \subseteq \{1,\ldots,n\}} c_S \, \mathbb{E}_{|x|=k} \left[\prod_{i \in S} x_i \right] \tag{21.9}$$

and it suffices to compute the expectation of each monomial. We find

$$\mathbb{E}_{|x|=k} \left[\prod_{i \in S} x_i \right] = \Pr_{|x|=k}[\forall\, i \in S,\ x_i = 1] \tag{21.10}$$

$$= \frac{\binom{n-|S|}{k-|S|}}{\binom{n}{k}} \tag{21.11}$$

$$= \frac{(n-|S|)!\,k!\,(n-k)!}{(k-|S|)!\,(n-k)!\,n!} \tag{21.12}$$

$$= \frac{(n-|S|)!}{n!} k(k-1)\cdots(k-|S|+1) \tag{21.13}$$

which is a polynomial in k of degree $|S|$. Since $c_S = 0$ whenever $|S| > \deg(p)$, we see that $\deg(P) \leq \deg(p)$. □

Thus the polynomial method is a particularly natural approach for symmetric functions, those that only depend on the Hamming weight of the input.

21.5 Parity

Let PARITY$\colon \{0,1\}^n \to \{0,1\}$ denote the symmetric function PARITY$(x) = x_1 \oplus \cdots \oplus x_n$. Recall that Deutsch's problem, which is the problem of computing the parity of 2 bits, can be solved exactly with only one quantum

query. Applying this algorithm to a pair of bits at a time and then taking the parity of the results, we see that $Q_0(\text{PARITY}) \leq n/2$.

What can we say about lower bounds for computing parity? Symmetrizing PARITY gives the function $P\colon \{0, 1, \ldots, n\} \to \mathbb{R}$ defined by

$$P(k) = \begin{cases} 0 & \text{if } k \text{ is even} \\ 1 & \text{if } k \text{ is odd.} \end{cases} \tag{21.14}$$

Since P changes direction n times, $\deg(P) \geq n$, so we see that $Q_0(\text{PARITY}) \geq n/2$. Thus Deutsch's algorithm is tight among zero-error algorithms.

What about bounded-error algorithms? To understand this, we would like to lower bound the approximate degree of PARITY. If $|p(x) - f(x)| \leq \epsilon$ for all $x \in \{0,1\}^n$, then

$$|P(k) - F(k)| = \left|\mathbb{E}_{|x|=k}(p(x) - f(x))\right| \leq \epsilon \tag{21.15}$$

for all $k \in \{0, 1, \ldots, n\}$, where P is the symmetrization of p and F is the symmetrization of f. Thus, a multilinear polynomial p that ϵ-approximates PARITY implies a univariate polynomial P satisfying $P(k) \leq \epsilon$ for k even and $P(k) \geq 1 - \epsilon$ for k odd. For any $\epsilon < 1/2$, this function still changes direction n times, so in fact we have $\widetilde{\deg}_\epsilon(f) \geq n$, and hence $Q_\epsilon(\text{PARITY}) \geq n/2$.

This shows that the strategy for computing parity using Deutsch's algorithm is optimal, even among bounded-error algorithms. This is an example of a problem for which a quantum computer cannot get a significant speedup—here the speedup is only by factor of 2. In fact, we need at least $n/2$ queries to succeed with any bounded error, even with very small advantage (e.g., even if we only want to be correct with probability $\frac{1}{2} + 10^{-100}$). In contrast, while the adversary method can prove an $\Omega(n)$ lower bound for parity, the constant factor that it establishes is error-dependent.

Note that this also shows we need $\Omega(n)$ queries to exactly count the number of marked items in an unstructured search problem, since exactly determining the number of 1s would in particular determine whether the number of 1s is odd or even.

21.6 Unstructured search

Next we will see how the polynomial method can be used to prove the $\Omega(\sqrt{n})$ lower bound for computing the logical OR of n bits. Symmetrizing OR gives a function $F(k)$ with $F(0) = 0$ and $F(1) = 1$. We also have $F(k) = 1$ for all $k > 1$, but we will not actually need to use this. This function is monotonic, so we cannot use the same simple argument we applied to parity. Nevertheless, we can prove that $\widetilde{\deg}(\text{OR}) = \Omega(\sqrt{n})$ using the following basic fact about polynomials, due to Markov.

Lemma 21.3. *Let $P\colon \mathbb{R} \to \mathbb{R}$ be a polynomial. Then*

$$\max_{x \in [0,n]} \frac{\mathrm{d}P(x)}{\mathrm{d}x} \leq \frac{\deg(P)^2}{n} \left(\max_{x \in [0,n]} P(x) - \min_{x \in [0,n]} P(x) \right). \tag{21.16}$$

In other words, if we let

$$h := \max_{x \in [0,n]} P(x) - \min_{x \in [0,n]} P(x) \tag{21.17}$$

denote the "height" of P in the range $[0, n]$, and

$$d := \max_{x \in [0,n]} \frac{\mathrm{d}P(x)}{\mathrm{d}x} \tag{21.18}$$

denote the largest derivative of P in that range, then we have $\deg(P) \geq \sqrt{nd/h}$.

Now let P be a polynomial that ϵ-approximates OR. Since $P(0) \leq \epsilon$ and $P(1) \geq 1 - \epsilon$, P must increase by at least $1 - 2\epsilon$ in going from $k = 0$ to $k = 1$, so $d \geq 1 - 2\epsilon$.

We have no particular bound on h, since we have no control over the value of P at non-integer points; the function could become arbitrarily large or small. However, since $P(k) \in [0, 1]$ for $k \in \{0, 1, \ldots, n\}$, a large

value of h implies a large value of d, since P must change fast enough to start from and return to values in the range $[0, 1]$. In particular, P must change by at least $(h-1)/2$ over a range of k of width at most $1/2$, so we have $d \geq h - 1$. Therefore,

$$\deg(P) \geq \sqrt{\frac{n \max\{1-2\epsilon, h-1\}}{h}} \tag{21.19}$$

$$= \Omega(\sqrt{n}). \tag{21.20}$$

It follows that $Q(\text{OR}) = \Omega(\sqrt{n})$.

Note that the same argument applies for a function that takes the value 0 whenever $|x| = w$ and the value 1 whenever $|x| = w + 1$, for any w; in particular, it applies to any non-constant symmetric function. (Of course, we can do better for some symmetric functions, such as PARITY and also MAJORITY, among others.)

Chapter 22

The collision problem

We now discuss the quantum lower bound for the collision problem. This lower bound is a more involved application of the polynomial method than the simple examples we've seen so far.

22.1 Problem statement

In the collision problem, we are given a black-box function $f\colon \{1,\ldots,n\} \to \Sigma$ that is promised to be either one-to-one or two-to-one. The goal is to determine which is the case. (Of course, we can assume that n is even, since otherwise a two-to-one function is impossible.)

Note that for this problem to be well-defined, we must have $|\Sigma| \geq n$. For concreteness, we consider the case where $\Sigma = \{1,\ldots,n\}$. Since this is a special case of a function with a larger range, this assumption can only make the problem slightly harder. However, we will present an upper bound that does not depend on $|\Sigma|$, and our lower bound will handle the case where $\Sigma = \{1,\ldots,n\}$, so it applies in general.

While it may be most natural to think in terms of one-to-one and two-to-one functions, we can easily cast the collision problem in the framework we used to formally define query complexity. The input alphabet is Σ, and an input is an oracle string $x \in S_0 \cup S_1 \subset \Sigma^n$, where

$$S_0 = \{x \in \Sigma^n \colon \forall\, i \neq j,\ x_i \neq x_j\} \tag{22.1}$$

$$S_1 = \{x \in \Sigma^n \colon \forall\, i \text{ there is a unique } j \text{ such that } x_i = x_j\}. \tag{22.2}$$

In other words, we are promised that $x \in S_0 \cup S_1$, i.e., that either every character of the string is distinct or that every character of x appears exactly twice. Again, the goal is to determine which of these is the case, so we are computing a function $\textsc{collision}\colon S_0 \cup S_1 \to \{0,1\}$ with $\textsc{collision}(x) = y$ if $x \in S_y$.

Why is the collision problem interesting? We know that the unstructured search problem takes $\Omega(\sqrt{n})$ queries, i.e., exponentially many in the number of bits required to specify an index. Thus quantum computers cannot provide a speedup for completely unstructured problems. The collision problem is a problem with slightly more structure: instead of looking for a needle in a haystack, we are merely looking for two identical items among a stack consisting of two pieces of hay, two needles, two toothpicks, two twigs, two cactus spines, etc. We will see that the quantum query complexity of the collision problem is $\Theta(n^{1/3})$, so while this more structured problem can be solved slightly faster than unstructured search, it still requires many queries. In other words, the kind of structure present in the collision problem is not sufficient to allow an exponential quantum speedup.

Note that the structure in the collision problem arises naturally in the hidden subgroup problem: if the hidden subgroup has order r, then the hiding function is r-to-one. But the HSP has additional structure (namely, that of the underlying group), which explains why its quantum query complexity is so much smaller.

22.2 Classical query complexity

Before investigating quantum algorithms and lower bounds, let's briefly consider the classical query complexity of the collision problem. A natural algorithm for finding a collision in a two-to-one function is as follows:

query random indices until we observe a collision. For two independently random indices $i, j \in \{1, \ldots, n\}$, we have $\Pr(x_i = x_j) = \frac{1}{n-1}$. If we query m indices in this way, there are $\binom{m}{2}$ pairs, so the expected number of collisions seen is $\binom{m}{2}/(n-1)$. With $m = \sqrt{n}$ this is $\Omega(1)$, so we expect to see a collision. Indeed, a second moment argument shows that this happens with constant probability.

In fact, this algorithm is optimal. Until we see a collision, there is no better strategy than to query randomly. By the union bound, the probability of seeing a collision after making m queries is at most $\binom{m}{2}/(n-1) = O(m^2/n)$, so $m = \Omega(\sqrt{n})$.

22.3 Quantum algorithm

Now let's consider quantum algorithms for the collision problem. One simple approach is a trivial application of Grover's algorithm: first query x_1, and then search over $x_2, x_3, \ldots, x_n$ for x_1. This approach uses $O(\sqrt{n})$ queries, so it is no better than the above classical algorithm, although it achieves the same running time in a very different way.

However, we can improve the performance by combining this approach with the main idea of the optimal classical algorithm. The approach is to query m distinct indices (it doesn't matter how we choose them) and check for a collision. If we find a collision then we are done; otherwise, we can search the remaining $n - m$ characters for one of the m values we have seen. Overall, this approach takes time $m + O(\sqrt{(n-m)/m})$. This is minimized by taking $m = \Theta(n^{1/3})$, showing that $Q(\text{COLLISION}) = O(n^{1/3})$ [28].

This is similar to an algorithm for element distinctness that we saw previously. The difference is that in element distinctness, we could have a unique collision, so a single run of the basic algorithm only succeeds with small probability; we address this by performing amplitude amplification, at the cost of a longer running time. Recall that the resulting algorithm for element distinctness took time $O(n^{3/4})$, which turned out to be suboptimal. In contrast, the above algorithm for the collision problem turns out to be optimal.

As a side remark, note that this algorithm uses a lot of space, since we need to store the $m = n^{1/3}$ items queried initially. It is an open problem to understand the quantum query complexity of the collision problem with restricted space (i.e., to find a time-space tradeoff).

22.4 Toward a quantum lower bound

How can we prove a quantum lower bound for the collision problem?

Note that the positive-weighted adversary method will not be sufficient. This is a consequence of the so-called "property-testing barrier": for a decision problem in which "yes" input strings differ from "no" input strings in an ϵ fraction of their characters, the best possible positive adversary lower bound goes like $1/\epsilon$. For the collision problem, this means that the adversary method can at best prove a constant lower bound. (There is now an explicit $\Omega(n^{1/3})$ lower bound for collision using the negative adversary method [20], although it is somewhat complicated.)

Thus our strategy will be to apply the polynomial method. To do this, we need to adapt the method to the case of a non-binary input alphabet. To do this, we define variables δ_{ij} for $i, j \in \{1, \ldots, n\}$, as follows:

$$\delta_{ij} := \begin{cases} 1 & \text{if } x_i = j \\ 0 & \text{otherwise.} \end{cases} \tag{22.3}$$

Then we claim that acceptance probability of a t-query quantum algorithm is a multilinear polynomial in the $\{\delta_{ij}\}$ of degree at most $2t$.

This can be proved along similar lines to the binary case. Suppose we use an addition modulo n query, where $|i, j\rangle \mapsto |i, j + x_i\rangle$. Then we have

$$|i, j + x_i\rangle = \sum_k \delta_{ik} |i, j + k\rangle \tag{22.4}$$

$$= \sum_\ell \delta_{i,\ell-j} |i, \ell\rangle, \tag{22.5}$$

so the degree of each amplitude can increase by at most 1 when we make a query.

Next we would like to obtain a simpler polynomial. We cannot directly symmetrize over the variables $\{\delta_{ij}\}$, as this would destroy too much of the structure of the problem.

The original idea leading to a nontrivial collision lower bound, due to Aaronson, was to express the acceptance probability as a bivariate polynomial in n and r, where the function is r-to-one. The main difficulty with this approach is that we need to have $r \mid n$ in order for such inputs to make sense (so that we can at least say that the acceptance probability of a quantum algorithm is defined, and hence a given approximating polynomial is bounded between 0 and 1). This approach originally gave a lower bound $\Omega(n^{1/5})$ [1]. Subsequently, Shi improved this to give the optimal result of $\Omega(n^{1/3})$ [104].

22.5 Constructing the functions

In this lecture we will describe a later argument due to Kutin [75] that seems to be simplest known approach to lower bounding the collision problem, yet that gives a tight lower bound, and even covers the case where the range is $\{1, \ldots, n\}$. We will follow Kutin's presentation closely, with only some slight changes of notation.

The basic idea of Kutin's proof is to consider functions that are a-to-one on part of the domain and b-to-one on the rest of the domain. Let us say that a triple (m, a, b) is valid if $m \in \{0, 1, \ldots, n\}$, $a \mid m$, and $b \mid n - m$. Then we define an input $x^{m,a,b}$ as

$$x_i^{m,a,b} = \begin{cases} \lceil i/a \rceil & \text{if } 1 \leq i \leq m \\ n - \lfloor (n-i)/b \rfloor & \text{if } m < i \leq n. \end{cases} \tag{22.6}$$

Given such an input, we can obtain a family of inputs by permuting the input alphabet and the characters of the string arbitrarily. Specifically, for any permutations σ, τ of $\{1, \ldots, n\}$, we define an input $x^{\sigma\tau}$ with $x_i^{\sigma\tau} := \tau(x_{\sigma(i)}^{m,a,b})$. This induces corresponding binary variables $\delta_{ij}^{\sigma\tau}$ with $\delta_{ij}^{\sigma\tau} = 1$ iff $x_i^{\sigma\tau} = j$.

Now we claim that the acceptance probability of a quantum algorithm presented with such an input is a polynomial in m, a, b.

Lemma 22.1. *Let $p(\{\delta_{ij}\})$ be a polynomial in the δ_{ij}. For any valid triple (m, a, b), let*

$$P(m, a, b) := \mathbb{E}_{\sigma,\tau}[p(\{\delta_{ij}^{\sigma\tau}\})]. \tag{22.7}$$

Then P is a polynomial in m, a, b with $\deg(P) \leq \deg(p)$.

Proof. Any polynomial p can be expanded as a sum of monomials of the form

$$I_S(\{\delta_{ij}\}) := \prod_{j=1}^{n} \prod_{i \in S_j} \delta_{ij} \tag{22.8}$$

where $S_1, \ldots, S_n$ are disjoint subsets of $\{1, \ldots, n\}$. This monomial is 1 if all the indices $i \in S_j$ satisfy $x_i = j$, and 0 otherwise. Its degree is $s := \sum_j |S_j|$. It suffices to handle the case where p is any particular monomial.

For any input $x^{\sigma\tau}$, we can identify the subset of function values in the range of the a-to-one part of the function and the b-to-one part of the function. Given $U \subseteq \{1, \ldots, n\}$, let X denote the event that all function values in U belong to the a-to-one part and all function values not in U belong to the b-to-one part (more formally, that $\tau(j) \in U$ for all $j \in \{1, \ldots, \frac{m}{a}\}$ with $S_j \neq \varnothing$ and $\tau(j) \notin U$ for all $j \in \{n, n-1, \ldots, n - \frac{n-m}{b} + 1\}$ with $S_j \neq \varnothing$). We collect together the inputs corresponding to a particular event X. Then we have

$$P(m, a, b) = \sum_{U \subseteq \{1,\ldots,n\}} \Pr_{\tau}[X] \, \mathbb{E}_{\sigma,\tau}[I_S(\{\delta_{ij}^{\sigma\tau}\}) \mid X] \tag{22.9}$$

$$= \sum_{U \subseteq \{1,\ldots,n\}} \Pr_{\tau}[X] \Pr_{\sigma,\tau}[\forall\, j,\ \forall\, i \in S_j,\ \delta_{ij}^{\sigma\tau} = 1 \mid X]. \tag{22.10}$$

Let $u := |U|$, and let t denote the number of values of j with $|S_j| \neq 0$. Then we have

$$\Pr_\tau[X] = \frac{\frac{m}{a}(\frac{m}{a}-1)\cdots(\frac{m}{a}-u+1)\cdot(\frac{n-m}{b})(\frac{n-m}{b}-1)\cdots(\frac{n-m}{b}-(t-u)+1)\cdot(n-t)!}{n!} \tag{22.11}$$

$$= \frac{(\frac{m}{a})_u(\frac{n-m}{b})_{t-u}}{(n)_t} \tag{22.12}$$

where

$$(n)_k := \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1). \tag{22.13}$$

Here the numerator of (22.11) has three contributions: the number of ways to permute the u function values in the a-to-one part is $(\frac{m}{a})_u$, the number of ways to permute the $t-u$ function values in the b-to-one part is $(\frac{n-m}{b})_{t-u}$, and the number of ways to permute the $n-t$ remaining function values is $(n-t)!$. Note that this expression is a rational function in m, a, b whose numerator has degree t and whose denominator is $a^u b^{t-u}$.

Now consider the latter term of (22.10). Given that X occurs, $\Pr_\tau[\forall\, j,\ \forall\, i \in S_j,\ \delta_{ij}^{\sigma\tau} = 1]$ is independent of σ, so suppose σ is any permutation such that X occurs. In other words, we only need to count consistent ways of permuting the indices i. Observe that the number of ways to permute the indices i such that $x_i = j$ for some $j \in U$ is

$$a(a-1)\cdots(a-|S_j|+1) = (a)_{|S_j|}. \tag{22.14}$$

Similarly, the number of ways to permute the indices i such that $x_i = j$ for some $j \notin U$ is $(b)_{|S_j|}$. In addition, we can permute the remaining $n-s$ indices however we like. Thus we have

$$\Pr_{\sigma,\tau}[\forall\, j,\ \forall\, i \in S_j,\ \delta_{ij}^{\sigma\tau} = 1 \mid X] = \frac{(n-s)!\prod_{j\in U}(a)_{|S_j}\prod_{j\notin U}(b)_{|S_j|}}{n!} \tag{22.15}$$

$$= \frac{\prod_{j\in U}(a)_{|S_j|}\prod_{j\notin U}(b)_{|S_j|}}{(n)_s}. \tag{22.16}$$

This expression is a polynomial in a, b of degree s. Also, it is divisible by a^u and b^{t-u}. Thus $P(m, a, b)$ is a polynomial in m, a, b of degree $t + s - u - (t-u) = s$. □

22.6 Finishing the proof

To prove the lower bound, we will use the following fact about real polynomials. (Note that this fact is also useful elsewhere, e.g., in proving a tight lower bound for general threshold functions.)

Lemma 22.2 (Paturi)**.** *Let $f \in \mathbb{R}[x]$, let $a, b \in \mathbb{Z}$ with $a < b$, and let $\xi \in [a, b]$. If there are constants c, d such that*

- *$|f(i)| \leq c$ for all integers $i \in [a, b]$*
- *$|f(\lfloor \xi \rfloor) - f(\xi)| \geq d$,*

then $\deg(f) = \Omega(\sqrt{(\xi - a + 1)(b - \xi + 1)})$.

Regardless of ξ, Paturi's lemma always shows that the degree is $\Omega(\sqrt{b-a})$. If ξ is near the middle of the range then it does much better, showing that the degree is $\Omega(b-a)$. Also note that by continuity, it is sufficient for f to differ by a constant amount between two consecutive integers (i.e., we can replace the second bullet point by $|f(\xi - 1) - f(\xi)| \geq d$ for some integer $\xi \in [a+1, b]$).

Now we are ready to prove that the quantum query complexity of the collision problem is $\Omega(n^{1/3})$. Let $p(\{\delta_{ij}\})$ be the acceptance probability of a t-query quantum algorithm that solves the collision problem with error probability at most $1/3$, and let $P(m, a, b)$ be as above. We know that $t \geq \deg(P)/2$. Furthermore, we know that P has the following properties:

- $0 \leq P(m, 1, 1) \leq 1/3$
- $2/3 \leq P(m, 2, 2) \leq 1$
- $0 \leq P(m, a, b) \leq 1$ for any valid triple (m, a, b).

Now consider inputs that are roughly half one-to-one and half two-to-one. For concreteness, let $m = 2\lfloor n/4 \rfloor$. Since $n - m$ is even (recall that n is always even by assumption since otherwise the problem is trivial), $(m, 1, 2)$ is valid. We consider two cases, depending on whether the algorithm is more likely to call this a yes or a no input.

First suppose $P(m, 1, 2) \geq 1/2$. Let r be the smallest positive integer such that $|P(m, 1, r)| \geq 2$. First we consider the polynomial $f(x) = P(m, 1, x)$. For all $x \in \{1, \ldots, r-1\}$, we have $-2 \leq P(m, 1, x) \leq 2$. But we also know that $|P(m, 1, 1) - P(m, 1, 2)| \geq \frac{1}{2} - \frac{1}{3} = \frac{1}{6}$. By Paturi's lemma with $a = 1$, $b = r - 1$, $\xi = 2$, $c = 2$, and $d = \frac{1}{6}$, we have $\deg(P) = \Omega(\sqrt{r})$.

On the other hand, consider the polynomial $f(x) = P(n - rx, 1, r)$. When $x \in \mathbb{Z}$ such that $rx \in \{0, \ldots, n\}$, the triple $(n - rx, 1, r)$ is valid, so we have $0 \leq f(x) \leq 1$ for all integers $x \in [0, \lfloor n/r \rfloor]$. However, $|f(\frac{n-m}{r})| = |P(m, 1, r)| \geq 2$, and $\left\lfloor \frac{n-m}{r} \right\rfloor$ is about halfway between 0 and $\lfloor n/r \rfloor$. Thus by Paturi's lemma with $a = 0$, $b = \lfloor n/r \rfloor$, $\xi = \frac{n-m}{r}$, $c = 1$, and $d = 1$, we have $\deg(P) = \Omega(n/r)$.

Since $\deg(P) = \Omega(\sqrt{r})$ and $\deg(P) = \Omega(n/r)$, we find $\deg(P) = \Omega(n/\deg(P)^2)$, so $\deg(P) = \Omega(n^{1/3})$.

It remains to consider the case where $P(m, 1, 2) < 1/2$. (Here we deviate slightly from the presention in [75] to correct an apparent error in that work.) The same conclusion holds here by a similar argument. Now let r be the smallest positive integer for which $|P(m, r, 2)| \geq 2$. On the one hand, $|P(m, 1, 2) - P(m, 2, 2)| \geq \frac{2}{3} - \frac{1}{2} = \frac{1}{6}$, so applying Paturi's lemma to $f(x) = P(m, x, 2)$ with $a = 1$, $b = r - 1$, $\xi = 2$, $c = 2$, and $d = \frac{1}{6}$ gives $\deg(P) = \Omega(\sqrt{r})$ as before.

On the other hand, consider the polynomial $f(x) = P(2rx, r, 2)$. The triples $(2rx, r, 2)$ are valid for $x \in \{0, \ldots, \lfloor n/2r \rfloor\}$, so $0 \leq f(x) \leq 1$ for all such x. Also, since $|P(m, r, 2)| \geq 2$, $f(x)$ jumps by at least 1 from $x = \lfloor m/2r \rfloor$ to $x = m/2r$ (note in particular that $2r$ must not divide m, since by assupmtion, $(m, r, 2)$ cannot be valid). Thus, applying Paturi's lemma with with $a = 0$, $b = \lfloor n/2r \rfloor$, $\xi = m/2r$, $c = 1$, and $d = 1$ shows that $\deg(P) = \Omega(n/r)$.

Overall, it follows that any quantum algorithm for solving the collision problem must use $\Omega(n^{1/3})$ queries.

Chapter 23

The quantum adversary method

We now discuss a second approach to proving quantum query lower bounds, the *quantum adversary method* [10]. In fact, we'll see later that the generalized version of the adversary method we consider here (allowing negative weights [63]) turns out to be an *upper bound* on quantum query complexity, up to constant factors [93, 78].

23.1 Quantum adversaries

Motivation for the quantum adversary method comes from the following construction. Suppose the oracle is operated by an adversarial party who holds a quantum state determining the oracle string, which is in some superposition $\sum_{x\in S} a_x|x\rangle$ over the possible oracles. To implement each query, the adversary performs the "super-oracle"

$$O := \sum_{x\in S} |x\rangle\langle x| \otimes O_x. \tag{23.1}$$

An algorithm does not have direct access to the oracle string, and hence can only perform unitary operations that act as the identity on the adversary's superposition. After t steps, an algorithm maps the overall state to

$$|\psi^t\rangle := (I\otimes U_t)O\ldots(I\otimes U_2)O(I\otimes U_1)O\left(\sum_{x\in S} a_x|x\rangle\otimes|\psi\rangle\right) \tag{23.2}$$

$$= \sum_{x\in S} a_x|x\rangle\otimes|\psi_x^t\rangle. \tag{23.3}$$

The main idea of the approach is that for the algorithm to learn x, this state must become very entangled. To measure the entanglement of the pure state $|\psi^t\rangle$, we can consider the reduced density matrix of the oracle,

$$\rho^t := \sum_{x,y\in S} a_x^* a_y \langle\psi_x^t|\psi_y^t\rangle\,|x\rangle\langle y|. \tag{23.4}$$

Initially, the state ρ^0 is pure. Our goal is to quantify how mixed it must become (i.e., how entangled the overall state must be) before we can compute f with error at most ϵ. To do this we could consider, for example, the entropy of ρ^t. However, it turns out that other measures are easier to deal with.

In particular, we have the following basic fact about the distinguishability of quantum states (for a proof, see for example [68, Section A.9]):

Fact 23.1. *Given one of two pure states $|\psi\rangle, |\phi\rangle$, we can make a measurement that determines which state we have with error probability at most $\epsilon \in [0, 1/2]$ if and only if $|\langle\psi|\phi\rangle| \leq 2\sqrt{\epsilon(1-\epsilon)}$.*

Thus it is convenient to consider measures that are linear in the inner products $\langle\psi_x^t|\psi_y^t\rangle$.

23.2 The adversary method

To obtain an adversary lower bound, we choose a matrix $\Gamma \in \mathbb{R}^{|S|\times|S|}$ with rows and columns indexed by the possible black-box inputs. The entry $\Gamma_{x,y}$ is meant to characterize how hard it is to distinguish between x and y. We say Γ is an *adversary matrix* if

1. $\Gamma_{xy} = \Gamma_{yx}$ and
2. if $f(x) = f(y)$ then $\Gamma_{xy} = 0$.

The second condition reflects that we do not need to distinguish between x and y if $f(x) = f(y)$.

The original adversary method made the additional assumption that $\Gamma_{xy} \geq 0$, but it turns out that this condition is not actually necessary [63]. Sometimes we refer to the *negative* or *generalized* adversary method to distinguish it from the original, positive-weighted method. While it may not be intuitively obvious what it would mean to give a negative weight to the entry characterizing distinguishability of two inputs, it turns out that this flexibility can lead to significantly improved lower bounds for some functions.

Given an adversary matrix Γ, we can define a weight function

$$W^j := \sum_{x,y\in S} \Gamma_{xy} a_x^* a_y \langle\psi_x^j|\psi_y^j\rangle. \tag{23.5}$$

Note that this is a simple function of the entries of ρ^j. The idea of the lower bound is to show that W^j starts out large, must become small in order to compute f, and cannot change by much if we make a query.

The initial value of the weight function is

$$W^0 = \sum_{x,y\in S} \Gamma_{xy} a_x^* a_y \langle\psi_x^0|\psi_y^0\rangle \tag{23.6}$$

$$= \sum_{x,y\in S} a_x^* \Gamma_{xy} a_y \tag{23.7}$$

since $|\psi_x^0\rangle$ cannot depend on x. To make this as large as possible, we take a to be a principal eigenvector of Γ, an eigenvector with eigenvalue $\pm\|\Gamma\|$. Then $|W^0| = \|\Gamma\|$.

The final value of the weight function is easier to bound if we assume a nonnegative adversary matrix. The final value is constrained by the fact that we must distinguish x from y with error probability at most ϵ whenever $f(x) \neq f(y)$. For this to hold after t queries, we need $|\langle\psi_x^t|\psi_y^t\rangle| \leq 2\sqrt{\epsilon(1-\epsilon)}$ for all pairs $x, y \in S$ with $f(x) \neq f(y)$ (by the above Fact). Thus, if Γ has nonnegative entries, we have

$$|W^t| \leq \sum_{x,y\in S} \Gamma_{xy} a_x^* a_y 2\sqrt{\epsilon(1-\epsilon)} \tag{23.8}$$

$$= 2\sqrt{\epsilon(1-\epsilon)}\|\Gamma\|. \tag{23.9}$$

Here we can include the terms where $f(x) = f(y)$ in the sum since $\Gamma_{xy} = 0$ for such pairs. We also used the fact that the principal eigenevector of a nonnegative matrix can be taken to have nonnegative entries (by the Perron-Frobenius theorem).

A similar bound holds if Γ has negative entries, but we need a different argument. In general, one can only show that $|W^t| \leq (2\sqrt{\epsilon(1-\epsilon)}+2\epsilon)\|\Gamma\|$. But if we assume that $f\colon S \to \{0,1\}$ has Boolean output, then we can prove the same bound as in the non-negative case, and the proof is simpler than for a general output space. We use the following simple result, stated in terms of the Frobenius norm $\|X\|_F^2 := \sum_{a,b} |X_{ab}|^2$:

Proposition 23.2. *For any $X \in \mathbb{C}^{m\times n}, Y \in \mathbb{C}^{n\times n}, Z \in \mathbb{C}^{n\times m}$, we have $|\mathrm{tr}(XYZ)| \leq \|X\|_F\|Y\|\|Z\|_F$.*

Proof. We have

$$\mathrm{tr}(XYZ) = \sum_{a,b,c} X_{ab} Y_{bc} Z_{ca} \tag{23.10}$$

$$= \sum_a (x^a)^\dagger Y z^a \tag{23.11}$$

where $(x^a)_b = X^*_{ab}$ and $(z^a)_c = Z_{ca}$. Thus

$$|\mathrm{tr}(XYZ)| \leq \sum_a \|x^a\|\|Yz^a\| \tag{23.12}$$

$$\leq \|Y\| \sum_a \|x^a\|\|z^a\| \tag{23.13}$$

$$\leq \|Y\| \sqrt{\sum_a \|x^a\|^2 \sum_{a'} \|z^{a'}\|^2} \tag{23.14}$$

$$= \|Y\|\|X\|_F\|Z\|_F \tag{23.15}$$

as claimed, where we used the Cauchy-Schwarz inequality in the first and third steps. □

To upper bound $|W^t|$ for the negative adversary with Boolean output, write $W^t = \mathrm{tr}(\Gamma V)$ where $V_{xy} := a^*_x a_y \langle\psi^t_x|\psi^t_y\rangle \delta[f(x) \neq f(y)]$. Define

$$C := \sum_{x\in S} a_x \Pi_{f(x)} |\psi^t_x\rangle\langle x| \tag{23.16}$$

$$\bar{C} := \sum_{x\in S} a_x \Pi_{1-f(x)} |\psi^t_x\rangle\langle x| \tag{23.17}$$

with Π_0, Π_1 denoting the projectors onto the subspaces indicating $f(x) = 0, 1$, respectively. Then

$$(C^\dagger \bar{C})_{xy} = a^*_x a_y \langle\psi^t_x|\Pi_{f(x)}\Pi_{1-f(y)}|\psi^t_y\rangle, \tag{23.18}$$

so

$$(C^\dagger\bar{C} + \bar{C}^\dagger C)_{xy} = a^*_x a_y \langle\psi^t_x|(\Pi_{f(x)}\Pi_{1-f(y)} + \Pi_{1-f(x)}\Pi_{f(y)})|\psi^t_y\rangle \tag{23.19}$$

$$= a^*_x a_y \langle\psi^t_x|\psi^t_y\rangle \delta[f(x) \neq f(y)], \tag{23.20}$$

i.e., $V = C^\dagger\bar{C} + \bar{C}^\dagger C$. Thus we have

$$W^t = \mathrm{tr}(\Gamma(C^\dagger\bar{C} + \bar{C}^\dagger C)) \tag{23.21}$$

$$= \mathrm{tr}(\bar{C}\Gamma C^\dagger) + \mathrm{tr}(C\Gamma\bar{C}^\dagger). \tag{23.22}$$

By the Proposition, $|W^t| \leq 2\|\Gamma\|\|C\|_F\|\bar{C}\|_F$. Finally, we upper bound $\|C\|_F$ and $\|\bar{C}\|_F$. We have

$$\|C\|^2_F + \|\bar{C}\|^2_F = \sum_{x,y\in S} |a_x|^2(|\langle y|\Pi_{f(x)}|\psi^t_x\rangle|^2 + |\langle y|\Pi_{1-f(x)}|\psi^t_x\rangle|^2) = 1 \tag{23.23}$$

$$\|\bar{C}\|^2_F = \sum_{x\in S} |a_x|^2 \|\Pi_{1-f(x)}|\psi^t_x\rangle\|^2 \leq \epsilon. \tag{23.24}$$

Therefore $\|C\|_F\|\bar{C}\|_F \leq \max_{x\in[0,\epsilon]} \sqrt{x(1-x)} = \sqrt{\epsilon(1-\epsilon)}$ (assuming $\epsilon \in [0, 1/2]$), and we find that $|W^t| \leq 2\sqrt{\epsilon(1-\epsilon)}\|\Gamma\|$, as claimed.

It remains to understand how much the weight function can decrease at each step of the algorithm. We have

$$W^{j+1} - W^j = \sum_{x,y\in S} \Gamma_{xy} a^*_x a_y (\langle\psi^{j+1}_x|\psi^{j+1}_y\rangle - \langle\psi^j_x|\psi^j_y\rangle). \tag{23.25}$$

Consider how the state changes when we make a query. We have $|\psi^{j+1}_x\rangle = U_{j+1}O_x|\psi^j_x\rangle$. Thus the elements of the Gram matrix of the states $\{|\psi^{j+1}_x\rangle : x \in S\}$ are

$$\langle\psi^{j+1}_x|\psi^{j+1}_y\rangle = \langle\psi^j_x|O^\dagger_x(U_{j+1})^\dagger U_{j+1}O_y|\psi^j_y\rangle \tag{23.26}$$

$$= \langle\psi^j_x|O_xO_y|\psi^j_y\rangle \tag{23.27}$$

since U_{j+1} is unitary and $O_x^\dagger = O_x$. Therefore

$$W^{j+1} - W^j = \sum_{x,y \in S} \Gamma_{xy} a_x^* a_y \langle \psi_x^j | (O_x O_y - I) | \psi_y^j \rangle. \tag{23.28}$$

Observe that $O_x O_y |i,b\rangle = (-1)^{b(x_i \oplus y_i)} |i,b\rangle$. Let $P_0 = I \otimes |0\rangle\langle 0|$ denote the projection onto the $b = 0$ states, and let P_i denote the projection $|i,1\rangle\langle i,1|$. (As with O_x, the projections P_i implicitly act as the identity on any ancilla registers, so $\sum_{i=0}^n P_i = I$.) Then $O_x O_y = P_0 + \sum_{i=1}^n (-1)^{x_i \oplus y_i} P_i$, so $O_x O_y - I = -2 \sum_{i:\, x_i \neq y_i} P_i$. Thus we have

$$W^{j+1} - W^j = -2 \sum_{x,y \in S} \sum_{i:\, x_i \neq y_i} \Gamma_{xy} a_x^* a_y \langle \psi_x^j | P_i | \psi_y^j \rangle. \tag{23.29}$$

Now for each $i \in \{1, \ldots, n\}$, let Γ_i be a matrix with

$$(\Gamma_i)_{xy} := \begin{cases} \Gamma_{xy} & \text{if } x_i \neq y_i \\ 0 & \text{if } x_i = y_i \end{cases} \tag{23.30}$$

Then we have

$$W^{j+1} - W^j = -2 \sum_{x,y \in S} \sum_{i=1}^n (\Gamma_i)_{xy} a_x^* a_y \langle \psi_x^j | P_i | \psi_y^j \rangle \tag{23.31}$$

$$= -2 \sum_{i=1}^n \operatorname{tr}(Q_i \Gamma_i Q_i^\dagger) \tag{23.32}$$

where $Q_i := \sum_x a_x P_i |\psi_x^j\rangle\langle x|$.

Using the triangle inequality and the above Proposition, we have

$$|W^{j+1} - W^j| \leq 2 \sum_{i=1}^n |\operatorname{tr}(Q_i \Gamma_i Q_i^\dagger)| \tag{23.33}$$

$$\leq 2 \sum_{i=1}^n \|\Gamma_i\| \|Q_i\|_F^2. \tag{23.34}$$

Since

$$\sum_{i=1}^n \|Q_i\|_F^2 = \sum_{i=1}^n \sum_{x \in S} |a_x|^2 \|P_i |\psi_x^j\rangle\|^2 \tag{23.35}$$

$$\leq \sum_{x \in S} |a_x|^2 \tag{23.36}$$

$$= 1, \tag{23.37}$$

we have

$$|W^{j+1} - W^j| \leq 2 \max_{i \in \{1,\ldots,n\}} \|\Gamma_i\|. \tag{23.38}$$

Combining these three facts gives the adversary lower bound. Since $|W^0| = \|\Gamma\|$, we have

$$|W^t| \geq \|\Gamma\| - 2t \max_{i \in \{1,\ldots,n\}} \|\Gamma_i\|. \tag{23.39}$$

Thus, to have $|W^t| \leq 2\sqrt{\epsilon(1-\epsilon)}\|\Gamma\|$, we require

$$t \geq \frac{1 - 2\sqrt{\epsilon(1-\epsilon)}}{2} \operatorname{Adv}(f). \tag{23.40}$$

where

$$\mathrm{Adv}(f) := \max_{\Gamma} \frac{\|\Gamma\|}{\max_{i\in\{1,\ldots,n\}} \|\Gamma_i\|} \tag{23.41}$$

with the maximum taken over all adversary matrices Γ for the function f. (Sometimes the notation $\mathrm{Adv}(f)$ is reserved for the maximization over nonnegative adversary matrices, with the notation $\mathrm{Adv}^{\pm}(f)$ for the generalized adversary method allowing negative weights.)

23.3 Example: Unstructured search

As a simple application of this method, we prove the optimality of Grover's algorithm. It suffices to consider the problem of distinguishing between the case of no marked items and the case of a unique marked item (in an unknown location). Thus, consider the partial function where S consists of the strings of Hamming weight 0 or 1, and f is the logical OR of the input bits. (Equivalently, we consider the total function OR but only consider adversary matrices with zero weight on strings of Hamming weight more than 1.)

For this problem, adversary matrices have the form

$$\Gamma = \begin{pmatrix} 0 & \gamma_1 & \cdots & \gamma_n \\ \gamma_1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \gamma_n & 0 & \cdots & 0 \end{pmatrix} \tag{23.42}$$

for some nonnegative coefficients $\gamma_1, \ldots, \gamma_n$. Symmetry suggests that we should take $\gamma_1 = \cdots = \gamma_n$. This can be formalized, but for the present purposes we can take this as an ansatz.

Setting $\gamma_1 = \cdots = \gamma_n = 1$ (since an overall scale factor does not affect the bound), we have

$$\Gamma^2 = \begin{pmatrix} n & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & \cdots & 1 \end{pmatrix} \tag{23.43}$$

which has norm $\|\Gamma^2\| = n$, and hence $\|\Gamma\| = \sqrt{n}$. We also have

$$\Gamma_1 = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix} \tag{23.44}$$

and similarly for the other Γ_i, so $\|\Gamma_i\| = 1$. Thus we find $\mathrm{Adv}(\mathrm{OR}) \geq \sqrt{n}$, and it follows that $Q_\epsilon(\mathrm{OR}) \geq \frac{1-2\sqrt{\epsilon(1-\epsilon)}}{2}\sqrt{n}$. This shows that Grover's algorithm is optimal up to a constant factor (recall that Grover's algorithm finds a unique marked item with probability $1 - o(1)$ in $\frac{\pi}{4}\sqrt{n} + o(1)$ queries).

Chapter 24

Span programs and formula evaluation

Having discussed lower bounds on quantum query complexity, we now turn our attention back to upper bounds. The framework of *span programs* is a powerful tool for understanding quantum query complexity [94, 92]. Span programs are closely related to the quantum adversary method, and can be used to show that the (generalized) adversary method actually characterizes quantum query complexity up to constant factors [93, 78].

For simplicity, we restrict our attention to the case of a (possibly partial) Boolean function $f\colon S \to \{0,1\}$ where $S \subseteq \{0,1\}^n$. Many (but not all) of the considerations for this case generalize to other kinds of functions.

24.1 The dual of the adversary method

Recall that the adversary method defines a quantity

$$\mathrm{Adv}^{\pm}(f) := \max_{\Gamma} \frac{\|\Gamma\|}{\max_{i\in\{1,\ldots,n\}} \|\Gamma_i\|} \tag{24.1}$$

such that $Q(f) = \Omega(\mathrm{Adv}^{\pm}(f))$. Although not immediately obvious from the above expression, it can be shown that $\mathrm{Adv}^{\pm}(f)$ is the value of a *semidefinite program* (SDP), a kind of optimization problem in which a linear objective function is optimized subject to linear and positive semidefiniteness constraints.

Unfortunately, the details of semidefinite programming are beyond the scope of this course. For a good introduction in the context of quantum information, see Watrous's lecture notes on Theory of Quantum Information [112, Lecture 7].

A useful feature of SDPs is that they can be solved efficiently. Thus, we can use a computer program to find the optimal adversary lower bound for a fixed (finite-size) function. However, while this may be useful for getting intuition about a problem, in general this does not give a strategy for determining asymptotic quantum query complexity.

Another key feature of SDPs is the concept of semidefinite programming duality. To every primal SDP, phrased as a maximization problem, there is a dual SDP, which is a minimization problem. Whereas feasible solutions of the primal SDP give lower bounds, feasible solutions of the dual SDP give upper bounds. The dual problem can be constructed from the primal problem by a straightforward (but sometimes tedious) process. Semidefinite programs satisfy *weak duality*, which says that the value of the primal problem is at most the value of the dual problem. Furthermore, almost all SDPs actually satisfy *strong duality*, which says that the primal and dual values are equal. (In particular, this holds under the *Slater conditions*, which essentially say that the primal or dual constraints are strictly feasible.)

To understand any SDP, one should always construct its dual. Carrying this out for the adversary method would require some experience with semidefinite programs, so we simply state the result here. The variables of the dual problem can be viewed as a set of vectors $|v_{x,i}\rangle \in \mathbb{C}^d$ for all inputs $x \in S$

and all indices $i \in [n] := \{1, \ldots, n\}$, for some dimension d. For $b \in \{0,1\}$, we define the b-complexity $C_b := \max_{x \in f^{-1}(b)} \sum_{i \in [n]} \||v_{x,i}\rangle\|^2$. Since strong duality holds, we have the following.

Theorem 24.1. *For any function $f\colon S \to \{0,1\}$ with $S \subseteq \{0,1\}^n$, we have*

$$\mathrm{Adv}^{\pm}(f) = \min_{\{|v_{x,i}\rangle\}} \max\{C_0, C_1\} \tag{24.2}$$

where the minimization is over all positive integers d and all sets of vectors $\{|v_{x,i}\rangle \in \mathbb{C}^d\colon x \in S, i \in [n]\}$ satisfying the constraint

$$\sum_{i\colon x_i \neq y_i} \langle v_{x,i}|v_{y,i}\rangle = 1 - \delta_{f(x),f(y)} \quad \forall\, x \neq y. \tag{24.3}$$

By constructing solutions of the adversary dual, we place upper bounds on the best possible adversary lower bound. But more surprisingly, one can construct an algorithm from a solution of the adversary dual, giving an upper bound on the quantum query complexity itself.

Observe that if we replace $|v_{x,i}\rangle \to \alpha|v_{x,i}\rangle$ for all $x \in f^{-1}(0)$ and $|v_{y,i}\rangle \to |v_{y,i}\rangle/\alpha$ for all $y \in f^{-1}(1)$, we don't affect the constraints (24.3), but we map $C_0 \to \alpha^2 C_0$ and $C_1 \to C_1/\alpha^2$. Taking $\alpha = (C_1/C_0)^{1/4}$, we make the two complexities equal. Thus we have

$$\mathrm{Adv}^{\pm}(f) = \min_{\{|v_{x,i}\rangle\}} \sqrt{C_0 C_1}. \tag{24.4}$$

Note that the constraint (24.3) for $f(x) = f(y)$, where the right-hand side is zero, can be removed without changing the value of the optimization problem. (For functions with non-Boolean output, one loses a factor strictly between 1 and 2 in the analogous relaxation.) To see this, suppose we have a set of vectors $\{|v_{x,i}\rangle\}$ satisfying the constraint (24.3) for $f(x) \neq f(y)$ but not for $f(x) = f(y)$. Simply let $|v'_{x,i}\rangle = |v_{x,i}\rangle|x_i \oplus f(x)\rangle$ for all $x \in S$ and all $i \in [n]$. Then $\||v'_{x,i}\rangle\| = \||v_{x,i}\rangle\|$, and for the terms where $x_i \neq y_i$, we have $\langle v'_{x,i}|v'_{y,i}\rangle = \langle v_{x,i}|v_{y,i}\rangle$ if $f(x) \neq f(y)$ and $\langle v'_{x,i}|v'_{y,i}\rangle = 0$ if $f(x) = f(y)$.

24.2 Span programs

The dual of the adversary method is equivalent to a linear-algebraic model of computation known as *span programs*. This model was first studied in the context of classical computational complexity. It was connected to quantum algorithms for formula evaluation by Reichardt and Špalek [94], and was subsequently related to the adversary method by Reichardt [92].

A span program for a function $f\colon \{0,1\}^n \to \{0,1\}$ consists of a target vector $|t\rangle \in \mathbb{C}^D$, sets of input vectors $I_{i,b} \subset \mathbb{C}^D$ for all $i \in [n]$ and $b \in \{0,1\}$, and a set of free input vectors $I_{\mathrm{free}} \subset \mathbb{C}^D$. The set of available input vectors for input x is $I(x) := I_{\mathrm{free}} \cup \bigcup_{i \in [n]} I_{i,x_i}$. We say that a span program computes f if $|t\rangle \in \operatorname{span} I(x)$ if and only if $f(x) = 1$.

The complexity of a span program is measured by its *witness size*. If $f(x) = 1$, then there is a linear combination of vectors from $I(x)$ that gives $|t\rangle$; the witness size of x is the smallest squared length of the coefficients for any such linear combination. If $f(x) = 0$, then there is a vector that has inner product 1 with $|t\rangle$ that is orthogonal to all available input vectors; the witness size of x is the smallest squared length of the vector of inner products of this vector with *all* input vectors (of course, these inner products are zero for the available input vectors). The witness size of f is the largest witness size of any $x \in S$, or equivalently, the geometric mean of the largest witness sizes of 0- and 1-inputs.

The smallest witness size of any span program computing f is precisely $\mathrm{Adv}^{\pm}(f)$, and there is a close relationship between span programs and dual adversary solutions. Given a dual adversary solution with vectors $|v_{x,i}\rangle$, one can construct a matrix whose rows are the vectors $\bigoplus_{i \in [n]} \langle \bar{x}_i| \langle v_{x,i}|$. Take the columns of this matrix in block i and subblock b to be the vectors in $I_{i,b}$, let the target vector be the all ones vector, and let there be no free input vectors. It can be shown that this gives a span program for f whose witness size is exactly the complexity of the dual adversary solution. Furthermore, every span program can be put into a canonical form for which this translation can be reversed to produce a dual adversary solution: taking the vectors of a canonical span program to be the columns of a matrix, the rows give dual adversary vectors

for $x \in f^{-1}(0)$ and the witness vectors for $x \in f^{-1}(1)$ give the remaining dual adversary vectors. For more detail on this translation, see [92, Lemma 6.5] (and see the rest of that paper for more than you ever wanted to know about span programs).

We focus on dual adversary solutions here, as these are simpler to work with for the applications we consider. However, for other applications it may be useful to work directly with span programs instead; in particular, (non-canonical) span programs offer more freedom when trying to devise upper bounds.

24.3 Unstructured search

We now give a simple example of an optimal dual adversary solution, namely for unstructured search. Let $f\colon S \to \{0,1\}$ be defined by $f(x) = \textsc{or}(x)$ with $S = \{x \in \{0,1\}^n \colon |x| \leq 1\}$ the set of inputs with Hamming weight at most 1. Take dimension $d = 1$; let $|v_{0,i}\rangle = 1$ for all $i \in [n]$ and $|v_{x,i}\rangle = x_i$. The constraint (24.3) gives

$$\sum_{i\colon 0 \neq (e_j)_i} \langle v_{0,i}|v_{e_j,i}\rangle = \langle v_{0,j}|v_{e_j,j}\rangle = 1 \tag{24.5}$$

for all $j \in [n]$ (where $e_j \in \mathbb{C}^n$ is the jth standard basis vector) and

$$\sum_{i\colon (e_j)_i \neq (e_k)_i} \langle v_{e_j,i}|v_{e_k,i}\rangle = \langle v_{e_j,j}|v_{e_k,j}\rangle + \langle v_{e_j,k}|v_{e_k,k}\rangle = 0 \tag{24.6}$$

for $j \neq k$, so the constraint is satisfied.

The 0- and 1-complexities of this solution are

$$C_0 = \sum_{i \in [n]} 1 = n \tag{24.7}$$

$$C_1 = \max_j \sum_{i \in [n]} \delta_{i,j} = 1. \tag{24.8}$$

Since $\sqrt{C_0 C_1} = \sqrt{n}$, we see that $\mathrm{Adv}^{\pm}(f) \leq \sqrt{n}$, demonstrating that the previously discussed adversary lower bound is the best possible adversary lower bound.

It is easy to extend this dual adversary solution to one for the total OR function. For any $x \neq 0$, simply let $|v_{x,i}\rangle = \delta_{i,j}$, where j is the index of any particular bit for which $x_j = 1$ (e.g., the first such bit). Then the constraints are still satisfied, and the complexity is the same. As an exercise, you should work out an optimal dual adversary for AND.

24.4 Formulas and games

Unstructured search can be viewed as the problem of evaluating the logical OR of n bits in $O(\sqrt{n})$ quantum queries. It is natural to ask about the quantum query complexity of other Boolean functions. Next we show how span programs can be used to show that the quantum query complexity of evaluating a balanced binary AND-OR tree with n leaves is $O(\sqrt{n})$. In fact, a similar approach shows that the quantum query complexity of evaluating *any* Boolean formula expressed in terms of AND, OR, and NOT gates is $O(\sqrt{n})$.

One way of motivating this query problem is as follows. Consider a two-player game between Andrea and Orlando in which the players alternate turns, with Andrea going first and Orlando going second. On each turn there are d possible moves, and there are a total of k turns. Suppose that the winner is determined by some black box function $f : \{1, 2, \ldots, d\}^k \to \{0, 1\}$ indicating who wins the game (0 = Andrea, 1 = Orlando) after a particular sequence of k moves. The goal is to determine who wins the game, assuming Andrea and Orlando both play optimally.

This problem is equivalent to evaluating an AND-OR formula. Consider the tree of moves (a balanced d-ary tree of height k) and let internal vertices evaluate to 0 or 1 depending on who wins the game if the players start from the corresponding (partial) sequence of moves. Orlando wins if he can make any move

that leads to the outcome 1, so vertices representing his moves correspond to OR gates. Andrea wins if she can make any move that gives 0—i.e., she only loses if all her moves give 1—so her vertices correspond to AND gates.

What is the query complexity of evaluating this balanced d-ary AND-OR tree? Let us first consider randomized classical algorithms. Notice that it is sometimes possible to avoid evaluating all the leaves: for example, if we learn that one input to an AND gate is 0, then we do not need to evaluate the other inputs to know that the gate evaluates to 0. In the case where all inputs are 1, we must evaluate all of them; but the inputs to an AND gate are given by the outputs of OR gates, and an OR gate evaluating to 1 is exactly the case where it is possible to learn the value of the gate without knowing all of its inputs. Similarly, the hardest input to the OR gate is precisely the output of an AND gate for which it is possible to learn the output without evaluating all inputs.

With these observations in mind, a sensible classical algorithm is as follows. Suppose that to evaluate any given vertex of the tree, we guess a random child and evaluate it (recursively), only evaluating other children when necessary. By analyzing a simple recurrence, one can show that this algorithm uses

$$O\left(\left(\frac{d-1+\sqrt{d^2+14d+1}}{4}\right)^k\right) = O\left(n^{\log_d \frac{d-1+\sqrt{d^2+14d+1}}{4}}\right) \tag{24.9}$$

queries, where $n = d^k$ is the input size (e.g., for $d = 2$, $O(n^{0.753})$) [108, 96]. In fact, it is possible to show that this algorithm is asymptotically optimal [97]. Notice, in particular, that the classical query complexity becomes larger as d is increased with n fixed. In the extreme case where $k = 1$, so that $n = d$, we are simply evaluating the AND gate, which is equivalent (by de Morgan's laws) to evaluating an OR gate, and which we know takes $\Omega(n)$ queries.

A quantum computer can evaluate such games faster if k is sufficiently small. Of course, the $k = 1$ case is solved in $O(\sqrt{n})$ queries by Grover's algorithm. By applying Grover's algorithm recursively, suitably amplifying the success probability, it is possible to evaluate the formula in $\sqrt{n}O(\log n)^{k-1}$ queries [29], which is nearly optimal for constant k. This can be improved slightly to $O(\sqrt{n}c^k)$ queries for some constant c using a variant of Grover's algorithm that allows noisy inputs [64]. But both of these algorithms are only close to tight when k is constant. Indeed, for very low degree (such as $d = 2$, so that $k = \log_2 n$), nothing better than the classical algorithm was known until 2007 [47]. Here we will describe how to solve that problem in only $O(\sqrt{n})$ quantum queries. However, rather than presenting the original algorithm, we show how a composition property of span programs offers a particularly simple analysis.

24.5 Function composition

A nice property of the adversary method (in both dual and primal formulations) is its behavior under function composition. Given functions $f\colon \{0,1\}^n \to \{0,1\}$ and $g\colon \{0,1\}^m \to \{0,1\}$, we define $f \circ g\colon \{0,1\}^{nm} \to \{0,1\}$ by $(f \circ g)(x) = f(g(x_1,\ldots,x_m),\ldots,g(x_{nm-m+1},\ldots,x_{nm}))$. Here we focus on upper bounds, for which we have the following [93, 78].

Theorem 24.2. $\mathrm{Adv}^{\pm}(f \circ g) \leq \mathrm{Adv}^{\pm}(f)\,\mathrm{Adv}^{\pm}(g)$.

Proof. Let $\{|v_{x,i}\rangle\colon x \in \{0,1\}^n, i \in [n]\}$ be an optimal dual adversary solution for f, and let $\{|u_{y,j}\rangle\colon y \in \{0,1\}^m, j \in [m]\rangle\}$ be an optimal dual adversary solution for g. Let $y = (y^1,\ldots,y^n)$ where each $y^i \in \{0,1\}^m$. Then define

$$|w_{y,(i,j)}\rangle = |v_{g(y),i}\rangle \otimes |u_{y^i,j}\rangle \tag{24.10}$$

where $g(y)$ denotes the vector with $g(y)_i = g(y^i)$.

We claim that this is a dual adversary solution for $f \circ g$. To see this, we compute

$$\sum_{(i,j)\colon y^i_j \neq z^i_j} \langle w_{y,(i,j)} | w_{z,(i,j)} \rangle = \sum_{i\in[n]} \langle v_{g(y),i} | v_{g(z),i} \rangle \sum_{j\colon y^i_j \neq z^i_j} \langle u_{y^i,j} | u_{z^i,j} \rangle \tag{24.11}$$

$$= \sum_{i\in[n]} \langle v_{g(y),i} | v_{g(z),i} \rangle (1 - \delta_{g(y^i),g(z^i)}) \tag{24.12}$$

$$= \sum_{i\colon g(y^i)\neq g(z^i)} \langle v_{g(y),i} | v_{g(z),i} \rangle \tag{24.13}$$

$$= 1 - \delta_{(f\circ g)(y),(f\circ g)(z)}. \tag{24.14}$$

Finally, since $\| |w_{y,(i,j)}\rangle \| = \| |v_{g(y),i}\rangle \| \cdot \| |u_{y^i,j}\rangle \|$, using (24.2) gives

$$\mathrm{Adv}^{\pm}(f \circ g) \leq \max_y \sum_i \| |v_{g(y),i}\rangle \|^2 \sum_j \| |u_{y^i,j}\rangle \|^2 \tag{24.15}$$

$$\leq \mathrm{Adv}^{\pm}(f)\, \mathrm{Adv}^{\pm}(g) \tag{24.16}$$

as claimed. □

Note that here we needed the constraint (24.3) in the case where $f(x) = f(y)$.

In particular, combining this with the dual adversary for OR and a similar solution for AND, this shows that $\mathrm{Adv}^{\pm}(f) \leq \sqrt{n}$ for the n-input balanced binary AND-OR tree.

24.6 An algorithm from a dual adversary solution

The dual adversary is significant not just because it gives upper bounds on $\mathrm{Adv}^{\pm}(f)$, but because it directly gives a quantum algorithm for evaluating f with quantum query complexity $O(\mathrm{Adv}^{\pm}(f))$ [93, 78]. (Note that the construction is not necessarily time-efficient—it may use many more elementary gates than queries—but many known algorithms developed using span programs have subsequently led to explicit, time-efficient algorithms.)

In particular, this shows that the quantum query complexity of the balanced binary AND-OR tree is $O(\sqrt{n})$. This was originally shown, up to some small overhead, using a continuous-time quantum walk algorithm based on scattering theory [47]. The classical query complexity of this problem is $O(n^{\log_2(\frac{1+\sqrt{33}}{4})}) = O(n^{0.753})$ [108, 96], and no better quantum algorithm was known for many years. From the perspective of span programs, the formula evaluation algorithm can be seen a method of recursive evaluation with no need for error reduction.

Similarly to the quantum walk search algorithms we discussed previously, the algorithm for the adversary dual uses a product of two reflections. Let $A = \mathrm{Adv}^{\pm}(f)$, and let Δ be the projector onto $\mathrm{span}\{|\psi_x\rangle \colon f(x) = 1\}$ where

$$|\psi_x\rangle := \frac{1}{\sqrt{\nu_x}} \Big(|0\rangle + \tfrac{1}{\sqrt{2A}} \sum_{i\in[n]} |i\rangle |v_{x,i}\rangle |x_i\rangle \Big) \tag{24.17}$$

with $\{|v_{x,i}\rangle\}$ an optimal dual adversary solution. Here the normalization factor is

$$\nu_x = 1 + \frac{1}{2A} \sum_{i\in[n]} \| |v_{x,i}\rangle \|^2 \leq \frac{3}{2}. \tag{24.18}$$

The reflection $2\Delta - I$ requires no queries to implement. Let $\Pi_x = |0\rangle\langle 0| + \sum_{i\in[n]} |i\rangle\langle i| \otimes I \otimes |x_i\rangle\langle x_i|$ be the projector onto $|0\rangle$ and states where the query and output registers are consistent. Then the reflection $2\Pi_x - I$ can be implemented using only two queries to the oracle O_x.

The algorithm runs phase estimation with precision $\Theta(1/A)$ on the unitary $U := (2\Pi_x - I)(2\Delta - I)$, with initial state $|0\rangle$. If the estimated phase is 1, then the algorithm reports that $f(x) = 1$; otherwise it

reports that $f(x) = 0$. This procedure uses $O(A)$ queries. It remains to see why the algorithm is correct with bounded error.

First, we claim that if $f(x) = 1$, then $|0\rangle$ is close to the 1-eigenspace of U. We have $\Pi_x|\psi_x\rangle = |\psi_x\rangle$ for all x and $\Delta|\psi_x\rangle = |\psi_x\rangle$ for $f(x) = 1$, so clearly $U|\psi_x\rangle = |\psi_x\rangle$. Furthermore, $|\langle 0|\psi_x\rangle|^2 = 1/\nu_x \geq 2/3$ for all x. Thus the algorithm is correct with probability at least $2/3$ when $f(x) = 1$.

On the other hand, we claim that if $f(x) = 0$, then $|0\rangle$ has small projection onto the subspace of eigenvectors with eigenvalue $e^{i\theta}$ for $\theta \leq c/A$, for some constant c. To prove this, we use the following [78]:

Lemma 24.3 (Effective spectral gap lemma)**.** *Let $|\phi\rangle$ be a unit vector with $\Delta|\phi\rangle = 0$; let P_ω be the projector onto eigenvectors of $U = (2\Pi - I)(2\Delta - I)$ with eigenvalues $e^{i\theta}$ with $|\theta| < \omega$ for some $\omega \geq 0$. Then $\|P_\omega \Pi|\phi\rangle\| \leq \omega/2$.*

Let

$$|\phi_x\rangle := \frac{1}{\sqrt{\mu_x}}\Big(|0\rangle - \sqrt{2A}\sum_{i\in[n]} |i\rangle|v_{x,i}\rangle|\bar{x}_i\rangle\Big), \tag{24.19}$$

where the normalization factor is

$$\mu_x = 1 + 2A\sum_{i\in[n]} \||v_{x,i}\rangle\|^2 \leq 1 + 2A^2. \tag{24.20}$$

For any y with $f(y) = 1$, we have

$$\langle\psi_y|\phi_x\rangle = \frac{1}{\sqrt{\nu_y\mu_x}}\Big(1 - \sum_{i:\ y_i\neq x_i} \langle v_{y,i}|v_{x,i}\rangle\Big) = 0, \tag{24.21}$$

so $\Delta|\phi_x\rangle = 0$. Also, observe that $\Pi_x|\phi_x\rangle = |0\rangle/\sqrt{\mu_x}$. By the effective spectral gap lemma, $\|P_\omega|0\rangle\| \leq \sqrt{\mu_x}\omega/2 \leq \sqrt{1+2A^2}\,\omega/2 \approx A\omega/\sqrt{2}$. Thus, choosing $\omega = \sqrt{\frac{2}{3}}\cdot\frac{1}{A}$ gives a projection of at most $1/\sqrt{3}$, so the algorithm fails with probability at most $1/3$ (plus the error of phase estimation, which can be made negligible, and the small error from approximating $1 + 2A^2 \approx 2A^2$, which is negligible if $A \gg 1$).

It remains to prove the lemma.

Proof. We apply Jordan's lemma, which says that for any two projections acting on the same finite-dimensional space, there is a decomposition of the space into a direct sum of one- and two-dimensional subspaces that are invariant under both projections.

We can assume without loss of generality that $|\phi\rangle$ only has support on 2×2 blocks of the Jordan decomposition in which Δ and Π both have rank one. If the block is 1×1, or if either projection has rank 0 or 2 within the block, then U acts as either $\pm I$ on the block; components with eigenvalue -1 are annihilated by P_ω, and components with eigenvalue $+1$ are annihilated by Π.

Now, by an appropriate choice of basis, restricting Δ and Π to any particular 2×2 block gives

$$\bar{\Delta} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \tag{24.22}$$

$$\bar{\Pi} = \begin{pmatrix} \cos\frac{\theta}{2} \\ \sin\frac{\theta}{2} \end{pmatrix}\begin{pmatrix} \cos\frac{\theta}{2} & \sin\frac{\theta}{2} \end{pmatrix} \tag{24.23}$$

where $\frac{\theta}{2}$ is the angle between the vectors projected onto within the two subspaces. A simple calculation shows that $(2\bar{\Pi} - I)(2\bar{\Delta} - I)$ is a rotation by an angle θ, so its eigenvalues are $e^{\pm i\theta}$. Since $\Delta|\phi\rangle = 0$, the component of $|\phi\rangle$ in the relevant subspace is proportional to $\binom{0}{1}$, and

$$\left\|\bar{\Pi}\begin{pmatrix} 0 \\ 1 \end{pmatrix}\right\| = \left\|\sin\tfrac{\theta}{2}\begin{pmatrix} \cos\frac{\theta}{2} \\ \sin\frac{\theta}{2} \end{pmatrix}\right\| = |\sin\tfrac{\theta}{2}| \leq \tfrac{\theta}{2} \tag{24.24}$$

as claimed. □

Chapter 25

Learning graphs

While span programs provide a powerful tool for proving upper bounds on quantum query complexity, they can be difficult to design. The model of learning graphs, introduced by Belovs [19], is a restricted class of span programs that are more intuitive to design and understand. This model has led to improved upper bounds for various problems, such as subgraph finding and k-distinctness.

25.1 Learning graphs and their complexity

A learning graph for an n-bit oracle is a directed, acyclic graph whose vertices are subsets of $[n]$, the set of indices of input bits. Edges of the learning graph can only connect vertices $\sigma \subset [n]$ and $\sigma \cup \{i\}$ for some $i \in [n] \setminus \sigma$. We interpret such an edge as querying index i, and we sometimes say that the edge $(\sigma, \sigma \cup \{i\})$ *loads* index i. Each edge e has an associated weight $w_e > 0$. We say that a learning graph computes $f\colon S \to \{0,1\}$ (where $S \subseteq \{0,1\}^n$) if, for all x with $f(x) = 1$, there is a path from $\varnothing$ to a 1-certificate for x (a subset of indices σ such that $f(y) = f(x)$ for all y such that $x_\sigma = y_\sigma$, where x_σ denotes the restriction of x to the indices in σ).

Associated to any learning graph for f is a complexity measure $\mathcal{C} = \sqrt{\mathcal{C}_0 \mathcal{C}_1}$, the geometric mean of the 0-complexity $\mathcal{C}_0$ and the 1-complexity $\mathcal{C}_1$. The 0-complexity is simply $\mathcal{C}_0 := \sum_e w_e$, where the sum is over all edges in the learning graph.

The definition of the 1-complexity is somewhat more involved. Associated to any x with $f(x) = 1$, we consider a *flow* in the learning graph, which assigns a value p_e to each edge so that for any vertex, the sum of all incoming flows equals the sum of all outgoing flows. There is a unit flow coming from vertex $\varnothing$; this is the only source. A vertex can be a sink if and only if it contains a 1-certificate for x. The complexity of any such flow is $\sum_e p_e^2 / w_e$. (Note that $w_e > 0$ for any edge in a learning graph, although we also have the possibility of omitting edges.) The complexity $\mathcal{C}_1(x)$ is the smallest complexity of any valid flow for x. Finally, we have $\mathcal{C}_1 := \max_{x \in f^{-1}(1)} \mathcal{C}_1(x)$, the largest complexity of any 1-input.

25.2 Unstructured search

Perhaps the simplest example of a learning graph is for the case of unstructured search. The learning graph simply loads an index. In other words, there is an edge of weight 1 from $\varnothing$ to $\{i\}$ for each $i \in [n]$. Clearly, we have $\mathcal{C}_0 = n$. To compute the 1-complexity, consider the input $x = e_i$ for some $i \in [n]$. For this input there is a unique 1-certificate, namely $\{i\}$. The only possible flow is the one with unit weight on the edge from $\varnothing$ to $\{i\}$. This gives $\mathcal{C}_1(e_i) = 1$ for all i, so $\mathcal{C}_1 = 1$. Therefore the complexity of this learning graph is $\mathcal{C} = \sqrt{\mathcal{C}_0 \mathcal{C}_1} = \sqrt{n}$.

It is not hard to see that the same learning graph works for the total function OR: for each x with $f(x) = 1$, we can send all flow to any particular i for which $x_i = 1$.

25.3 From learning graphs to span programs

We now show that every learning graph implies a dual adversary solution of the same complexity, so that the learning graph complexity is an upper bound on quantum query complexity, up to constant factors.

We construct vectors $|v_{x,i}\rangle$ for all $x \in S$. These vectors consist of a block for each vertex σ of the learning graph, with the coordinates within each block labeled by possible assignments of the bits in σ. Since we fix a particular index i, we can think of the blocks as labeling edges $e_{\sigma,i} := (\sigma, \sigma \cup \{i\})$. We define

$$|v_{x,i}\rangle = \begin{cases} \sum_{\sigma} \sqrt{w_{e_{\sigma,i}}} |\sigma, x_\sigma\rangle & \text{if } f(x) = 0 \\ \sum_{\sigma} \frac{p_{e_{\sigma,i}}}{\sqrt{w_{e_{\sigma,i}}}} |\sigma, x_\sigma\rangle & \text{if } f(x) = 1 \end{cases} \tag{25.1}$$

where the sums only run over those $\sigma \subset [n]$ for which $e_{\sigma,i}$ is an edge of the learning graph.

It is easy to check that this definition satisfies the dual adversary constraints. For any $x, y \in S$ with $f(x) = 0$ and $f(y) = 1$, we have

$$\sum_{i\colon x_i \neq y_i} \langle v_{x,i} | v_{y,i} \rangle = \sum_{i\colon x_i \neq y_i} \sum_{\sigma} \sqrt{w_{e_\sigma,i}} \frac{p_{e_{\sigma,i}}}{\sqrt{w_{e_{\sigma,i}}}} \langle x_\sigma | y_\sigma \rangle \tag{25.2}$$

$$= \sum_{i\colon x_i \neq y_i} \sum_{\sigma\colon x_\sigma = y_\sigma} p_{e_{\sigma,i}}. \tag{25.3}$$

Now observe that the set of edges $\{e_{\sigma,i}\colon x_\sigma = y_\sigma, x_i \neq y_i\}$ forms a cut in the graph between the vertex sets $\{\sigma\colon x_\sigma = y_\sigma\}$ and $\{\sigma\colon x_\sigma \neq y_\sigma\}$ Since $\varnothing$ is in the former set and all sinks are in the latter set, the total flow through the cut must be 1.

Recall that we do not have to satisfy the constraint for $f(x) = f(y)$ since there is a construction that enforces this condition without changing the complexity provided the condition for $f(x) \neq f(y)$ is satisfied.

It remains to see that the complexity of this dual adversary solution equals the original learning graph complexity. For $b \in \{0, 1\}$, we have

$$C_b = \max_{x \in f^{-1}(b)} \sum_{i \in [n]} \| |v_{x,i}\rangle \|^2 \tag{25.4}$$

$$= \max_{x \in f^{-1}(b)} \sum_{i \in [n]} \sum_{\sigma} \begin{cases} w_{e_{\sigma,i}} & \text{if } b = 0 \\ \frac{p^2_{e_{\sigma,i}}}{w_{e_{\sigma,i}}} & \text{if } b = 1 \end{cases} \tag{25.5}$$

$$= \begin{cases} \mathcal{C}_0 & \text{if } b = 0 \\ \max_{x \in f^{-1}(1)} \mathcal{C}_1(x) & \text{if } b = 1 \end{cases} \tag{25.6}$$

$$= \mathcal{C}_b. \tag{25.7}$$

Therefore $\sqrt{C_0 C_1} = \sqrt{\mathcal{C}_0 \mathcal{C}_1} = \mathcal{C}$ as claimed. In particular, $\mathrm{Adv}^{\pm}(f) \leq \mathcal{C}$, so $Q(f) = O(\mathcal{C})$.

Learning graphs are simpler to design than span programs: the constraints are automatically satisfied, so one can focus on optimizing the objective value. In contrast, span programs have exponentially many constraints (in n, if f is a total function), and in general it is not obvious how to even write down a solution satisfying the constraints.

Note, however, that learning graphs are not equivalent to general span programs. For example, learning graphs (as defined above) only depend on the 1-certificates of a function, so two functions with the same 1-certificates have the same learning graph complexity. The 2-threshold function (the symmetric Boolean function that is 1 iff two or more input bits are 1) has the same certificates as element distinctness, so its learning graph complexity is $\Omega(n^{2/3})$, whereas its query complexity is $O(\sqrt{n})$. This barrier can be circumvented by modifying the learning graph model, but even such variants are apparently less powerful than general span programs.

25.4 Element distinctness

We conclude by giving another simple example of a learning graph, one for element distinctness. (This requires generalizing learning graphs to non-Boolean input alphabet, but this generalization is straightforward.) We assume for simplicity that there is a unique collision—in fact, the analysis of the learning graph works for the general case by simply fixing one particular collision when designing a flow.

A convenient simplification is to break up the learning graph into k *stages*, which are simply subsets of the edges. To compute the complexity of a stage, we sum over only the edges in that stage. It is easy to see that there is a learning graph whose complexity is at most the sum of the complexities of the stages times the square root of the number of stages (which we will take to be constant). Let $\mathcal{C}_b^j$ denote the b-complexity of stage j. By dividing the weight of every edge in stage j by $\mathcal{C}_0^j$, we send $\mathcal{C}_0^j \to 1$ and $\mathcal{C}_1^j \to \mathcal{C}_0^j\mathcal{C}_1^j$. Then the total 0-complexity becomes $\mathcal{C}_0 = k$ and the total 1-complexity becomes

$$\mathcal{C}_1 = \sum_{j=1}^{k} \mathcal{C}_0^j\mathcal{C}_1^j \leq \left(\sum_{j=1}^{k} \sqrt{\mathcal{C}_0^j\mathcal{C}_1^j}\right)^2 \tag{25.8}$$

(since the 1-norm upper bounds the 2-norm), so $\mathcal{C} \leq \sqrt{k}\sum_{j=1}^{k}\sqrt{\mathcal{C}_0^j\mathcal{C}_1^j}$.

Another useful modification is to allow multiple vertices corresponding to the same subset of indices. It is straightforward to show that such learning graphs can be converted to span programs at the same cost, or to construct a new learning graph with no multiple vertices and the same or better complexity.

The learning graph for element distinctness has three stages. For the first stage, we load subsets of size $r-2$. We do this by first adding edges from $\varnothing$ to $\binom{n-i}{r-3}$ copies of vertex $\{i\}$, so that there are $\sum_{i=1}^{n}\binom{n-i}{r-3} = \binom{n}{r-2}$ singleton vertices. Then, from each of these singleton vertices, we load the remaining indices of each possible subset of size $r-2$, one index at a time. Every edge has weight 1. Then the 0-complexity of the first stage is $(r-2)\binom{n}{r-2}$.

To upper bound the 1-complexity of the first stage, we route flow only through vertices that do not contain the indices of a collision, sending equal flow of $\binom{n-2}{r-2}^{-1}$ to all subsets of size $r-2$. This gives 1-complexity of at most $(r-2)\binom{n-2}{r-2}\binom{n-2}{r-2}^{-2} = (r-2)\binom{n-2}{r-2}^{-1}$.

Overall, the complexity of the first stage is at most

$$\sqrt{(r-2)^2\binom{n}{r-2}\binom{n-2}{r-2}^{-1}} = (r-2)\sqrt{\frac{n(n-1)}{(n-r+2)(n-r+1)}} = O(r). \tag{25.9}$$

The second and third stages each include all possible edges that load one additional index from the terminal vertices of the previous stage. Again every edge has unit weight. Thus, the 0-complexity is $(n-r+2)\binom{n}{r-2}$ for the second stage and $(n-r+1)\binom{n}{r-1}$ for the third stage. We send the flow through vertices that contain the collision pair (namely, that contain the first index of a collision in the second stage and the second index of a collision in the third stage). Thus, the 1-complexity is $\binom{n-2}{r-2}\binom{n-2}{r-2}^{-2} = \binom{n-2}{r-2}^{-1}$ in both the second and the third stages. This gives total complexity

$$\sqrt{(n-r+2)\binom{n}{r-2}\binom{n-2}{r-2}^{-1}} = O(\sqrt{n}) \tag{25.10}$$

for the second stage and

$$\sqrt{(n-r+1)\binom{n}{r-1}\binom{n-2}{r-2}^{-1}} = \sqrt{\frac{n(n-1)}{(r-1)}} = O(n/\sqrt{r}) \tag{25.11}$$

for the third stage.

Overall, the complexity is $O(r + \sqrt{n} + n/\sqrt{r})$. This optimized by choosing r to equate the first and last terms, giving $r = n^{2/3}$. The overall complexity is $O(n^{2/3})$, matching the optimal quantum walk search algorithm.

25.5 Other applications

The simple examples discussed above only involve problems for which the optimal query complexity was previously known using other techniques. However, several new quantum query upper bounds have been given using learning graphs. These include improved algorithms for the triangle problem [19, 77] (and more generally, subgraph finding [76, 115, 77], with an application to associativity testing [77]) and the k-distinctness problem [19]. (Note that the algorithm for k-distinctness uses a subtle modification of the learning graph framework.) Unfortunately, the details of these algorithms are beyond the scope of the course.

Chapter 26

The compressed oracle method

We now discuss another lower bound technique called the *compressed oracle method* [114]. This approach is specifically adapted to the case where the oracle is chosen uniformly at random (as in the *random oracle model*, a well-studied setting for cryptographic hardness proofs), but it can also be used to prove lower bound results of the kind discussed in previous lectures, since average-case hardness implies worst-case hardness.

The compressed oracle method gives an arguably simpler proof of the lower bound for collision finding than the one based on the polynomial method. It has also been used to prove lower bounds for k-collision, k-sum, models of parallel query complexity, and in a variety of cryptographic scenarios.

26.1 The compressed oracle

Suppose we have oracle access to a uniformly random function $f\colon [n] \to \Sigma$, where $\Sigma = \{0,1\}^m$. As usual, a quantum query to this function acts as $|x\rangle|z\rangle \mapsto |x\rangle|z \oplus f(x)\rangle$.

We can implement queries to this oracle by adding a register encoding the truth table of the oracle as $|y_1, \dots, y_n\rangle$, where $y_x = f(x)$, and having a query act according to the unitary

$$\Lambda\colon |x\rangle|z\rangle|y_1,\dots,y_n\rangle \mapsto |x\rangle|z \oplus y_x\rangle|y_1,\dots,y_n\rangle \tag{26.1}$$

that looks up the function value in the appropriate oracle register and adds it into the output register. If the oracle registers are inaccessible to the algorithm, then this is indistinguishable from having black-box access to f. (This is essentially the same as the "super-oracle" we considered when discussing the adversary method, just with slightly different notation.) Furthermore, if we start the oracle registers in a uniform superposition, then this is indistinguishable from an oracle for a uniformly random function.

The idea of the compressed oracle is for the oracle to generate responses as queries are received, recording what has been queried in a way that cannot be detected by the algorithm. We introduce a symbol $\perp$ for the oracle registers, indicating that a given entry has not yet been queried. All oracle registers begin in the $\perp$ state. When a query is made, the oracle effectively acts in a random way if that item has not been queried previously, and acts consistently with previous results if that item has been queried. Of course, we have to implement this unitarily to have a valid quantum process.

More concretely, we define the compressed oracle as follows. For $y \in \Sigma$, define $|\hat{y}\rangle := H^{\otimes m}|y\rangle$ where H is the Hadamard gate. Define a "compression/decompression" unitary C acting on $\operatorname{span}(\{|y\rangle : y \in \Sigma\} \cup \{|\perp\rangle\})$, where $|\perp\rangle$ is a normalized state orthogonal to all $|y\rangle$ for $y \in \Sigma$, as

$$C|\perp\rangle = |\hat{0}\rangle = \frac{1}{\sqrt{|\Sigma|}} \sum_{y\in\Sigma} |y\rangle \tag{26.2}$$

$$C|\hat{0}\rangle = |\perp\rangle \tag{26.3}$$

$$C|\hat{y}\rangle = |\hat{y}\rangle \text{ for all } y \in \Sigma \setminus \{0\}. \tag{26.4}$$

The compressed oracle acts as $(I \otimes I \otimes C^{\otimes n})\Lambda(I \otimes I \otimes C^{\otimes n})$, where we extend Λ to act as the identity on $|x\rangle|z\rangle|\perp\rangle$ for all $x \in [n]$, $z \in \Sigma$. In other words, it performs the (de)compression operation (which is its own inverse) on each oracle register before and after every query.

We can think of the compressed oracle as reversibly "recording" queries made by the algorithm. The symbol $\perp$ in register x indicates that the value y_x has not been queried by the algorithm. When presented with a new query, the oracle first "decompresses" by mapping $|\perp\rangle$ onto a uniform superposition $|\hat{0}\rangle$ of possible oracle contents, corresponding to a uniformly random assignment of y_x. Then the oracle acts acccording to a normal query, and finally "compresses" the contents of the oracle, mapping $|\hat{0}\rangle$ back to $|\perp\rangle$. On the other hand, the oracle should not change the contents of an oracle register that has been queried, so it leaves states $|\hat{y}\rangle$ for $y \neq 0$ unchanged. Note that since a query can only map one oracle register in the unspecified state $|\perp\rangle$ to a state in the specified subspace $\text{span}\{|y\rangle : y \in \Sigma\}$, after t queries the oracle registers can contain at most t non-$\perp$ entries.

To an algorithm that cannot access the oracle register, this is indistinguishable from the behavior of a random oracle because the compression operation commutes with the non-query operations, so the compression operations cancel—except at the beginning of the algorithm, to map $|\perp\rangle$ to $|\hat{0}\rangle$, and at the end of the algorithm, performing a change of basis that does not affect the state of the non-query registers.

Since the compressed oracle is merely a change of basis in the final step, it may not seem to provide much new insight. But it turns out that this is a very useful basis for understanding the progress of an algorithm.

26.2 Relating the standard and compressed oracles

We now prove a key technical lemma that connects the behavior of the standard and compressed oracles. Of course, as observed above, the behaviors of these two settings are exactly the same as far as the algorithm is concerned, and differ only in a transformation applied to the oracle register at the end of the algorithm. Nevertheless, we will prove a result that relates the success probability of an algorithm with the standard oracle to outcomes obtained when measuring the oracle register in the compressed oracle setting, which is useful for showing query lower bounds.

Lemma 26.1. *Let $R \subseteq [n] \times \Sigma$ be a set of input-output pairs. Suppose a quantum algorithm making queries to a random oracle outputs k input-output pairs $(x_1, z_1), \ldots, (x_k, z_k)$ such that, with probability p, $f(x_i) = z_i$ for all i and the pairs come from R. Alternatively, suppose the algorithm is run with the compressed oracle and, upon a measurement of the oracle register, we obtain result y_i in each oracle register x_i for $i \in [k]$. Let p' be the probability that $z_i = y_i$ for all i and the input-output pairs come from R. Then $\sqrt{p} \leq \sqrt{p'} + \sqrt{k/2^m}$.*

We follow the proof presented in [39, Lemma 4.1 and Corollary 4.2], except that we take a different (and tighter) approach to the final calculation.

Proof. Let $|\psi\rangle$ be the state prepared by the algorithm in the simulation of a random oracle where the oracle begins in the state $|\hat{0}\rangle^{\otimes n}$ and the algorithm alternates between lookup unitaries Λ and arbitrary non-query operations. The only difference between the two procedures is in the last step, where the compressed oracle applies the transformation $C^{\otimes n}$ to the oracle register.

Let $\Pi_x = |y_1\rangle\langle y_1| \otimes \cdots \otimes |y_k\rangle\langle y_k|$ be the projector onto states of the oracle registers corresponding to the evaluations of f at $x = (x_1, \ldots, x_k)$. Then (supposing without loss of generality that the algorithm also measures its final workspace registers, say in the computational basis, giving an outcome w) we have

$$p = \sum_{x,y \in R} \sum_{w} \Pr(\text{algorithm outputs } x, y, w) \Pr(f(x) = y \mid \text{algorithm outputs } x, y, w) \tag{26.5}$$

$$= \sum_{x,y \in R} \sum_{w} p_{x,y,w} \|\Pi_x |\phi_{x,y,w}\rangle\|^2, \tag{26.6}$$

where we define $p_{x,y,w} := \Pr(\text{algorithm outputs } x, y, w)$, and $|\phi_{x,y,w}\rangle$ is the state of the oracle registers, postselected on the algorithm outputting x, y, w. Similarly,

$$p' = \sum_{x,y \in R} \sum_{w} p_{x,y,w} \|\Pi_x C^{\otimes n} |\phi_{x,y,w}\rangle\|^2. \tag{26.7}$$

For any state $|\phi\rangle$ with no support on states $|\perp\rangle$ in the oracle registers (as for the states $|\phi_{x,y,w}\rangle$, which are produced without use of the compressed oracle), we claim that

$$\|\Pi_x |\phi\rangle\| \leq \|\Pi_x C^{\otimes n} |\phi\rangle\| + \sqrt{k/2^m}. \tag{26.8}$$

Assuming this claim holds, we have

$$p = \mathbb{E}_{x,y,w} \|\Pi_x|\phi_{x,y,w}\rangle\|^2 \tag{26.9}$$
$$\leq \mathbb{E}_{x,y,w}(\|\Pi_x C^{\otimes n}|\phi_{x,y,w}\rangle\| + \sqrt{k/2^m})^2 \tag{26.10}$$

where the expectation is over the probability distribution $p_{x,y,w}$. Now we observe that for any random variable A and constant c,

$$\mathbb{E}(A+c)^2 = \mathbb{E}(A^2 + 2Ac + c^2) \tag{26.11}$$
$$\leq \mathbb{E}\, A^2 + 2\sqrt{\mathbb{E}\, A^2}c + c^2 \tag{26.12}$$
$$= (\sqrt{\mathbb{E}\, A^2} + c)^2 \tag{26.13}$$

where we have used the fact that A^2 is a convex function of A, so $(\mathbb{E}\, A)^2 \leq \mathbb{E}\, A^2$. Applying this to (26.10) gives

$$p \leq \left(\sqrt{\mathbb{E}_{x,y,w} \|\Pi_x C^{\otimes n}|\phi_{x,y,w}\rangle\|^2} + \sqrt{k/2^m}\right)^2 \tag{26.14}$$
$$= (\sqrt{p'} + \sqrt{k/2^m})^2 \tag{26.15}$$

as claimed.

It remains to show (26.8). We can focus on the registers storing $y_1, \ldots, y_k$, since Π_x acts as the identity on the other registers and C acts unitarily there, so dropping those registers does not affect the spectral norm. Letting C_x denote the corresponding operator on the relevant registers, we have

$$\||\phi\rangle\| - \|\Pi_x C^{\otimes n}|\phi\rangle\| = \||\phi\rangle\| - \|\Pi_x C_x|\phi\rangle\| \tag{26.16}$$
$$\leq \|(\Pi_x - \Pi_x C_x)|\phi\rangle\| \tag{26.17}$$

by the triangle inequality. Now use the fact that $\|M\|^2 = \|MM^\dagger\|$ to write

$$\|\Pi_x - \Pi_x C_x\|^2 = \|\Pi_x - \Pi_x C_x \Pi_x - \Pi_x C_x^\dagger \Pi_x + \Pi_x C_x C_x^\dagger \Pi_x\|. \tag{26.18}$$

Since we only apply C to states with no support on $|\perp\rangle$ in any oracle register, and Π_x has no support on $|\perp\rangle$, we have

$$\Pi_x C_x = \bigotimes_i |y_i\rangle\langle y_i|(I - |\hat{0}\rangle\langle\hat{0}|) \tag{26.19}$$

where I represents the identity on $\mathrm{span}\{|y\rangle : y \in \Sigma\}$, so

$$\Pi_x C_x \Pi_x = \bigotimes_i |y_i\rangle\langle y_i|(I - |\hat{0}\rangle\langle\hat{0}|)|y_i\rangle\langle y_i| \tag{26.20}$$
$$= \bigotimes_i |y_i\rangle\langle y_i|(1 - \tfrac{1}{2^m}) \tag{26.21}$$
$$= (1 - \tfrac{1}{2^m})^k \Pi_x. \tag{26.22}$$

We similarly find $\Pi_x C_x^\dagger \Pi_x = (1 - \frac{1}{2^m})^k \Pi_x$. Finally,

$$\Pi_x C_x C_x^\dagger \Pi_x = \bigotimes_i |y_i\rangle\langle y_i|(I - |\hat{0}\rangle\langle\hat{0}|)^2|y_i\rangle\langle y_i| \tag{26.23}$$
$$= \bigotimes_i |y_i\rangle\langle y_i|(I - |\hat{0}\rangle\langle\hat{0}|)|y_i\rangle\langle y_i| \tag{26.24}$$
$$= (1 - \tfrac{1}{2^m})^k \Pi_x \tag{26.25}$$

is again the same operator. Putting this together in (26.18), we have

$$\|\Pi_x - \Pi_x C_x\|^2 = \left\|\left(1 - (1 - \tfrac{1}{2^m})^k\right)\Pi_x\right\| \tag{26.26}$$
$$= 1 - (1 - \tfrac{1}{2^m})^k \tag{26.27}$$
$$\leq k/2^m, \tag{26.28}$$

which completes the proofs. □

This lets us connect easier-to-calculate probabilities for the compressed oracle to probabilities for the actual algorithm, as we will see in two example.

26.3 Unstructured search

As a first example, we consider the unstructured search problem. To model unstructured search in the context of a random oracle, we consider the task of finding an x for which $f(x) = 0$, with parameters chosen so that this happens with constant probability (say, take $n = 2^m$).

We will show that the probability of a t-query algorithm outputting a pair $(x, 0)$ is $O(t^2/2^m)$. To do this, we show that the algorithm using the compressed oracle can only increase the projection onto a state with 0^n in one of the oracle registers by at most $\sqrt{2/2^m}$ per query. By Lemma 26.1, an algorithm using the standard oracle can output such a pair with probability larger than this by at most $O(t^2/2^m)$, so this shows that the success probability is $O(t^2/2^m)$. Then it follows that $\Omega(2^{m/2})$ queries are needed to find a preimage of 0 (i.e., solve unstructured search) with constant probability.

It is convenient to work with the phase oracle, which conjugates the output register by a Hadamard transform to act as $|x, z\rangle \mapsto (-1)^{z \cdot f(x)}|x, z\rangle$. The compressed phase oracle is

$$\Phi := (I \otimes H^{\otimes m} \otimes C^{\otimes n})\Lambda(I \otimes H^{\otimes m} \otimes C^{\otimes n}) \tag{26.29}$$

where H is the Hadamard gate.

We can expand a general state of the algorithm as

$$|\psi\rangle = \sum_{x,z,w,y} \alpha_{x,z,w,y}|x, z, w, y\rangle \tag{26.30}$$

where $x \in [n]$ is the input, $z \in \Sigma$ is the output, w is the workspace (consisting of any number of qubits), and $y \in (\Sigma \cup \{\perp\})^n$ is the oracle register. We partition the computational basis states into four types, defining four corresponding projections:

- P: oracle contains a zero ($\exists x^* : y_{x^*} = 0$);
- Q: oracle contains no zero, not making a null query ($z \neq 0$), querying an unspecified position ($y_x = \perp$);
- R: oracle contains no zero, not making a null query ($z \neq 0$), querying a specified position ($y_x \neq \perp$); and
- S: oracle contains no zero, making a null query ($z = 0$).

Clearly $P + Q + R + S = I$.

Our goal is to show the following.

Fact 26.2. $\|P\Phi|\psi\rangle\| \leq \|P|\psi\rangle\| + \sqrt{2/2^m}$.

Proof. We have

$$\|P\Phi|\psi\rangle\| = \|P\Phi(P + Q + R + S)|\psi\rangle\| \tag{26.31}$$
$$\leq \|P\Phi P|\psi\rangle\| + \|P\Phi Q|\psi\rangle\| + \|P\Phi R|\psi\rangle\| + \|P\Phi S|\psi\rangle\|. \tag{26.32}$$

For the first term, clearly $\|P\Phi P|\psi\rangle\| \leq \|P|\psi\rangle\|$.

For the second term, consider a basis state $|x, z, \perp_x\rangle$ in the support of Q (ignoring the workspace, and only writing the oracle register corresponding to x, using a subscript to emphasize its register). Observe that $C =$

$|\perp\rangle\langle\hat{0}|+|\hat{0}\rangle\langle\perp|+\sum_{y\neq 0}|\hat{y}\rangle\langle\hat{y}| = I+|\perp\rangle\langle\hat{0}|+|\hat{0}\rangle\langle\perp|-|\hat{0}\rangle\langle\hat{0}|$ (where I is over Σ), so $C|y\rangle = |y\rangle + \frac{1}{\sqrt{2^m}}(|\perp\rangle - |\hat{0}\rangle)$. Therefore, the phase query Φ maps $|x,z,\perp_x\rangle$ to

$$\frac{1}{\sqrt{2^m}}\sum_y (-1)^{z\cdot y}|x,z\rangle C|y_x\rangle = \frac{1}{\sqrt{2^m}}\sum_y (-1)^{z\cdot y}|x,z\rangle(|y_x\rangle + \frac{1}{\sqrt{2^m}}(|\perp_x\rangle - |\hat{0}_x\rangle)) \tag{26.33}$$

$$= \frac{1}{\sqrt{2^m}}\sum_y (-1)^{z\cdot y}|x,z,y_x\rangle. \tag{26.34}$$

Applying P to this state gives $\frac{1}{\sqrt{2^m}}|x,z,0_x\rangle$ (since none of the suppressed oracle registers are in the state 0). Since different basis states are mapped to orthogonal states, it follows that $\|P\Phi Q|\psi\rangle\| \leq \frac{1}{\sqrt{2^m}}\|Q|\psi\rangle\|$.

For the third term, for a basis state $|x,z,y_x\rangle$ in the support of R, we find

$$(I\otimes I\otimes C)\Phi|x,z\rangle C|y_x\rangle = (I\otimes I\otimes C)\Phi|x,z\rangle(|y_x\rangle + \frac{1}{\sqrt{2^m}}(|\perp_x\rangle - |\hat{0}_x\rangle)) \tag{26.35}$$

$$= (I\otimes I\otimes C)\Phi|x,z\rangle(|y_x\rangle + \frac{1}{\sqrt{2^m}}|\perp_x\rangle - \frac{1}{2^m}\sum_v |v_x\rangle)) \tag{26.36}$$

$$= (I\otimes I\otimes C)[|x,z\rangle((-1)^{z\cdot y}|y_x\rangle + \frac{1}{\sqrt{2^m}}|\perp_x\rangle - \frac{1}{2^m}\sum_v (-1)^{z\cdot v}|v_x\rangle))] \tag{26.37}$$

$$= |x,z\rangle[(-1)^{z\cdot y}(|y_x\rangle + \frac{1}{\sqrt{2^m}}(|\perp_x\rangle - |\hat{0}_x\rangle)) + \frac{1}{\sqrt{2^m}}|\hat{0}_x\rangle$$
$$- \frac{1}{2^m}\sum_v (-1)^{z\cdot v}(|v_x\rangle + \frac{1}{\sqrt{2^m}}(|\perp_x\rangle - |\hat{0}_x\rangle)] \tag{26.38}$$

$$= |x,z\rangle[(-1)^{z\cdot y}|y_x\rangle + \frac{(-1)^{z\cdot y}}{\sqrt{2^m}}|\perp_x\rangle + \frac{1-(-1)^{z\cdot y}}{\sqrt{2^m}}|\hat{0}_x\rangle - \frac{1}{\sqrt{2^m}}|\hat{z}_x\rangle]. \tag{26.39}$$

Since $y\neq 0$, applying P to this state gives $(\frac{1-(-1)^{z\cdot y}}{2^m} - \frac{1}{2^m})|x,z,0_x\rangle = -\frac{(-1)^{z\cdot y}}{2^m}|x,z,0_x\rangle$.

In this case we find that different basis states can be mapped to the same output under the projection. But for a general input state $\sum_{x,z,w,y}\alpha_{x,z,w,y}|x,z,w,y\rangle$, we have output $-\frac{1}{2^m}\sum_{x,z,w,y}\alpha_{x,z,w,y}(-1)^{z\cdot y_x}|x,z,w,\bar{y}\rangle$ (where $\bar{y}$ has y_x replaced by 0), which has squared norm

$$\frac{1}{4^m}\sum_{x,z,w,\bar{y}}\left|\sum_{y_x}\alpha_{x,z,w,y}(-1)^{z\cdot y_x}\right|^2 \leq \frac{1}{2^m}\sum_{x,w,y}\sum_z |\alpha_{x,z,w,\bar{y}}|^2 = \frac{1}{2^m}\|R|\psi\rangle\|^2 \tag{26.40}$$

by Cauchy-Schwarz. Therefore $\|P\Phi R|\psi\rangle\| \leq \frac{1}{\sqrt{2^m}}\|R|\psi\rangle\|$.

Finally, for the fourth term, $\Phi S|\psi\rangle = S|\phi\rangle$, so we have $P\Phi S|\phi\rangle = 0$.

Combining the above facts, we have

$$\|P\Phi|\psi\rangle\| \leq \|P|\psi\rangle\| + \frac{1}{\sqrt{2^m}}(\|Q|\psi\rangle\| + \|R|\psi\rangle\|) \tag{26.41}$$

$$\leq \|P|\psi\rangle\| + \sqrt{\frac{2}{2^m}} \tag{26.42}$$

since Q and R project onto orthogonal subspaces. □

26.4 Collision finding

While the above lower bound for unstructured search is a bit more complicated than the adversary or polynomial method lower bounds for that problem, we can give an analogous lower bound for collision finding that is arguably simpler than the collision lower bound by the polynomial method.

Consider the problem of finding a collision in a random function $f\colon [n]\to\{0,1\}^m$. If we take $n = \Theta(2^{m/2})$ then we expect some m-bit string to have multiple preimages with constant probability (by the birthday

problem), and we can show that $\Omega(n^{2/3})$ quantum queries are necessary to find such a collision, establishing the lower bound for element distinctness. On the other hand, if we take $n = \Theta(2^m)$, then each string in $\{0,1\}^m$ has a constant number of preimages in expectation, and we can show that $\Omega(n^{1/3})$ quantum queries are necessary to find a collision. This is analogous to the collision lower bound proven in Chapter 22, though not in the setting where the function is exactly r-to-1 for some constant r.

To establish the bound, we now define P to project onto states for which the oracle registers contain a collision, and define Q, R, S analogously as before.

A similar calculation as for unstructured search shows that after t queries, $\|P\Phi Q|\psi\rangle\| \leq \sqrt{t/2^m}\|Q|\psi\rangle\|$. In particular, the same calculation as in (26.34) shows that a phase query maps a basis state $|x, z, \perp_x, y_{\bar{x}}\rangle$ in the support of Q, where $y_{\bar{x}}$ denotes the contents of the oracle for registers other than x, to $P\Phi Q|x, z, \perp_x, y_{\bar{x}}\rangle = \frac{1}{\sqrt{2^m}} \sum_{y_x \in y_{\bar{x}}} |x, z, y_x, y_{\bar{x}}\rangle$, where $y_x \in y_{\bar{x}}$ means that y_x is any of the non-$\perp$ entries of $y_{\bar{x}}$. Since there are at most t non-$\perp$ entries in $y_{\bar{x}}$, the squared norm of this state is at most $t/2^m$, so its norm is at most $\sqrt{t/2^m}$.

Similarly, it can be shown that $\|P\Phi R|\psi\rangle\| \leq \sqrt{t/2^m}\|R|\psi\rangle\|$. Also, we again have $\|P\Phi P|\psi\rangle\| \leq \|P|\psi\rangle\|$ and $P\Phi S|\phi\rangle = 0$.

Putting these facts together, we find that

$$\|P\Phi|\psi\rangle\| \leq \|P\Phi P|\psi\rangle\| + \|P\Phi Q|\psi\rangle\| + \|P\Phi R|\psi\rangle\| + \|P\Phi S|\psi\rangle\| \tag{26.43}$$

$$\leq \|P|\psi\rangle\| + \sqrt{\frac{2t}{2^m}}. \tag{26.44}$$

It follows that $\|P|\psi\rangle\| \leq \sqrt{2t^3/2^m}$ after t queries, so $t = \Omega(2^{m/3})$ to find a collision with constant probability. In particular, with $n = \Theta(2^{m/2})$, this gives the $t = \Omega(n^{2/3})$ lower bound for element distinctness.

Part V

Quantum simulation

Chapter 27

Simulating Hamiltonian dynamics

Another major potential application of quantum computers is the simulation of quantum dynamics. Indeed, this was the idea that first led Feynman to propose the concept of a quantum computer [50]. In this lecture we will see how a universal quantum computer can efficiently simulate several natural families of Hamiltonians. These simulation methods could be used either to simulate actual physical systems or to implement quantum algorithms defined in terms of Hamiltonian dynamics, such as continuous-time quantum walks (Part III) and adiabatic quantum algorithms (Part VI).

27.1 Hamiltonian dynamics

In quantum mechanics, time evolution of the wave function $|\psi(t)\rangle$ is governed by the Schrödinger equation,

$$i\hbar\frac{\mathrm{d}}{\mathrm{d}t}|\psi(t)\rangle = H(t)|\psi(t)\rangle. \tag{27.1}$$

Here $H(t)$ is the *Hamiltonian*, an operator with units of energy, and $\hbar$ is Planck's constant. For convenience it is typical to choose units in which $\hbar = 1$. Given an initial wave function $|\psi(0)\rangle$, we can solve this differential equation to determine $|\psi(t)\rangle$ at any later (or earlier) time t.

For H independent of time, the solution of the Schrödinger equation is $|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle$. For simplicity we will only consider this case. There are many situations in which time-dependent Hamiltonians arise, not only in physical systems but also in computational applications such as adiabatic quantum computing. In such cases, the evolution cannot in general be written in such a simple form, but nevertheless similar ideas can be used to simulate the dynamics.

27.2 Efficient simulation

We will say that a Hamiltonian H acting on n qubits can be *efficiently simulated* if for any $t > 0$, $\epsilon > 0$ there is a quantum circuit U consisting of $\mathrm{poly}(n, t, 1/\epsilon)$ gates such that $\|U - e^{-iHt}\| < \epsilon$. Clearly, the problem of simulating Hamiltonians in general is BQP-hard, since we can implement any quantum computation by a sequence of Hamiltonian evolutions. In fact, even with natural restrictions on the kind of Hamiltonians we consider, it is easy to specify Hamiltonian simulation problems that are BQP-complete (or more precisely, PromiseBQP-complete).

You might ask why we define the notion of efficient simulation to be polynomial in t; if t is given as part of the input, this means that the running time is, strictly speaking, not polynomial in the input size. However, one can show that a running time polynomial in $\log t$ is impossible; running time $\Omega(t)$ is required in general (intuitively, one cannot "fast forward" the evolution according to a generic Hamiltonian) [21]. The dependence on ϵ is more subtle. In fact, it is possible to achieve running time logarithmic in $1/\epsilon$, as we discuss further in Chapter 28.

We would like to understand the conditions under which a Hamiltonian can be efficiently simulated. Of course, we cannot hope to efficiently simulate arbitrarily Hamiltonians, just as we cannot hope to efficiently

implement arbitrary unitaries. Instead, we will simply describe a few classes of Hamiltonian that can be efficiently simulated. Our strategy will be to start from simple Hamiltonians that are easy to simulate and define ways of combining the known simulations to give more complicated ones.

There are a few cases where a Hamiltonian can obviously simulated efficiently. For example, this is the case if H only acts nontrivially on a constant number of qubits, simply because any unitary evolution on a constant number of qubits can be approximated with error at most ϵ using $\mathrm{poly}(\log \frac{1}{\epsilon})$ one- and two-qubit gates, using the Solovay-Kitaev theorem.

Note that since we require a simulation for an arbitrary time t (with $\mathrm{poly}(t)$ gates), we can rescale the evolution by any polynomial factor: if H can be efficiently simulated, then so can cH for any $c = \mathrm{poly}(n)$. This holds even if $c < 0$, since any efficient simulation is expressed in terms of quantum gates, and can simply be run in reverse.

In addition, we can rotate the basis in which a Hamiltonian is applied using any unitary transformation with an efficient decomposition into basic gates. In other words, if H can be efficiently simulated and the unitary transformation U can be efficiently implemented, then $UHU^\dagger$ can be efficiently simulated. This follows from the simple identity

$$e^{-iUHU^\dagger t} = Ue^{-iHt}U^\dagger. \tag{27.2}$$

Another simple but useful trick for simulating Hamiltonians is the following. Suppose H is diagonal in the computational basis, and any diagonal element $d(a) = \langle a|H|a\rangle$ can be computed efficiently. Then H can be simulated efficiently using the following sequence of operations, for any input computational basis state $|a\rangle$:

$$|a, 0\rangle \mapsto |a, d(a)\rangle \tag{27.3}$$
$$\mapsto e^{-itd(a)}|a, d(a)\rangle \tag{27.4}$$
$$\mapsto e^{-itd(a)}|a, 0\rangle \tag{27.5}$$
$$= e^{-iHt}|a\rangle|0\rangle. \tag{27.6}$$

By linearity, this process simulates H for time t on an arbitrary input.

Note that if we combine this simulation with the previous one, we have a way to simulate any Hamiltonian that can be efficiently diagonalized, and whose eigenvalues can be efficiently computed.

27.3 Product formulas

Many natural Hamiltonians have the form of a sum of terms, each of which can be simulated by the techniques described above. For example, consider the Hamiltonian of a particle in a potential:

$$H = \frac{p^2}{2m} + V(x).$$

To simulate this a digital quantum computer, we can imagine discretizing the x coordinate. The operator $V(x)$ is diagonal, and natural discretizations of $p^2 = -\mathrm{d}^2/\mathrm{d}x^2$ are diagonal in the discrete Fourier basis. Thus we can efficiently simulate both $V(x)$ and $p^2/2m$. Similarly, consider the Hamiltonian of a spin system, say of the form

$$H = \sum_i h_i X_i + \sum_{ij} J_{ij} Z_i Z_j$$

(or more generally, any k-local Hamiltonian, a sum of terms that each act on at most k qubits). This consists of a sum of terms, each of which acts only only a constant number of qubits and hence is easy to simulate.

In general, if H_1 and H_2 can be efficiently simulated, then $H_1 + H_2$ can also be efficiently simulated. If the two Hamiltonians commute, then this is trivial, since $e^{-iH_1t}e^{-iH_2t} = e^{-i(H_1+H_2)t}$. However, in the general case where the two Hamiltonians do not commute, we can still simulate their sum as a consequence of the Lie product formula

$$e^{-i(H_1+H_2)t} = \lim_{m\to\infty} \left(e^{-iH_1t/m}e^{-iH_2t/m}\right)^m. \tag{27.7}$$

A simulation using a finite number of steps can be achieved by truncating this expression to a finite number of terms, which introduces some amount of error that must be kept small. In particular, if we want to have

$$\left\| \left(e^{-iH_1 t/m} e^{-iH_2 t/m} \right)^m - e^{-i(H_1+H_2)t} \right\| \leq \epsilon, \tag{27.8}$$

it suffices to take $m = O((\nu t)^2/\epsilon)$, where $\nu := \max\{\|H_1\|, \|H_2\|\}$. (The requirement that H_1 and H_2 be efficiently simulable means that ν can be at most $\mathrm{poly}(n)$.)

It is somewhat unappealing that to simulate an evolution for time t, we need a number of steps proportional to t^2. Fortunately, the situation can be improved if we use higher-order approximations of (27.7). For example, one can show that

$$\left\| \left(e^{-iH_1 t/2m} e^{-iH_2 t/m} e^{-iH_1 t/2m} \right)^m - e^{-i(H_1+H_2)t} \right\| \leq \epsilon \tag{27.9}$$

with a smaller value of m. In fact, by using even higher-order approximations, it is possible to show that $H_1 + H_2$ can be simulated for time t with only $O(t^{1+\delta})$, for any fixed $\delta > 0$, no matter how small [32, 21].

A Hamiltonian that is a sum of polynomially many terms can be efficiently simulated by composing the simulation of two terms, or by directly using an approximation to the identity

$$e^{-i(H_1+\cdots+H_k)t} = \lim_{m\to\infty} \left(e^{-iH_1 t/m} \cdots e^{-iH_k t/m} \right)^m. \tag{27.10}$$

Another way of combining Hamiltonians comes from commutation: if H_1 and H_2 can be efficiently simulated, then $i[H_1, H_2]$ can be efficiently simulated. This is a consequence of the identity

$$e^{[H_1,H_2]t} = \lim_{m\to\infty} \left(e^{-iH_1\sqrt{t/m}} e^{-iH_2\sqrt{t/m}} e^{iH_1\sqrt{t/m}} e^{iH_2\sqrt{t/m}} \right)^m, \tag{27.11}$$

which can again be approximated with a finite number of terms. However, I don't know of any algorithmic application of such a simulation.

27.4 Sparse Hamiltonians

We will say that an $N \times N$ Hermitian matrix is *sparse* (in a fixed basis) if, in any fixed row, there are only $\mathrm{poly}(\log N)$ nonzero entries [7]. The simulation techniques described above allow us to efficiently simulate sparse Hamiltonians. More precisely, suppose that for any a, we can efficiently determine all of the bs for which $\langle a|H|b\rangle$ is nonzero, as well as the values of the corresponding matrix elements; then H can be efficiently simulated. In particular, this gives an efficient implementation of the continuous-time quantum walk on any graph $G = (V, E)$ whose maximum degree is $\mathrm{poly}(\log |V|)$.

The basic idea of the simulation is to edge-color the graph, simulate the edges of each color separately, and combine these pieces using (27.7). The main new technical ingredient in the simulation is a means of coloring the edges of the graph of nonzero matrix elements of H. A classic result in graph theory (Vizing's Theorem) says that a graph of maximum degree d has an edge coloring with at most $d + 1$ colors (in fact, the edge chromatic number is either d or $d + 1$). If we are willing to accept a polynomial overhead in the number of colors used, then we can actually find an edge coloring using only local information about the graph. Here we describe a simple d^2-coloring for the case of a bipartite graph, which is sufficient for general Hamiltonian simulation using a simple reduction [23].

Lemma 27.1. *Suppose we are given an undirected, bipartite graph G with N vertices and maximum degree d, and that we can efficiently compute the neighbors of any given vertex. Then there is an efficiently computable edge coloring of G with at most d^2 colors.*

Proof. Number the vertices of G from 1 through N. For any vertex α, let $\mathrm{idx}(\alpha, \beta)$ denote the index of vertex β in the list of neighbors of α. Define the color of the edge $\alpha\beta$, where α is from the left part of the bipartition and β is from the right, to be the ordered pair $(\mathrm{idx}(\alpha, \beta), \mathrm{idx}(\beta, \alpha))$. Clearly there are at most d^2 such colors. This is a valid coloring since if (α, β) and (α, δ) have the same color, then $\mathrm{idx}(\alpha, \beta) = \mathrm{idx}(\alpha, \delta)$, so $\beta = \delta$. Similarly, if (α, β) and (γ, β) have the same color, then $\mathrm{idx}(\beta, \alpha) = \mathrm{idx}(\beta, \gamma)$, so $\alpha = \gamma$. □

Given this lemma, the simulation proceeds as follows. First, to ensure that the graph of H is bipartite, we actually simulate evolution according to the Hamiltonian $\sigma_x \otimes H$, which is bipartite and has the same sparsity as H. Since $e^{-i(\sigma_x \otimes H)t}|+\rangle|\psi\rangle = |+\rangle e^{-iHt}|\psi\rangle$, we can recover a simulation of H from a simulation of $\sigma_x \otimes H$.

Now write H as a diagonal matrix plus a matrix with zeros on the diagonal. We have already shown how to simulate the diagonal part, so we can assume H has zeros on the diagonal without loss of generality.

It suffices to simulate the term corresponding to the edges of a particular color c. We show how to make the simulation work for any particular vertex x; then it works in general by linearity. By computing the complete list of neighbors of x and computing each of their colors, we can reversibly compute $v_c(x)$, the vertex adjacent to x via an edge with color c, along with the associated matrix element:

$$|x\rangle \mapsto |x, v_c(x), H_{x,v_c(x)}\rangle. \tag{27.12}$$

Then we can simulate the H-independent Hamiltonian defined by the map

$$|x, y, h\rangle \mapsto h|y, x, h^*\rangle \tag{27.13}$$

since it is easily diagonalized, as it consists of a direct sum of two-dimensional blocks. Finally, we can uncompute the second and third registers. Before the uncomputation, the simulation produces a linear combination of the states $|x, v_c(x), H_{x,v_c(x)}\rangle$ and $|v_c(x), x, H^*_{x,v_c(x)}\rangle$. Since

$$|v_c(x), x, H^*_{x,v_c(x)}\rangle = |v_c(x), v_c(v_c(x)), H_{v_c(x),x}\rangle, \tag{27.14}$$

the uncomputation works identically for both components.

27.5 Measuring an operator

We can view a Hermitian operator not just as the generator of dynamics, but also as a quantity to be measured. In a practical quantum simulation, the desired final measurement might be of this type. For example, we might want to measure the energy of the system, and the Hamiltonian could be a complicated sum of noncommuting terms.

It turns out that any Hermitian operator that can be efficiently simulated (viewing it as the Hamiltonian of a quantum system) can also be efficiently measured using a formulation of the quantum measurement process given by von Neumann. In fact, von Neumann's procedure is essentially the same as quantum phase estimation.

In von Neumann's description of the measurement process, a measurement is performed by coupling the system of interest to an ancillary system, which we call the *pointer*. Suppose that the pointer is a one-dimensional free particle and that the system-pointer interaction Hamiltonian is $H \otimes p$, where p is the momentum of the particle. Furthermore, suppose that the mass of the particle is sufficiently large that we can neglect the kinetic term. Then the resulting evolution is

$$e^{-itH \otimes p} = \sum_a \left[|E_a\rangle\langle E_a| \otimes e^{-itE_a p} \right], \tag{27.15}$$

where $|E_a\rangle$ are the eigenstates of H with eigenvalues E_a. Suppose we prepare the pointer in the state $|x = 0\rangle$, a narrow wave packet centered at $x = 0$. Since the momentum operator generates translations in position, the above evolution performs the transformation

$$|E_a\rangle \otimes |x = 0\rangle \mapsto |E_a\rangle \otimes |x = tE_a\rangle. \tag{27.16}$$

If we can measure the position of the pointer with sufficiently high precision that all relevant spacings $x_{ab} = t|E_a - E_b|$ can be resolved, then measurement of the position of the pointer—a fixed, easy-to-measure observable, independent of H—effects a measurement of H.

Von Neumann's measurement protocol makes use of a continuous variable, the position of the pointer. To turn it into an algorithm that can be implemented on a digital quantum computer, we can approximate the evolution (27.15) using r quantum bits to represent the pointer. The full Hilbert space is thus a tensor

product of a 2^n-dimensional space for the system and a 2^r-dimensional space for the pointer. We let the computational basis of the pointer, with basis states $\{|z\rangle\}$, represent the basis of momentum eigenstates. The label z is an integer between 0 and $2^r - 1$, and the r bits of the binary representation of z specify the states of the r qubits. In this basis, p acts as

$$p|z\rangle = \frac{z}{2^r}|z\rangle. \tag{27.17}$$

In other words, the evolution $e^{-itH\otimes p}$ can be viewed as the evolution e^{-itH} on the system for a time controlled by the value of the pointer.

Expanded in the momentum eigenbasis, the initial state of the pointer is

$$|x = 0\rangle = \frac{1}{2^{r/2}} \sum_{z=0}^{2^r-1} |z\rangle. \tag{27.18}$$

The measurement is performed by evolving under $H \otimes p$ for some appropriately chosen time t. After this evolution, the position of the simulated pointer can be measured by measuring the qubits that represent it in the x basis, i.e., the Fourier transform of the computational basis.

Note that this discretized von Neumann measurement procedure is equivalent to phase estimation. Recall that in the phase estimation problem, we are given an eigenvector $|\psi\rangle$ of a unitary operator U and asked to determine its eigenvalue $e^{i\phi}$. The algorithm uses two registers, one that initially stores $|\psi\rangle$ and one that will store an approximation of the phase ϕ. The first and last steps of the algorithm are Fourier transforms on the phase register. The intervening step is to perform the transformation

$$|\psi\rangle \otimes |z\rangle \mapsto U^z|\psi\rangle \otimes |z\rangle, \tag{27.19}$$

where $|z\rangle$ is a computational basis state. If we take $|z\rangle$ to be a momentum eigenstate with eigenvalue z (i.e., if we choose a different normalization than in (27.17)) and let $U = e^{-iHt}$, this is exactly the transformation induced by $e^{-i(H\otimes p)t}$. Thus we see that the phase estimation algorithm for a unitary operator U is exactly von Neumann's prescription for measuring $i \ln U$.

Chapter 28

Fast quantum simulation algorithms

While product formulas provide the most straightforward approach to Hamiltonian simulation, alternative approaches can offer improved performance. Here we begin to explore Hamiltonian simulation beyond product formulas.

28.1 No fast-forwarding

Before introducing improved upper bounds, we establish a limitation on the ability of quantum algorithms to simulate sparse Hamiltonians. Specifically, as mentioned in Chapter 27, we show that no general procedure can simulate a sparse Hamiltonian acting for time t using $o(t)$ queries [21].

The lower bound is based on a reduction from parity. Recall from Section 21.5 that computing the parity of n bits requires $\Omega(n)$ queries. Given an input string $x \in \{0,1\}^n$, construct a graph on vertices (i, b) for $i \in \{0, 1, \ldots, n\}$ and $b \in \{0, 1\}$, such that $(i-1, b)$ is adjacent to $(i, b \oplus x_i)$ for all $i \in \{1, \ldots, n\}$ and $b \in \{0, 1\}$. This graph is the disjoint union of two paths of length n, and $(0, 0)$ is connected to (n, b) for exactly one value of b, namely $b = x_1 \oplus \cdots \oplus x_n$, the parity of the input string. The main idea of the proof is to construct a Hamiltonian whose nonzero entries correspond to this graph, such that the dynamics for some time $t = O(n)$ map the state $|0, 0\rangle$ to the state $|n, x_1 \oplus \cdots \oplus x_n\rangle$. Then a simulation of the Hamiltonian dynamics for time t using $o(t)$ queries would violate the parity lower bound.

The most obvious choice is to simply use the adjacency matrix of the graph as the Hamiltonian. However, then the dynamics generate a continuous-time quantum walk on a finite path, which does not reach the opposite end of the path with constant amplitude after linear time.

Instead, we choose the matrix elements of the Hamiltonian H so that

$$\langle i-1, b|H|i, b \oplus x_i\rangle = \sqrt{i(n-i+1)}/n. \tag{28.1}$$

Clearly, a black box for this 2-sparse Hamiltonian can be implemented using $O(1)$ queries to the black box for the input string to answer each neighbor query. The weights are chosen to reflect the transitions between column states (in the sense of Section 17.5) for an unweighted hypercube. Specifically, letting Q denote the adjacency matrix of the hypercube and $|\mathrm{wt}_k\rangle := \binom{n}{k}^{-1/2} \sum_{|x|=k} |x\rangle$, we have

$$Q|\mathrm{wt}_k\rangle = \binom{n}{k}^{-1/2} \left((n-k+1) \sum_{|x|=k-1} |x\rangle + (k+1) \sum_{|x|=k+1} |x\rangle \right) \tag{28.2}$$

$$= \sqrt{k(n-k+1)}|\mathrm{wt}_{k-1}\rangle + \sqrt{(k+1)(n-k)}\,|\mathrm{wt}_{k+1}\rangle. \tag{28.3}$$

Thus, with these weights on the edges, the dynamics behave just as the walk on the hypercube within its column subspace. In particular, since the dynamics on the hypercube map a vertex into the opposite corner in time $\pi/2$ (as shown in Section 17.2), the chosen Hamiltonian maps $|0, 0\rangle$ to $|n, x_1 \oplus \cdots \oplus x_n\rangle$ in time $O(n)$.

It follows that a generic procedure for simulating sparse Hamiltonians for time t must have complexity $\Omega(t)$ in general. In other words, one cannot "fast-forward" the dynamics of arbitrary Hamiltonians.

28.2 Quantum walk

As we saw in Section 27.3, simulations based on product formulas have superlinear complexity. A conceptually different approach to Hamiltonian simulation uses the notion of a discrete-time quantum walk (specifically, the Szegedy framework introduced in Chapter 18). Here, one defines a quantum walk that is closely related to any given time-independent Hamiltonian and applies phase estimation in order to simulate the Schrödinger dynamics [33]. This approach gives a simulation of sparse Hamiltonians acting for time t with complexity $O(t)$, matching the lower bound of the previous section.

The quantum walk approach is based on a construction analogous to Theorem 18.1, except we start from a Hamiltonian instead of a stochastic matrix. Define states

$$|\psi_j\rangle := \frac{1}{\sqrt{X}} \sum_{k=1}^{N} \sqrt{H_{jk}^*} |j,k\rangle + \sqrt{1 - \frac{1}{X} \sum_{k=1}^{N} |H_{jk}|} |j, N+1\rangle \tag{28.4}$$

where $X \geq \max_j \sum_{k=1}^{N} |H_{jk}|$. Define the operators S, T as in the proof of Theorem 18.1. Then we get $T^\dagger S T = H/X$, so the walk has eigenvalues $e^{\pm i \arccos(\lambda/X)}$, where λ is an eigenvalue of H. The eigenvectors corresponding to these two eigenvalues can be found within the subspace $\mathrm{span}\{T|\lambda\rangle, ST|\lambda\rangle\}$, where $|\lambda\rangle$ is the eigenvector of H with eigenvalue λ.

To simulate H on a given input state $|\psi\rangle$, we proceed as follows:

1. Apply the isometry T to produce the state $T|\psi\rangle$.
2. Perform phase estimation on the quantum walk with precision δ (to be determined).
3. Given a value approximating $\arccos(\lambda/X)$, compute an estimate of λ.
4. Introduce the phase $e^{-i\lambda t}$.
5. Uncompute the estimate of λ.
6. Invert the phase estimation procedure.
7. Apply $T^\dagger$ to return to a state in the original Hilbert space.

Since a step of the quantum walk can be implemented using two applications of the isometry T, this procedure makes $O(1/\delta)$ calls to T. In turn, T can be implemented using a number of queries that is polynomial in the sparsity of the Hamiltonian, so up to factors of the sparsity, the query complexity of simulation is simply $O(1/\delta)$. Thus it remains to determine what value of δ suffices to ensure that the overall procedure reproduces the dynamics up to error at most ϵ.

The details of this analysis are presented in [33, 22], but we can understand it roughly as follows. Suppose the estimate of $\arccos(\lambda/X)$ deviates from its true value by of order δ. Since the cosine function has Lipschitz constant 1 (i.e., $|\cos(x+\Delta) - \cos(x)| \leq |\Delta|$), the resulting error in the value of λ/X is also of order δ. In other words, the error in the value of λ is of order $X\delta$. To ensure that $e^{-i\lambda t}$ deviates by at most ϵ from its true value, we take $X\delta t = \Theta(\epsilon)$, i.e., $1/\delta = \Theta(Xt/\epsilon)$. Thus we see that the complexity is linear in t and polynomial in $1/\epsilon$. Note if H is d-sparse, then we can choose $X \leq \sqrt{d}\|H\| \leq d\|H\|_{\max}$, so the factor of X just introduces polynomial overhead with respect to the sparsity.

Using a more refined implementation and analysis of this approach, one can achieve query complexity $O(\frac{\|H\|t}{\sqrt{\epsilon}} + d\|H\|_{\max} t) = O(d\|H\|_{\max} t/\sqrt{\epsilon})$ for a d-sparse Hamiltonian H [22].

28.3 Linear combinations of unitaries

While the quantum walk approach described in the previous section gives optimal complexity as a function of the simulation time t, its performance as a function of the allowed error ϵ is worse than using high-order product formulas. It is natural to ask how efficiently we can simulate Hamiltonian dynamics as a function of ϵ, and in particular, whether we can achieve complexity $\mathrm{poly}(\log(1/\epsilon))$.

This can indeed be achieved by another approach to Hamiltonian simulation [23, 24], which is based on techniques for implementing linear combinations of unitary operators on a quantum computer. This approach strictly improves over direct use of product formulas, giving faster performance without the need

for higher-order formulas. The query complexity of this approach is $O(\tau \frac{\log(\tau/\epsilon)}{\log\log(\tau/\epsilon)})$, where $\tau := d^2\|H\|_{\max}t$ (with $\|H\|_{\max}$ denoting the largest magnitude of an entry of H).

Denote the Taylor series for the evolution up to time t, truncated at order K, by

$$\tilde{U}(t) := \sum_{k=0}^{K} \frac{(-iHt)^k}{k!}. \tag{28.5}$$

For sufficiently large K, the operator $\tilde{U}(t)$ is a good approximation of $\exp(-iHt)$. Specifically, by Taylor's theorem, we have

$$\|\tilde{U}(t) - \exp(-iHt)\| \leq \frac{\exp(\|H\|t)(\|H\|t)^{K+1}}{(K+1)!}, \tag{28.6}$$

so we can ensure that the error is at most ϵ by taking $K = O(\log(\|H\|t/\epsilon)/\log\log(\|H\|t/\epsilon))$. If we take $\|H\|t$ constant, then we get an approximation with $K = O(\log(1/\epsilon)/\log\log(1/\epsilon))$. If we could implement the evolution for constant time with this complexity, then by reducing the error to ϵ/t, we could repeat the process $O(t)$ times and get a simulation with complexity $O(t\log(t/\epsilon)/\log\log(t/\epsilon))$ and overall error at most ϵ.

Now suppose we can decompose the given Hamiltonian in the form

$$H = \sum_{\ell=1}^{L} \alpha_\ell H_\ell \tag{28.7}$$

for some coefficients $\alpha_\ell \in \mathbb{R}$, where the individual terms H_ℓ are both unitary and Hermitian. This is straightforward if H is k-local, since we can express the local terms as linear combinations of Pauli operators. If H is sparse, then such a decomposition can also be constructed efficiently [23].

To implement $\tilde{U}(t)$, we begin by writing it as a linear combination of unitaries, namely

$$\tilde{U}(t) = \sum_{k=0}^{K} \frac{(-iHt)^k}{k!} \tag{28.8}$$

$$= \sum_{k=0}^{K} \sum_{\ell_1,\ldots,\ell_k=1}^{L} \frac{t^k}{k!} \alpha_{\ell_1}\cdots\alpha_{\ell_k}(-i)^k H_{\ell_1}\cdots H_{\ell_k} \tag{28.9}$$

$$= \sum_{j=0}^{m-1} \beta_j V_j, \tag{28.10}$$

where the V_j are products of the form $(-i)^k H_{\ell_1}\cdots H_{\ell_k}$, and the β_j are the corresponding coefficients.

How can we implement such a linear combination of unitaires? Let B be an operation that prepares the state

$$|\beta\rangle := \frac{1}{\sqrt{s}} \sum_{j=0}^{m-1} \sqrt{\beta_j}|j\rangle \tag{28.11}$$

from $|0\rangle$ (i.e., $B|0\rangle = |\beta\rangle$), where

$$s := \sum_{j=0}^{m-1} |\beta_j| \tag{28.12}$$

$$= \sum_{k=0}^{K} \sum_{\ell_1,\ldots,\ell_k=1}^{L} \frac{t^k}{k!} |\alpha_{\ell_1}\cdots\alpha_{\ell_k}| \tag{28.13}$$

$$= \sum_{k=0}^{K} \frac{(t\sum_{\ell=1}^{L}|\alpha_\ell|))^k}{k!}. \tag{28.14}$$

Let

$$W := B^\dagger \operatorname{select}(V) B \tag{28.15}$$

with

$$\operatorname{select}(V) := \sum_{j=0}^{m-1} |j\rangle\langle j| \otimes V_j. \tag{28.16}$$

Then we have

$$(\langle 0| \otimes I) W (|0\rangle \otimes |\psi\rangle) = \frac{1}{\sqrt{s}} (\langle 0| \otimes I) B^\dagger \operatorname{select}(V) \sum_j \sqrt{\beta_j} |j\rangle |\psi\rangle \tag{28.17}$$

$$= \frac{1}{\sqrt{s}} (\langle 0| \otimes I) B^\dagger \sum_j \sqrt{\beta_j} |j\rangle V_j |\psi\rangle \tag{28.18}$$

$$= \frac{1}{s} \sum_j \beta_j V_j |\psi\rangle \tag{28.19}$$

$$= \frac{1}{s} \tilde{U}(t) |\psi\rangle. \tag{28.20}$$

In other words, if we postselect the state $W(|0\rangle \otimes |\psi\rangle)$ on having its first register in the state $|0\rangle$, we obtain the desired result. However, this postselection only succeeds with probability (approximately) $1/s^2$.

Considering the action of W on the full space, we have

$$W|0\rangle|\psi\rangle = \frac{1}{s}|0\rangle \otimes \tilde{U}(t)|\psi\rangle + \sqrt{1 - \frac{1}{s^2}}|\Phi\rangle \tag{28.21}$$

for $|\psi\rangle \in \mathcal{H}$ and some $|\Phi\rangle$ whose ancillary state is supported in the subspace orthogonal to $|0\rangle$. To boost the chance of success, we might like to apply amplitude amplification to W. However, the initial state $|\psi\rangle$ is unknown, so we cannot reflect about it. Fortunately, something similar can be achieved using the reflection

$$R := (I - 2|0\rangle\langle 0|) \otimes I. \tag{28.22}$$

about the subspace with $|0\rangle$ in the first register. Specifically, letting $P := |0\rangle\langle 0|$, we have

$$\begin{aligned}(\langle 0| \otimes I) W R W^\dagger R W (|0\rangle \otimes I) &= (\langle 0| \otimes I)\big(W W^\dagger W - 2 W W^\dagger P W - 2 W P W^\dagger W \\ &\quad + 4 W P W^\dagger P W\big)(|0\rangle \otimes I) \qquad (28.23)\\ &= (\langle 0| \otimes I)\big(-3W + 4 W P W^\dagger P W\big)(|0\rangle \otimes I). \qquad (28.24)\end{aligned}$$

Therefore

$$(\langle 0| \otimes I) W R W^\dagger R W |0\rangle|\psi\rangle = -\frac{3}{s}\tilde{U}(t) + \frac{4}{s^3}\tilde{U}(t)\tilde{U}(t)^\dagger \tilde{U}(t), \tag{28.25}$$

which is close to $-(\frac{3}{s} - \frac{4}{s^3})\tilde{U}(t)$ since $\tilde{U}(t)$ is close to unitary. In particular, if $s = 2$ then this process boosts the amplitude from $1/2$ to 1, analogous to Grover search with a single marked item out of 4. For the purpose of Hamiltonian simulation, we can choose the parameters such that a single segment of the evolution has this value of s, and we repeat the process as many times as necessary to simulate the full evolution.

More generally, the operation $WRW^\dagger R$ is analogous to the Grover iterate, and it can be applied many times to boost the amplitude for success from something small to a value close to 1. Using this *oblivious amplitude amplification*, a general linear combination of unitaries as in (28.10) can be implemented with complexity $O(1/s)$.

Chapter 29

Quantum signal processing

In this lecture, we describe the powerful framework of *quantum signal processing* [80]. Given an operator represented using a so-called *block encoding*, we can encode spectral information about it in a qubit through the concept of *qubitization* [81] and then transform it by quantum signal processing to implement a function of the block-encoded operator. Applying this method to the Hamiltonian simulation problem, and using the method of oblivious amplitude amplification discussed in the last lecture to boost the amplitude of the block-encoded evolution, this approach can be used to simulate sparse Hamiltonians with an optimal complexity tradeoff. These techniques can also be applied in a wide variety of other algorithms [54].

29.1 Block encoding

We say a unitary transformation U is a block encoding of a matrix A if

$$U = \begin{pmatrix} A & \cdot \\ \cdot & \cdot \end{pmatrix} = |\mathbf{0}\rangle\langle\mathbf{0}| \otimes A + \cdots \tag{29.1}$$

where $|\mathbf{0}\rangle$ denotes the first computational basis state of the ancilla register. Equivalently, we can write $A = (\langle\mathbf{0}| \otimes I)U(|\mathbf{0}\rangle \otimes I)$.

Note that for A to have a block encoding, it must satisfy $\|A\| \leq 1$. However, we often consider block encodings under rescaling. Observe that we can block encode A/α for any $\alpha \geq \|A\|$. We can think of α as a parameter that measures the quality of the block encoding, where smaller values are better.

Obviously an efficient quantum circuit block-encodes itself, but we can give efficient block encodings of many other kinds of matrices. In particular, we can efficiently block-encode sparse matrices. Suppose $A \in \mathbb{C}^{N\times N}$ is d-sparse and efficiently row- and column-computable (in the sense of Section 27.4). Furthermore, suppose $\max_{i,j} |A_{ij}| \leq 1$ (this assumption is without loss of generality since we can always rescale A with a corresponding change to the block-encoding constant α). Then we can efficiently implement unitary operations R and C acting on $\mathbb{C}^{3\times N\times N}$ as

$$R\colon |0\rangle|0\rangle|i\rangle \mapsto |0\rangle\frac{1}{\sqrt{d}}\sum_{k=1}^{N}\sqrt{A_{ik}^*}|i\rangle|k\rangle + |1\rangle|i\rangle|\mu_i\rangle \tag{29.2}$$

$$C\colon |0\rangle|0\rangle|j\rangle \mapsto |0\rangle\frac{1}{\sqrt{d}}\sum_{\ell=1}^{N}\sqrt{A_{\ell j}}|\ell\rangle|j\rangle + |2\rangle|j\rangle|\nu_j\rangle \tag{29.3}$$

for some states $|\mu_i\rangle, |\nu_j\rangle \in \mathbb{C}^N$ for all $i, j \in [N]$. (This can be done by similar techniques as those for implementing the quantum walk corresponding to a Hamiltonian, as discussed in Section 28.2.) Then we

have

$$\langle 0|\langle 0|\langle i|R^\dagger C|0\rangle|0\rangle|j\rangle = \frac{1}{d}\sum_{k,\ell=1}^{N} \sqrt{A_{ik}A_{\ell j}}\langle i,k|\ell,j\rangle \tag{29.4}$$

$$= \frac{1}{d}A_{ij} \tag{29.5}$$

so $R^\dagger C$ is a block encoding of A/d.

Note that the operator W constructed in Section 28.3 is a block encoding of the scaled-down (approximately) unitary matrix $\tilde{U}/s$, as shown in (28.21). The oblivious amplitude amplification technique discussed there gives a way of turning this into a block encoding of $\tilde{U}$ with cost $O(1/s)$. (Strictly speaking, we need a robust version of amplitude amplification to handle the case that $\tilde{U}$ is not exactly unitary [25, Lemma 6].)

Block encodings also have nice closure properties. In particular, given efficient block encodings of A and B, we can construct efficient block encodings of AB [54, Section 4.3] and $\alpha A + \beta B$ [54, Section 4.4].

29.2 Quantum signal processing

A key problem is to transform a block encoding of one matrix into a block encoding of a related matrix. In particular, given a block encoding of A, when can we produce a block encoding of $f(A)$, and at what cost? This problem is addressed by *quantum signal processing*.

In this section, we describe a result of Low, Yoder, and Chuang [82] that shows how to perform such a transformation of a 2×2 matrix. This result is the core of the quantum signal processing concept and can be lifted to effectively perform quantum signal processing of higher-dimensional operators.

Suppose we are given a single-qubit rotation

$$W(x) := \begin{pmatrix} x & i\sqrt{1-x^2} \\ i\sqrt{1-x^2} & x \end{pmatrix} = e^{i \arccos(x)\sigma_x}. \tag{29.6}$$

Our goal is to generate a matrix whose entries are polynomials in x. We can do this by interspersing $W(x)$ gates with z rotations, giving a circuit

$$W_\Phi(x) := e^{i\phi_0\sigma_z}W(x)e^{i\phi_1\sigma_z}W(x)\cdots W(x)e^{i\phi_k\sigma_z} \tag{29.7}$$

where $\Phi := (\phi_0, \phi_1, \ldots, \phi_k)$. The functions of $W(x)$ that can be realized in this way are captured by the following lemma.

Lemma 29.1 ([54, Theorem 3])**.** *There exists $\Phi \in \mathbb{R}^{k+1}$ such that*

$$W_\Phi(x) = \begin{pmatrix} P(x) & iQ(x)\sqrt{1-x^2} \\ iQ^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix} \tag{29.8}$$

if and only if $P, Q \in \mathbb{C}[x]$ satisfy

(i) $\deg(P) \le k$ *and* $\deg(Q) \le k-1$,
(ii) *P has parity $k \bmod 2$ and Q has parity $k-1 \bmod 2$, and*
(iii) $\forall x \in [-1,1]$, $|P(x)|^2 + (1-x^2)|Q(x)|^2 = 1$.

(Here we say an even function has parity 0 and an odd function has parity 1.)

Proof. First we show by induction on k that (29.8) implies the three conditions.

For the base case, if $k = 0$, then $W_{(\phi_0)} = e^{i\phi_0\sigma_z}$ gives $P(x) = e^{i\phi_0}$ and $Q(x) = 0$, which satisfies (i)–(iii).

For the induction step, suppose (29.8) satisfies the three conditions. Then

$$W_{(\phi_0,\dots,\phi_{k+1})} = \begin{pmatrix} P(x) & iQ(x)\sqrt{1-x^2} \\ iQ^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix} W(x) e^{i\phi_k \sigma_z} \tag{29.9}$$

$$= \begin{pmatrix} P(x) & iQ(x)\sqrt{1-x^2} \\ iQ^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix} \begin{pmatrix} xe^{i\phi_k} & ie^{-i\phi_k}\sqrt{1-x^2} \\ ie^{i\phi_k}\sqrt{1-x^2} & xe^{-i\phi_k} \end{pmatrix} \tag{29.10}$$

$$= \begin{pmatrix} xe^{i\phi_k}P(x) - e^{i\phi_k}(1-x^2)Q(x) & ie^{-i\phi_k}(P(x)+xQ(x))\sqrt{1-x^2} \\ ie^{i\phi_k}(P(x)+xQ(x))\sqrt{1-x^2} & xe^{i\phi_k}P(x) - e^{i\phi_k}(1-x^2)Q(x) \end{pmatrix} \tag{29.11}$$

$$= \begin{pmatrix} \tilde{P}(x) & i\tilde{Q}(x)\sqrt{1-x^2} \\ i\tilde{Q}^*(x)\sqrt{1-x^2} & \tilde{P}^*(x) \end{pmatrix} \tag{29.12}$$

where

$$\tilde{P}(x) := e^{i\phi_k}\big(xP(x) - (1-x^2)Q(x)\big) \tag{29.13}$$

$$\tilde{Q}(x) := e^{-i\phi_k}\big(P(x) + xQ(x)\big) \tag{29.14}$$

clearly satisfy the three conditions (the third follows from the fact that a product of unitary matrices is unitary).

For the converse, we show by induction on k that the three conditions suffice to construct the decomposition (29.8).

For $k = 0$, we have $\deg(P) = 0$, so (iii) implies that $P(x) = e^{i\phi_0}$ for some $\phi_0 \in \mathbb{R}$ and $Q(x) = 0$.

For the induction step, observe that the quantity in condition (iii) is

$$|P(x)|^2 + (1-x^2)|Q(x)|^2 = P(x)P^*(x) + (1-x^2)Q(x)Q^*(x) \tag{29.15}$$

which is a polynomial in x. Since this polynomial is identically 1, we must have $\ell := \deg(P) = \deg(Q-1)$ and their leading coefficients must satisfy $|p_\ell| = |q_{\ell-1}|$ so that they cancel in (29.15). Choose $\phi_k \in \mathbb{R}$ so that $e^{2i\phi_k} = p_\ell / q_{\ell-1}$, and consider

$$\begin{pmatrix} P(x) & iQ(x)\sqrt{1-x^2} \\ iQ^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix} e^{-i\phi_k\sigma_z} W^\dagger(x)$$

$$= \begin{pmatrix} P(x) & iQ(x)\sqrt{1-x^2} \\ iQ^*(x)\sqrt{1-x^2} & P^*(x) \end{pmatrix} \begin{pmatrix} xe^{i\phi_k} & -ie^{i\phi_k}\sqrt{1-x^2} \\ -ie^{-i\phi_k}\sqrt{1-x^2} & xe^{-i\phi_k} \end{pmatrix} \tag{29.16}$$

$$= \begin{pmatrix} xe^{i\phi_k}P(x) + e^{-i\phi_k}Q(x)(1-x^2) & \big(-ie^{i\phi_k}P(x) + ie^{-i\phi_k}xQ(x)\big)\sqrt{1-x^2} \\ \big(ie^{i\phi_k}xQ^*(x) - ie^{-i\phi_k}P^*(x)\big)\sqrt{1-x^2} & e^{i\phi_k}Q^*(x)(1-x^2) + e^{-i\phi_k}xP^*(x) \end{pmatrix} \tag{29.17}$$

$$= \begin{pmatrix} \bar{P}(x) & i\bar{Q}(x)\sqrt{1-x^2} \\ i\bar{Q}^*(x)\sqrt{1-x^2} & \bar{P}^*(x) \end{pmatrix} \tag{29.18}$$

where

$$\bar{P}(x) := xe^{i\phi_k}P(x) + e^{-i\phi_k}(1-x^2)Q(x) \tag{29.19}$$

$$\bar{Q}(x) := -e^{i\phi_k}P(x) + e^{-i\phi_k}xQ(x). \tag{29.20}$$

While $\bar{P}$ might appear to be a polynomial of degree $\ell+1$, in fact it has degree $\ell-1$, because the coefficient of the $x^{\ell+1}$ term in (29.19) is $e^{i\phi_k}p_\ell - e^{-i\phi_k}q_{\ell-1} = 0$ by the choice of ϕ_k, and the coefficient of the x^ℓ term is 0 by condition (ii). Similarly, $\bar{Q}$ has leading coefficient $-e^{i\phi_k}p_\ell + e^{-i\phi_k}q_{\ell-1} = 0$ and has degree $\ell-2$. The parity condition (ii) is easy to see from the form of $\bar{P}$ and $\bar{Q}$, and the condition (iii) follows from unitarity. Thus by the induction hypothesis, (29.18) can be written in the form of (29.8), and therefore so can the matrix on the left-hand side of (29.16). □

When lifting this decomposition to higher-dimensional cases via qubitization, it will be useful to employ a variant of quantum signal processing using reflections. Observe that

$$W(x) = ie^{-i\sigma_z\pi/4} R(x) e^{i\sigma_z\pi/4} \tag{29.21}$$

where

$$R(x) := \begin{pmatrix} x & \sqrt{1-x^2} \\ \sqrt{1-x^2} & -x \end{pmatrix} \tag{29.22}$$

is a single-qubit reflection operator. Thus we have

$$W_\Phi(x) = e^{i\phi_0\sigma_z} \prod_{j=1}^{k} W(x) e^{i\phi_j\sigma_z} \tag{29.23}$$

$$= i^k e^{i\phi_0\sigma_z} \prod_{j=1}^{k} e^{-i\sigma_z\pi/4} R(x) e^{i\sigma_z(\phi_j+\pi/4)} \tag{29.24}$$

so we can also realize the functions described in Lemma 29.1 using a product of $R(x)$ gates and σ_z rotations.

29.3 Qubitization

Qubitization is a tool that effectively lets us map a high-dimensional block encoding to a single qubit, on which we can apply the quantum signal processing framework of the previous section. This mapping relies on a decomposition of the block encoding into two-dimensional subspaces.

Consider a block encoding (29.1) of a Hermitian matrix A. Let $|\psi_\lambda\rangle$ be an eigenvector of A, with $A|\psi_\lambda\rangle = \lambda|\psi_\lambda\rangle$. Let $\Pi := |\mathbf{0}\rangle\langle\mathbf{0}| \otimes I$ be such that $\Pi U \Pi = |\mathbf{0}\rangle\langle\mathbf{0}| \otimes A = A \oplus 0I$. Let $|\lambda\rangle := |\mathbf{0}\rangle \otimes |\psi_\lambda\rangle$, so that $\Pi U \Pi|\lambda\rangle = \lambda|\lambda\rangle$. Then we claim that U maps the span of $|\lambda\rangle$ and $(I-\Pi)U^\dagger|\lambda\rangle$ to the span of $|\lambda\rangle$ and $(I-\Pi)U|\lambda\rangle$, and vice versa if we interchange the roles of $U^\dagger$ and U.

To see this, we calculate

$$U|\lambda\rangle = (\Pi + (I-\Pi))U|\lambda\rangle \tag{29.25}$$

$$= \lambda|\lambda\rangle + (I-\Pi)U|\lambda\rangle \tag{29.26}$$

and

$$U(I-\Pi)U^\dagger|\lambda\rangle = (I - U\Pi U^\dagger)|\lambda\rangle \tag{29.27}$$

$$= |\lambda\rangle - \lambda U|\lambda\rangle \tag{29.28}$$

$$= (1-\lambda^2)|\lambda\rangle - \lambda(I-\Pi)U|\lambda\rangle. \tag{29.29}$$

Defining the normalized states

$$|\lambda^\perp\rangle := \frac{(I-\Pi)U}{\sqrt{1-\lambda^2}}|\lambda\rangle \qquad |\lambda^\ddagger\rangle := \frac{(I-\Pi)U^\dagger}{\sqrt{1-\lambda^2}}|\lambda\rangle, \tag{29.30}$$

we can re-express this as

$$U|\lambda\rangle = \lambda|\lambda\rangle + \sqrt{1-\lambda^2}|\lambda^\perp\rangle \tag{29.31}$$

$$U|\lambda^\ddagger\rangle = \sqrt{1-\lambda^2}|\lambda\rangle - \lambda|\lambda^\perp\rangle. \tag{29.32}$$

In other words, we see that U has the block-diagonal form

$$U = \bigoplus_\lambda \begin{matrix} |\lambda\rangle & |\lambda^\perp\rangle \end{matrix} \begin{pmatrix} \lambda & \sqrt{1-\lambda^2} \\ \sqrt{1-\lambda^2} & -\lambda \end{pmatrix} \begin{matrix} \langle\lambda| \\ \langle\lambda^\ddagger| \end{matrix} \tag{29.33}$$

and $U^\dagger$ has the same form, but with the bases reversed. (Here we ignore some irrelevant subspaces.) In other words, these operators are direct sums of the reflections $R(\lambda)$ acting between these two bases.

To respect these invariant subspaces, we have to alternate U and $U^\dagger$ gates. Thus we consider the following sequence of operations:

$$U_\Theta := e^{i\theta_1(2\Pi-1)}U^\dagger e^{i\theta_2(2\Pi-1)}U e^{i\theta_3(2\Pi-1)}U^\dagger e^{i\theta_4(2\Pi-1)}U \cdots e^{i\theta_{k-1}(2\Pi-1)}U^\dagger e^{i\theta_k(2\Pi-1)}U \tag{29.34}$$

where $\Theta = (\theta_1, \ldots, \theta_k)$ (here k is even). With an appropriate choice of the phases θ_j, this sequence can realize the reflection-based quantum signal processing sequence (29.34) within each two-dimensional subspace, so that U_Θ can be a block encoding of $P(A)$ for any function P satisfying the conditions of Lemma 29.1.

To implement this sequence with a quantum circuit, we can use a Π-*controlled not gate* that flips the value of an ancilla bit conditioned on the state of the main register being in the space projected onto by Π. Conjugating the gate $e^{i\phi\sigma_z}$ acting on the ancilla by Π-controlled not gates, with the ancilla initially in the state $|0\rangle$, effectively implements the $e^{i\phi(2\Pi-1)}$ operation. This explains the name "qubitization": we transfer the information about which part of the block encoding the state is in to the qubit, so that by performing quantum signal processing on that qubit, we effectively perform quantum signal processing in superposition on the high-dimensional target space.

To apply this method, there are a few more details not yet accounted for. In practice we are only interested in choosing P (the function that acts on the block-encoded matrix) and we can allow any Q that allows for a quantum signal processing decomposition. Also, we may want to implement a general function (not necessarily one of a particular parity), which we can do by adding together block encodings of the even and odd parts. We do not describe these issues in detail here, and instead simply state the following final result.

Theorem 29.2 ([54, Theorem 56])**.** *Let $f \in \mathbb{R}[x]$ be a polynomial of degree d with $|f(x)| \leq 1/2$ for all $x \in [-1, 1]$. Let U be a block encoding of a Hermitian matrix A. Then there is a circuit that gives a block encoding of $f(A)$ using d applications of U and $U^\dagger$ and one controlled-U gate.*

Further generalizations are possible to handle the situation where A is not necessarily Hermitian, in which case one can apply transformations to its singular values [54].

29.4 Application to Hamiltonian simulation

Finally, we describe how quantum signal processing can be used to give an optimal algorithm for simulating sparse Hamiltonians.

As described in Section 29.1, we can construct an efficient block encoding of a sparse Hamiltonian H (scaled down by its sparsity times the largest magnitude of a matrix element). Our goal is to turn this into a block encoding of the evolution operator e^{-iHt}.

To do this, we make use of the Jacobi-Anger expansion

$$e^{it\cos\theta} = \sum_{k=-\infty}^{\infty} i^k J_k(t) e^{ik\theta} \tag{29.35}$$

$$= J_0(t) + 2\sum_{k=1}^{\infty} i^k J_k(t)\cos(k\theta) \tag{29.36}$$

$$= J_0(t) + 2\sum_{k=1}^{\infty} i^k J_k(t) T_k(\cos\theta) \tag{29.37}$$

where J_k is a Bessel function and $T_k(\theta) = \cos(k\theta)$ is a Chebyshev polynomial. By truncating this expression to the first K terms, we get a degree-K polynomial in x,

$$J_0(-t) + 2\sum_{k=1}^{K} i^k J_k(-t) T_k(x) \approx e^{-itx}. \tag{29.38}$$

Using this polynomial as f in Theorem 29.2 (scaled down by a factor of 2 to satisfy the condition $f(x) \leq 1/2$), where U is the block encoding of the sparse Hamiltonian H, we get (a good approximation of) a block encoding of $e^{-iHt}/2$.

To understand the quality of the approximation, we must bound the error incurred by truncating the infinite series. We omit the details of this analysis here since it is technical and not very illuminating, but one can show that the error is $O((t/2)^K/K!)$ [25]. (Note that this is very similar to the error analysis for the linear combination of unitaries method as in (28.6), but without the exponential-in-t prefactor that prevents us from directly taking large values of t with that method.) To make this $O(\epsilon)$, one can show that it suffices to take [54, Lemma 59]

$$K = \left(t + \frac{\ln(1/\epsilon)}{\ln(e + \ln(1/\epsilon)/t)} \right) \tag{29.39}$$

(and indeed, this expression is tight).

Since this is scaled down by a factor of 2, we need to scale it back up to achieve the desired (deterministic) simulation. We can do this using (robust) oblivious amplitude amplification with only constant-factor overhead. Overall, this gives a quantum algorithm for sparse Hamiltonian simulation with optimal query complexity as a function of both t and ϵ.

Part VI

Adiabatic quantum computing

Chapter 30

The quantum adiabatic theorem

In the last part of this course, we will discuss an approach to quantum computation based on the concept of *adiabatic evolution*. According to the quantum adiabatic theorem, a quantum system that begins in the nondegenerate ground state of a time-dependent Hamiltonian will remain in the instantaneous ground state provided the Hamiltonian changes sufficiently slowly. In this lecture we will prove the quantum adiabatic theorem, which quantifies this statement.

30.1 Adiabatic evolution

When the Hamiltonian of a quantum system does not depend on time, the dynamics of that system are fairly straightforward. Given a time-independent Hamiltonian H, the solution of the Schrödinger equation

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}t}|\psi(t)\rangle = H|\psi(t)\rangle \tag{30.1}$$

with the initial quantum state $|\psi(0)\rangle$ is given by

$$|\psi(t)\rangle = \exp(-\mathrm{i}Ht)|\psi(0)\rangle. \tag{30.2}$$

So any eigenstate $|E\rangle$ of the Hamiltonian, with $H|E\rangle = E|E\rangle$, simply acquires a phase $\exp(-\mathrm{i}Et)$. In particular, there are no transitions between eigenstates.

If the Hamiltonian varies in time, the evolution it generates can be considerably more complicated. However, if the change in the Hamiltonian occurs sufficiently slowly, the dynamics remain relatively simple: roughly speaking, if the system begins close to an eigenstate, it remains close to an eigenstate. The *quantum adiabatic theorem* is a formal description of this phenomenon.

For a simple example of adiabatic evolution in action, consider a spin in a magnetic field that is rotated from the x direction to the z direction in a total time T:

$$H(t) = -\cos\left(\frac{\pi t}{2T}\right)\sigma_x - \sin\left(\frac{\pi t}{2T}\right)\sigma_z. \tag{30.3}$$

Suppose that initially, the spin points in the x direction: $|\psi(0)\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, the ground state of $H(0)$. As the magnetic field is slowly rotated toward the z direction, the spin begins to precess about the new direction of the field, moving it toward the z axis (and also producing a small component out of the xz plane). If T is made larger and larger, so that the rotation of the field direction happens more and more slowly (as compared to the speed of precession), the state will precess in a tighter and tighter orbit about the field direction. In the limit of arbitrarily slow rotation of the field, the state simply tracks the field, remaining in the instantaneous ground state of $H(t)$.

More generally, for $s \in [0, 1]$, let $H(s)$ be a Hermitian operator that varies smoothly as a function of s. (The notion of smoothness will be made precise in the following section.) Let $s := t/T$. Then for T arbitrarily large, $H(t)$ varies arbitrarily slowly as a function of t. An initial quantum state $|\psi(0)\rangle$ evolves

according to the Schrödinger equation,

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}t}|\psi(t)\rangle = H(t)|\psi(t)\rangle, \tag{30.4}$$

or equivalently,

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}s}|\psi(s)\rangle = TH(s)|\psi(s)\rangle. \tag{30.5}$$

Now suppose that $|\psi(0)\rangle$ is an eigenstate of $H(0)$, which we assume for simplicity is the ground state, and is nondegenerate. Furthermore, suppose that the ground state of $H(s)$ is nondegenerate for all values of $s \in [0,1]$. Then the adiabatic theorem says that in the limit $T \to \infty$, the final state $|\psi(T)\rangle$ obtained by the evolution (30.4) will be the ground state of $H(1)$.

Of course, evolution for an infinite time is rather impractical. For computational purposes, we need a quantitative version of the adiabatic theorem: we would like to understand how large T must be so that the final state is guaranteed to differ from the adiabatically evolved state by at most some fixed small amount. In particular, we would like to understand how the required evolution time depends on spectral properties of the interpolating Hamiltonian $H(s)$. We will see that the timescale for adiabaticity is intimately connected to the energy gap between the ground and first excited states.

30.2 Proof of the adiabatic theorem

We now give a proof of the adiabatic theorem, following [110] (see also [66] for a similar approach).

We would like to compare the evolution according to (30.5) to the corresponding *(exactly) adiabatic evolution*, in which the initial ground state $|\phi(0)\rangle = |\psi(0)\rangle$ evolves into the instantaneous ground state $|\phi(s)\rangle$ of $H(s)$ satisfying

$$H(s)|\phi(s)\rangle = E(s)|\phi(s)\rangle \tag{30.6}$$

where $E(s)$ is the instantaneous ground state energy. (We assume for simplicity that the ground state is unique.) Let

$$P(s) := |\phi(s)\rangle\langle\phi(s)| \tag{30.7}$$

denote the projector onto the ground state of $H(s)$. Then we claim that the Hamiltonian

$$H_a(s) := TH(s) + \mathrm{i}[\dot{P}(s), P(s)] \tag{30.8}$$

generates exactly adiabatic evolution, where we use a dot to denote differentiation with respect to s. In other words, we claim that the differential equation

$$\mathrm{i}\frac{\mathrm{d}}{\mathrm{d}s}|\xi(s)\rangle = H_a(s)|\xi(s)\rangle \tag{30.9}$$

with $|\xi(0)\rangle = |\phi(0)\rangle$ has the solution $|\xi(s)\rangle = e^{\mathrm{i}\theta(s)}|\phi(s)\rangle$ for some time-dependent phase $\theta(s)$. Equivalently, the density matrix $P(s) = |\phi(s)\rangle\langle\phi(s)| = |\xi(s)\rangle\langle\xi(s)|$ satisfies the differential equation

$$\mathrm{i}\dot{P}(s) = \mathrm{i}\left[\left(\frac{\mathrm{d}}{\mathrm{d}s}|\xi(s)\rangle\right)\langle\xi(s)| + |\xi(s)\rangle\left(\frac{\mathrm{d}}{\mathrm{d}s}\langle\xi(s)|\right)\right] \tag{30.10}$$

$$= [H_a(s), P(s)]. \tag{30.11}$$

To see this, we compute

$$[H_a, P] = T[H, P] + \mathrm{i}[[\dot{P}, P], P] \tag{30.12}$$

$$= \mathrm{i}(\dot{P}P - 2P\dot{P}P + P\dot{P}) \tag{30.13}$$

(dropping the argument s when it is clear from context). Differentiating the identity $P^2 = P$ gives

$$\dot{P} = \dot{P}P + P\dot{P}, \tag{30.14}$$

and multiplying this identity by P on both sides gives

$$P\dot{P}P = 0. \tag{30.15}$$

Thus we see that $[H_a, P] = \mathrm{i}\dot{P}$ as claimed.

Recall that our goal is to understand the evolution according to (30.5), which can be written as

$$|\psi(s)\rangle = U(s)|\psi(0)\rangle \tag{30.16}$$

for some unitary operator $U(s)$. It is helpful to write the evolution in terms of a differential equation for $U(s)$. We have

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}s}U(s)|\psi(0)\rangle &= \frac{\mathrm{d}}{\mathrm{d}s}|\psi(s)\rangle &(30.17)\\ &= -\mathrm{i}TH(s)|\psi(s)\rangle &(30.18)\\ &= -\mathrm{i}TH(s)U(s)|\psi(0)\rangle, &(30.19)\end{aligned}$$

and since this holds for any initial state $|\psi(0)\rangle$, we see that $U(s)$ satisfies the differential equation

$$\mathrm{i}\dot{U}(s) = TH(s)U(s). \tag{30.20}$$

Similarly, we have

$$\mathrm{i}\dot{U}_a(s) = H_a(s)U_a(s) \tag{30.21}$$

for the corresponding adiabatic evolution.

We would like to show that the difference between U and U_a is small. Thus we consider

$$\begin{aligned}U(1) - U_a(1) &= -U(1)\int_0^1 \frac{\mathrm{d}}{\mathrm{d}s}(U^\dagger U_a)\,\mathrm{d}s &(30.22)\\ &= \mathrm{i}U(1)\int_0^1 U^\dagger[H_a - TH]U_a\,\mathrm{d}s &(30.23)\\ &= -U(1)\int_0^1 U^\dagger[\dot{P}, P]U_a\,\mathrm{d}s &(30.24)\end{aligned}$$

where the first line follows from the fundamental theorem of calculus, the second from (30.20) and (30.21), and the third from the definition of H_a.

It turns out that the expression $[\dot{P}, P]$ can be written as a commutator with the Hamiltonian, $[\dot{P}, P] = [H, F]$, where

$$F := R\dot{P}P + P\dot{P}R \tag{30.25}$$

where we have defined the *resolvent*

$$R := \frac{1}{H - E} \tag{30.26}$$

(which has poles at the eigenvalues of H). This can be seen as follows: noting that $(H - E)R = 1$ so that $HR = 1 + ER$, and $PH = EP$, we have

$$\begin{aligned}[H, F] &= HR\dot{P}P + HP\dot{P}R - R\dot{P}PH - P\dot{P}RH &(30.27)\\ &= \dot{P}P + ER\dot{P}P + EP\dot{P}R - ER\dot{P}P - P\dot{P} - EP\dot{P}R &(30.28)\\ &= [\dot{P}, P] &(30.29)\end{aligned}$$

as claimed.

Now let us define

$$\tilde{F} := U^\dagger F U. \tag{30.30}$$

Using (30.20), we have

$$\dot{\tilde{F}} = \mathrm{i}TU^\dagger[H, F]U + U^\dagger\dot{F}U; \tag{30.31}$$

therefore

$$U^\dagger[\dot{P},P]U = U^\dagger[H,F]U \tag{30.32}$$

$$= \frac{1}{\mathrm{i}T}(\dot{\tilde{F}} - U^\dagger \dot{F} U). \tag{30.33}$$

Now we insert this into (30.24) and integrate the first term by parts:

$$U(1) - U_a(1) = \frac{\mathrm{i}}{T}U(1)\int_0^1 [(\dot{\tilde{F}} - U^\dagger \dot{F} U)U^\dagger U_a]\,\mathrm{d}s \tag{30.34}$$

$$= \frac{\mathrm{i}}{T}U(1)\left(\left[\tilde{F}U^\dagger U_a\right]_0^1 - \int_0^1 \left(\tilde{F}\frac{\mathrm{d}}{\mathrm{d}s}(U^\dagger U_a) - U^\dagger \dot{F} U_a\right)\mathrm{d}s\right) \tag{30.35}$$

$$= \frac{\mathrm{i}}{T}U(1)\left(\left[\tilde{F}U^\dagger U_a\right]_0^1 - \int_0^1 \left(\tilde{F}U^\dagger[\dot{P},P]U_a - U^\dagger \dot{F} U_a\right)\mathrm{d}s\right) \tag{30.36}$$

where we compute the derivative of $U^\dagger U_a$ as in (30.24). Thus we have

$$T\|U(1) - U_a(1)\| \le \|F(0)\| + \|F(1)\| + \int_0^1 \mathrm{d}s\,\left(2\|F\|\cdot\|\dot{P}\| + \|\dot{F}\|\right). \tag{30.37}$$

Now

$$\|F\| \le 2\|R\dot{P}P\| \tag{30.38}$$

$$= 2\|R(1-P)\dot{P}\| \tag{30.39}$$

$$\le 2\|R(1-P)\|\cdot\|\dot{P}\| \tag{30.40}$$

$$\le \frac{2\|\dot{P}\|}{\Delta} \tag{30.41}$$

where we have used (30.14) to see that $\dot{P}P = (1-P)\dot{P}$, and where $\Delta(s)$ is the gap between the smallest eigenvalue $E(s)$ of $H(s)$ and the nearest distinct eigenvalue of $H(s)$. Also,

$$\dot{F} = \dot{R}\dot{P}P + R\ddot{P}P + R\dot{P}^2 + P\dot{P}\dot{R} + P\ddot{P}R + \dot{P}^2 R \tag{30.42}$$

and

$$\dot{R} = -\frac{1}{(H-E)}\dot{H}\frac{1}{(H-E)} \tag{30.43}$$

(to see this, differentiate the identity $(H-E)R = 1$), so (by similar calculations as above)

$$\|\dot{F}\| \le 2\left(\frac{\|\dot{H}\|\cdot\|\dot{P}\|}{\Delta^2} + \frac{\|\ddot{P}\|}{\Delta} + \frac{\|\dot{P}\|^2}{\Delta}\right). \tag{30.44}$$

Thus we have

$$\frac{T}{2}\|U(1) - U_a(1)\| \le \frac{\|\dot{P}(0)\|}{\Delta(0)} + \frac{\|\dot{P}(1)\|}{\Delta(1)} + \int_0^1 \mathrm{d}s\,\left(3\frac{\|\dot{P}\|^2}{\Delta} + \frac{\|\dot{H}\|\cdot\|\dot{P}\|}{\Delta^2} + \frac{\|\ddot{P}\|}{\Delta}\right). \tag{30.45}$$

Finally, we would like to express $\|\dot{P}\|$ and $\|\ddot{P}\|$ in terms of H. We can obtain upper bounds for these quantities using first and second order perturbation theory. Intuitively, if the Hamiltonian changes slowly, and if its eigenvalues are not close to degenerate, then its eigenvectors should also change slowly. At first order, we have

$$\|\dot{P}\| \le c_1\frac{\|\dot{H}\|}{\Delta} \tag{30.46}$$

for some constant c_1, and at second order,

$$\|\ddot{P}\| \le c_2\frac{\|\ddot{H}\|}{\Delta} + c_3\frac{\|\dot{H}\|^2}{\Delta^2} \tag{30.47}$$

for some constants c_2, c_3. Plugging these estimates into (30.45), we have

$$\frac{T}{2}\|U(1) - U_a(1)\| \leq c_1 \frac{\|\dot{H}(0)\|}{\Delta(0)^2} + c_1 \frac{\|\dot{H}(1)\|}{\Delta(1)^2} + \int_0^1 \mathrm{d}s \left((3c_1^2 + c_1 + c_3) \frac{\|\dot{H}\|^2}{\Delta^3} + c_2 \frac{\|\ddot{H}\|}{\Delta^2} \right). \tag{30.48}$$

Overall, we have proved the following quantitative version of the adiabatic theorem:

Theorem 30.1. *Suppose $H(s)$ has a nondegenerate ground state for all $s \in [0,1]$, and suppose that the total evolution time satisfies*

$$T \geq \frac{2}{\epsilon} \left[c_1 \frac{\|\dot{H}(0)\|}{\Delta(0)^2} + c_1 \frac{\|\dot{H}(1)\|}{\Delta(1)^2} + \int_0^1 \mathrm{d}s \left((3c_1^2 + c_1 + c_3) \frac{\|\dot{H}\|^2}{\Delta^3} + c_2 \frac{\|\ddot{H}\|}{\Delta^2} \right) \right]. \tag{30.49}$$

Then evolution of the initial state $|\psi(0)\rangle = |\phi(0)\rangle$ under the Schrödinger equation (30.5) produces a final state $|\psi(1)\rangle$ satisfying

$$\| |\psi(1)\rangle - |\phi(1)\rangle \| \leq \epsilon. \tag{30.50}$$

Chapter 31

Adiabatic optimization

Having established the quantum adiabatic theorem, we will now see how it can be applied to solve optimization problems. After describing the general framework [48], we will see how this approach gives an alternative $O(\sqrt{N})$-time algorithm for unstructured search.

31.1 An adiabatic optimization algorithm

Many computational problems can be cast as the minimization of some cost function. For concreteness, suppose we are given a function

$$h : \{0,1\}^n \to \mathbb{R} \tag{31.1}$$

from strings of n bits to real numbers. A natural question is, does there exist a string $z \in \{0,1\}^n$ such that $h(z) = 0$? Or alternatively, can you find a string z that globally minimizes $h(z)$? In general, such questions can be very difficult to answer—the first problem is NP-complete, and the second is NP-hard. (For example, $h(z)$ could be the number of clauses violated by some CNF formula.) But of course, specific instances of such a problem can be more tractable than the general case.

The quantum adiabatic theorem suggests a natural approach to minimizing functions such as (31.1). The basic idea is to encode the solutions of the minimization problem in the ground state of a Hamiltonian and to adiabatically evolve into this ground state, starting from a known ground state. According to the adiabatic theorem, the probability of finding a solution will be high provided the evolution is sufficiently slow.

To cast the problem of minimizing (31.1) in quantum mechanical terms, consider a Hamiltonian that is diagonal in the computational basis, with eigenvalues $h(z)$:

$$H_P := \sum_{z \in \{0,1\}^n} h(z) \, |z\rangle\langle z| \tag{31.2}$$

We refer to H_P as the *problem Hamiltonian*, since it corresponds to the problem of minimizing h. Clearly, its ground state consists of strings z such that $h(z)$ is minimized. Therefore, if we could prepare the ground state of H_P, we could solve the minimization problem.

To prepare the ground state of H_P, we will adiabatically evolve from the ground state of a simpler Hamiltonian. Let the *beginning Hamiltonian* H_B be some Hamiltonian whose ground state is easy to prepare. Then let $H_T(t)$ be a smoothly varying time-dependent Hamiltonian with $H_T(0) = H_B$ and $H_T(T) = H_P$, where T is the total run time of the evolution. Assuming the evolution is sufficiently close to adiabatic, the initial ground state will evolve into a state close to the final ground state, thereby solving the problem.

For any given H_B and H_P, there are many possible choices for the interpolation $H_T(t)$. One simple choice is a time-dependent Hamiltonian of the form

$$H_T(t) = H(t/T) := \big(1 - f(t/T)\big)H_B + f(t/T)H_P \tag{31.3}$$

where $f(s)$ is a smooth, monotonic function of $s \in [0,1]$ satisfying $f(0) = 0$ and $f(1) = 1$, so that $H(0) = H_B$ and $H(1) = H_P$. In other words, the interpolating function $f(t/T)$ should vary smoothly from 0 to 1 as the

time t varies from 0 to T. If $f(s)$ is twice differentiable, and if the ground state of $H(s)$ is nondegenerate for all $s \in [0,1]$, then the adiabatic theorem guarantees that the evolution will become arbitrarily close to adiabatic in the limit $T \to \infty$. An especially simple choice for this interpolation schedule is the linear interpolation $f(s) = s$, but many other choices are possible.

Finally, how should we choose the beginning Hamiltonian? If we choose an interpolation of the form (31.3), then H_B clearly should not commute with H_P, or else no evolution will occur. One natural choice for H_B is

$$H_B = -\sum_{j=1}^{n} \sigma_x^{(j)} \tag{31.4}$$

where $\sigma_x^{(j)}$ is the Pauli x operator on the jth qubit. This beginning Hamiltonian has the ground state

$$|S\rangle := \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle, \tag{31.5}$$

a uniform superposition of all possible solutions $S = \{0,1\}^n$. But as for the method of interpolation, many other choices for H_B are possible.

To summarize, a quantum adiabatic optimization algorithm works as follows:

1. Prepare the quantum computer in the ground state of the beginning Hamiltonian H_B.
2. Evolve the state with the Hamiltonian $H(t)$ for a total time T, ending with the problem Hamiltonian H_P.
3. Measure in the computational basis.

Step 1 can be performed efficiently if H_B has a sufficiently simple ground state—for example, if it is the state (31.5). Step 2 can be simulated efficiently on a universal quantum computer, assuming the Hamiltonian is of a suitable form (say, if it is sparse) and the run time T is not too large. Step 3 is straightforward to implement, and will yield a state close to the ground state assuming the simulation of the evolution is sufficiently good and the evolution being simulated meets the conditions of the adiabatic theorem.

31.2 The running time and the gap

The quantum adiabatic optimization algorithm described above is guaranteed to produce the correct answer with high probability provided the run time is sufficiently large. But how long is long enough? Unfortunately, this question is difficult to answer for almost all interesting problems. However, using the adiabatic theorem, it can at least be rephrased as a statement about spectral properties of the Hamiltonian.

From the adiabatic theorem, we see that the run time depends crucially on the gap $\Delta(s)$ between the ground and first excited states of $H(s)$. Suppose for simplicity that we use linear interpolation between H_B and H_P, i.e., (31.3) with $f(s) = s$. Clearly

$$\dot{H} = H_P - H_B \tag{31.6}$$

$$\ddot{H} = 0. \tag{31.7}$$

Now let

$$\Delta_{\min} := \min_{s \in [0,1]} \Delta(s) \tag{31.8}$$

be the *minimum gap* between the ground and first excited states. Then we have

Theorem 31.1. *Suppose the evolution time satisfies*

$$T \geq \frac{2}{\epsilon} \left[2c_1 \frac{\|H_P - H_B\|}{\Delta_{\min}^2} + (3c_1^2 + c_1 + c_3) \frac{\|H_P - H_B\|^2}{\Delta_{\min}^3} \right]. \tag{31.9}$$

Then $\| |\psi(1)\rangle - |\phi(1)\rangle \| \leq \epsilon$.

Recall that to be efficiently simulable, H_B and H_P should not have very large norm. Thus we see that if the minimum gap $\Delta_{\min}$ is not too small, the run time need not be too large. In particular, to show that the adiabatic algorithm runs in polynomial time, it sufficies to show that the minimum gap is only polynomially small, i.e., that $1/\Delta_{\min}$ is upper bounded by a polynomial in n.

Of course, this does not answer the question of whether the adiabatic algorithm runs in polynomial time unless the minimum gap can be estimated. In general, calculating the gap for a particular Hamiltonian is a difficult problem, which makes the adiabatic algorithm difficult to analyze. Nevertheless, there are a few examples of interest for which the gap can indeed be estimated.

31.3 Adabatic optimization algorithm for unstructured search

Recall that the unstructured search problem for N items can be solved in time $O(\sqrt{N})$ (by Grover's algorithm, or by a quantum walk search algorithm), but has classical query complexity $\Omega(N)$. Let's show that adiabatic optimization can also solve this problem in $O(\sqrt{N})$, demonstrating that it can indeed provide quantum speedup [95, 42].

Unstructured search is equivalent to minimizing the black box function $h : \{0, 1, \ldots, N-1\} \to \{0, 1\}$ defined as

$$h(z) := \begin{cases} 0 & z \text{ is marked} \\ 1 & z \text{ is unmarked.} \end{cases} \tag{31.10}$$

For simplicity, let's focus on the case of a single marked item m. Then the problem Hamiltonian (31.2) corresponding to (31.10) is

$$H_P = 1 - |m\rangle\langle m|. \tag{31.11}$$

A natural starting point for adiabatic evolution is the uniform superposition state $|S\rangle$, where now $S = \{0, 1, \ldots, N-1\}$. Then a particularly simple choice of the beginning Hamiltonian is the projector onto states orthogonal to $|S\rangle$,

$$H_B = 1 - |S\rangle\langle S|. \tag{31.12}$$

(Note that such a beginning Hamiltonian is not a good choice in general, since one can show that it allows at most a quadratic speedup over brute force search. We could instead use the initial Hamiltonian (31.4) for unstructured search, corresponding to adiabiatic optimization over the hypercube, but the analysis would be more complicated.) For this beginning Hamiltonian, $|S\rangle$ is the ground state, with energy 0, and all orthogonal states have energy 1. Finally, suppose we use the interpolation Hamiltonian (31.3), so that

$$H(s) = 1 - \big[\big(1 - f(s)\big)|S\rangle\langle S| + f(s)|m\rangle\langle m|\big] \tag{31.13}$$

for some as yet undetermined function $f : [0, 1] \to [0, 1]$.

Just as in Grover's algorithm, $H_T(t)$ acts nontrivially only on the subspace spanned by $|m\rangle$ and $|S\rangle$, making its spectrum straightforward to calculate. Working in the $\{|m\rangle, |m^\perp\rangle\}$ basis we defined when analyzing the continuous-time quantum walk algorithm for unstructured search on the complete graph (with $|m^\perp\rangle = (|S\rangle - a|m\rangle)/\sqrt{1-a^2}$, where $a := \langle S|m\rangle = 1/\sqrt{N}$),

$$H = \begin{pmatrix} (1-f)(1-a^2) & -(1-f)a\sqrt{1-a^2} \\ -(1-f)a\sqrt{1-a^2} & 1-(1-f)(1-a^2) \end{pmatrix} \tag{31.14}$$

$$= \frac{1}{2} - (1-f)a\sqrt{1-a^2}\,\sigma_x + \Big[(1-f)(1-a^2) - \frac{1}{2}\Big]\sigma_z. \tag{31.15}$$

Then it is straightforward to compute the eigenvalues

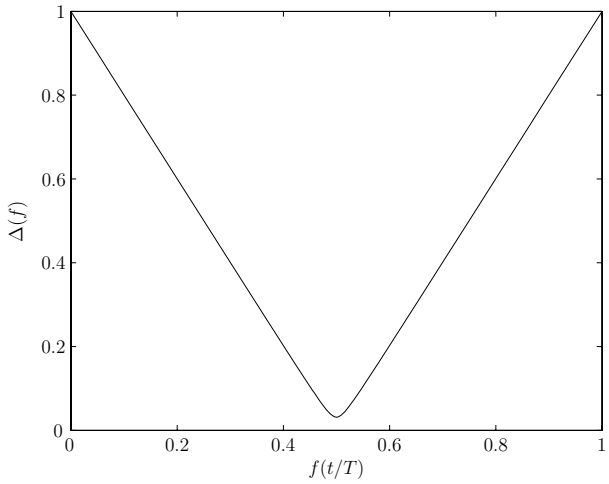
$$E_0 = \frac{1}{2}\Big(1 - \sqrt{1 - 4f(1-f)(1-a^2)}\Big) \tag{31.16}$$

$$E_1 = \frac{1}{2}\Big(1 + \sqrt{1 - 4f(1-f)(1-a^2)}\Big). \tag{31.17}$$

In other words, the gap between the ground and first excited states is

$$\Delta = \sqrt{1 - 4f(1-f)(1-a^2)}. \tag{31.18}$$

For example, with $N = 1000$, we have the following:



In general, the minimum value occurs at $f = 1/2$, where we have $\Delta_{\min} = a = 1/\sqrt{N}$.

To finish specifying the algorithm, we must choose a particular interpolation function (or *schedule*) $f(s)$. The simplest choice is to use the linear interpolation $f(s) = s$, but it turns out that this simple choice does not work. Applying (31.9), which pessimistically depends solely on the minimum value of the gap, only shows it is sufficient to take $T = O(1/\Delta^3) = O(N^{3/2})$. But even if we use the full adiabatic theorem, we only find that it is sufficient to take the run time to be large compared to

$$\int_0^1 \frac{\mathrm{d}f}{\Delta^3} = \int_0^1 \frac{\mathrm{d}f}{[1 - 4f(1-f)(1-a^2)]^{3/2}} = \frac{1}{a^2} = N. \tag{31.19}$$

While the adiabatic theorem only gives an upper bound on the running time, it turns out that the bound is essentially tight in this case: with linear interpolation, the run time must be $\Omega(N)$ for the evolution to remain approximately adiabatic.

However, we can do better by choosing a different interpolation schedule $f(s)$. Intuitively, since the gap is smallest when $f(s)$ is close to $1/2$, we should evolve more slowly for such values. The fact that the gap is only of order $1/\sqrt{N}$ for values of $|f - 1/2|$ of order $1/\sqrt{N}$ ultimately means that it is possible to choose a schedule for which a total run time of $O(\sqrt{N})$ suffices. Since we should evolve most slowly when the gap is smallest, it is reasonable to let $\dot{f} \propto \Delta^p$ for some power p. For concreteness, we will use $p = 3/2$, although any $p \in (1, 2)$ would work.

If we let

$$\dot{f} = \alpha \Delta^{3/2}, \tag{31.20}$$

then the coefficient α is fixed by the equation $\int_0^1 \mathrm{d}s = \int_0^1 \mathrm{d}f/\dot{f} = 1$, i.e.,

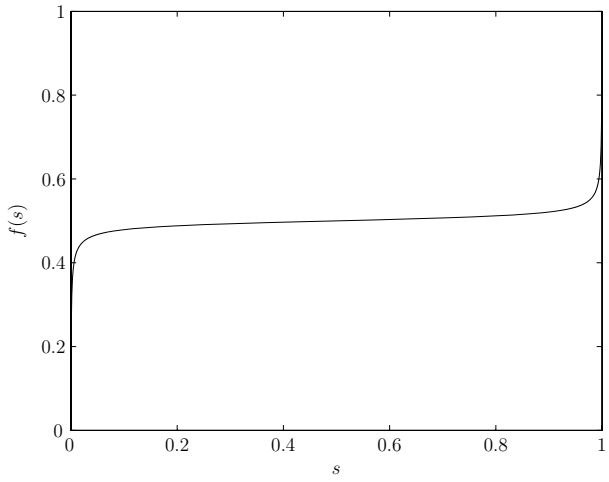
$$\alpha = \int_0^1 \frac{\mathrm{d}f}{\Delta^{3/2}} \tag{31.21}$$

$$= \int_0^1 \frac{\mathrm{d}f}{[1 - 4f(1-f)(1-a^2)]^{3/4}} \tag{31.22}$$

$$= \operatorname{Im} B_{(1+i\sqrt{N-1})/2}(1/4, 1/4) \frac{N^{3/4}}{\sqrt{2(N-1)}} \tag{31.23}$$

$$= 2\sqrt{\pi} \frac{\Gamma(5/4)}{\Gamma(3/4)} N^{1/4} + O(1) \tag{31.24}$$

where $B_z(a, b)$ denotes the incomplete beta function, and $\Gamma(z)$ denotes the gamma function. Then for example, with $N = 1000$, the schedule obtained by integrating (31.20) looks as follows:



Now we want to evaluate the terms appearing in the adiabatic theorem. For the first three terms, we need to calculate

$$\|\dot{H}(s)\| = \|\dot{f}(s)(H_P - H_B)\| \tag{31.25}$$
$$= |\dot{f}(s)|\sqrt{1-a^2} \tag{31.26}$$
$$= \alpha\sqrt{1-a^2}\Delta^{3/2}. \tag{31.27}$$

The first and second terms are

$$\frac{\|\dot{H}(0)\|}{\Delta(0)^2} = \frac{\|\dot{H}(1)\|}{\Delta(1)^2} = \alpha\sqrt{1-a^2} \tag{31.28}$$
$$= O(N^{1/4}), \tag{31.29}$$

so they will turn out to be negligible. The third term is

$$\int_0^1 \frac{\|\dot{H}\|^2}{\Delta^3}\mathrm{d}s = (1-a^2)\int_0^1 \frac{\dot{f}^2}{\Delta^3}\frac{\mathrm{d}f}{\dot{f}} \tag{31.30}$$
$$= \alpha(1-a^2)\int_0^1 \frac{\mathrm{d}f}{\Delta^{3/2}} \tag{31.31}$$
$$= \alpha^2(1-a^2) \tag{31.32}$$
$$= O(\sqrt{N}). \tag{31.33}$$

To calculate the final term, we need to compute

$$\|\ddot{H}\| = \|\ddot{f}(s)(H_P - H_B)\| \tag{31.34}$$
$$= |\ddot{f}(s)|\sqrt{1-a^2} \tag{31.35}$$
$$= \frac{3}{2}\alpha\Delta^{1/2}|\dot{\Delta}|\sqrt{1-a^2} \tag{31.36}$$
$$= \frac{3}{2}\alpha\sqrt{1-a^2}\Delta^{1/2}\dot{f}\Big|\frac{\mathrm{d}\Delta}{\mathrm{d}f}\Big|, \tag{31.37}$$

and

$$\frac{\mathrm{d}\Delta}{\mathrm{d}f} = \frac{2(2f-1)(1-a^2)}{\Delta}. \tag{31.38}$$

Then we have

$$\int_0^1 \frac{\|\ddot{H}\|}{\Delta^2}\mathrm{d}s = \int_0^1 \frac{\|\ddot{H}\|}{\Delta^2}\frac{\mathrm{d}f}{\dot{f}} \tag{31.39}$$

$$= \frac{3}{2}\alpha\sqrt{1-a^2}\int_0^1 \frac{1}{\Delta^{3/2}}\left|\frac{\mathrm{d}\Delta}{\mathrm{d}f}\right|\mathrm{d}f \tag{31.40}$$

$$= 3\alpha(1-a^2)^{3/2}\int_0^1 \mathrm{d}f\,\frac{|2f-1|}{[1-4f(1-f)(1-a^2)]^{5/4}} \tag{31.41}$$

$$= \frac{6\alpha(1-a^2)^{3/2}}{\sqrt{a}(1+\sqrt{a})(1+a)} \tag{31.42}$$

$$= O(\sqrt{N}). \tag{31.43}$$

Overall, we find a total run time of $T = O(\sqrt{N})$ suffices to make the evolution arbitrarily close to adiabatic.

In the above analysis, it was essential to understand the behavior of the gap as a function of f. In particular, since the spectrum of the Hamiltonian (31.13) does not depend on which item m is marked, we can choose a schedule that is simultaneously good for all possible marked items. For general instances of adiabatic optimization, this may not be the case.

To implement this adiabatic optimization algorithm for unstructured search in the conventional quantum query model, we must simulate evolution according to this Hamiltonian. Using the fact that $[H_B, H_P] = O(1/\sqrt{N})$, it is possible to perform this simulation using $O(\sqrt{N})$ queries to a black box for $h(z)$.

Chapter 32

An example of the success of adiabatic optimization

In this lecture, we describe a simple example of a function that can be minimized by adiabatic optimization in polynomial time [48].

32.1 The ring of agrees

Consider n bits $z_1, z_2, \ldots, z_n$ arranged on a ring. For each adjacent pair of bits, we include a clause that is satisfied if and only if the two bits are the same. This instance has exactly two satisfying assignments, namely those for which all the bits agree: $z_1 = z_2 = \cdots = z_n = 0$ and $z_1 = z_2 = \cdots = z_n = 1$. But even though it does not present a computational challenge, it is interesting to ask how well adiabatic optimization does on this simple "problem."

Summing over the n clauses, the cost function is

$$h(z) = \sum_{j=1}^{n} (1 - \delta_{z_j, z_{j+1}}) \tag{32.1}$$

$$= \sum_{j=1}^{n} \frac{1 - (2z_j - 1)(2z_{j+1} - 1)}{2} \tag{32.2}$$

where we make the identification $z_{n+1} := z_1$. Thus, the problem Hamiltonian can be written in terms of Pauli operators as

$$H_P := \sum_z h(z) |z\rangle\langle z| \tag{32.3}$$

$$= \frac{1}{2} \sum_{j=1}^{n} (1 - \sigma_z^{(j)} \sigma_z^{(j+1)}) \tag{32.4}$$

where we make the similar identification $\sigma_z^{(n+1)} := \sigma_z^{(1)}$. To prepare the ground state of H_P, we will use linear interpolation from a magnetic field in the x direction (i.e., the adjacency matrix of the hypercube), giving

$$H(s) = -(1-s) \sum_{j=1}^{n} \sigma_x^{(j)} + \frac{s}{2} \sum_{j=1}^{n} (1 - \sigma_z^{(j)} \sigma_z^{(j+1)}) \,. \tag{32.5}$$

To understand how well the resulting adiabatic algorithm performs, we would like to calculate the gap $\Delta(s)$ of this Hamiltonian as a function of s. Strictly speaking, this gap is zero, since the final ground state is degenerate: any state in the two-dimensional subspace $\mathrm{span}\{|0 \ldots 0\rangle, |1 \ldots 1\rangle\}$ has zero energy. However,

note that the Hamiltonian commutes with the operator

$$G := \prod_{j=1}^{n} \sigma_x^{(j)} , \tag{32.6}$$

and that the initial state $|S\rangle$ (where $S = \{0,1\}^n$) is an eigenstate of G with eigenvalue $+1$. The evolution takes place entirely within the $+1$ eigenspace of G, so we can restrict our attention to this subspace. So let $\Delta(s)$ denote the gap between the ground state of $H(s)$ and the first excited state in the $+1$ eigenspace of G. This is the relevant gap for adiabatic evolution starting in $|S\rangle$, with the ultimate goal of producing the unique $G = +1$ ground state of H_P, the GHZ state

$$\frac{|0\ldots0\rangle + |1\ldots1\rangle}{\sqrt{2}} . \tag{32.7}$$

Measurement of this state in the computational basis will yield one of the two satisfying assignments of the n bits, each occurring with probability $1/2$.

The Hamiltonian (32.5) is well-known in statistical mechanics, where it is referred to as a ferromagnetic Ising model in a transverse magnetic field. It can be diagonalized using the Jordan-Wigner transform, which we describe next.

32.2 The Jordan-Wigner transformation: From spins to fermions

The Jordan-Wigner transformation is a way of mapping a one-dimensional spin system to a system of free fermions. Since finding the spectrum of the resulting system of noninteracting fermions only requires diagonalizing an $n \times n$ matrix, whereas determining the spectrum of a generic system of n spins requires diagonalizing a $2^n \times 2^n$ matrix, the Jordan-Wigner transformation shows that one-dimensional spin systems are particularly simple, and provides a powerful tool for analyzing them.

We will focus on a one-dimensional Ising spin system in a transverse magnetic field, with nearest-neighbor couplings and magnetic fields that can vary arbitrarily from site to site. In other words,

$$H = \sum_{i=1}^{n} J_i \, \sigma_z^{(i)} \sigma_z^{(i+1)} + \sum_{i=1}^{n} h_i \, \sigma_x^{(i)} \tag{32.8}$$

for some values of the real numbers J_i and h_i. We may either have periodic boundary conditions (by identifying $\sigma_z^{(n+1)}$ with $\sigma_z^{(1)}$) or open boundary conditions (by setting $J_n = 0$).

The Jordan-Wigner transformation consists of the definition

$$a_j := \sigma_x^{(1)} \sigma_x^{(2)} \cdots \sigma_x^{(j-1)} \tilde{\sigma}_-^{(j)} 1^{(j+1)} \cdots 1^{(n)} \tag{32.9}$$

(which will turn out to be a fermion annihilation operator), where we have defined spin raising and lowering operators in the x basis,

$$\tilde{\sigma}_\pm := R \frac{\sigma_x \mp \mathrm{i}\sigma_y}{2} R \tag{32.10}$$

$$= |\mp\rangle\langle\pm| \tag{32.11}$$

where

$$R := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{32.12}$$

is the Hadamard transformation, and $|\pm\rangle := (|0\rangle \pm |1\rangle)/\sqrt{2}$ are the eigenvectors of σ_x.

To see that the a_j's correspond to fermion annihilation operators, we observe that a_j and

$$a_j^\dagger = \sigma_x^{(1)} \sigma_x^{(2)} \cdots \sigma_x^{(j-1)} \tilde{\sigma}_+^{(j)} 1^{(j+1)} \cdots 1^{(n)} \tag{32.13}$$

obey fermion anticommutation relations. In particular, let

$$\{A, B\} := AB + BA \tag{32.14}$$

denote the anticommutator. For $j < k$, we have

$$\{a_j, a_k\} = \{\tilde{\sigma}_-^{(j)}, \sigma_x^{(j)}\}\sigma_x^{(j+1)} \cdots \sigma_x^{(k-1)}\tilde{\sigma}_-^{(k)} = 0 \tag{32.15}$$

and

$$\{a_j, a_k^\dagger\} = \{\tilde{\sigma}_-^{(j)}, \sigma_x^{(j)}\}\sigma_x^{(j+1)} \cdots \sigma_x^{(k-1)}\tilde{\sigma}_+^{(k)} = 0. \tag{32.16}$$

On the other hand, for $j = k$ we have

$$\{a_j, a_j\} = \{\tilde{\sigma}_-^{(j)}, \tilde{\sigma}_-^{(j)}\} = 2(|+\rangle\langle -|+\rangle\langle -|)^{(j)} = 0 \tag{32.17}$$

$$\{a_j, a_j^\dagger\} = \{\tilde{\sigma}_-^{(j)}, \tilde{\sigma}_+^{(j)}\} = (|+\rangle\langle -|-\rangle\langle +| + |-\rangle\langle +|+\rangle\langle -|)^{(j)} = 1. \tag{32.18}$$

Thus we find the fermion anticommutation relations

$$\{a_j, a_k\} = 0 \tag{32.19}$$

$$\{a_j, a_k^\dagger\} = \delta_{j,k}. \tag{32.20}$$

To fermionize H, we need to express $\sigma_x^{(j)}$ and $\sigma_z^{(j)}\sigma_z^{(j+1)}$ in terms of fermion operators. The important point is that even though the a_j's and $a_j^\dagger$'s are highly nonlocal spin operators, certain local combinations of them correspond to local spin operators, and vice versa. For the magnetic field, we have

$$a_j^\dagger a_j = \tilde{\sigma}_+^{(j)}\tilde{\sigma}_-^{(j)} \tag{32.21}$$

$$= (|-\rangle\langle -|)^{(j)} \tag{32.22}$$

$$= \frac{1}{2}(1 - \sigma_x^{(j)}), \tag{32.23}$$

so

$$\sigma_x^{(j)} = 1 - 2a_j^\dagger a_j \tag{32.24}$$

$$= a_j a_j^\dagger - a_j^\dagger a_j . \tag{32.25}$$

For the Ising coupling term, we have (for $j = 1, 2, \ldots, n-1$),

$$(a_j^\dagger - a_j)(a_{j+1}^\dagger + a_{j+1}) = (\tilde{\sigma}_+^{(j)} - \tilde{\sigma}_-^{(j)})\sigma_x^{(j)}(\tilde{\sigma}_+^{(j+1)} + \tilde{\sigma}_-^{(j+1)}) \tag{32.26}$$

$$= \sigma_z^{(j)}\sigma_z^{(j+1)} . \tag{32.27}$$

If we want to use periodic boundary conditions, including the operator $\sigma_z^{(n)}\sigma_z^{(1)}$, then we have to treat it separately. We have

$$(a_n^\dagger - a_n)(a_1^\dagger + a_1) = \left(\prod_{j=1}^{n-1} \sigma_x^{(j)}\right)(\tilde{\sigma}_+^{(n)} - \tilde{\sigma}_-^{(n)})(\tilde{\sigma}_+^{(1)} + \tilde{\sigma}_-^{(1)}) \tag{32.28}$$

$$= -G\sigma_z^{(n)}\sigma_z^{(1)} \tag{32.29}$$

where G is the spin flip operator defined in (32.6). Since σ_x anticommutes with σ_z, the operator G commutes with each Ising coupling term, and thus commutes with any H of the form (32.8). Therefore, to find the spectrum of H, it suffices to separately determine the spectra in the subspaces with $G = +1$ and $G = -1$.

Note that since $\sigma_x = (-1)^{\frac{1}{2}(1-\sigma_x)}$, we can write

$$G = (-1)^{\sum_{j=1}^n \frac{1}{2}(1-\sigma_x^{(j)})} \tag{32.30}$$

$$= (-1)^{\sum_{j=1}^n a_j^\dagger a_j} . \tag{32.31}$$

Thus the cases $G = +1$, $G = -1$ correspond to the cases of an even or an odd number of occupied fermion modes, respectively.

Overall, the Jordan-Wigner transformation results in the expression

$$H = \sum_{i=1}^{n} J_i'(a_i^\dagger - a_i)(a_{i+1}^\dagger + a_{i+1}) - \sum_{i=1}^{n} h_i(a_i^\dagger a_i - a_i a_i^\dagger) \tag{32.32}$$

where $a_{n+1} := a_1$, and where

$$J_i' := \begin{cases} J_i & i = 1, 2, \ldots, n-1 \\ -GJ_n & i = n\,. \end{cases} \tag{32.33}$$

Since this Hamiltonian is quadratic in the fermion operators, it corresponds to a collection of n free fermions. Now it remains to diagonalize such a Hamiltonian.

32.3 Diagonalizing a system of free fermions

Consider the most general quadratic fermion Hamiltonian,

$$H = \sum_{j,k=1}^{n} (\mu_{jk}\, a_j^\dagger a_k + \nu_{jk}\, a_j a_k) + \text{h.c.} \tag{32.34}$$

Using the fermion anticommutation relations (32.19) and (32.20), we can rewrite this Hamiltonian as

$$H = \underline{a}^\dagger \begin{pmatrix} \mu & -\nu^* \\ \nu & -\mu^* \end{pmatrix} \underline{a} + \operatorname{tr} \mu \tag{32.35}$$

where μ and ν denote the matrices whose j, k entries are μ_{jk} and ν_{jk}, respectively, and $\underline{a}$ denotes the column vector whose first block has entries $a_1, \ldots, a_n$ and whose second block has entries $a_1^\dagger, \ldots, a_n^\dagger$. Since H is hermitian, we can always choose μ, ν so that $\mu = \mu^\dagger$ and $\nu = -\nu^T$.

We would like to define a change of basis to a new set of fermion operators $b_j, b_j^\dagger$ in which the Hamiltonian is diagonal. If we let

$$b_j := \sum_{k=1}^{n} (\kappa_{jk}\, a_k + \lambda_{jk}\, a_k^\dagger) \tag{32.36}$$

(sometimes referred to as a Bogoliubov transformation), then we have

$$\underline{b} = \begin{pmatrix} \kappa & \lambda \\ \lambda^* & \kappa^* \end{pmatrix} \underline{a}\,. \tag{32.37}$$

The matrices κ and λ are not arbitrary, since we require that the transformed b_j's and $b_j^\dagger$'s remain fermion operators, i.e., that they satisfy the fermion anticommutation relations

$$\{b_j, b_k\} = 0 \tag{32.38}$$

$$\{b_j, b_k^\dagger\} = \delta_{j,k}\,. \tag{32.39}$$

It is a good exercise to check that the condition that these relations are satisfied if an only if the matrix in (32.37) is unitary.

Although we will not describe the proof here,[1] it turns out that any quadratic fermion Hamiltonian can be diagonalized by such a transformation. In particular, it is always possible to choose κ, λ so that

$$H = \underline{b}^\dagger \begin{pmatrix} \omega & 0 \\ 0 & -\omega \end{pmatrix} \underline{b} + \operatorname{tr} \mu \tag{32.40}$$

[1] The diagonalization of H in the case of real μ, ν appears in [79]. For general μ, ν, as well as the case where we include terms that are linear in the fermion operators, see [41].

where ω is a diagonal matrix whose diagonal entries are the positive eigenvalues of the 2×2 block matrix (representing a $2n \times 2n$ matrix whose eigenvalues occur in $\pm$ pairs) appearing in (32.35). Expanding this expression, we have

$$H = \sum_{j=1}^{n} \omega_j (2 b_j^\dagger b_j - 1) + \operatorname{tr} \mu \tag{32.41}$$

where we have again used the fermion anticommutation relations. Since the $b_j^\dagger b_j$'s are commuting operators with eigenvalues 0 and 1, we see that spectrum of H is given by the 2^n numbers

$$\sum_{j=1}^{n} s_j \omega_j + \operatorname{tr} \mu \tag{32.42}$$

for each of the 2^n possible assignments of $s_1, s_2, \ldots, s_n = \pm 1$.

Calculating the eigenvalues ω_j is especially simple when μ, ν are real, as they are in the case of (32.32). In this case, we have (now treating R in (32.12) as a block matrix)

$$R \begin{pmatrix} \mu & -\nu \\ \nu & -\mu \end{pmatrix} R = \begin{pmatrix} 0 & \mu + \nu \\ \mu - \nu & 0 \end{pmatrix} . \tag{32.43}$$

Since the square of this matrix is

$$\begin{pmatrix} 0 & \mu + \nu \\ \mu - \nu & 0 \end{pmatrix}^2 = \begin{pmatrix} (\mu + \nu)(\mu - \nu) & 0 \\ 0 & (\mu - \nu)(\mu + \nu) \end{pmatrix} , \tag{32.44}$$

we see that the ω_j's are simply the positive square roots of the eigenvalues of the $n \times n$ matrix $(\mu + \nu)(\mu - \nu)$ (or equivalently, of $(\mu - \nu)(\mu + \nu)$).

Finally, we specialize to a Hamiltonian of the Ising form (32.32). Here we have

$$\mu = \frac{1}{2} \begin{pmatrix} 0 & J_1 & 0 & \cdots & 0 & J_n' \\ J_1 & 0 & J_2 & 0 & \cdots & 0 \\ 0 & J_2 & 0 & J_3 & \ddots & \vdots \\ \vdots & 0 & J_3 & \ddots & \ddots & 0 \\ 0 & \vdots & \ddots & \ddots & 0 & J_{n-1} \\ J_n' & 0 & \cdots & 0 & J_{n-1} & 0 \end{pmatrix} - \begin{pmatrix} h_1 & 0 & \cdots & 0 \\ 0 & h_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & h_n \end{pmatrix} \tag{32.45}$$

$$\nu = \frac{1}{2} \begin{pmatrix} 0 & J_1 & 0 & \cdots & 0 & -J_n' \\ -J_1 & 0 & J_2 & 0 & \cdots & 0 \\ 0 & -J_2 & 0 & J_3 & \ddots & \vdots \\ \vdots & 0 & -J_3 & \ddots & \ddots & 0 \\ 0 & \vdots & \ddots & \ddots & 0 & J_{n-1} \\ J_n' & 0 & \cdots & 0 & -J_{n-1} & 0 \end{pmatrix} , \tag{32.46}$$

so the matrix $(\mu + \nu)(\mu - \nu)$ is given by

$$\begin{pmatrix} J_1^2 + h_1^2 & -J_1 h_2 & 0 & \cdots & 0 & -J_n' h_1 \\ -J_1 h_2 & J_2^2 + h_2^2 & -J_2 h_3 & 0 & \cdots & 0 \\ 0 & -J_2 h_3 & J_3^2 + h_3^2 & -J_3 h_4 & \ddots & \vdots \\ \vdots & 0 & -J_3 h_4 & \ddots & \ddots & 0 \\ 0 & \vdots & \ddots & \ddots & J_{n-1}^2 + h_{n-1}^2 & -J_{n-1} h_n \\ -J_n' h_1 & 0 & \cdots & 0 & -J_{n-1} h_n & J_n^2 + h_n^2 \end{pmatrix} . \tag{32.47}$$

The eigenvalues corresponding to eigenstates with $G = \pm 1$ can be identified as follows. The transformation (32.37) is invertible, so any quadratic expression in the a_j's and $a_j^\dagger$'s can be written as a quadratic expression in the b_j's and $b_j^\dagger$'s. Since quadratic fermion operators do not change the parity of the total number of occupied modes, this means that the parity of the a modes is the same as the parity of the b modes. In other words,

$$G = (-1)^{\sum_{j=1}^n b_j^\dagger b_j} . \tag{32.48}$$

Thus the eigenvalues with $G = +1$ are those with an even number of s_j's equal to $+1$ in (32.42), whereas the eigenvalues with $G = -1$ are those with an odd number of s_j's equal to $+1$. In particular, we see that the gap between the ground and first excited states in the $G = +1$ subspace is equal to $2(\omega_1 + \omega_2)$, where ω_1 and ω_2 are the square roots of the two smallest eigenvalues of (32.47).

In the case of periodic boundary conditions, note that we have two distinct matrices (32.47), one for each value of G. However, with a fixed value of G, only half the possible assignments of the s_j's give rise to eigenvalues of the Hamiltonian, so we still find the correct number of eigenvalues. Here again, the gap between the ground and first excited states in the $G = +1$ subspace is equal to $2(\omega_1 + \omega_2)$, where now ω_1 and ω_2 are the square roots of the two smallest eigenvalues of (32.47) with $G = +1$.

32.4 Diagonalizing the ring of agrees

The Hamiltonian (32.5) is of the form (32.8) with $J_i = -s/2$ and $h_i = -(1-s)$ for each $i = 1, 2, \ldots, n$. (Note that we can neglect terms proportional to the identity, since they do not affect the gap.) Then, according to (32.47), the gap is given by twice the sum of the square roots of the two smallest eigenvalues of the matrix

$$(J_i^2 + h_i^2) - J_i h_i (D + D^{-1}) = \frac{1}{4}\left[s^2 + 4(1-s)^2 - 2s(1-s)(D + D^{-1})\right] , \tag{32.49}$$

where D is the skew-circulant matrix

$$D := \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & & & 0 & 1 \\ -1 & 0 & \cdots & 0 & 0 \end{pmatrix} \tag{32.50}$$

$$= \sum_{x=0}^{n-2} |x+1\rangle\langle x| - |0\rangle\langle n-1| . \tag{32.51}$$

Now just as the circulant matrix

$$C := \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & & & 0 & 1 \\ 1 & 0 & \cdots & 0 & 0 \end{pmatrix} \tag{32.52}$$

$$= \sum_{x=0}^{n-1} |x+1 \bmod n\rangle\langle x| \tag{32.53}$$

(and hence any circulant matrix) is diagonal in the Fourier basis

$$|\phi_k\rangle := \frac{1}{\sqrt{n}} \sum_{x=0}^{n-1} e^{2\pi i k x/n} |x\rangle \tag{32.54}$$

for $k = 0, 1, \ldots, n-1$, one can show that the matrix D (and hence any skew-circulant matrix) is diagonal in the *skew-Fourier basis*

$$|\chi_k\rangle := \frac{1}{\sqrt{n}} \sum_{x=0}^{n-1} e^{\pi \mathrm{i}(2k+1)x/n} |x\rangle \,, \tag{32.55}$$

also for $k = 0, 1, \ldots, n-1$. In particular,

$$D|\chi_k\rangle = e^{-\pi \mathrm{i}(2k+1)/n} |\chi_k\rangle \,. \tag{32.56}$$

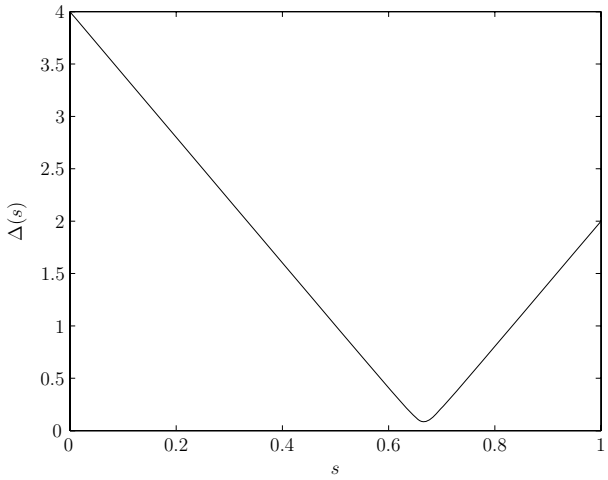
Thus, the eigenvalues of (32.49) are given by

$$\frac{1}{4}\left[s^2 + 4(1-s)^2 - 4s(1-s)\cos\frac{\pi(2k+1)}{n}\right] . \tag{32.57}$$

The smallest two eigenvalues (which are equal) occur for $k = 0$ and $k = n-1$, so the gap as a function of the interpolating parameter is

$$\Delta(s) = 2\sqrt{s^2 + 4(1-s)^2 - 4s(1-s)\cos\frac{\pi}{n}} \,, \tag{32.58}$$

which looks like this for $n = 50$:



For large n,

$$\cos\frac{\pi}{n} = 1 - \frac{\pi^2}{2n^2} + O(1/n^4) \,, \tag{32.59}$$

so

$$\Delta(s) = 2\sqrt{(2-3s)^2 + s(1-s)\frac{2\pi^2}{n^2} + O(1/n^4)} \,. \tag{32.60}$$

Setting $\mathrm{d}\Delta(s)^2/\mathrm{d}s$ equal to zero, we see that the minimum occurs at $s = 2/3 + O(1/n^2)$, at which the minimum gap is

$$\Delta = \frac{4\pi}{3n} + O(1/n^3) \,. \tag{32.61}$$

Since the minimum gap decreases only as $1/\operatorname{poly}(n)$, we see that adiabatic optimization can efficiently find a satisfying assignment for the ring of agrees. Even though the ring of agrees is not by itself an interesting computational problem, we can take this as preliminary evidence that adiabatic optimization sometimes succeeds.

However, it is also possible for the adiabatic algorithm to fail (at least for certain natural choices of the interpolating Hamiltonian), even for cost functions that are almost as simple as the ring of agrees. For example, suppose we have $4n$ spins arranged on a ring, and we define the cost function

$$h'(z) = \sum_{j=1}^{n}(1-\delta_{z_j,z_{j+1}}) + 2\sum_{j=n+1}^{2n}(1-\delta_{z_j,z_{j+1}}) + \sum_{j=2n+1}^{3n}(1-\delta_{z_j,z_{j+1}}) + 2\sum_{j=3n+1}^{4n}(1-\delta_{z_j,z_{j+1}}). \tag{32.62}$$

In other words, we again penalize a string when adjacent bits disagree, but the penalty is either 1 or 2 for contiguous blocks of n pairs of spins. In this case one can show that the gap is exponentially small. Unfortunately, we did not have time to discuss the details of this calculation.

Chapter 33

Universality of adiabatic quantum computation

In this final chapter, we see how adiabatic evolution can be used to implement an arbitrary quantum circuit [8]. In particular, this can be done with a local, linearly interpolated Hamiltonian. We may think of such Hamiltonians as describing a model of quantum computation. We know that this model can be efficiently simulated in the quantum circuit model. In this lecture we will see how the circuit model can be efficiently simulated by the adiabatic model, so that in fact the two models have equivalent computational power (up to polynomial factors).

This does not necessarily mean that there is an efficient adiabatic *optimization* algorithm for any problem that can be solved efficiently by a quantum computer. For example, Shor's algorithm shows that quantum computers can factor integers efficiently, yet we do not know if there is an adiabatic factoring algorithm that works by optimizing some cost function (such as the squared difference between the integer and a product of smaller integers). In general, it does not seem that the constructions of universal adiabatic quantum computers give much insight into how one might design efficient quantum adiabatic optimization algorithms. Nevertheless, they show that there is some sense in which the idea of adiabatic evolution captures much of the power of quantum computation.

33.1 The Feynman quantum computer

In a classic paper from the mid-1980s, Feynman presented a quantum mechanical model of a computer using local, time-independent Hamiltonian dynamics [51]. (Feynman's Hamiltonian has also been useful in quantum complexity, namely in formulating a complete problem for a quantum analog of the complexity class NP [70].) The motivation for this model was to show that quantum mechanics does not pose barriers to building a classical computer, despite quantum effects such as the uncertainty principle. Feynman showed that any sequence of reversible classical logic gates can be efficiently simulated using local Hamiltonian dynamics. However, his model applies equally well to simulate a quantum circuit.

Given a k-gate quantum circuit on n qubits, $U_k \cdots U_2 U_1$, let

$$H_F := \sum_{j=1}^{k} H_j \tag{33.1}$$

where

$$H_j := U_j \otimes |j\rangle\langle j-1| + U_j^\dagger \otimes |j-1\rangle\langle j| \,. \tag{33.2}$$

Here the first register consists of n qubits, and the second register stores a quantum state in a $(k+1)$-dimensional space spanned by states $|j\rangle$ for $j \in \{0, 1, \ldots, k\}$. The second register acts as a clock that records the progress of the computation. Later, we will show how to represent the clock using qubits, but for now, we treat it as a convenient abstraction.

If we start the computer in the state $|\psi\rangle \otimes |0\rangle$, then the evolved state remains in the subspace spanned by the $k+1$ states

$$|\psi_j\rangle := U_j \cdots U_1 |\psi\rangle \otimes |j\rangle \tag{33.3}$$

for $j \in \{0, 1, \ldots, k\}$. In this subspace, the nonzero matrix elements of H_F are

$$\langle \psi_j | H_F | \psi_{j\pm 1} \rangle = 1\,, \tag{33.4}$$

so the evolution is the same as that of a free particle propagating on a discretized line segment. Such a particle moves with constant speed, so in a time proportional to k, the initial state $|\psi_0\rangle$ will evolve to a state with substantial overlap on the state $|\psi_k\rangle = U_k \cdots U_1 |\psi\rangle |k\rangle$, corresponding to the final state of the computation. For large k, one can show that

$$|\langle \psi_k | e^{-\mathrm{i} H_F k/2} | \psi_0 \rangle|^2 = \Omega(k^{-2/3})\,, \tag{33.5}$$

so that after time $k/2$, a measurement of the clock will yield the result k, and hence give the final state of the computation, with a probability that is only polynomially small in the total number of gates in the original circuit.

The success probability of Feynman's computer can be made close to 1 by a variety of techniques. The simplest approach is to repeat the process $O(k^{2/3})$ times. Or we could pad the end of the computation with a large number of identity gates, boosting the probability that we reach a state in which the entire computation has been performed. Alternatively, as Feynman suggested, the success probability can be made arbitrarily close to 1 in single shot by preparing the initial state in a narrow wave packet that will propagate ballistically without substantial spreading. But perhaps the best approach is to make the process perfect by changing the Hamiltonian to

$$H_{FG} := \sum_{j=1}^{k} \sqrt{j(k+1-j)}\, H_j. \tag{33.6}$$

In this case, the choice $t = \pi$ gives the exact transformation $e^{-\mathrm{i} H_{FG} t} |\psi_0\rangle = |\psi_k\rangle$. This can be understood by viewing $|\psi_j\rangle$ as a state of total angular momentum $\frac{k}{2}(\frac{k}{2}+1)$ with z component $j - \frac{k}{2}$. Then H_{FG} is simply the x component of angular momentum, which rotates between the states with z component $\pm\frac{k}{2}$ in time π. Equivalently, H_{FG} can be viewed as the Hamiltonian in the Hamming weight subspace of a hypercube.

In the Hamiltonians (33.1) and (33.6), the clock space is not represented using qubits. However, we can easily create a Hamiltonian expressed entirely in terms of $k+1$ qubits using a unary representation of the clock. Let

$$|j\rangle := |\underbrace{0 \cdots 0}_{j} 1 \underbrace{0 \cdots 0}_{k-j}\rangle\,. \tag{33.7}$$

Then suppose we make the replacement

$$|j\rangle\langle j-1| \to \big(|01\rangle\langle 10|\big)^{(j-1,j)} \tag{33.8}$$

(and similarly for the adjoint), where the parenthesized superscript indicates which qubits are acted on. Then the subspace of states for which the clock register has the form (33.7) is invariant under the Hamiltonian, and within this subspace, its action is identical to that of the original Hamiltonian.

Notice that if the quantum circuit consists of one- and two-qubit gates, then the Hamiltonians (33.1) and (33.6) are local in the sense that the interactions involve at most four qubits. We call such a Hamiltonian 4-*local*.

This construction shows that even a time-independent Hamiltonian of a particularly simple form can be universal for quantum computation. Now let's see how we can modify the construction to use adiabatic evolution instead of a time-independent Hamiltonian.

33.2 An adiabatic variant

The construction of an adiabatic quantum computer will again involve two registers, the first holding the state of the quantum computation and the second representing a clock. The idea is to start from a Hamiltonian

whose ground state is the initial state of the computation together with the initial configuration of the clock, and to slowly evolve to a Hamiltonian (essentially, minus the Feynman Hamiltonian (33.1)) whose ground state encodes not the final state of the computation, but rather a uniform superposition over the entire history of the computation.

As before, we will find it convenient to start with an abstract description of the clock register in terms of $k+1$ basis states $|0\rangle, |1\rangle, \ldots, |k\rangle$, without worrying about how these states are represented in terms of qubits. Later, we will consider issues of locality in this type of construction.

For the beginning Hamiltonian, we will use

$$H_B := -I \otimes |0\rangle\langle 0| + H_{\text{penalty}} \tag{33.9}$$

where

$$H_{\text{penalty}} := \sum_{j=1}^{n} \left(|1\rangle\langle 1|\right)^{(j)} \otimes |0\rangle\langle 0|. \tag{33.10}$$

Here the parenthesized superscript again indicates which qubit is acted on. The first term of (33.9) says that the energy is lower if the clock is in the initial state $|0\rangle$. Adding H_{penalty} gives an energy penalty to states whose clock is in the state $|0\rangle$, yet for which the state of the computation is not the initial state $|00\ldots0\rangle$. Thus the unique ground state of H_B is $|00\ldots0\rangle \otimes |0\rangle$.

For the final Hamiltonian (which we denote H_C, since it encodes the final result of an arbitrary circuit, rather than the solution of a particular problem), we will use

$$H_C := -H_F + H_{\text{penalty}} \tag{33.11}$$

where H_F is the Feynman Hamiltonian defined in (33.1). From (33.4), we see that the $-H_F$ has a degenerate ground state subspace, where any state of the form

$$|\eta\rangle := \frac{1}{\sqrt{k+1}} \sum_{j=0}^{k} |\psi_j\rangle \tag{33.12}$$

(with $|\psi_j\rangle$ defined in (33.3)), with an arbitrary initial state $|\psi\rangle$, has minimal energy. Adding H_{penalty} penalizes those states for which the initial state of the computation is not $|00\ldots0\rangle$, so that (33.12) with $|\psi\rangle = |00\ldots0\rangle$ is the unique ground state of H_C. This state is almost as good as the final state of the computation, since if we measure the clock, we obtain the result k with probability $1/(k+1)$, which is $1/\operatorname{poly}(n)$ assuming the length of the circuit is only $k = \operatorname{poly}(n)$. By repeating the entire process $\operatorname{poly}(k)$ times, we can obtain the final state of the computation with high probability.

Finally, we use linear interpolation to get from H_B to H_C, defining

$$H(s) := (1-s)H_B + sH_C\,. \tag{33.13}$$

If we begin in the state $|00\ldots0\rangle \otimes |0\rangle$ and evolve according to $H_T(t) := H(t/T)$ for a sufficiently large time T, the adiabatic theorem guarantees that the final state will be close to $|\eta\rangle$. It remains to estimate the gap $\Delta(s)$ to show that $T = \operatorname{poly}(k)$ is sufficient.

In fact, the $(k+1)$-dimensional *computational subspace* spanned by the states $|\psi_j\rangle$ with $|\psi\rangle = |00\ldots0\rangle$ is invariant under $H(s)$, so it suffices to compute the gap within this subspace. Let us examine how $H(s)$ acts within the computational subspace. Note that $H_{\text{penalty}}|\psi_j\rangle = 0$ for all $j \in \{0, 1, \ldots, k\}$. We have

$$\langle\psi_j|H_B|\psi_{j'}\rangle = -\delta_{j,j'}\delta_{j,0} \tag{33.14}$$

and

$$\langle\psi_j|H_C|\psi_{j'}\rangle = -(\delta_{j,j'+1} + \delta_{j,j'-1})\,, \tag{33.15}$$

so we need to lower bound the gap between the smallest and second smallest eigenvalues of the matrix

$$\begin{pmatrix} s-1 & -s & 0 & \cdots & 0 \\ -s & 0 & -s & \ddots & \vdots \\ 0 & -s & 0 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -s \\ 0 & \cdots & 0 & -s & 0 \end{pmatrix}. \tag{33.16}$$

We will show

Lemma 33.1. *The gap between the smallest and second smallest eigenvalues of the matrix (33.16) for $s \in [0,1]$ is $\Omega(1/k^2)$*

Proof. The reduced Hamiltonian (33.16) essentially describes a free particle on a finite, discrete line, with a nonzero potential at one end. Thus the eigenstates are simply plane waves with a quantization condition determining the allowed values of the momentum. We will show the lower bound on the gap by analyzing this quantization condition.

We claim that the (unnormalized) eigenstates of (33.16), denoted $|E_p\rangle$, are given by

$$\langle \psi_j | E_p \rangle = \sin(p(k-j+1)) \tag{33.17}$$

for $j = 0, 1, \ldots, k$, and where p is yet to be determined. It is straightforward to verify that these states satisfy

$$\langle \psi_j | H(s) | E_p \rangle = E_p \langle \psi_j | E_p \rangle \tag{33.18}$$

for $j = 1, 2, \ldots, k$, with the energy given by

$$E_p = -2s \cos p. \tag{33.19}$$

(where p may be either real or imaginary). The allowed values of p are determined by the quantization condition obtained by demanding that (33.18) also holds at $j = 0$, i.e., that we have

$$-s \sin(kp) + (s-1) \sin((k+1)p) = E_p \sin((k+1)p)\,. \tag{33.20}$$

Using trigonometric identities, we can rewrite this condition as

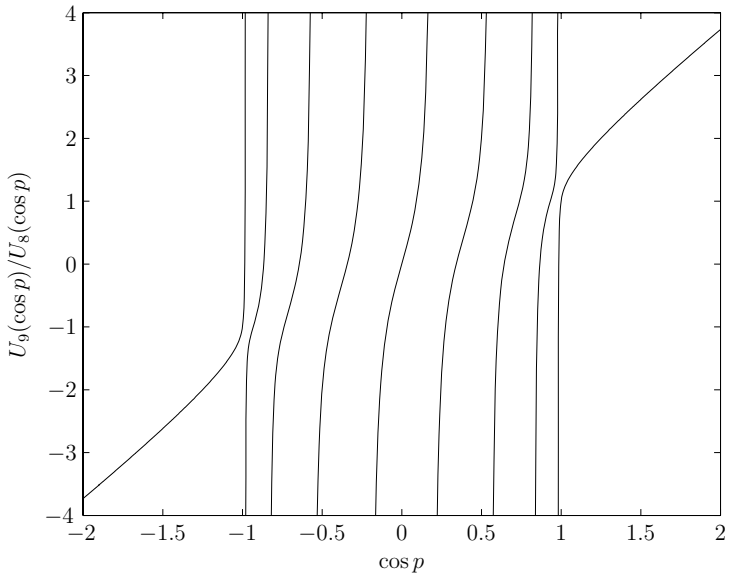
$$s \sin((k+2)p) = (1-s) \sin((k+1)p)\,, \tag{33.21}$$

or equivalently, in terms of Chebyshev polynomials, as

$$\frac{U_{k+1}(\cos p)}{U_k(\cos p)} = \frac{1-s}{s} \tag{33.22}$$

where $U_k(x)$ is the kth Chebyshev polynomial of the second kind, satisfying $U_k(\cos\theta) = \sin((k+1)\theta)/\sin\theta$.

The left hand side of (33.22) is shown below for $k = 8$. The intersections of this curve with the constant function $(1-s)/s$, when multiplied by $-2s$, give the eigenvalues E_p. Note that since p can be imaginary, $\cos p$ can be larger than 1 or smaller than -1.



Since the roots of $U_k(x)$ are given by $\cos\frac{j\pi}{k+1}$ for $j = 1, 2, \ldots, k$, the left hand side of (33.22) has simple poles at those values (and zeros at $\cos\frac{j\pi}{k+2}$ for $j = 1, 2, \ldots, k+1$). One can show that left hand side of (33.22) is strictly increasing. So there is one solution of (33.22) to the left of the leftmost pole, one between each pair of poles, and one to the right of the rightmost pole, giving a total of $k+1$ solutions, and thus accounting for all the eigenvalues of (33.16).

It remains to show that the gap between the two rightmost solutions of (33.22) is not too small. It is easy to see that the gap is $\Omega(1/k^3)$, because the ground state has $\cos p \geq \cos\frac{\pi}{k+2}$ (since it must occur to the right of the rightmost root), and the first excited state has $\cos p \leq \cos\frac{\pi}{k+1}$ (since it must occur to the left of the rightmost pole). This shows the gap is at least $2s(\cos\frac{\pi}{k+2} - \cos\frac{\pi}{k+1}) = \Omega(1/k^3)$ for constant s (and it is easy to show that the gap is a constant for $s = o(1)$).

However, we might like to prove a tighter result. To do this, we can separately consider the cases where the value of p corresponding the ground state is real (giving a plane wave) and where it is imaginary (giving a bound state). Since $U_{k+1}(1)/U_k(1) = (k+2)/(k+1)$, the value of s separating these two regimes is $s^* := (k+1)/(2k+3)$.

For $s \leq s^*$, the ground state has $\cos p \geq 1$, whereas the first excited state has $\cos p \leq \cos\frac{\pi}{k+1}$ (as observed above). Therefore, the gap satisfies

$$\Delta(s) \geq 2s\Big(1 - \cos\frac{\pi}{k+1}\Big) = \Omega(1/k^2) \tag{33.23}$$

for constant s (and as mentioned above, it is easy to see that $\Delta(s) = \Omega(1)$ for $s = o(1)$).

For $s > s^*$, the ground state has $\cos p \geq \cos\frac{\pi}{k+2}$ (as mentioned above). For the first excited state, we will show that the solution of (33.22) not only lies to the left of the rightmost pole, but that its distance from that pole is at least a constant fraction more than the distance of that pole from $\cos p = 1$. In particular, for any constant $a > 0$, we have

$$\frac{U_{k+1}(1-(1+a)(1-\cos\frac{\pi}{k+1}))}{U_k(1-(1+a)(1-\cos\frac{\pi}{k+1}))} = \frac{\sin((k+2)\cos^{-1}((1+a)\cos\frac{\pi}{k+1} - a))}{\sin((k+1)\cos^{-1}((1+a)\cos\frac{\pi}{k+1} - a))} \tag{33.24}$$

$$= 1 + \frac{\pi\sqrt{1+a}\cot(\pi\sqrt{1+a})}{k} + O(1/k^2) \tag{33.25}$$

where the second line follows by Taylor expansion. In comparison,

$$\frac{k+2}{k+1} = 1 + \frac{1}{k} + O(1/k^2)\,. \tag{33.26}$$

So if we fix (say) $a = 1$, then for k sufficiently large, (33.25) is larger than (33.26), which implies that the first excited state has $\cos p \leq 2\cos\frac{\pi}{k+1} - 1$. In turn, this implies that

$$\Delta(s) \geq 2s\left(\cos\frac{\pi}{k+2} - 2\cos\frac{\pi}{k+1} + 1\right) = \Omega(1/k^2)\,, \tag{33.27}$$

which completes the proof. □

33.3 Locality

The Hamiltonian (33.13) is local in terms of the computational qubits, but not in terms of the clock. However, it is possible to make the entire construction local.

The basic idea is again to use a unary representation of the clock, as in (33.7). We saw above that this makes H_F 4-local. However, H_B and H_{penalty} remain nonlocal with this clock, since they include the projector $|0\rangle\langle 0|$ acting on the clock register, which involves all $k+1$ of the clock qubits. Thus we must modify the construction slightly.

Let's try adding a term to H_{penalty} that penalizes clock states which are not of the correct form. To do this, it will be useful to change the unary representation from (33.7) to a form that can be checked locally, this time with $k+2$ qubits:

$$|j\rangle := |\underbrace{0\cdots0}_{j+1}\underbrace{1\cdots1}_{k-j+1}\rangle \tag{33.28}$$

for $j \in \{0, 1, \ldots, k\}$. (Note that the first qubit is always in the state $|0\rangle$, and the last qubit is always in the state $|1\rangle$.) Now we can verify that the clock state is of the form (33.28) by ensuring that there is no occurrence of the string "10" in the clock register, that the first bit is not "1", and that the last bit is not "0"; then we can check whether the clock is in its initial state by checking whether the second clock qubit is in the state $|1\rangle$. Thus, let us redefine

$$H_{\text{penalty}} := \sum_{j=1}^{n} \left(|1\rangle\langle 1|\right)^{(j)} \otimes (|0\rangle\langle 0|)^{(1)} + I \otimes (|1\rangle\langle 1|)^{(0)} + \sum_{j=1}^{k} I \otimes \left(|10\rangle\langle 10|\right)^{(j,j+1)} + I \otimes (|0\rangle\langle 0|)^{(k+2)} \quad (33.29)$$

where the parenthesized superscripts again indicate which qubits are acted on. We redefine the beginning Hamiltonian as

$$H_B := -I \otimes (|0\rangle\langle 0|)^{(1)} + H_{\text{penalty}}\,, \quad (33.30)$$

and in the Feynman term H_F of the computational Hamiltonian H_C, we make the replacement

$$|j\rangle\langle j-1| \to \left(|001\rangle\langle 011|\right)^{(j-1,j,j+1)} \quad (33.31)$$

(and similarly for the adjoint). With these redefinitions, the overall Hamiltonian $H(s) = (1-s)H_B + sH_C$ is 5-local, assuming as before that the gates in the quantum circuit to be simulated involve at most two qubits each.

As with the original nonlocal-clock construction, H_B and H_C have unique ground states $|0\ldots0\rangle \otimes |01\ldots1\rangle$ and $\frac{1}{\sqrt{k+1}} \sum_{j=0}^{k} U_j \cdots U_1 |0\ldots0\rangle \otimes |0^{j+1}1^{k-j+1}\rangle$, respectively. Again, the computational subspace spanned by the states $|\psi_j\rangle$ from (33.3) (but now with the clock representation (33.28)) is invariant under $H(s)$; and within this subspace, the Hamiltonian acts according to (33.16), which has a gap of $\Omega(1/k^2)$. Overall, this shows that there is a 5-local Hamiltonian $H(s)$ implementing an arbitrary quantum circuit by adiabatic evolution.

By suitable engineering, it's possible to produce variants of this construction with even better locality properties. One can even make the Hamiltonian *spatially* local, with nearest-neighbor interactions between qubits on a two-dimensional square lattice [90]. (In fact, one can even use a one-dimensional array of quantum systems, although not necessarily with qubits, but with higher-dimensional particles [5].)

Bibliography

[1] Scott Aaronson, *Quantum lower bound for the collision problem*, Proc. 34th ACM Symposium on Theory of Computing, pp. 635–642, 2002, quant-ph/0111102. [p. 105]

[2] Scott Aaronson and Andris Ambainis, *Quantum search of spatial regions*, Theory of Computing **1** (2005), 47–79, quant-ph/0303041, preliminary version in FOCS 2003. [p. 89]

[3] Scott Aaronson and Yaoyun Shi, *Quantum lower bounds for the collision and the element distinctness problems*, Journal of the ACM **51** (2004), no. 4, 595–605, quant-ph/0111102 and quant-ph/0112086, preliminary versions in STOC 2002 and FOCS 2002. [p. 91]

[4] Dorit Aharonov, Itai Arad, Elad Eban, and Zeph Landau, *Polynomial quantum algorithms for additive approximations of the Potts model and other points of the Tutte plane*, quant-ph/0702008. [p. 70]

[5] Dorit Aharonov, Daniel Gottesman, Sandy Irani, and Julia Kempe, *The power of quantum systems on a line*, Commun. Math. Phys. **287** (2009), no. 1, 41–65, arXiv:0705.4077. [p. 176]

[6] Dorit Aharonov, Vaughan Jones, and Zeph Landau, *A polynomial quantum algorithm for approximating the jones polynomial*, Proc. 38th ACM Symposium on Theory of Computing, pp. 427–436, 2006. [p. 67]

[7] Dorit Aharonov and Amnon Ta-Shma, *Adiabatic quantum state generation and statistical zero knowledge*, Proceedings of the 35th ACM Symposium on Theory of Computing, pp. 20–29, 2003, quant-ph/0301023. [p. 135]

[8] Dorit Aharonov, Wim van Dam, Julia Kempe, Zeph Landau, Seth Lloyd, and Oded Regev, *Adiabatic quantum computation is equivalent to standard quantum computation*, SIAM Journal on Computing **37** (2007), no. 1, 166–194, quant-ph/0405098, preliminary version in FOCS 2004. [p. 171]

[9] Gorjan Alagic, Cristopher Moore, and Alexander Russell, *Quantum algorithms for Simon's problem over general groups*, Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms, pp. 1217–1224, 2007, quant-ph/0603251. [p. 58]

[10] Andris Ambainis, *Quantum lower bounds by quantum arguments*, Journal of Computer and System Sciences **64** (2002), no. 4, 750–767, quant-ph/0002066, Preliminary version in STOC 2000. [p. 109]

[11] ———, *Quantum walk algorithm for element distinctness*, SIAM J. Comput. **37** (2007), no. 1, 210–239, quant-ph/0311001, preliminary version in FOCS 2004. [p. 91]

[12] Andris Ambainis, Julia Kempe, and Alexander Rivosh, *Coins make quantum walks faster*, Proceedings of the 16th ACM-SIAM Symposium on Discrete Algorithms, pp. 1099–1108, 2005, quant-ph/0402107. [p. 89]

[13] Itai Arad and Zeph Landau, *Quantum computation and the evaluation of tensor networks*, arXiv:0805.0040. [p. 70]

[14] D. Bacon, I. L. Chuang, and A. W. Harrow, *The quantum Schur transform: I. efficient qudit circuits*, Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 1235–1244, 2007, quant-ph/0601001. [p. 64]

[15] Dave Bacon, Andrew M. Childs, and Wim van Dam, *From optimal measurement to efficient quantum algorithms for the hidden subgroup problem over semidirect product groups*, Proceedings of the 46th IEEE Symposium on Foundations of Computer Science, pp. 469–478, 2005, quant-ph/0504083. [p. 59]

[16] Howard Barnum and Emanuel Knill, *Reversing quantum dynamics with near-optimal quantum and classical fidelity*, Journal of Mathematical Physics **43** (2002), no. 5, 2097–2106, quant-ph/0004088. [p. 45]

[17] R. Beals, *Quantum computation of Fourier transforms over symmetric groups*, Proc. 29th ACM Symposium on Theory of Computing, pp. 48–53, 1997. [p. 65]

[18] Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca, and Ronald de Wolf, *Quantum lower bounds by polynomials*, Journal of the ACM **48** (2001), no. 4, 778–797, quant-ph/9802049, preliminary version in FOCS 1998. [pp. 89, 98]

[19] Aleksandrs Belovs, *Span programs for functions with constant-sized 1-certificates*, Proceedings of the 44th Symposium on Theory of Computing, pp. 77–84, 2012, arXiv:1105.4024. [pp. 121, 124]

[20] Aleksandrs Belovs and Ansis Rosmanis, *Adversary lower bounds for the collision and the set equality problems*, arXiv:1310.5185. [p. 104]

[21] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders, *Efficient quantum algorithms for simulating sparse Hamiltonians*, Communications in Mathematical Physics **270** (2007), no. 2, 359–371, quant-ph/0508139. [pp. 133, 135, 139]

[22] Dominic W. Berry and Andrew M. Childs, *Black-box Hamiltonian simulation and unitary implementation*, Quantum Information and Computation **12** (2012), no. 1-2, 29–62, arXiv:0910.4157. [p. 140]

[23] Dominic W. Berry, Andrew M. Childs, Richard Cleve, Robin Kothari, and Rolando D. Somma, *Exponential improvement in precision for simulating sparse Hamiltonians*, Proceedings of the 46th ACM Symposium on Theory of Computing, pp. 283–292, 2014, arXiv:1312.1414. [pp. 135, 140, 141]

[24] ———, *Simulating Hamiltonian dynamics with a truncated Taylor series*, Physical Review Letters **114** (2015), no. 9, 090502, arXiv:1412.4687. [p. 140]

[25] Dominic W. Berry, Andrew M. Childs, and Robin Kothari, *Hamiltonian simulation with nearly optimal dependence on all parameters*, Proceedings of the 56th IEEE Symposium on Foundations of Computer Science, pp. 792–809, 2015, arXiv:1501.01715. [pp. 144, 148]

[26] Adam Bouland and Tudor Giurgica-Tiron, *Efficient universal quantum compilation: An inverse-free Solovay-Kitaev algorithm*, arXiv:2112.02040. [p. 10]

[27] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp, *Quantum amplitude amplification and estimation*, Quantum Computation and Information (S. J. Lomonaco and H. E. Brandt, eds.), AMS Contemporary Mathematics Series, vol. 305, AMS, Providence, RI, 2002, quant-ph/0005055. [p. 89]

[28] Gilles Brassard, Peter Høyer, and Alain Tapp, *Quantum algorithm for the collision problem*, quant-ph/9705002. [p. 104]

[29] Harry Buhrman, Richard Cleve, and Avi Wigderson, *Quantum vs. classical communication and computation*, Proceedings of the 30th ACM Symposium on Theory of Computing, pp. 63–68, 1998, quant-ph/9802040. [p. 118]

[30] Harry Buhrman, Christoph Dürr, Mark Heiligman, Peter Høyer, Frédéric Magniez, Miklos Santha, and Ronald de Wolf, *Quantum algorithms for element distinctness*, SIAM Journal on Computing **34** (2005), no. 6, 1324–1330, quant-ph/0007016, preliminary version in CCC 2001. [p. 91]

[31] Kevin K. H. Cheung and Michele Mosca, *Decomposing finite abelian groups*, Quantum Information and Computation **1** (2001), no. 3, 26–32, cs.DS/0101004. [p. 27]

[32] Andrew M. Childs, *Quantum information processing in continuous time*, Ph.D. thesis, Massachusetts Institute of Technology, 2004. [p. 135]

[33] ———, *On the relationship between continuous- and discrete-time quantum walk*, Communications in Mathematical Physics **294** (2010), no. 2, 581–603, arXiv:0810.0312. [p. 140]

[34] Andrew M. Childs, Richard Cleve, Enrico Deotto, Edward Farhi, Sam Gutmann, and Daniel A. Spielman, *Exponential algorithmic speedup by quantum walk*, Proceedings of the 35th ACM Symposium on Theory of Computing, pp. 59–68, 2003, quant-ph/0209131. [p. 75]

[35] Andrew M. Childs and Jeffrey Goldstone, *Spatial search by quantum walk*, Physical Review A **70** (2004), no. 2, 022314, quant-ph/0306054. [p. 89]

[36] Andrew M. Childs, Aram W. Harrow, and Pawel Wocjan, *Weak Fourier-Schur sampling, the hidden subgroup problem, and the quantum collision problem*, Proceedings of the 24th Symposium on Theoretical Aspects of Computer Science, Lecture Notes in Computer Science, vol. 4393, pp. 598–609, 2007, quant-ph/0609110. [p. 63]

[37] Andrew M. Childs, David Jao, and Vladimir Soukharev, *Constructing elliptic curve isogenies in quantum subexponential time*, Journal of Mathematical Cryptology **8** (2014), no. 1, 1–29, arXiv:1012.4019. [p. 58]

[38] Andrew M. Childs and Wim van Dam, *Quantum algorithms for algebraic problems*, Reviews of Modern Physics **82** (2010), no. 1, 1–52, arXiv:0812.0380. [p. vii]

[39] Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao, *On the compressed-oracle technique, and post-quantum security of proofs of sequential work*, Advances in Cryptology – EUROCRYPT 2021, pp. 598–629, 2021, arXiv:2010.11658. [p. 126]

[40] Richard Cleve, Artur Ekert, Chiara Macchiavello, and Michele Mosca, *Quantum algorithms revisited*, Proceedings of the Royal Society of London A **454** (1998), no. 1969, 339–354, quant-ph/9708016. [pp. 19, 89]

[41] J. H. P. Colpa, *Diagonalisation of the quadratic fermion hamiltonian with a linear part*, J. Phys. A **12** (1979), no. 4, 469–488. [p. 166]

[42] Wim van Dam, Michele Mosca, and Umesh Vazirani, *How powerful is adiabatic quantum computation?*, Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science, pp. 279–287, 2001, quant-ph/0206003. [p. 159]

[43] Christopher M. Dawson and Michael A. Nielsen, *The Solovay-Kitaev algorithm*, Quantum Information and Computation **6** (2006), no. 1, 81–95, quant-ph/0505030. [p. 7]

[44] Kirsten Eisenträger, Sean Hallgren, Alexei Kitaev, and Fang Song, *A quantum algorithm for computing the unit group of an arbitrary degree number field*, Proceedings of the 46th ACM Symposium on Theory of Computing, pp. 293–302, 2014. [p. 41]

[45] Mark Ettinger and Peter Høyer, *On quantum algorithms for noncommutative hidden subgroups*, Advances in Applied Mathematics **25** (2000), 239–251, quant-ph/9807029. [pp. 54, 58]

[46] Mark Ettinger, Peter Høyer, and Emanuel Knill, *The quantum query complexity of the hidden subgroup problem is polynomial*, Information Processing Letters **91** (2004), no. 1, 43–48, quant-ph/0401083. [p. 45]

[47] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann, *A quantum algorithm for the Hamiltonian NAND tree*, Theory of Computing **4** (2008), no. 1, 169–190, quant-ph/0702144. [pp. 118, 119]

[48] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser, *Quantum computation by adiabatic evolution*, quant-ph/0001106. [pp. 157, 163]

[49] Edward Farhi and Sam Gutmann, *Quantum computation and decision trees*, Physical Review A **58** (1998), no. 2, 915–928, quant-ph/9706062. [p. 73]

[50] Richard P. Feynman, *Simulating physics with computers*, International Journal of Theoretical Physics **21** (1982), no. 6-7, 467–488. [p. 133]

[51] ———, *Quantum mechanical computers*, Optics News **11** (1985), 11–20. [p. 171]

[52] M. H. Freedman, A. Yu. Kitaev, M. J. Larsen, and Z. Wang, *Topological quantum computation*, Bull. Amer. Math. Soc. **40** (2003), 31–38, quant-ph/0101025. [p. 67]

[53] Brett Giles and Peter Selinger, *Remarks on Matsumoto and Amano's normal form for single-qubit Clifford+T operators*, arXiv:1312.6584. [pp. 11, 14]

[54] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe, *Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics*, Proceedings of the 51st Annual ACM Symposium on Theory of Computing, pp. 193–204, 2019, arXiv:1806.01838. [pp. 143, 144, 147, 148]

[55] Daniel Gottesman, *An introduction to quantum error correction and fault-tolerant quantum computation*, Quantum Information Science and Its Contributions to Mathematics (Jr. Samuel J. Lomonaco, ed.), Proceedings of Symposia in Applied Mathematics, vol. 68, AMS, 2010, arXiv:0904.2557. [p. 4]

[56] M. Grigni, L. J. Schulman, M. Vazirani, and U. Vazirani, *Quantum mechanical algorithms for the nonabelian hidden subgroup problem*, Combinatorica **24** (2004), no. 1, 137–154, preliminary version in STOC 2001. [p. 53]

[57] Lov K. Grover, *Quantum mechanics helps in searching for a needle in a haystack*, Physical Review Letters **79** (1997), no. 2, 325–328, quant-ph/9706033, preliminary version in STOC 1996. [p. 87]

[58] S. Hallgren, C. Moore, M. Rötteler, A. Russell, and P. Sen, *Limitations of quantum coset states for graph isomorphism*, Proc. 38th ACM Symposium on Theory of Computing, pp. 604–617, 2006, quant-ph/0511148, quant-ph/0511149. [p. 54]

[59] S. Hallgren, A. Russell, and A. Ta-Shma, *The hidden subgroup problem and quantum computation using group representations*, SIAM Journal on Computing **32** (2003), no. 4, 916–934, preliminary version in STOC 2000. [p. 52]

[60] Sean Hallgren, *Fast quantum algorithms for computing the unit group and class group of a number field*, Proc. 37th ACM Symposium on Theory of Computing, pp. 468–474, 2005. [p. 41]

[61] ———, *Polynomial-time quantum algorithms for Pell's equation and the principal ideal problem*, Journal of the ACM **54** (2007), no. 1, article 4, preliminary version in STOC 2002. [pp. 33, 41]

[62] A. W. Harrow, *Applications of coherent classical communication and the Schur transform to quantum information theory*, Ph.D. thesis, Massachusetts Institute of Technology, September 2005, quant-ph/0512255. [p. 64]

[63] P. Høyer, T. Lee, and R. Špalek, *Negative weights make adversaries stronger*, Proc. 39th ACM Symposium on Theory of Computing, pp. 526–535, 2007, quant-ph/0611054. [pp. 109, 110]

[64] P. Høyer, M. Mosca, and R. de Wolf, *Quantum search on bounded-error inputs*, Proc. 30th International Colloquium on Automata, Languages, and Programming, Lecture Notes in Computer Science, vol. 2719, pp. 291–299, 2003, quant-ph/0304052. [p. 118]

[65] P. Høyer and R. Špalek, *Lower bounds on quantum query complexity*, Bulletin of the European Association for Theoretical Computer Science **87** (2005), 78–103, quant-ph/0509153. [p. 97]

[66] Sabine Jansen, Mary-Beth Ruskai, and Ruedi Seiler, *Bounds for the adiabatic approximation with applications to quantum computation*, Journal of Mathematical Physics **48** (2007), 102111, quant-ph/0603175. [p. 152]

[67] Richard Jozsa, *Quantum computation in algebraic number theory: Hallgren's efficient quantum algorithm for solving Pell's equation*, Annals of Physics **306** (2003), no. 2, 241–279, quant-ph/0302134. [p. 33]

[68] Phillip Kaye, Raymond Laflamme, and Michele Mosca, *An introduction to quantum computing*, Oxford University Press, 2007. [pp. 1, 109]

[69] Kiran S. Kedlaya, *Quantum computation of zeta functions of curves*, Computational Complexity **15** (2006), no. 1, 1–19, math.NT/0411623. [p. 30]

[70] Alexei Yu. Kitaev, Alexander H. Shen, and Mikhail N. Vyalyi, *Classical and quantum computation*, AMS, 2002. [pp. 1, 7, 19, 171]

[71] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca, *Fast and efficient exact synthesis of single qubit unitaries generated by Clifford and T gates*, Quantum Information and Computation **13** (2013), no. 7-8, 607–630, arXiv:1206.5236. [p. 11]

[72] Emanuel Knill, *Approximation by quantum circuits*, Tech. Report LAUR-95-2225, Los Alamos National Laboratory, 1995, quant-ph/9508006. [p. 11]

[73] Hari Krovi, *An efficient high dimensional quantum Schur transform*, Quantum **3** (2019), 122, arXiv:1804.00055. [p. 64]

[74] Greg Kuperberg, *A subexponential-time quantum algorithm for the dihedral hidden subgroup problem*, SIAM Journal on Computing **35** (2005), no. 1, 170–188, quant-ph/0302112. [p. 55]

[75] Samuel Kutin, *Quantum lower bound for the collision problem with small range*, Theory of Computing **1** (2005), no. 2, 29–36, quant-ph/0304162. [pp. 105, 107]

[76] Troy Lee, Frédéric Magniez, and Miklos Santha, *Learning graph based quantum query algorithms for finding constant-size subgraphs*, Chicago Journal of Theoretical Computer Science (2012), no. 10, arXiv:1109.5135. [p. 124]

[77] ———, *Improved quantum query algorithms for triangle finding and associativity testing*, Proceedings of the 24th ACM-SIAM Symposium on Discrete Algorithms, pp. 1486–1502, 2013, arXiv:1210.1014. [p. 124]

[78] Troy Lee, Rajat Mittal, Ben W. Reichardt, Robert Špalek, and Mario Szegedy, *Quantum query complexity of state conversion*, Proceedings of the 52nd IEEE Symposium on Foundations of Computer Science, pp. 344–353, 2011, arXiv:1011.3020. [pp. 109, 115, 118, 119, 120]

[79] E. Lieb, T. Schultz, and D. Mattis, *Two soluble models of an antiferromagnetic chain*, Ann. Phys. **16** (1961), no. 3, 407–466. [p. 166]

[80] Guang Hao Low and Isaac L. Chuang, *Optimal Hamiltonian simulation by quantum signal processing*, Physical Review Letters **118** (2017), 010501, arXiv:1606.02685. [p. 143]

[81] ———, *Hamiltonian simulation by qubitization*, Quantum **3** (2019), 163, arXiv:1610.06546. [p. 143]

[82] Guang Hao Low, Theodore J. Yoder, and Isaac L. Chuang, *The methodology of resonant equiangular composite quantum gates*, Physical Review X **6** (2016), 041067, arXiv:1603.03996. [p. 144]

[83] Frédéric Magniez, Ashwin Nayak, Jérémie Roland, and Miklos Santha, *Search via quantum walk*, SIAM Journal on Computing **40** (2011), no. 1, 142–164, quant-ph/0608026. [pp. 91, 93]

[84] Ken Matsumoto and Kazuyuki Amano, *Representation of quantum circuits with Clifford and $\pi/8$ gates*, arXiv:0806.3834. [p. 11]

[85] C. Moore, A. Russell, and L. J. Schulman, *The symmetric group defies strong Fourier sampling*, Proc. 46th IEEE Symposium on Foundations of Computer Science, pp. 479–490, 2005, quant-ph/0501056. [p. 54]

[86] Cristopher Moore, Daniel N. Rockmore, Alexander Russell, and Leonard J. Schulman, *The power of strong fourier sampling: Quantum algorithms for affine groups and hidden shifts*, SIAM J. Comput. **37** (2007), no. 3, 938–958, quant-ph/0503095, preliminary version in SODA 2004. [p. 53]

[87] Cristopher Moore, Alexander Russell, and Piotr Sniady, *On the impossibility of a quantum sieve algorithm for graph isomorphism*, Proc. 29th ACM Symposium on Theory of Computing, pp. 536–545, 2007, quant-ph/0612089. [p. 58]

[88] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*, Cambridge University Press, Cambridge, 2000. [pp. 1, 7, 11]

[89] Ryan O'Donnell and John Wright, *Quantum spectrum testing*, Proceedings of the 47th Annual ACM Symposium on Theory of Computing, pp. 529–538, 2015. [p. 65]

[90] R. Oliveira and B. M. Terhal, *The complexity of quantum spin systems on a two-dimensional square lattice*, Quantum Information and Computation **8** (2008), no. 10, 900–924, quant-ph/0504050. [p. 176]

[91] Oded Regev, *A subexponential time algorithm for the dihedral hidden subgroup problem with polynomial space*, quant-ph/0406151. [p. 58]

[92] B. W. Reichardt, *Span programs and quantum query complexity: The general adversary bound is nearly tight for every boolean function*, Proc. 50th IEEE Symposium on Foundations of Computer Science, pp. 544–551, 2009, 0904.2759. [pp. 115, 116, 117]

[93] ———, *Reflections for quantum query algorithms*, Proceedings of the 22nd ACM-SIAM Symposium on Discrete Algorithms, pp. 560–569, 2011, 1005.1601. [pp. 109, 115, 118, 119]

[94] B. W. Reichardt and R. Špalek, *Span-program-based quantum algorithm for evaluating formulas*, Proc. 40th ACM Symposium on Theory of Computing, pp. 103–112, 2008, arXiv:0710.2630. [pp. 115, 116]

[95] J. Roland and N. J. Cerf, *Quantum search by local adiabatic evolution*, Physical Review A **65** (2002), no. 4, 042308, quant-ph/0107015. [p. 159]

[96] M. Saks and A. Wigderson, *Probabilistic boolean decision trees and the complexity of evaluating game trees*, Proc. 27th IEEE Symposium on Foundations of Computer Science, pp. 29–38, 1986. [pp. 118, 119]

[97] Miklos Santha, *On the Monte Carlo Boolean decision tree complexity of read-once formulae*, Random Structures and Algorithms **6** (1995), no. 1, 75–87. [p. 118]

[98] Miklos Santha, *Quantum walk based search algorithms*, Theory and Applications of Models of Computation, Lecture Notes in Computer Science, vol. 4978, Springer, 2008, arXiv:0808.0059, pp. 31–46. [p. 91]

[99] Arthur Schmidt and Ulrich Vollmer, *Polynomial time quantum algorithm for the computation of the unit group of a number field*, Proc. 37th ACM Symposium on Theory of Computing, pp. 475–480, 2005. [p. 41]

[100] Pranab Sen, *Random measurement bases, quantum state distinction and applications to the hidden subgroup problem*, Proc. 21st IEEE Conference on Computational Complexity (2006), 274–287, quant-ph/0512085. [p. 53]

[101] Jean-Pierre Serre, *Linear representations of finite groups*, Graduate Texts in Mathematics, vol. 42, Springer, 1977. [p. 47]

[102] Simone Severini, *On the digraph of a unitary matrix*, SIAM Journal on Matrix Analysis and Applications **25** (2003), no. 1, 295–300, math.CO/0205187. [p. 81]

[103] Neil Shenvi, Julie Kempe, and K. Birgitta Whaley, *A quantum random walk search algorithm*, Physical Review A **67** (2003), no. 5, 052307, quant-ph/0210064. [p. 89]

[104] Yaoyun Shi, *Quantum lower bounds for the collision and the element distinctness problems*, Proceedings of the 43rd IEEE Symposium on Foundations of Computer Science, pp. 513–519, 2002, quant-ph/0112086. [p. 105]

[105] Peter W. Shor, *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*, SIAM Journal on Computing **26** (1997), no. 5, 1484–1509, quant-ph/9508027, preliminary version in FOCS 1994. [p. 23]

[106] D. Simon, *On the power of quantum computation*, Proc. 35th IEEE Symposium on Foundations of Computer Science, pp. 116–123, 1994. [p. 22]

[107] D. R. Simon, *On the power of quantum computation*, SIAM Journal on Computing **26** (1997), no. 5, 1474–1483, preliminary version in FOCS 1994. [p. 17]

[108] M. Snir, *Lower bounds on probabilistic linear decision trees*, Theoretical Computer Science **38** (1985), 69–82. [pp. 118, 119]

[109] Mario Szegedy, *Quantum speed-up of Markov chain based algorithms*, Proceedings of the 45th IEEE Symposium on Foundations of Computer Science, pp. 32–41, 2004, quant-ph/0401053. [pp. 82, 84]

[110] Stefan Teufel, *Adiabatic perturbation theory in quantum dynamics*, Lecture Notes in Mathematics, vol. 1821, Springer-Verlag, 2003. [p. 152]

[111] John Watrous, *Quantum simulations of classical random walks and undirected graph connectivity*, Journal of Computer and System Sciences **62** (2001), no. 2, 376–391, cs.CC/9812012. [p. 81]

[112] ———, *Theory of quantum information*, lecture notes, 2011, http://cs.uwaterloo.ca/~watrous/LectureNotes.html. [p. 115]

[113] Pawel Wocjan and Jon Yard, *The Jones polynomial: Quantum algorithms and applications in quantum complexity theory*, quant-ph/0603069. [p. 70]

[114] Mark Zhandry, *How to record quantum queries, and applications to quantum indifferentiability*, Advances in Cryptology – CRYPTO 2019, pp. 239–268, 2019. [p. 125]

[115] Yechao Zhu, *Quantum query complexity of subgraph containment with constant-sized certificates*, International Journal of Quantum Information **10** (2012), no. 3, 1250019, arXiv:1109.4165. [p. 124]